

FastAPI 프로젝트 학습 — 주문·이커머스 API 만들기

목차

1장. FastAPI 기초: 첫 앱과 자동 문서 [기초]

- 도입: 왜 FastAPI인가 — 타입 힌트로 검증·문서까지
- 정의: FastAPI는 Starlette+Pydantic 위의 ASGI 프레임워크
- 동작 원리: 클라이언트→uvicorn→앱→경로 동작→응답
- 개발 환경 세팅: Windows 11·VS Code·Docker·PostgreSQL
- worked example: 헬스체크와 상품 목록 엔드포인트
- 실행과 동작 테스트: 앱 띄우기와 동작 확인
- 자동 문서: /docs-/redoc/openapi.json 살펴보기
- 즉시 연습(2종 1세트): 상품 카탈로그 엔드포인트 채우기
- 흔한 오류와 오개념: 앱 vs 서버·경로 인자·문서 자동생성
- 단원 미니 프로젝트: 상품 카탈로그 첫 API(2종 1세트)
- 자주 묻는 질문(FAQ)

2장. 경로·쿼리 파라미터: 입력을 타입으로 받기 [기초]

- 도입: URL에 담겨 오는 입력 다루기(c01 상기)
- 정의: 경로 파라미터와 쿼리 파라미터
- 동작 원리: URL 분해와 타입 변환·검증 흐름
- worked example: GET /products/{product_id} 단건 조회
- 쿼리 파라미터와 기본값·선택값(| None)
- Query()로 제약: ge·le·min_length·pattern
- worked example: 카테고리 필터+페이지네이션 목록
- Enum 경로 파라미터와 경로 순서·충돌
- 즉시 연습(2종 1세트): 페이지 크기 제약 채우기
- 흔한 오류와 오개념: 자동 변환·경로 순서·필수/선택
- 단원 미니 프로젝트: 상품 조회 파라미터 API(2종 1세트)

3장. Pydantic 모델: 요청 본문 검증 [기초]

- 도입: 본문(JSON)을 안전한 객체로(c02 상기)
- 정의: Pydantic v2 BaseModel과 요청 본문
- 동작 원리: JSON→파싱/검증→모델 객체 or 422
- worked example: ProductCreate로 POST 본문 검증
- Field()로 제약·기본값·예시
- 중첩 모델과 list[Model]: OrderCreate(items)
- worked example: field_validator로 가격 규칙 강제
- 즉시 연습(2종 1세트): 검증 규칙 채우기
- 흔한 오류와 오개념: v1→v2 변화·수동검사 불필요·필수 필드
- 단원 미니 프로젝트: 상품 등록 검증 API(2종 1세트)

4장. 응답 모델·상태 코드: 출력을 통제하기 [중급]

- 도입: 들어온 모델과 내보낼 모델은 다르다(c03 상기)
- 정의: response_model과 status_code
- 동작 원리: 반환 객체→필터/직렬화→상태코드 응답
- worked example: 201과 ProductRead로 내부 필드 숨기기
- 입력·출력 모델 분리(Create vs Read)
- worked example: UserRead로 password_hash 제외
- exclude_unset·responses=로 응답 가공·문서화
- 즉시 연습(2종 1세트): 응답 모델 채우기
- 흔한 오류와 오개념: 필터링·기본 200·from_attributes
- 단원 미니 프로젝트: 안전한 응답 모델 API(2종 1세트)

5장. async/await: 비동기 동시성 [중급]

- 도입: I/O를 기다리는 동안 다른 일 하기
- 정의: 이벤트 루프와 async/await
- 동작 원리: 동기 직렬 vs 비동기 인터리브 타임라인
- worked example: httpx.AsyncClient로 배송 추정 await
- asyncio.gather로 여러 외부 호출 동시 실행
- 일반 def 핸들러와 스레드풀·블로킹 회피
- worked example: 블로킹 vs 비동기 응답시간 비교
- 즉시 연습(2종 1세트): gather 동시 호출 채우기
- 흔한 오류와 오개념: 무조건 async-await 누락·스레드풀
- 단원 미니 프로젝트: 비동기 외부 조회 엔드포인트(2종 1세트)

6장. 의존성 주입: Depends로 공통 로직 분리 [중급]

- 도입: 반복되는 공통 로직을 주입으로(c05 상기)
- 정의: Depends와 의존성 함수

- 동작 원리: 의존성 그래프 합성과 yield 정리 타임라인
- worked example: get_pagination 공통 쿼리 의존성
- Annotated[T, Depends()]와 타입 별칭(PaginationDep)
- worked example: yield로 세션 setup/teardown
- 하위 의존성·클래스 의존성·의존성 캐시
- 즉시 연습(2종 1세트): yield 의존성 채우기
- 흔한 오류와 오개념: 괄호 없이 넘기기·캐시·정리 시점
- 단원 미니 프로젝트: 공통 의존성 모듈(2종 1세트)

7장. 데이터베이스 연동: SQLAlchemy로 영속화 [중급]

- 도입: 메모리가 아니라 DB에 남기기(c06 상기)
- 정의: SQLAlchemy(Pydantic)과 engine·Session
- 동작 원리: 요청→세션→트랜잭션→테이블 커밋/롤백
- worked example: Product 테이블과 상품 저장
- select()로 필터·정렬 조회
- worked example: get_session 의존성으로 트랜잭션 관리
- 테이블/입력/출력 모델 분리와 마이그레이션 개요
- 즉시 연습(2종 1세트): 저장·조회 채우기
- 흔한 오류와 오개념: commit/refresh·모델 노출·세션 공유
- 단원 미니 프로젝트: 상품 DB 영속화(2종 1세트)

8장. CRUD 라우터: APIRouter·페이지네이션·필터 [실무]

- 도입: 큰 앱을 자원별 모듈로(c07 상기)
- 정의: APIRouter와 prefix·tags·include_router
- 동작 원리: main ← include_router 모듈 구조도
- worked example: products 라우터로 표준 CRUD
- limit/offset 페이지네이션
- worked example: 필터·정렬·PATCH 부분 갱신
- 404 처리와 일관된 자원 응답
- 즉시 연습(2종 1세트): 페이지네이션·PATCH 채우기
- 흔한 오류와 오개념: 접두어 중복·PATCH vs PUT·전체 반환
- 단원 미니 프로젝트: 상품 CRUD 라우터(2종 1세트)

9장. 인증·인가: OAuth2와 JWT [실무]

- 도입: 누구의 요청인가·권한이 있는가(c08 상기)
- 정의: OAuth2PasswordBearer·JWT·해시
- 동작 원리: 로그인→해시검증→JWT→Bearer→검증 시퀀스
- worked example: 비밀번호 해시와 /token 발급

- JWT 생성·검증(PyJWT, exp·서명·알고리즘)
- worked example: get_current_user로 보호 라우트
- 권한(관리자 vs 일반)과 401 vs 403
- 즉시 연습(2종 1세트): 토큰 검증 채우기
- 흔한 오류와 오개념: JWT는 서명·해시 저장·인증 vs 인가
- 단원 미니 프로젝트: 로그인·보호 라우트(2종 1세트)

10장. 에러 처리·미들웨어·CORS: 견고한 API [실무]

- 도입: 오류를 일관된 응답으로(c08 상기)
- 정의: HTTPException·예외 핸들러·미들웨어·CORS
- 동작 원리: 미들웨어 체인→라우트→예외 핸들러 변환
- worked example: 재고 부족 도메인 예외→409
- RequestValidationError 가공과 일관 오류 포맷
- worked example: X-Process-Time 미들웨어
- CORSMiddleware 설정(출처·자격증명)
- 즉시 연습(2종 1세트): 예외 핸들러 채우기
- 흔한 오류와 오개념: raise vs return·미들웨어 순서·CORS 정책
- 단원 미니 프로젝트: 도메인 예외·CORS 설정(2종 1세트)

11장. 테스트: pytest와 TestClient [실무]

- 도입: 손이 아니라 코드로 검증하기(c10 상기)
- 정의: TestClient와 pytest 픽스처
- 동작 원리: 픽스처→오버라이드→요청→단언 흐름
- worked example: 상품 생성→조회 일치 테스트
- dependency_overrides로 인메모리 DB 교체
- worked example: 토큰 발급→보호 200·무토큰 401
- 검증 오류(422) 단언과 테스트 격리
- 즉시 연습(2종 1세트): 오버라이드·단언 채우기
- 흔한 오류와 오개념: 운영 DB·async 호출·픽스처 초기화
- 단원 미니 프로젝트: 엔드포인트 테스트 스위트(2종 1세트)

12장. 프로덕션: 백그라운드 작업·설정·배포 [실무]

- 도입: 응답 후 작업과 운영 준비(c11 상기)
- 정의: BackgroundTasks·lifespan·pydantic-settings
- 동작 원리: 응답→백그라운드, .env→Settings→앱→Docker 배포
- worked example: 주문 확인 메일 백그라운드(모킹)
- lifespan으로 시작/종료 자원 관리
- worked example: Settings로 환경변수·시크릿 로드

- uvicorn 워커와 멀티스테이지 Dockerfile
- 즉시 연습(2종 1세트): BackgroundTasks·Settings 채우기
- 흔한 오류와 오개념: 같은 프로세스·시크릿 하드코딩·reload
- 단원 미니 프로젝트: 백그라운드 작업과 도커 배포(2종 1세트)

13장. 종합 프로젝트: 주문·이커머스 API — 분석(요구사항·범위) [실무]

- 드라이빙 퀘스천과 POV 다시 보기
- 기능 요구사항(FR): 상품·주문·인증·결제
- 비기능 요구사항(NFR): 검증·트랜잭션·비동기·테스트·배포
- 사용자 스토리와 유스케이스(주문 생성·조회·취소)
- 범위 In/Out 확정과 결제·메일 모킹 결정
- 추적성: 요구사항 ↔ 단원 개념 매핑
- 작성형 연습(2종 1세트): 요구사항 표 채우기·자가체크

14장. 종합 프로젝트: 주문·이커머스 API — 설계(데이터·모듈·ADR) [실무]

- 설계 개요: 라우터·서비스·DB·인증의 분리
- 데이터 모델·ERD: User·Order·OrderItem·Product
- 모듈 분해도(C4 컴포넌트): 라우터·서비스·의존성
- 설계 결정 기록: ADR 5건(모델분리·세션·JWT·모킹·재고)
- 주문 생성 시퀀스도: 인증→재고차감→결제→저장→메일
- 작성형 연습(2종 1세트): ADR·ERD 작성·자가체크

15장. 종합 프로젝트: 주문·이커머스 API — 구현(마일스톤 증분) [실무]

- 구현 전략과 마일스톤 개요
- M1 앱 골격·설정·DB(lifespan·세션)
- M2 상품 CRUD 라우터(페이지네이션·입출력 모델)
- M3 인증·인가(JWT·get_current_user·관리자)
- M4 주문 생성·재고 차감·결제(모킹·비동기·409)
- M5 예외·미들웨어·백그라운드 메일·Dockerfile
- M6 테스트 스위트 통합과 통합 실행 확인
- 작성형 연습(2종 1세트): 빈칸 코드 채우기·자가체크

16장. 종합 프로젝트: 주문·이커머스 API — 발표·평가·성찰 [실무]

- 산출물 시연: 주문 흐름 데모와 /docs
- 평가 루브릭
- 동료 피드백(코드 리뷰·갤러리 워크)
- 성찰 회고: 12단원 개념의 통합 적용
- 작성형 연습(2종 1세트): 성찰 템플릿·자가체크

FastAPI 기초: 첫 앱과 자동 문서

학습 목표

- FastAPI가 Starlette(ASGI)와 Pydantic 위에서 동작하는 구조를 설명할 수 있습니다.
- `FastAPI()` 앱과 경로 동작(path operation) 데코레이터로 첫 엔드포인트를 만들 수 있습니다.
- `uvicorn` (또는 `fastapi dev`)으로 앱을 실행하고 요청을 보낼 수 있습니다.
- `/docs` · `/redoc` · `/openapi.json`으로 자동 생성된 문서를 확인할 수 있습니다.

이 장은 이 책 전체에서 만들 **주문·이커머스 API**의 첫 벽돌을 놓는 장입니다. 우리는 앞으로 상품 카탈로그·재고·주문·인증·결제(모킹)까지 갖춘 한 개의 REST API를 단계적으로 완성합니다. 그 출발점은 의외로 단순합니다. "요청을 받아서, 함수로 처리하고, JSON으로 돌려주는" 엔드포인트 하나를 띄우는 것입니다. 이 장에서는 **FastAPI 기초와 자동 문서**라는 단 하나의 개념을 충실히 익혀, `GET /health` 헬스체크와 `GET /products` 상품 목록을 응답하는 최소 서비스를 직접 띄우고 `/docs`에서 호출해 봅니다.

이 단원의 선수 지식: **없습니다**. 다만 독자가 HTTP 메서드(GET·POST)와 JSON, 그리고 파이썬 함수·타입 힌트를 다뤄 본 실무 백엔드 개발자임을 전제합니다. Flask·Django·Express 등 다른 웹 프레임워크 경험이 있다면 "그 프레임워크의 무엇에 대응하는가"를 짚어 가며 읽으면 빠릅니다.

1. 도입: 왜 FastAPI인가 — 타입 힌트로 검증·문서까지

웹 API를 만들 때 우리가 매번 반복하는 일이 있습니다. 들어온 입력이 올바른지 **검증**하고, 응답을 JSON으로 **직렬화**하고, 다른 팀이 쓸 수 있도록 **문서**를 작성하는 일입니다. 전통적인 프레임워크에서는 이 세 가지를 대부분 손으로 합니다. 입력 검증은 핸들러 안에서 `if`로 일일이 확인하고, 문서는 별도 도구(Swagger 주식·위키)로 따로 관리합니다. 코드와 문서가 어긋나는 일은 시간문제입니다.

FastAPI의 핵심 가치는 이 반복을 **파이썬 타입 힌트 하나로 흡수**한다는 점입니다. 함수 인자에 `product_id: int`라고만 적으면, FastAPI가 그 타입 힌트를 읽어서 다음을 자동으로 해 줍니다.

1. 들어온 값을 `int`로 **변환**하고, 변환할 수 없으면 명확한 `422` 오류를 돌려줍니다(검증).
2. 함수가 돌려준 값을 JSON으로 **직렬화**합니다(응답 변환).

3. 이 엔드포인트가 어떤 입력을 받고 무엇을 돌려주는지 **OpenAPI 스키마**로 기술해 대화형 문서를 만듭니다(자동 문서).

즉, 우리가 "코드를 정확히 쓰는 것"과 "검증·문서를 만드는 것"이 **하나의 행위**가 됩니다. 코드와 문서가 어긋날 수 없습니다. 둘이 같은 원천(타입 힌트)에서 나오기 때문입니다.

한눈에 보는 이 장의 지도

- 앱을 만든다 → `app = FastAPI()`
- 경로에 함수를 연결한다 → `@app.get("/health")`
- 앱을 실제로 띄운다 → `uvicorn` 또는 `fastapi dev` (앱은 서버가 아닙니다)
- 문서를 본다 → `/docs` (Swagger UI) · `/redoc` (ReDoc) · `/openapi.json` (기계용 스키마)

여기에 더해 FastAPI는 **비동기(async)**를 1급으로 지원합니다. 외부 결제 게이트웨이나 배송 추정 API를 기다리는 동안 다른 요청을 처리할 수 있어, 동시 처리량이 높습니다(비동기는 5장에서 본격적으로 다룹니다). 그래서 FastAPI는 "타입으로 검증·문서까지 자동화되는, 빠른 비동기 API 프레임워크"로 요약할 수 있습니다.

2. 정의: FastAPI는 Starlette+Pydantic 위의 ASGI 프레임워크

먼저 한 문장으로 정의하겠습니다.

FastAPI 기초와 자동 문서 — FastAPI는 **Starlette**(ASGI 웹 토대)와 **Pydantic**(데이터 검증) 위에 선 현대적 비동기 웹 API 프레임워크로, 함수에 **타입 힌트**로 **경로 동작(path operation)**을 선언하면 요청 검증·응답 직렬화·대화형 문서(Swagger UI·ReDoc)와 **OpenAPI** 스키마가 자동으로 생성됩니다. 앱 자체는 서버가 아니며 **uvicorn** 같은 ASGI 서버가 실행합니다.

정의에 등장한 용어를 하나씩 풀어 보겠습니다.

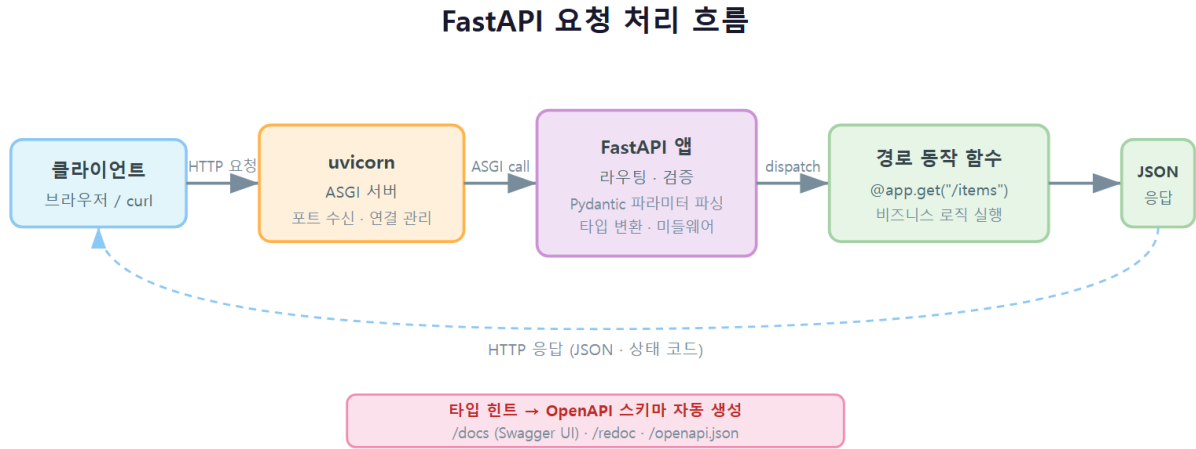
용어	한글(영문)	역할
ASGI	비동기 서버 게이트웨이 인터페이스 (Asynchronous Server Gateway Interface)	비동기 파이썬 웹 앱과 서버가 주고받는 표준 규약. WSGI(동기)의 비동기 후신입니다
Starlette	스타렛(Starlette)	FastAPI가 기반으로 삼는 경량 ASGI 토대. 라우팅·요청/응답·미들웨어를 담당합니다
Pydantic	파이덴틱(Pydantic)	타입 힌트 기반 데이터 검증·직렬화 라이브러리(v2). 입력 검증과 응답 모델을 담당합니다
uvicorn	유비콘(uvicorn)	실제로 포트를 열고 요청을 받는 ASGI 서버. 앱을 "실행"하는 주체입니다
OpenAPI	오픈API(OpenAPI)	REST API를 기계가 읽을 수 있게 기술하는 표준 스펙. <code>/openapi.json</code> 이 이것입니다
경로 동작	경로 동작(path operation)	"경로(<code>/products</code>) + HTTP 메서드(GET)"의 조합과 그것을 처리하는 함수의 묶음

여기서 가장 중요한 구분은 **앱(app)**과 **서버(server)**는 다르다는 점입니다. `FastAPI()` 로 만든 `app` 은 "요청이 들어오면 어떻게 처리할지"를 담은 **애플리케이션 객체**일 뿐, 스스로 포트를 열고 네트워크 요청을 받지 않습니다. 실제로 8000번 포트를 열어 요청을 받아 `app` 에 넘기는 일은 **uvicorn** 같은 ASGI 서버가 합니다. Flask의 `app.run()` 이 개발용 서버를 내장한 것과 달리, FastAPI는 앱과 서버의 역할이 명확히 분리되어 있습니다. 이 분리 덕분에 같은 `app` 을 개발에서는 `fastapi dev` 로, 운영에서는 워커를 여럿 띄운 `uvicorn`으로 실행할 수 있습니다(배포는 12장에서 다룹니다).

다른 프레임워크에서 온 독자를 위한 대응표

- Flask의 `@app.route("/x", methods=["GET"])` → FastAPI의 `@app.get("/x")`
- Flask의 `request.args.get("q")` 수동 파싱 → FastAPI의 함수 인자 타입 힌트 자동 파싱
- DRF의 Serializer·문서 도구 → Pydantic 모델 + 자동 OpenAPI 문서
- Flask의 내장 개발 서버 → `uvicorn`(ASGI 서버, 별도)

3. 동작 원리: 클라이언트 → uvicorn → 앱 → 경로 동작 → 응답



요청 하나가 처리되는 과정을 단계로 따라가 보겠습니다. 브라우저나 `curl` 이 `GET /products` 요청을 보냈다고 합시다.

- 클라이언트 → uvicorn:** 클라이언트가 `http://127.0.0.1:8000/products` 로 HTTP 요청을 보냅니다. 포트 8000을 듣고 있던 uvicorn(ASGI 서버)이 이 요청을 받습니다.
- uvicorn → 앱:** uvicorn은 받은 요청을 ASGI 규약에 맞는 형태로 바꿔 `app` (FastAPI 인스턴스)에 넘깁니다. 여기서 서버와 앱의 경계가 드러납니다. 네트워크는 서버가, 처리는 앱이 맡습니다.
- 앱 → 경로 매칭:** 앱은 요청의 **메서드(GET)**와 **경로(/products)**를 보고, 등록된 경로 동작 중 일치하는 것을 찾습니다. `@app.get("/products")`로 등록된 함수가 선택됩니다.
- 입력 검증·변환:** 그 함수의 인자에 타입 힌트가 있다면, FastAPI가 요청에서 해당 값을 뽑아 타입에 맞게 변환·검증합니다. 실패하면 핸들러를 부르지 않고 곧장 `422`를 돌려줍니다(2장 주제).
- 함수 실행:** 검증을 통과하면 경로 동작 함수가 실행되어 파이썬 객체(예: `list[dict]`)를 반환합니다.
- 응답 직렬화:** 반환값을 JSON으로 직렬화하고, 상태 코드·헤더와 함께 응답을 만듭니다.
- 앱 → uvicorn → 클라이언트:** 응답이 uvicorn을 거쳐 클라이언트에게 전달됩니다.

이와 **병렬로** 일어나는 또 하나의 흐름이 자동 문서입니다. 앱이 시작될 때(정확히는 처음 필요할 때) FastAPI는 등록된 모든 경로 동작의 타입 힌트를 읽어 **OpenAPI 스키마**(`/openapi.json`)를 생성합니다. 그리고 `/docs` (Swagger UI)와 `/redoc` (ReDoc)은 그 스키마를 읽어 사람이 보기 좋은 대화형 문서로 렌더합니다. 즉 문서는 "따로 쓰는 것"이 아니라 "코드에서 자동으로 파생되는 것"입니다.

단계	담당	핵심 질문
포트 열기·요청 수신	uvicorn(서버)	누가 네트워크를 듣는가
경로·메서드 매칭	FastAPI 앱	어떤 함수를 부를까
입력 검증·변환	Pydantic	입력이 타입에 맞는가
처리	경로 동작 함수	무엇을 돌려줄까
직렬화·문서화	FastAPI + OpenAPI	JSON·스키마는 어떻게 생기는가

4. 개발 환경 세팅: Windows 11 · VS Code · Docker · PostgreSQL

원리를 코드로 옮기기 전에, 코드를 작성하고 실행할 **개발 환경**을 먼저 갖춰야 합니다. 이 절은 **Windows 11**에서 실무 백엔드 개발자가 흔히 쓰는 구성(**VS Code · Docker · PostgreSQL**)을 기준으로, 실제 명령과 함께 단계별로 진행합니다. 한 번 갖춰 두면 이 책의 나머지 장에서 그대로 재사용합니다. 아래 명령은 모두 **Windows 11**의 **PowerShell**(VS Code 통합 터미널 포함)에서 실행하는 것을 기준으로 합니다.

이 절에서 갖추는 6가지

1. Python 3.12 — 코드를 실행할 런타임
2. 가상환경(venv) — 프로젝트별 의존성 격리
3. VS Code + 확장 — 편집기와 도구
4. 의존성 설치 — `fastapi[standard]` 등
5. Docker Desktop — 컨테이너 실행 기반
6. PostgreSQL(Docker) — 데이터베이스(7장에서 본격 사용, 지금은 컨테이너만 준비)

4.1 Python 3.12 설치

FastAPI는 최신 타입 힌트 문법(`list[dict]`, `X | None` 등)을 적극적으로 활용하므로 **Python 3.12**를 권장합니다. Windows 11에서 설치하는 방법은 두 가지입니다.

방법 1 — python.org 설치 프로그램. python.org에서 Windows용 3.12 설치 프로그램을 내려받아 실행합니다. 첫 화면 하단의 **Add python.exe to PATH** 체크박스를 반드시 켜 뒤 설치합니다(이 체크를 빠뜨리면 터미널에서 `python` 명령을 찾지 못합니다).

방법 2 — winget(Windows 11 기본 패키지 관리자). PowerShell에서 한 줄로 설치합니다.

```
winget install Python.Python.3.12
```

설치가 끝나면 새 **PowerShell 창**을 열어(PATH 갱신 반영) 버전을 확인합니다.

```
python --version
```

```
Python 3.12.7
```

`python` 대신 `py` 런처(`py -3.12 --version`)로도 확인할 수 있습니다. 버전이 `3.12.x`로 보이면 준비된 것입니다.

4.2 프로젝트 폴더와 가상환경(venv)

프로젝트마다 의존성을 격리하기 위해 **가상환경(virtual environment)**을 만듭니다. 전역 파이썬을 더럽히지 않고, 이 프로젝트의 패키지를 한 폴더 안에 가둘 수 있습니다. 먼저 작업 폴더를 만들고 그 안으로 이동합니다.

```
mkdir shop-api
cd shop-api
python -m venv .venv
```

`.venv` 폴더가 생기면 가상환경을 **활성화**합니다.

```
.\.venv\Scripts\Activate.ps1
```

활성화되면 프롬프트 앞에 `(.venv)`가 붙습니다.

```
(.venv) PS C:\Users\you\shop-api>
```

실행 정책에 막힐 때. Windows 11에서 `Activate.ps1` 실행이 이 시스템에서 스크립트를 실행할 수 없으므로 라는 오류로 막히면, PowerShell의 스크립트 실행 정책 때문입니다. 현재 사용자 범위로 한 번만 풀어 주면 됩니다(관리자 권한 불필요).

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

이후 다시 `.\.venv\Scripts\Activate.ps1`을 실행하면 활성화됩니다. 가상환경을 끌 때는 `deactivate`를 입력합니다.

4.3 VS Code와 확장 설치

편집기는 **VS Code**를 사용합니다. code.visualstudio.com에서 내려받아 설치한 뒤, 프로젝트 폴더를 엽니다(터미널에서 `code .`). 다음 확장을 설치합니다(왼쪽 Extensions 패널에서 검색).

확장	구분	역할
Python (Microsoft)	필수	인터프리터 선택·실행·디버깅·테스트 통합
Pylance	필수	타입 힌트 기반 자동완성·타입 검사(IntelliSense)
Docker	권장	컨테이너·이미지를 GUI로 관리
REST Client	권장	<code>.http</code> 파일로 API 요청을 편집기 안에서 호출
Even Better TOML	권장	<code>pyproject.toml</code> 등 TOML 파일 문법 강조

설치 후 **인터프리터로 방금 만든 `.venv`를 선택**하는 것이 중요합니다. `Ctrl+Shift+P`로 명령 팔레트를 열고 **Python: Select Interpreter**를 실행한 뒤, 경로에 `.venv`가 포함된 항목(예: `.\venv\Scripts\python.exe`)을 고릅니다. 이렇게 해야 VS Code의 자동완성·실행·디버깅이 프로젝트 가상환경을 사용합니다.

4.4 의존성 설치와 `requirements.txt`

가상환경이 활성화된 상태(`.venv` 표시)에서 FastAPI와 표준 의존성을 설치합니다.

```
pip install "fastapi[standard]" "uvicorn[standard]"
```

`fastapi[standard]`는 FastAPI 본체와 함께 **uvicorn 서버**·`fastapi CLI`·`httpx`·`Jinja2` 등 개발에 필요한 표준 묶음을 함께 설치합니다(따라서 `uvicorn[standard]`는 사실상 함께 들어오지만, 의도를 분명히 하려고 명시했습니다). DB·인증 같은 패키지는 해당 장에서 그때그때 추가합니다(예: 7장에서 `sqlmodel`, 9장에서 인증 라이브러리).

설치가 끝나면 **버전을 고정**하기 위해 `requirements.txt`로 현재 패키지 목록을 적어 둡니다.

```
pip freeze > requirements.txt
```

이렇게 하면 다른 사람(또는 다른 PC·CI)이 `pip install -r requirements.txt` 한 줄로 동일한 환경을 재현할 수 있습니다. 설치된 FastAPI 버전은 다음으로 확인합니다.

```
python -c "import fastapi; print(fastapi.__version__)"
```

```
0.118.0
```

(버전 숫자는 설치 시점에 따라 다를 수 있습니다. 0.11x 이상이면 이 책의 내용을 그대로 따라갈 수 있습니다.)

4.5 Docker Desktop 설치(Windows 11)

데이터베이스(PostgreSQL)를 PC에 직접 설치하는 대신 **Docker 컨테이너**로 띄우면, 버전을 깔끔하게 통제하고 필요 없을 때 한 번에 지울 수 있습니다. Windows 11에서는 **Docker Desktop**을 사용하며, 내부적으로 **WSL2(Windows Subsystem for Linux 2)** 백엔드를 씁니다.

1. docker.com에서 Docker Desktop for Windows를 내려받아 설치합니다. 설치 중 **Use WSL 2 instead of Hyper-V** 옵션을 켭니다(권장).
2. 설치 후 Docker Desktop을 실행해 엔진이 **Running** 상태가 되게 합니다(작업 표시줄의 고래 아이콘).
3. PowerShell에서 설치를 확인합니다.

```
docker --version
```

```
Docker version 27.3.1, build ce12230
```

WSL2가 처음이라면. Docker Desktop 설치 과정에서 WSL2가 없으면 자동 설치를 안내합니다. 수동으로 준비하려면 관리자 PowerShell에서 `wsl --install` 을 한 번 실행하고 재부팅하면 됩니다. 사내 정책으로 Docker Desktop을 쓸 수 없다면, PostgreSQL을 직접 설치해도 이 책의 7장 이후 내용을 따라갈 수 있습니다(연결 정보만 맞추면 됩니다).

4.6 PostgreSQL을 Docker로 실행

이제 **PostgreSQL 16**을 컨테이너로 띄웁니다. 중요한 점부터 짚겠습니다. **데이터베이스는 7장에서 본격적으로 사용하며**, 지금은 "환경을 미리 준비"하는 차원에서 컨테이너만 띄워 둡니다. 이 장의 예제(`/health` · `/products`)는 고정 데이터만 쓰므로 DB가 없어도 동작합니다. 다만 한 번 띄워 두면 7장에서 곧바로 연결만 하면 됩니다.

방법 1 — `docker run` 한 줄. PowerShell에서 다음을 실행합니다.

```
docker run --name shop-postgres -e POSTGRES_PASSWORD=devpass -e POSTGRES_DB=shop -p 5432:5432 -v shop
```

각 옵션의 뜻은 다음과 같습니다.

옵션	의미
<code>--name shop-postgres</code>	컨테이너 이름(이후 명령에서 이 이름으로 가리킵니다)
<code>-e POSTGRES_PASSWORD=devpass</code>	관리자(postgres) 비밀번호(개발용 값, 운영에서는 시크릿으로 관리)
<code>-e POSTGRES_DB=shop</code>	시작 시 자동 생성할 데이터베이스 이름
<code>-p 5432:5432</code>	호스트 5432 포트를 컨테이너 5432로 연결(앱이 접속할 통로)
<code>-v shop_pgdata:/...</code>	데이터를 이름 있는 볼륨에 보존(컨테이너를 지워도 데이터 유지)
<code>-d postgres:16</code>	postgres:16 이미지를 백그라운드(-d)로 실행

방법 2 — docker-compose.yml (권장). 옵션이 많아질수록 파일로 관리하는 편이 재현성·가독성이 좋습니다. 프로젝트 폴더에 `docker-compose.yml` 을 만듭니다.

```
# docker-compose.yml
services:
  db:
    image: postgres:16
    container_name: shop-postgres
    environment:
      POSTGRES_PASSWORD: devpass
      POSTGRES_DB: shop
    ports:
      - "5432:5432"
    volumes:
      - shop_pgdata:/var/lib/postgresql/data

volumes:
  shop_pgdata:
```

그리고 한 줄로 띄웁니다(끝 때는 `docker compose down`).

```
docker compose up -d
```

연결 확인. 컨테이너가 떠 있는지 먼저 봅니다.

```
docker ps
```


CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
a1b2c3d4e5f6	postgres:16	Up 10 seconds	0.0.0.0:5432->5432/tcp	shop-postgres

STATUS 가 Up ... 이면 정상입니다. 실제로 DB에 접속해 보려면 컨테이너 안의 `psql` 을 실행합니다.

```
docker exec -it shop-postgres psql -U postgres -d shop
```

```
psql (16.4)
Type "help" for help.

shop=# \l
shop=# \q
```

`\l` 은 데이터베이스 목록, `\q` 는 종료입니다. 명령행이 익숙하지 않다면 VS Code의 Docker 확장이거나 **DBeaver** 같은 GUI 도구로 `localhost:5432` (사용자 `postgres`, 비밀번호 `devpass`, DB `shop`)에 접속해 확인해도 됩니다. 다시 강조하면, 지금 이 연결을 앱 코드에서 쓰지는 않습니다. 7장에서 이 정보를 그대로 사용해 `SQLModel`로 연결합니다.

직접 해보기 (환경 점검 체크리스트)

위 단계를 끝낸 뒤 다음을 스스로 점검하세요. 모두 통과해야 다음 절의 코드를 무리 없이 따라갈 수 있습니다.

자가체크 (스스로 점검)

- `[] python --version` 이 `3.12.x` 를 출력하는가
- `[]` 가상환경 활성화 시 프롬프트에 `(.venv)` 가 보이는가
- `[]` VS Code의 인터프리터가 `.venv` 로 선택되어 있는가(우하단 상태바 확인)
- `[] python -c "import fastapi; print(fastapi.__version__)"` 가 버전을 출력하는가
- `[] requirements.txt` 가 생성되어 있는가
- `[] docker --version` 이 버전을 출력하는가
- `[] docker ps` 에서 `shop-postgres` 가 Up 상태로 보이는가

5. worked example: 헬스체크와 상품 목록 엔드포인트

이제 정의와 원리를 코드로 옮깁니다. 주문·이커머스 API의 가장 작은 출발점으로, 서비스가 살아 있는지 알려 주는 `GET /health` 와 고정 상품 목록을 돌려주는 `GET /products` 를 만듭니다.

```
# main.py
from fastapi import FastAPI

# 1) 애플리케이션 객체를 만듭니다. 이 app이 모든 경로 동작을 등록받습니다.
app = FastAPI(
    title="Shop API",
    description="주문·이커머스 API의 첫 걸음",
    version="0.1.0",
)

# 2) 루트 경로: 서비스 안내 메시지를 돌려줍니다.
@app.get("/")
async def read_root() -> dict:
    return {"service": "shop-api", "docs": "/docs"}

# 3) 헬스체크: 로드밸런서·모니터링이 주기적으로 호출해 생존을 확인합니다.
@app.get("/health")
async def health_check() -> dict:
    return {"status": "ok"}

# 4) 상품 목록: 지금은 고정 데이터를 돌려주고, 7장에서 DB로 바꿉니다.
@app.get("/products")
async def list_products() -> list[dict]:
    return [
        {"id": 1, "name": "기계식 키보드", "price": 89000, "category": "주변기기"},
        {"id": 2, "name": "27인치 모니터", "price": 235000, "category": "디스플레이"},
        {"id": 3, "name": "USB-C 허브", "price": 32000, "category": "주변기기"},
    ]
```

줄별 해설

- `from fastapi import FastAPI` — 프레임워크의 핵심 클래스를 가져옵니다.
- `app = FastAPI(title=..., version=...)` — 앱 객체를 만듭니다. `title`·`description`·`version`은 자동 문서 상단에 그대로 노출됩니다. 문서가 코드에서 나온다는 점이 여기서 드러납니다.
- `@app.get("/health")` — **경로 동작 데코레이터**입니다. "GET 메서드로 `/health` 경로에 요청이 오면 바로 아래 함수를 부르라"는 등록입니다. 경로를 정하는 것은 **데코레이터의 인자**이지 함수 이름이 아닙니다. 함수 이름 `health_check`는 우리만 쓰는 이름일 뿐 URL과 무관합니다.
- `async def health_check() -> dict:` — 핸들러를 `async def`로 선언했습니다. 이 함수는 `await`를 쓰지 않으므로 일반 `def`여도 됩니다. 다만 FastAPI 핸들러는 `async def`가 기본 관례이고, 비동기 I/O를 호출할 때 진가를 발휘합니다(5장).
- `return {"status": "ok"}` — 파이썬 `dict`를 반환하면 FastAPI가 JSON `{"status": "ok"}`으로 직렬화합니다. 우리가 `json.dumps`를 부를 필요가 없습니다.
- `list_products()`의 반환 타입 힌트 `-> list[dict]` — 이 엔드포인트가 객체 목록을 돌려준다는 사실이 문서에도 반영됩니다.

실행한 뒤(실행 방법은 6절) 각 경로를 호출하면 다음과 같은 응답을 얻습니다.

```
$ curl http://127.0.0.1:8000/health
{"status":"ok"}

$ curl http://127.0.0.1:8000/products
[{"id":1,"name":"기계식 키보드","price":89000,"category":"주변기기"},
 {"id":2,"name":"27인치 모니터","price":235000,"category":"디스플레이"},
 {"id":3,"name":"USB-C 허브","price":32000,"category":"주변기기"}]
```

이 작은 예제가 이미 FastAPI의 핵심을 모두 담고 있습니다. 앱을 만들고(`FastAPI()`), 경로 동작을 등록하고(`@app.get`), 파이썬 객체를 반환하면 JSON 응답과 문서가 따라온다는 것입니다. 두 번째 worked example로, 같은 핸들러에 응답 메타데이터를 곁들여 문서 품질을 높여 보겠습니다.

```
# main.py 에 이어서 — 카탈로그 요약 엔드포인트
from fastapi import FastAPI

app = FastAPI(title="Shop API", version="0.1.0")

# 모듈 수준의 고정 데이터(나중에 DB로 대체)
CATALOG = [
    {"id": 1, "name": "기계식 키보드", "price": 89000, "category": "주변기기"},
    {"id": 2, "name": "27인치 모니터", "price": 235000, "category": "디스플레이"},
    {"id": 3, "name": "USB-C 허브", "price": 32000, "category": "주변기기"},
]

# summary.tags는 순수 문서용 메타데이터입니다. /docs에 그대로 나타납니다.
@app.get("/products", summary="상품 목록 조회", tags=["products"])
async def list_products() -> list[dict]:
    return CATALOG

@app.get("/catalog/summary", summary="카탈로그 요약", tags=["products"])
async def catalog_summary() -> dict:
    total = len(CATALOG)
    cheapest = min(CATALOG, key=lambda p: p["price"])
    return {"total": total, "cheapest": cheapest["name"]}
```

줄별 해설

- CATALOG 를 모듈 수준 상수로 빼서 여러 핸들러가 공유하게 했습니다. 실무에서는 이 자리가 DB 조회로 바뀝니다(7장).
- `@app.get("/products", summary="상품 목록 조회", tags=["products"])` — `summary` 는 문서의 짧은 제목, `tags` 는 문서에서 엔드포인트를 그룹으로 묶는 라벨입니다. 둘 다 코드에 적으면 문서에 자동 반영됩니다.
- `catalog_summary()` — `min(..., key=...)` 으로 최저가 상품을 계산해 요약 객체를 돌려줍니다. 반환값이 `dict` 이므로 그대로 JSON이 됩니다.

```
$ curl http://127.0.0.1:8000/catalog/summary
{"total":3,"cheapest":"USB-C 허브"}
```

6. 실행과 동작 테스트: 앱 띄우기와 동작 확인

환경이 준비됐으니 앱을 띄우고 **동작을 확인·테스트**합니다. 이 절은 Windows 11·VS Code 기준으로, 앱을 실행하는 방법부터 자동 문서·curl·REST Client로 호출해 보는 방법, 그리고 PostgreSQL 연결 확인까지 다룹니다. 모든 명령은 **.venv**가 **활성화된**(프롬프트에 **(.venv)**) VS Code 통합 터미널에서 실행한다고 가정합니다.

6.1 앱 실행: fastapi dev · uvicorn

앞서 강조했듯 **app**은 스스로 뜨지 않습니다. 실행할 ASGI 서버가 필요합니다. 개발 단계에서 쓰는 방법은 두 가지입니다.

방법 1 — fastapi dev (권장, 개발용). FastAPI CLI가 제공하는 명령으로, 내부적으로 uvicorn을 자동 리로드와 함께 띄웁니다. 코드를 저장하면 서버가 알아서 재시작합니다.

```
$ fastapi dev main.py
INFO     Using import string: main:app
INFO     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO     Started reloader process using WatchFiles
INFO     Application startup complete.
```

방법 2 — uvicorn 을 직접 호출. 더 명시적이며, 운영 설정과 가까운 형태입니다.

```
$ uvicorn main:app --reload
INFO     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO     Started reloader process
INFO     Application startup complete.
```

여기서 **main:app**은 "**main.py** 모듈의 **app** 객체를 실행하라"는 뜻입니다. 콜론 앞은 모듈(파일명에서 **.py**를 뺀 것), 뒤는 그 안의 FastAPI 인스턴스 변수명입니다. 만약 변수명을 **application**으로 지었다면 **main:application**이 됩니다.

--reload는 **개발용**입니다. 자동 리로드는 파일 변경을 감시하느라 비용이 들고 단일 프로세스로 돕니다. 운영에서는 **--reload** 없이 워커 수를 지정해 띄우거나 Gunicorn·컨테이너로 관리합니다(12장에서 자세히 다룹니다). 지금은 "개발은 **fastapi dev**, 운영은 다르다"만 기억하면 됩니다.

앱이 뜨면 다음 주소들이 살아 있습니다.

주소	내용
<code>http://127.0.0.1:8000/</code>	우리가 만든 루트 엔드포인트
<code>http://127.0.0.1:8000/health</code>	헬스체크
<code>http://127.0.0.1:8000/products</code>	상품 목록
<code>http://127.0.0.1:8000/docs</code>	Swagger UI(대화형 문서)
<code>http://127.0.0.1:8000/redoc</code>	ReDoc(읽기용 문서)
<code>http://127.0.0.1:8000/openapi.json</code>	OpenAPI 스키마(기계용)

VS Code에서는 통합 터미널(`Ctrl+``)에 위 명령을 입력해 앱을 띄우는 것이 편합니다. 터미널 로그에 출력된 `http://127.0.0.1:8000` 링크를 `Ctrl+클릭` 하면 기본 브라우저로 바로 열립니다.

6.2 자동 문서(Swagger UI)로 테스트

앱을 가장 빠르게 시험하는 방법은 **자동 문서**입니다. 브라우저로

`http://127.0.0.1:8000/docs` (Swagger UI)를 엽니다. 우리가 등록한 엔드포인트가 목록으로 보입니다. 시험할 엔드포인트(예: `GET /products`)를 펼치고 **Try it out** 버튼을 누른 뒤 **Execute**를 누르면, 브라우저가 실제 요청을 보내고 **응답 본문·상태 코드·응답 헤더**를 그 자리에서 보여 줍니다. 별도 도구 없이 동작을 확인할 수 있어, 개발 중 가장 자주 쓰는 수동 테스트 방법입니다.

6.3 curl · VS Code REST Client로 테스트

명령행이나 편집기 안에서 직접 요청을 보내고 싶을 때가 있습니다. Windows 11의 PowerShell에는 `curl` 이 기본 포함되어 있어 그대로 호출합니다.

```
curl http://127.0.0.1:8000/health
```

```
{"status":"ok"}
```

PowerShell의 curl 주의. Windows PowerShell에서 `curl` 은 기본적으로 `Invoke-WebRequest` 의 별칭입니다. 출력 형식이 진짜 `curl` 과 다르게 보일 수 있고, 본문만 깔끔히 보려면 `curl.exe` 를 명시하거나 (`Invoke-RestMethod http://127.0.0.1:8000/health`) 를 쓰면 됩니다. 단순 동작 확인에는 별칭 `curl` 로도 충분합니다.

더 편한 방법은 VS Code의 **REST Client** 확장입니다. 프로젝트에 `requests.http` 파일을 만들고 요청을 적은 뒤, 요청 위에 뜨는 **Send Request**를 누르면 응답이 옆 패널에 표시됩니다.

```
### 헬스체크
GET http://127.0.0.1:8000/health

### 상품 목록
GET http://127.0.0.1:8000/products
```

으로 구분된 각 블록이 하나의 요청이며, 편집기를 떠나지 않고 요청·응답을 반복 확인할 수 있어 회귀 점검에 유용합니다.

6.4 PostgreSQL 연결 확인

이 장의 엔드포인트는 DB를 쓰지 않지만, 4.6절에서 띄운 PostgreSQL 컨테이너가 살아 있는지는 미리 확인해 두면 7장에서 매끄럽게 이어집니다.

```
docker ps
```

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
a1b2c3d4e5f6	postgres:16	Up 5 minutes	0.0.0.0:5432->5432/tcp	shop-postgres

STATUS 가 Up ... 이면 정상입니다. 멈춰 있다면 `docker start shop-postgres` (또는 `docker compose up -d`)로 다시 띄웁니다. 접속까지 확인하려면 4.6절의 `docker exec ... psql` 명령을 사용합니다.

6.5 자동화 테스트는 11장에서

지금까지는 모두 **사람이 손으로** 동작을 확인하는 수동 테스트입니다. 엔드포인트가 늘어나면 매년 손으로 확인하기 어려우므로, 코드로 자동 검증하는 **자동화 테스트(pytest·TestClient)**가 필요합니다. 이는 11장에서 본격적으로 다룹니다. 이 장에서는 "자동 문서·curl·REST Client로 직접 호출해 동작을 눈으로 확인한다"는 수동 흐름에 익숙해지는 것에 집중합니다.

7. 자동 문서: /docs-/redoc-/openapi.json 살펴보기

브라우저로 `http://127.0.0.1:8000/docs` 를 열면 **Swagger UI**가 나타납니다. 우리가 등록한 `GET /`, `GET /health`, `GET /products` 가 모두 목록으로 보이고, 각 엔드포인트를 펼쳐 **Try it out** 버튼으로 실제 요청을 보내 응답을 확인할 수 있습니다. 이 문서는 우리가 단 한 줄도 따로 쓰지 않았는데도 만들어졌습니다. 코드의 경로·메서드·타입 힌트 `summary` · `tags` 에서 자동으로 파생되었기 때문입니다.

세 문서의 차이는 다음과 같습니다.

경로	도구	용도
/docs	Swagger UI	브라우저에서 직접 호출·시험하는 대화형 문서
/redoc	ReDoc	읽기 위주의 정적 문서(스크롤형, 인쇄·공유에 적합)
/openapi.json	OpenAPI 스키마 (JSON)	사람이 아니라 도구가 읽는 원천. 클라이언트 SDK 생성·테스트·게이트웨이 연동에 쓰입니다

/openapi.json 을 직접 열어 보면, 우리 앱의 구조가 표준 스펙으로 기술된 것을 볼 수 있습니다(아래는 일부를 발췌·정리한 모습입니다).

```
$ curl http://127.0.0.1:8000/openapi.json
{
  "openapi": "3.1.0",
  "info": {"title": "Shop API", "version": "0.1.0"},
  "paths": {
    "/health": {
      "get": {
        "summary": "Health Check",
        "responses": {"200": {"description": "Successful Response"}}
      }
    },
    "/products": {
      "get": {
        "summary": "상품 목록 조회",
        "tags": ["products"],
        "responses": {"200": {"description": "Successful Response"}}
      }
    }
  }
}
```

/docs 와 /redoc 은 결국 이 JSON을 사람이 보기 좋게 그린 것에 불과합니다. 그래서 "문서를 좋게 만든다"는 곧 "코드의 타입 힌트·메타데이터를 잘 적는다"와 같은 말이 됩니다. 다음 장들에서 Pydantic 모델·응답 모델을 배우면 이 스키마가 훨씬 풍부해지는 것을 보게 됩니다.

자동 문서가 실무에서 주는 이득

- **프론트엔드와의 계약:** 백엔드가 엔드포인트를 추가하면 /docs 가 즉시 갱신되어, 프론트엔드가 별도 문서를 기다리지 않고 곧장 스펙을 확인합니다. 코드가 곧 계약서입니다.
- **클라이언트 SDK 자동 생성:** /openapi.json 을 입력으로 TypeScript·Kotlin 등 클라이언트 코드를 생성하는 도구가 많습니다. 손으로 모델을 다시 옮길 필요가 없습니다.
- **테스트·게이트웨이 연동:** API 게이트웨이·계약 테스트 도구가 OpenAPI 스키마를 그대로 읽어 검증·라우팅 규칙을 만듭니다.

- **문서 표류(drift) 방지:** 코드와 문서가 한 원천에서 나오므로, "문서는 옛날 그대로인데 코드만 바뀐" 전형적 사고가 구조적으로 사라집니다.

이 이득은 우리가 종합 프로젝트에서 상품·주문·인증 엔드포인트를 늘려 갈수록 더 커집니다. 엔드포인트가 수십 개가 되어도 문서는 항상 코드와 동기화된 채 유지됩니다.

운영 단계에서는 보안상 문서를 끄거나 인증 뒤로 숨기기도 합니다. `FastAPI(docs_url=None, redoc_url=None)` 처럼 생성자 인자로 문서 경로를 비활성화할 수 있고, `openapi_url=None` 으로 스키마 자체를 끌 수도 있습니다. 지금 학습 단계에서는 기본값(켜짐)을 그대로 두지만, "문서 노출 여부도 코드로 통제한다"는 점을 기억해 두면 됩니다.

8. 즉시 연습(2종 1세트): 상품 카탈로그 엔드포인트 채우기

배운 것을 손으로 굳힐 차례입니다. 아래는 **모범 답안**과 **직접 작성**할 빈칸 코드, 그리고 **자가체크**입니다.

모범 답안. 재고가 있는 상품만 골라 돌려주는 `GET /products/in-stock` 을 만든 모습입니다.

```
# main.py
from fastapi import FastAPI

app = FastAPI(title="Shop API", version="0.1.0")

CATALOG = [
    {"id": 1, "name": "기계식 키보드", "price": 89000, "stock": 5},
    {"id": 2, "name": "27인치 모니터", "price": 235000, "stock": 0},
    {"id": 3, "name": "USB-C 허브", "price": 32000, "stock": 12},
]

@app.get("/products/in-stock", tags=["products"])
async def list_in_stock() -> list[dict]:
    # 재고가 1개 이상인 상품만 추립니다.
    return [p for p in CATALOG if p["stock"] > 0]
```

```
$ curl http://127.0.0.1:8000/products/in-stock
[{"id":1,"name":"기계식 키보드","price":89000,"stock":5},
 {"id":3,"name":"USB-C 허브","price":32000,"stock":12}]
```

직접 작성 (빈칸 코드)

품절이 아닌 상품의 **개수**만 돌려주는 `GET /products/in-stock/count` 를 완성하세요. `_____` 부분을 채우면 됩니다.


```

from fastapi import FastAPI

app = FastAPI(title="Shop API", version="0.1.0")

CATALOG = [
    {"id": 1, "name": "기계식 키보드", "price": 89000, "stock": 5},
    {"id": 2, "name": "27인치 모니터", "price": 235000, "stock": 0},
    {"id": 3, "name": "USB-C 허브", "price": 32000, "stock": 12},
]

# (1) 경로 동작 데코레이터를 GET /products/in-stock/count 로 선언하세요.
@app.get("/in-stock/count")
async def count_in_stock() -> dict:
    # (2) 재고가 1개 이상인 상품의 개수를 세어 dict로 돌려주세요.
    available = [p for p in CATALOG if p["stock"] > 0]
    return {"count": len(available)}

```

자가체크 (스스로 점검)

- [] `fastapi dev main.py` 로 앱이 오류 없이 뜨는가
- [] `/docs` 에 `GET /products/in-stock/count` 가 보이는가
- [] 호출하면 `{"count": 2}` 가 반환되는가
- [] 경로를 정하는 것이 함수 이름이 아니라 데코레이터 인자임을 설명할 수 있는가

참고 답: `@app.get("/products/in-stock/count")`, `available = [p for p in CATALOG if p["stock"] > 0]`, `return {"count": len(available)}`.

9. 흔한 오류와 오개념: 앱 vs 서버·경로 인자·문서 자동생성

오개념 1 — "FastAPI가 곧 서버다." 아닙니다. `FastAPI()` 로 만든 `app` 은 처리 규칙을 담은 애플리케이션 객체일 뿐, 포트를 열어 요청을 받지 않습니다. 실제 실행은 **uvicorn 같은 ASGI 서버**가 합니다. `python main.py` 로 파일만 실행해도 아무 일이 일어나지 않는 이유가 이것입니다. 반드시 `fastapi dev main.py` 또는 `uvicorn main:app` 으로 서버를 띄워야 합니다.

오개념 2 — "경로는 함수 이름이 정한다." 아닙니다. `@app.get("/products")` 의 경로는 **데코레이터의 인자**가 정합니다. 함수 이름(`list_products`)은 URL과 무관한, 우리끼리 쓰는 이름입니다. 같은 함수 이름이어도 데코레이터 인자가 다르면 다른 경로입니다.

오개념 3 — `/docs` 문서는 따로 작성해야 한다." 아닙니다. `/docs` · `/redoc` 은 코드의 경로·메서드·타입 힌트·summary·tags 에서 **자동 생성**됩니다. 문서를 좋게 하려면 별도 문서를 쓰는 게 아니라 코드의 타입 힌트와 메타데이터를 충실히 적으면 됩니다.

이 밖에 입문 단계에서 자주 부딪히는 실수를 정리합니다.

증상	원인	해결
<code>uvicorn main:app</code> 실행 시 <code>Error loading ASGI app</code>	모듈명·변수명이 틀림 (<code>main:app</code> 의 좌/우)	파일이 <code>main.py</code> 이고 변수가 <code>app</code> 인지 확인
코드를 고쳐도 응답이 그대로	자동 리로드 없이 띄움	<code>fastapi dev</code> 또는 <code>uvicorn ... --reload</code> 사용
<code>/products</code> 가 405(Method Not Allowed)	<code>@app.get</code> 이 아니라 다른 메서드로 등록	호출 메서드와 데코레이터 메서드를 맞춤
한글이 <code>\uXXXX</code> 로 보임	터미널 표시 문제(실제 JSON은 정상)	브라우저· <code>/docs</code> 에서 확인하면 정상 표시

10. 단원 미니 프로젝트: 상품 카탈로그 첫 API(2종 1세트)

이 단원의 개념(FastAPI 기초와 자동 문서, c01)만으로 완성하는 작은 산출물입니다. 목표는 헬스체크와 고정 상품 목록 엔드포인트를 갖춘 최소 앱을 띄우고 `/docs` 에서 호출하는 것입니다. 마일스톤(증분)으로 나눠 진행합니다.

마일스톤	할 일	사용 개념
M1	<code>FastAPI()</code> 앱과 <code>GET /health</code> 작성·실행	앱 인스턴스·경로 동작·서버 실행
M2	<code>GET /products</code> 로 고정 상품 목록 반환	경로 동작·JSON 직렬화
M3	<code>/docs</code> · <code>/openapi.json</code> 에서 자동 문서 확인	자동 문서

모범 답안 (완성 `main.py`).

```
# main.py — 상품 카탈로그 첫 API
from fastapi import FastAPI

app = FastAPI(
    title="Shop API",
    description="주문·이커머스 API의 첫 단원 산출물",
    version="0.1.0",
)

CATALOG = [
    {"id": 1, "name": "기계식 키보드", "price": 89000, "category": "주변기기"},
    {"id": 2, "name": "27인치 모니터", "price": 235000, "category": "디스플레이"},
    {"id": 3, "name": "USB-C 허브", "price": 32000, "category": "주변기기"},
]

# M1: 헬스체크
@app.get("/health", tags=["system"])
async def health_check() -> dict:
    return {"status": "ok"}

# M2: 상품 목록
@app.get("/products", summary="상품 목록 조회", tags=["products"])
async def list_products() -> list[dict]:
    return CATALOG
```

실행과 확인:

```
$ fastapi dev main.py
INFO Uvicorn running on http://127.0.0.1:8000

$ curl http://127.0.0.1:8000/health
{"status":"ok"}

$ curl http://127.0.0.1:8000/products
[{"id":1,"name":"기계식 키보드","price":89000,"category":"주변기기"}, ... ]
```

브라우저에서 `http://127.0.0.1:8000/docs` 를 열면 `system` · `products` 태그 아래로 두 엔드포인트가 정리되어 보이고, **Try it out**으로 직접 호출됩니다.

직접 작성 (빈 스타터)

아래 빈 스타터를 채워 미니 프로젝트를 완성하세요. `products` 핸들러의 반환 데이터를 비워 두었습니다.

```
# main.py
from fastapi import FastAPI

# (M1) 앱을 만들고 title.version을 지정하세요.
app = ____

# (M2) 최소 2개 이상의 상품을 담은 고정 목록을 정의하세요.
CATALOG = [
    # {"id": ..., "name": ..., "price": ..., "category": ...},
]

# (M1) 헬스체크 엔드포인트
@app.get("____", tags=["system"])
async def health_check() -> dict:
    return {"status": ____}

# (M2) 상품 목록 엔드포인트
@app.get("____", tags=["products"])
async def list_products() -> list[dict]:
    return ____
```

자가체크 (체크리스트)

- [] `fastapi dev main.py` 또는 `uvicorn main:app --reload` 로 앱이 뜨는가
- [] `/docs` 에서 `health` 와 `products` 두 엔드포인트가 보이는가
- [] `GET /products` 가 상품 JSON 목록을 반환하는가
- [] `GET /health` 가 `{"status": "ok"}` 를 반환하는가
- [] `/openapi.json` 에 두 경로가 기술되어 있는가
- [] "앱은 서버가 아니다"를 한 문장으로 설명할 수 있는가

11. 자주 묻는 질문(FAQ)

Q. 핸들러를 `async def` 로 써야 하나요, 그냥 `def` 로 써도 되나요? A. 둘 다 동작합니다. 이번 장처럼 `await` 를 쓰지 않는 핸들러는 `def` 로 써도 무방하며, FastAPI가 일반 `def` 핸들러는 스레드풀에서 자동 실행합니다. 다만 외부 I/O를 `await` 로 호출할 때 `async def` 가 진가를 발휘합니다(5장에서 깊이 다룹니다). 입문 단계에서는 `async def` 를 기본으로 쓰되 차이를 5장에서 정확히 익히면 됩니다.

Q. `fastapi dev` 와 `uvicorn main:app` 은 무엇이 다른가요? A. `fastapi dev` 는 FastAPI CLI가 자동 리로드를 켜 채 `uvicorn` 을 대신 띄워 주는 개발 편의 명령입니다. `uvicorn main:app` 은 서버를 더 명시적으로 제어하는 호출이고, 운영 설정과 가깝습니다. 개발 중에는 `fastapi dev` 가 편하고, 배포 구성은 12장에서 다룹니다.

Q. `/docs` 가 404로 안 보입니다. A. 앱이 실제로 떠 있는지, 그리고 주소가 `http://127.0.0.1:8000/docs` 인지 확인하세요. 포트를 바꿔 띄웠다면 그 포트로 접속해야 합니다. 또

한 운영 환경에서는 `FastAPI(docs_url=None)` 처럼 문서를 끌 수도 있는데, 지금 단계에서는 끄지 않았으니 기본값으로 켜져 있습니다.

Q. 반환값으로 `dict`·`list` 만 돌려줄 수 있나요? A. 지금은 그렇게 했지만, 다음 장들에서 Pydantic 모델을 반환하면 검증·필터링·문서화가 훨씬 강력해집니다(3·4장). `dict`·`list` 반환은 가장 단순한 출발점일 뿐입니다.

이 장에서 우리는 **FastAPI 기초와 자동 문서**라는 개념 하나를 충분히 익혔습니다. 앱(`FastAPI()`)과 서버(`uvicorn`)의 분리, 경로 동작 데코레이터(`@app.get`), 그리고 타입 힌트에서 자동으로 파생되는 `/docs`·`/redoc`·`/openapi.json` 까지 살펴보았습니다. 다음 장에서는 이 엔드포인트들이 **입력**을 받기 시작합니다. URL 경로와 쿼리 스트링으로 들어오는 값을 타입 힌트로 받아 자동 변환·검증하는 **경로·쿼리 파라미터**로 넘어갑니다.

경로·쿼리 파라미터: 입력을 타입으로 받기

학습 목표

- 경로 파라미터를 타입 힌트로 선언해 자동 변환·검증되게 할 수 있습니다.
- 쿼리 파라미터로 선택·필수·기본값을 구분해 받을 수 있습니다.
- `Query()` · `Path()` 로 범위·길이·패턴 제약을 강제할 수 있습니다.
- 경로 충돌(고정 경로 vs 변수 경로)을 피하도록 순서를 설계할 수 있습니다.

1장에서 우리는 입력이 없는 엔드포인트(`GET /health`, `GET /products`)를 만들었습니다. 이제 API가 **입력을 받기** 시작합니다. "3번 상품을 보여 줘", "주변기기 카테고리만, 20개씩 보여 줘" 같은 요청이 들어오면, 그 "3", "주변기기", "20"을 URL에서 뽑아 처리해야 합니다. 이 장에서는 단 하나의 개념 **경로·쿼리 파라미터와 타입 검증(c02)**을 충실히 익혀, URL에 담겨 오는 입력을 타입 힌트만으로 안전하게 받아 처리합니다.

이 단원의 선수 지식: 1장의 **FastAPI 기초와 자동 문서(c01)**입니다. 앱(`FastAPI()`)·경로 동작 데코레이터(`@app.get`)·타입 힌트에서 자동 문서가 나온다는 점을 이미 알고 있다고 전제합니다. 이번 장은 그 경로 동작 함수에 **인자**를 더해 입력을 받는 이야기입니다.

1. 도입: URL에 담겨 오는 입력 다루기

REST API에서 클라이언트가 원하는 것을 표현하는 가장 흔한 통로가 URL입니다. URL은 두 곳에 입력을 씁니다.

- **경로(path):** `/products/3` 의 `3` 처럼 **무엇을** 가리키는지(어떤 자원인지)를 경로 자체에 박아 넣습니다.
- **쿼리 스트링(query string):** `/products?category=주변기기&limit=20` 의 `?` 뒤처럼 **어떻게** 거를 지(필터·정렬·페이지) 옵션을 키-값으로 붙입니다.

1장에서 본 것처럼, 전통적 프레임워크에서는 이 값들을 핸들러 안에서 손으로 꺼내고 (`request.args.get("limit")`) 손으로 정수 변환·검증(`int(...)`, `if limit < 1: ...`)합니다. FastAPI는 이 일을 **함수 인자의 타입 힌트**로 흡수합니다. 함수에 `product_id: int` 라고만 적으면, FastAPI가 URL에서 값을 뽑아 `int` 로 변환하고, 변환·검증에 실패하면 핸들러를 부르지 않고 곧장 422를 돌려줍니다. 우리는 "올바른 입력만 들어온다"고 가정하고 본문 로직에 집중할 수 있습니다.

한눈에 보는 이 장의 지도

- `{...}` 로 받는다 → **경로 파라미터**(어떤 자원인가)
- `?key=value` 로 받는다 → **쿼리 파라미터**(어떻게 거를까)
- 기본값을 준다 → **선택**, 안 준다 → **필수**
- 더 엄격하게 → `Path()` · `Query()` 로 `ge` · `le` · `min_length` · `pattern` 제약
- 허용 값을 못 박는다 → **Enum** 경로 파라미터
- 함정 하나 → 고정 경로를 변수 경로보다 먼저 두기

2. 정의: 경로 파라미터와 쿼리 파라미터

한 문장으로 정의하겠습니다.

경로·쿼리 파라미터와 타입 검증 — 경로 파라미터(path parameter)는 URL 경로에 포함된 변수이고, 쿼리 파라미터(query parameter)는 `?` 뒤의 키-값입니다. 함수 인자에 **타입 힌트**로 선언하면 FastAPI가 자동으로 형을 변환·검증하고, 실패하면 **422** 응답을 돌려줍니다. `Path()` · `Query()` 로 길이·범위·정규식 같은 제약을 추가합니다.

두 파라미터의 구분을 표로 정리합니다.

구분	경로 파라미터(path)	쿼리 파라미터(query)
위치	경로 안 <code>{name}</code>	<code>?</code> 뒤 <code>key=value</code>
예시	<code>/products/3</code> 의 3	<code>/products?limit=20</code> 의 20
선언	경로 문자열의 <code>{...}</code> 와 같은 이름의 인자	경로에 없는 이름의 인자
의미	자원을 식별(무엇을)	자원을 거름(어떻게)
필수성	경로에 박혀 항상 존재(필수)	기본값을 주면 선택, 안 주면 필수
제약 도구	<code>Path()</code>	<code>Query()</code>

핵심 규칙은 **선언 위치로 종류가 결정된다**는 것입니다. 함수 인자 이름이 경로 문자열의 `{...}` 안에 있으면 경로 파라미터, 없으면(그리고 본문 모델이 아니라 단순 타입이면) 쿼리 파라미터로 해석됩니다. 우리가 별도 설정을 하지 않아도 FastAPI가 이름 매칭으로 자동 분류합니다.


```

from fastapi import FastAPI

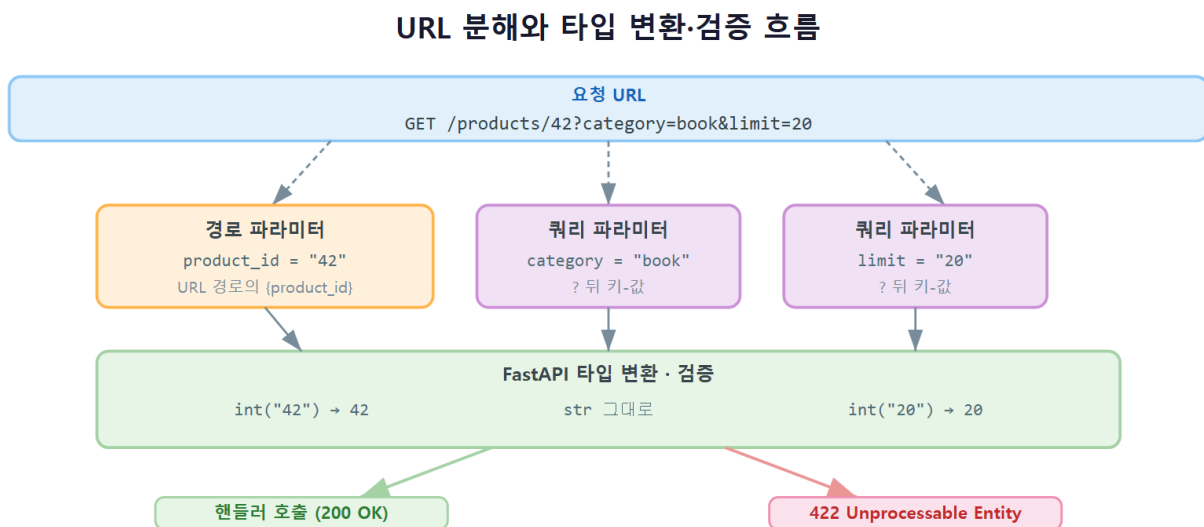
app = FastAPI()

# product_id 는 경로의 {product_id} 와 이름이 같으므로 '경로 파라미터'
# limit 는 경로에 없는 이름이므로 '쿼리 파라미터'
@app.get("/products/{product_id}/reviews")
async def list_reviews(product_id: int, limit: int = 10) -> dict:
    return {"product_id": product_id, "limit": limit}

```

GET /products/7/reviews?limit=5 를 호출하면 `product_id=7` (경로), `limit=5` (쿼리)로 들어옵니다.
`limit` 을 생각하면 기본값 `10` 이 쓰입니다.

3. 동작 원리: URL 분해와 타입 변환·검증 흐름



요청 URL `GET /products/42?category=book&limit=20` 이 처리되는 과정을 단계로 따라가 보겠습니다.

- 경로 매칭:** 앱이 등록된 경로 중 `/products/{product_id}` 패턴에 `/products/42` 를 맞춥니다. 패턴의 `{product_id}` 자리에 `42` (문자열)가 잡힙니다.
- 쿼리 분해:** `?` 뒤를 `category=book`, `limit=20` 두 쌍으로 쪼갭니다. 모두 처음에는 **문자열**입니다.
- 인자 매핑:** 핸들러의 인자 이름과 대조합니다. `product_id` 는 경로에서, `category` · `limit` 은 쿼리에서 값을 가져옵니다.
- 타입 변환·검증:** 각 인자의 타입 힌트에 맞춰 변환합니다. `product_id: int` 이면 `"42" → 42`, `limit: int` 이면 `"20" → 20`. 변환할 수 없으면(예: `/products/abc`) 그 인자에서 검증이 실패합니다.

5. **분기**: 모든 인자가 통과하면 핸들러를 호출합니다. 하나라도 실패하면 **핸들러를 부르지 않고** 422 Unprocessable Entity 와 함께 "어떤 값이 왜 틀렸는지" 담은 오류 본문을 돌려줍니다.

여기서 가장 중요한 개념은 **검증은 핸들러 앞에서 끝난다**는 것입니다. 우리 함수 본문이 실행될 시점에는 `product_id` 가 이미 올바른 `int` 임이 보장됩니다. 그래서 본문에서 다시 타입을 검사할 필요가 없습니다.

단계	입력	출력
경로 매칭	<code>/products/42</code>	<code>product_id = "42"</code>
쿼리 분해	<code>?category=book&limit=20</code>	<code>category="book", limit="20"</code>
타입 변환	타입 힌트 <code>int</code>	<code>product_id=42, limit=20</code>
검증 통과 시	—	핸들러 호출
검증 실패 시	<code>/products/abc</code>	422 + 오류 상세

검증 실패 시 응답 본문은 다음과 같은 모습입니다(어떤 위치의 어떤 값이 왜 틀렸는지 구조적으로 알려 줍니다).

```
$ curl http://127.0.0.1:8000/products/abc
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["path", "product_id"],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "abc"
    }
  ]
}
```

4. worked example: GET /products/{product_id} 단건 조회

경로 파라미터로 상품 하나를 조회하는 엔드포인트를 만듭니다. 핵심은 `product_id: int` 하나로 자동 변환과 422가 따라온다는 점입니다.

```
# main.py
from fastapi import FastAPI, HTTPException

app = FastAPI(title="Shop API", version="0.2.0")

CATALOG = {
    1: {"id": 1, "name": "기계식 키보드", "price": 89000, "category": "주변기기"},
    2: {"id": 2, "name": "27인치 모니터", "price": 235000, "category": "디스플레이"},
    3: {"id": 3, "name": "USB-C 허브", "price": 32000, "category": "주변기기"},
}

@app.get("/products/{product_id}", tags=["products"])
async def get_product(product_id: int) -> dict:
    # 이 시점에서 product_id 는 이미 올바른 int 임이 보장됩니다.
    product = CATALOG.get(product_id)
    if product is None:
        # 형식은 맞지만 존재하지 않는 자원 → 404 (검증 422와 구분)
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    return product
```

줄별 해설

- `@app.get("/products/{product_id}")` — 경로 문자열에 `{product_id}` 를 넣어 경로 파라미터 자리를 만듭니다.
- `async def get_product(product_id: int)` — 함수 인자 이름이 `{product_id}` 와 같아 경로 파라미터로 묶입니다. 타입 힌트 `int` 덕분에 `"3"` 이 `3` 으로 자동 변환되고, `"abc"` 처럼 변환 불가능한 값에는 핸들러 진입 전에 422가 납니다.
- `CATALOG.get(product_id)` — 변환이 끝난 정수 키로 딕셔너리를 조회합니다. 본문에서 타입 걱정 없이 바로 씁니다.
- `raise HTTPException(status_code=404, ...)` — "형식은 맞지만 없는 상품"은 검증 오류(422)가 아니라 **404**입니다. 두 상황을 구분하는 것이 실무 포인트입니다. 422는 입력 형식 문제, 404는 존재하지 않는 자원입니다.

호출 결과:

```
$ curl http://127.0.0.1:8000/products/3
{"id":3,"name":"USB-C 허브","price":32000,"category":"주변기기"}

$ curl http://127.0.0.1:8000/products/999
{"detail":"상품을 찾을 수 없습니다"} # 404 (없는 자원)

$ curl http://127.0.0.1:8000/products/abc
{"detail":[{"type":"int_parsing","loc":["path","product_id"], ...}]} # 422 (형식 오류)
```

여기에 `Path()` 로 제약을 더하면 더 견고해집니다. 상품 ID는 1 이상이어야 하므로 `ge=1` (greater than or equal)을 강제합니다.

```
# main.py — Path() 제약 추가 버전
from typing import Annotated
from fastapi import FastAPI, Path, HTTPException

app = FastAPI(title="Shop API", version="0.2.0")

CATALOG = {1: {"id": 1, "name": "기계식 키보드"}, 2: {"id": 2, "name": "모니터"}}

@app.get("/products/{product_id}", tags=["products"])
async def get_product(
    product_id: Annotated[int, Path(ge=1, description="조회할 상품 ID(1 이상)")],
) -> dict:
    product = CATALOG.get(product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    return product
```

줄별 해설

- `from typing import Annotated` — FastAPI 권장 형식인 `Annotated` 를 가져옵니다. 타입과 메타 데이터(제약)를 한 인자에 함께 묶는 현대적 방식입니다.
- `Annotated[int, Path(ge=1, description=...)]` — 타입은 `int`, 제약은 `Path(ge=1)` 입니다. `ge=1` 은 "1 이상"을 강제하므로 `/products/0` 이나 `/products/-5` 에는 422 가 납니다. `description` 은 자동 문서에 그대로 노출됩니다.
- 이 한 줄로 "정수 변환 + 1 이상" 두 검증이 핸들러 진입 전에 끝납니다.

```
$ curl http://127.0.0.1:8000/products/0
{"detail":[{"type":"greater_than_equal","loc":["path","product_id"],
      "msg":"Input should be greater than or equal to 1","input":"0"}]}
```

5. 쿼리 파라미터와 기본값·선택값(`| None`)

쿼리 파라미터는 "선택"과 "필수"를 **기본값**으로 구분합니다. 규칙은 단순합니다.

- 인자에 **기본값**을 주면 **선택**입니다. 클라이언트가 생략하면 기본값이 쓰입니다.
- 인자에 **기본값**을 주지 않으면 **필수**입니다. 생략하면 422 가 납니다.
- "있어도 되고 없어도 되며, 없으면 `None`"을 표현하려면 `str | None = None` 처럼 **Optional** 타입 + **기본값** `None` 으로 선언합니다.

```
# main.py
from fastapi import FastAPI

app = FastAPI(title="Shop API", version="0.2.0")

CATALOG = [
    {"id": 1, "name": "기계식 키보드", "category": "주변기기"},
    {"id": 2, "name": "27인치 모니터", "category": "디스플레이"},
    {"id": 3, "name": "USB-C 허브", "category": "주변기기"},
]

@app.get("/products", tags=["products"])
async def list_products(
    category: str | None = None,  # 선택: 없으면 전체
    limit: int = 20,              # 선택: 기본 20
    q: str | None = None,         # 선택: 이름 부분 검색어
) -> list[dict]:
    items = CATALOG
    if category is not None:
        items = [p for p in items if p["category"] == category]
    if q is not None:
        items = [p for p in items if q in p["name"]]
    return items[:limit]
```

줄별 해설

- `category: str | None = None` — 타입이 `str | None` 이고 기본값이 `None` 이므로 **선택 파라미터** 입니다. 클라이언트가 `?category=주변기기` 를 보내면 그 값, 안 보내면 `None` 이 들어옵니다.
- `limit: int = 20` — 기본값 `20` 이 있으므로 선택이며, 생략 시 `20` 이 쓰입니다. `?limit=5` 처럼 보내면 `5` 로 변환됩니다.
- `q: str | None = None` — 이름 부분 검색용 선택 파라미터입니다.
- 본문에서는 `if category is not None` 처럼 "보냈는지"를 확인해 필터를 단계적으로 좁힙니다.
- `return items[:limit]` — 최종적으로 `limit` 개까지만 잘라 돌려줍니다.

```
$ curl "http://127.0.0.1:8000/products?category=주변기기"
[{"id":1,"name":"기계식 키보드","category":"주변기기"},
 {"id":3,"name":"USB-C 허브","category":"주변기기"}]

$ curl "http://127.0.0.1:8000/products?q=모니터&limit=1"
[{"id":2,"name":"27인치 모니터","category":"디스플레이"}]

$ curl "http://127.0.0.1:8000/products"
[ ...전체 3개, limit 기본값 20 적용... ]
```

만약 어떤 쿼리 파라미터를 **필수**로 만들고 싶다면 기본값을 주지 않으면 됩니다. 예컨대 `keyword: str` (기본값 없음)으로 선언하면 `?keyword=...` 를 생략한 요청은 `422` 로 거부됩니다.

6. Query()로 제약: ge·le·min_length·pattern

선택/필수 구분만으로는 부족합니다. "페이지 크기는 1 이상 100 이하", "검색어는 최소 2글자", "정렬 키는 price 나 name 만" 같은 규칙을 강제해야 합니다. `Query()` 가 이 일을 합니다. 마찬가지로 `Annotated` 로 묶는 것이 권장 형식입니다.

제약	의미	적용 대상
ge / le	이상(greater/equal) / 이하(less/equal)	숫자
gt / lt	초과 / 미만	숫자
min_length / max_length	최소/최대 길이	문자열
pattern	정규식 패턴 일치(Pydantic v2)	문자열

```
# main.py
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI(title="Shop API", version="0.2.0")

CATALOG = [{"id": i, "name": f"상품{i}", "price": i * 1000} for i in range(1, 51)]

@app.get("/products", tags=["products"])
async def list_products(
    # 페이지 크기: 1 이상 100 이하, 기본 20
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    # 건너뛴 개수: 0 이상, 기본 0
    offset: Annotated[int, Query(ge=0)] = 0,
    # 검색어: 최소 2글자, 선택
    q: Annotated[str | None, Query(min_length=2, max_length=50)] = None,
    # 정렬 키: price 또는 name 만 허용(정규식)
    sort: Annotated[str, Query(pattern="^(price|name)$")] = "id",
) -> dict:
    items = CATALOG
    if q is not None:
        items = [p for p in items if q in p["name"]]
    if sort in ("price", "name"):
        items = sorted(items, key=lambda p: p[sort])
    page = items[offset: offset + limit]
    return {"total": len(items), "limit": limit, "offset": offset, "items": page}
```

줄별 해설

- `limit: Annotated[int, Query(ge=1, le=100)] = 20` — `ge=1, le=100` 으로 1~100 범위를 강제합니다. `?limit=0` 이나 `?limit=500` 은 422 로 거부됩니다. 페이지네이션에서 과도한 조회를 막는

핵심 안전장치입니다.

- `offset: Annotated[int, Query(ge=0)] = 0` — 음수 offset을 막습니다.
- `q: Annotated[str | None, Query(min_length=2, ...)] = None` — 선택이면서, 보낼 경우 최소 2 글자여야 합니다. `?q=a` 는 422 입니다.
- `sort: Annotated[str, Query(pattern="^(price|name)$")] = "id"` — 정규식으로 허용 값을 `price·name` 으로 제한합니다(기본값 `id` 는 패턴 검사를 받지 않고 명시 입력만 검사됨에 유의). 임의의 컬럼명을 정렬에 넘기는 것을 막아, 의도치 않은 동작을 예방합니다.
- 본문은 검색·정렬 후 `offset:offset+limit` 으로 페이지를 잘라 메타데이터와 함께 반환합니다.

```
$ curl "http://127.0.0.1:8000/products?limit=3&sort=price"
{"total":50,"limit":3,"offset":0,
 "items":[{"id":1,"name":"상품1","price":1000}, ...3개... ]}

$ curl "http://127.0.0.1:8000/products?limit=0"
{"detail":[{"type":"greater_than_equal","loc":["query","limit"],
          "msg":"Input should be greater than or equal to 1"}]} # 422

$ curl "http://127.0.0.1:8000/products?sort=secret_column"
{"detail":[{"type":"string_pattern_mismatch","loc":["query","sort"], ...}]} # 422
```

7. worked example: 카테고리 필터+페이지네이션 목록

앞의 조각들을 합쳐, 실무에서 바로 쓰는 형태의 **카테고리 필터 + 가격 범위 + 페이지네이션** 목록을 완성합니다. 이것은 8장 CRUD 라우터와 종합 프로젝트 상품 목록의 기반이 됩니다.

```
# main.py
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI(title="Shop API", version="0.2.0")

CATALOG = [
    {"id": 1, "name": "기계식 키보드", "price": 89000, "category": "주변기기"},
    {"id": 2, "name": "27인치 모니터", "price": 235000, "category": "디스플레이"},
    {"id": 3, "name": "USB-C 허브", "price": 32000, "category": "주변기기"},
    {"id": 4, "name": "4K 모니터", "price": 410000, "category": "디스플레이"},
    {"id": 5, "name": "무선 마우스", "price": 45000, "category": "주변기기"},
]

@app.get("/products", tags=["products"], summary="상품 목록(필터+페이지네이션)")
async def list_products(
    category: Annotated[str | None, Query(max_length=30)] = None,
    min_price: Annotated[int | None, Query(ge=0)] = None,
    max_price: Annotated[int | None, Query(ge=0)] = None,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
) -> dict:
    items = CATALOG
    if category is not None:
        items = [p for p in items if p["category"] == category]
    if min_price is not None:
        items = [p for p in items if p["price"] >= min_price]
    if max_price is not None:
        items = [p for p in items if p["price"] <= max_price]
    total = len(items)
    page = items[offset: offset + limit]
    return {"total": total, "limit": limit, "offset": offset, "items": page}
```

출별 해설

- `category` · `min_price` · `max_price` 는 모두 선택(`| None = None`)이라, 보낸 필터만 차례로 적용됩니다. 아무것도 안 보내면 전체 목록입니다.
- `min_price` · `max_price` 에 `ge=0` 을 걸어 음수 가격 필터를 막습니다.
- `limit` 은 `ge=1`, `le=100` 으로 페이지 크기 상한을 강제합니다. 클라이언트가 한 번에 수만 건을 끌어가는 것을 구조적으로 차단합니다.
- 반환에 `total` (필터 적용 후 전체 수)을 포함해, 클라이언트가 다음 페이지 존재 여부를 판단하게 합니다. 이것이 실무 목록 응답의 표준 형태입니다.

```
$ curl "http://127.0.0.1:8000/products?category=디스플레이&min_price=300000"
{"total":1,"limit":20,"offset":0,
 "items":[{"id":4,"name":"4K 모니터","price":410000,"category":"디스플레이"}]}

$ curl "http://127.0.0.1:8000/products?limit=2&offset=2"
{"total":5,"limit":2,"offset":2,"items":[ ...3,4번째 상품... ]}
```


8. Enum 경로 파라미터와 경로 순서·충돌

허용 값을 못 박는 Enum 경로 파라미터

경로 파라미터가 정해진 몇 가지 값만 받아야 한다면(예: 카테고리 슬러그), 파이썬 `Enum` 을 타입으로 쓰면 FastAPI가 허용 값을 강제하고 문서의 드롭다운으로 노출합니다.

```
# main.py
from enum import Enum
from fastapi import FastAPI

app = FastAPI(title="Shop API", version="0.2.0")

class Category(str, Enum):
    peripherals = "주변기기"
    display = "디스플레이"
    storage = "저장장치"

@app.get("/categories/{category}/products", tags=["products"])
async def products_by_category(category: Category) -> dict:
    # category 는 Category 멤버임이 보장됩니다. .value 로 문자열을 얻습니다.
    return {"category": category.value}
```

출별 해설

- `class Category(str, Enum)` — `str` 을 함께 상속해 JSON 직렬화·문서화가 매끄럽게 됩니다. 멤버 값이 곧 허용 문자열입니다.
- `category: Category` — 경로 파라미터 타입을 Enum으로 선언하면, 정의된 값(주변기기·디스플레이·저장장치) 외의 값에는 422 가 납니다. /docs 에는 선택 가능한 값이 드롭다운으로 나타납니다.
- `category.value` — 핸들러 안에서는 Enum 멤버로 들어오므로 `.value` 로 원래 문자열을 꺼냅니다.

```
$ curl "http://127.0.0.1:8000/categories/주변기기/products"
{"category": "주변기기"}

$ curl "http://127.0.0.1:8000/categories/없는카테고리/products"
{"detail": [{"type": "enum", "loc": ["path", "category"],
    "msg": "Input should be '주변기기', '디스플레이' or '저장장치'"}]} # 422
```

경로 순서와 충돌 — 고정 경로를 먼저

같은 접두어에 고정 경로(`/products/featured`)와 변수 경로(`/products/{product_id}`)가 함께 있으면 등록 순서가 중요합니다. FastAPI는 경로를 위에서 아래로 먼저 일치하는 것에 매칭합니다. 변수 경

로를 먼저 등록하면, `/products/featured` 요청이 `product_id="featured"` 로 잘못 잡혀 `int` 변환에 실패하거나 엉뚱하게 동작합니다.

```
# main.py
from fastapi import FastAPI, HTTPException

app = FastAPI(title="Shop API", version="0.2.0")

CATALOG = {1: {"id": 1, "name": "키보드"}, 2: {"id": 2, "name": "모니터"}}

# (1) 고정 경로를 먼저 등록합니다.
@app.get("/products/featured", tags=["products"])
async def featured_products() -> list[dict]:
    return [CATALOG[1]]

# (2) 변수 경로는 그 뒤에 둡니다.
@app.get("/products/{product_id}", tags=["products"])
async def get_product(product_id: int) -> dict:
    product = CATALOG.get(product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    return product
```

줄별 해설

- `@app.get("/products/featured")` 를 먼저 등록했습니다. 그래서 `/products/featured` 요청은 이 핸들러에 정확히 매칭됩니다.
- `@app.get("/products/{product_id}")` 는 그 다음입니다. `featured` 가 위에서 이미 잡혔으므로, 변수 경로는 `/products/3` 같은 숫자 경로만 처리합니다.
- 만약 순서를 뒤집으면, `/products/featured` 가 `{product_id}` 로 먼저 잡혀 `"featured"` 를 `int` 로 변환하려다 422 가 납니다. 순서 하나로 동작이 갈립니다.

```
$ curl "http://127.0.0.1:8000/products/featured"
[{"id":1,"name":"키보드"}]           # 고정 경로가 먼저라 정상

$ curl "http://127.0.0.1:8000/products/2"
{"id":2,"name":"모니터"}           # 변수 경로가 숫자만 처리
```

9. 즉시 연습(2종 1세트): 페이지 크기 제약 채우기

모범 답안. 리뷰 목록에 `rating` 필터와 페이지네이션을 적용한 모습입니다.

```

from typing import Annotated
from fastapi import FastAPI, Path, Query

app = FastAPI(title="Shop API", version="0.2.0")

REVIEWS = [{"id": i, "product_id": (i % 3) + 1, "rating": (i % 5) + 1} for i in range(30)]

@app.get("/products/{product_id}/reviews", tags=["reviews"])
async def list_reviews(
    product_id: Annotated[int, Path(ge=1)],
    min_rating: Annotated[int, Query(ge=1, le=5)] = 1,
    limit: Annotated[int, Query(ge=1, le=50)] = 10,
) -> dict:
    items = [r for r in REVIEWS if r["product_id"] == product_id and r["rating"] >= min_rating]
    return {"total": len(items), "items": items[:limit]}

```

직접 작성 (빈칸 코드)

상품 목록을 **가격 오름차순**으로, **페이지 크기 1~100 제약**과 함께 돌려주는 핸들러를 완성하세요.
 ____ 를 채우면 됩니다.

```

from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI(title="Shop API", version="0.2.0")

CATALOG = [
    {"id": 1, "name": "키보드", "price": 89000},
    {"id": 2, "name": "모니터", "price": 235000},
    {"id": 3, "name": "허브", "price": 32000},
]

@app.get("/products", tags=["products"])
async def list_products(
    # (1) limit: 1 이상 100 이하, 기본 20 으로 제약하세요.
    limit: Annotated[int, Query(____, ____)] = 20,
    # (2) offset: 0 이상, 기본 0 으로 제약하세요.
    offset: Annotated[int, Query(____)] = 0,
) -> dict:
    # (3) price 기준 오름차순 정렬하세요.
    items = sorted(CATALOG, key=lambda p: ____ )
    # (4) offset 부터 limit 개 잘라 페이지를 만드세요.
    page = items[____ : ____]
    return {"total": len(items), "items": page}

```

자가체크 (체크리스트)

- [] ?limit=0 또는 ?limit=500 에 422 가 나는가
- [] ?offset=-1 에 422 가 나는가
- [] 기본 호출(/products)이 가격 오름차순으로 정렬되는가
- [] ?limit=2&offset=1 이 2번째부터 2개를 돌려주는가

- [] 제약을 `Annotated` 로 묶는 권장 형식을 설명할 수 있는가

참고 답: `Query(ge=1, le=100), Query(ge=0), key=lambda p: p["price"], items[offset: offset + limit]`.

10. 흔한 오류와 오개념: 자동 변환·경로 순서·필수/선택

오개념 1 — "경로 파라미터는 문자열이라 직접 `int()` 로 바꿔야 한다." 아닙니다. URL에서는 문자열로 들어오지만, 타입 힌트가 `int` 이면 FastAPI가 **자동 변환**합니다. 핸들러 본문이 실행될 때 `product_id` 는 이미 `int` 입니다. 본문에서 `int(product_id)` 를 다시 부를 필요가 없고, 변환 불가능한 값은 핸들러 진입 전에 `422` 로 걸러집니다.

오개념 2 — "경로 등록 순서는 상관없다." 아닙니다. `/products/{product_id}` 를 `/products/featured` 보다 먼저 두면, `featured` 가 `{product_id}` 로 잘못 잡힙니다. **고정 경로를 변수 경로보다 위에** 등록해야 합니다. 순서 하나가 동작을 가릅니다.

오개념 3 — "쿼리 파라미터는 다 선택이다." 아닙니다. **기본값을 주면 선택, 안 주면 필수**입니다. 기본값 없는 쿼리 파라미터(`keyword: str`)를 생략한 요청은 `422` 로 거부됩니다. "있어도 없어도 됨"은 `str | None = None` 으로 명시해야 합니다.

추가로 자주 부딪히는 실수입니다.

증상	원인	해결
없는 상품에 422 가 남	검증 오류와 존재하지 않는 자원을 혼동	형식 OK:존재 X는 <code>raise HTTPException(404)</code>
<code>Query(ge=1)</code> 이 안 먹힘	<code>Annotated</code> 없이 타입만 적음	<code>Annotated[int, Query(ge=1)]</code> 로 묶 기
한글 쿼리값이 깨짐	URL 인코딩 문제(터미널)	<code>curl</code> 에 따옴표·인코딩, <code>/docs</code> 로 확인
<code>sort=임의값</code> 이 통과	허용 값 제한 안 함	<code>Query(pattern=...)</code> 또는 Enum으 로 제한
필수로 만들고 싶은데 선택이 됨	기본값을 줌	기본값을 제거해 필수로

11. 단원 미니 프로젝트: 상품 조회 파라미터 API(2종 1세트)

이 단원의 개념(경로·쿼리 파라미터와 타입 검증, `c02`)만으로 완성하는 산출물입니다. 목표는 경로 파라미터로 단건 조회, 쿼리 파라미터로 카테고리 필터·페이지네이션을 받고, 잘못된 값에 422를 반환하는 앱입니다.

마일스 톤	할 일	사용 개념
M1	<code>GET /products/{product_id}</code> 단건 조회(int 자동 변환·없으면 404)	경로 파라미터·자동 변환
M2	<code>GET /products?category=</code> 로 카테고리 필터	선택 쿼리 파라미터
M3	<code>Query(ge=1, le=100)</code> 로 페이지 크기 제약·422 확인	<code>Query()</code> 제약·페이지네이션

모범 답안 (완성 `main.py`).

```
# main.py — 상품 조회 파라미터 API
from typing import Annotated
from fastapi import FastAPI, Path, Query, HTTPException

app = FastAPI(title="Shop API", version="0.2.0")

CATALOG = [
    {"id": 1, "name": "기계식 키보드", "price": 89000, "category": "주변기기"},
    {"id": 2, "name": "27인치 모니터", "price": 235000, "category": "디스플레이"},
    {"id": 3, "name": "USB-C 허브", "price": 32000, "category": "주변기기"},
    {"id": 4, "name": "4K 모니터", "price": 410000, "category": "디스플레이"},
]

# M1: 단건 조회
@app.get("/products/{product_id}", tags=["products"])
async def get_product(
    product_id: Annotated[int, Path(ge=1)],
) -> dict:
    for p in CATALOG:
        if p["id"] == product_id:
            return p
    raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")

# M2 + M3: 필터 + 페이지네이션
@app.get("/products", tags=["products"])
async def list_products(
    category: Annotated[str | None, Query(max_length=30)] = None,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
) -> dict:
    items = CATALOG
    if category is not None:
        items = [p for p in items if p["category"] == category]
    total = len(items)
    page = items[offset: offset + limit]
    return {"total": total, "limit": limit, "offset": offset, "items": page}
```

실행과 확인:

```
$ curl http://127.0.0.1:8000/products/2
{"id":2,"name":"27인치 모니터","price":235000,"category":"디스플레이"}

$ curl "http://127.0.0.1:8000/products?category=주변기기&limit=1"
{"total":2,"limit":1,"offset":0,"items":[{"id":1,"name":"기계식 키보드", ...}]}

$ curl "http://127.0.0.1:8000/products?limit=0"
{"detail":[{"type":"greater_than_equal","loc":["query","limit"], ...}]} # 422

$ curl http://127.0.0.1:8000/products/abc
{"detail":[{"type":"int_parsing","loc":["path","product_id"], ...}]} # 422
```

직접 작성 (빈 스타터)

아래 빈 스타터를 채워 미니 프로젝트를 완성하세요. `Query` 제약과 필터 로직을 비워 두었습니다.

```
# main.py
from typing import Annotated
from fastapi import FastAPI, Path, Query, HTTPException

app = FastAPI(title="Shop API", version="0.2.0")

CATALOG = [
    {"id": 1, "name": "키보드", "price": 89000, "category": "주변기기"},
    {"id": 2, "name": "모니터", "price": 235000, "category": "디스플레이"},
    {"id": 3, "name": "허브", "price": 32000, "category": "주변기기"},
]

# M1: 단건 조회 — product_id 를 1 이상 정수로 받고, 없으면 404 를 던지세요.
@app.get("/products/{product_id}", tags=["products"])
async def get_product(
    product_id: Annotated[int, Path(____)],
) -> dict:
    for p in CATALOG:
        if p["id"] == product_id:
            return p
    raise HTTPException(status_code=____, detail="상품을 찾을 수 없습니다")

# M2+M3: 필터 + 페이지네이션
@app.get("/products", tags=["products"])
async def list_products(
    category: Annotated[str | None, Query(max_length=30)] = None,
    # limit 를 1~100, 기본 20 으로 제약하세요.
    limit: Annotated[int, Query(____, ____)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
) -> dict:
    items = CATALOG
    if category is not None:
        # 카테고리 필터를 작성하세요.
        items = ____
    total = len(items)
    page = items[____ : ____]
    return {"total": total, "limit": limit, "offset": offset, "items": page}
```

자가체크 (체크리스트)

- [] `/products/abc` 처럼 문자열 id에 422 가 나는가
- [] `/products/999` 처럼 없는 id에 404 가 나는가(검증 오류와 구분)
- [] `?category=주변기기` 필터가 카테고리별로 동작하는가
- [] `?limit=0` · `?limit=500` 에 422 가 나며 상한이 강제되는가
- [] `?limit=1&offset=1` 이 페이지를 올바르게 자르는가
- [] 고정 경로와 변수 경로가 함께 있을 때 등록 순서를 설명할 수 있는가

참고 답: `Path(ge=1)`, `status_code=404`, `Query(ge=1, le=100)`, `items = [p for p in items if p["category"] == category]`, `items[offset: offset + limit]`.

이 장에서 우리는 **경로·쿼리 파라미터와 타입 검증** 개념을 충분히 익혔습니다. 경로 파라미터로 자원을 식별하고(자동 `int` 변환·404 / 422 구분), 쿼리 파라미터로 필터·페이지네이션을 받으며(기본값으로 선택/필수 구분), `Path()` · `Query()` 로 `ge` · `le` · `min_length` · `pattern` 제약을 강제하고, Enum과 경로 순서로 안전하게 설계하는 법까지 살펴보았습니다. 다음 장에서는 URL이 아니라 **요청 본문(JSON)**을 받습니다. Pydantic v2 `BaseModel` 로 본문 구조를 선언해 자동 검증하는 **요청 본문 검증**으로 넘어갑니다.

Pydantic 모델: 요청 본문 검증

학습 목표

- `BaseModel` 로 요청 본문(request body)의 구조를 선언하고 자동으로 검증되게 할 수 있습니다.
- `Field()` 로 제약·기본값·예시 메타데이터를 부여할 수 있습니다.
- 중첩 모델과 `list[Model]` 필드로 주문-주문항목 같은 복합 구조를 표현할 수 있습니다.
- `field_validator` (Pydantic v2)로 도메인 규칙을 강제하고, 위반 시 422가 나가는 흐름을 설명할 수 있습니다.

앞 단원까지 우리는 URL에 실려 오는 입력, 즉 경로 파라미터와 쿼리 파라미터를 타입 힌트로 받았습니다. 그러나 상품 등록이나 주문 생성처럼 **여러 필드를 가진 구조화된 데이터**는 URL이 아니라 HTTP 요청의 본문(body)에 JSON으로 담겨 옵니다. 이번 장의 주인공은 그 JSON 본문을 **타입 안전한 파이썬 객체로 바꾸고 동시에 검증하는** Pydantic v2 모델입니다.

선수 개념 상기: 앞 단원(c02)의 경로·쿼리 파라미터에서, 타입 힌트만 붙이면 FastAPI가 문자열을 `int` 로 변환하고 잘못된 값에 422를 돌려준 것을 떠올리십시오. 요청 본문 검증은 그 아이디어를 **여러 필드를 가진 객체 전체**로 확장한 것입니다. 파라미터 하나에 `int` 를 선언하던 것을, 이제는 상품 한 건의 구조(이름·가격·재고...)를 클래스로 통째로 선언합니다.

1. 도입: 본문(JSON)을 안전한 객체로

실무에서 상품을 등록하는 요청은 보통 이렇게 생겼습니다.

```
POST /products
Content-Type: application/json

{
  "name": "기계식 키보드",
  "category": "주변기기",
  "price": 89000,
  "stock": 25
}
```

이 JSON을 손으로 다룬다면 어떻게 될까요. Pydantic 없이 순수하게 처리하면 다음과 같은 방어 코드가 핸들러를 가득 채웁니다.

```
# Pydantic을 쓰지 않을 때 — 검증을 직접 다 짜야 합니다(권장하지 않는 방식)
@app.post("/products")
async def create_product(payload: dict):
    if "name" not in payload:
        raise HTTPException(422, "name은 필수입니다")
    if not isinstance(payload["name"], str):
        raise HTTPException(422, "name은 문자열이어야 합니다")
    if "price" not in payload or payload["price"] <= 0:
        raise HTTPException(422, "price는 0보다 커야 합니다")
    if "stock" not in payload or payload["stock"] < 0:
        raise HTTPException(422, "stock은 0 이상이어야 합니다")
    # ... 필드마다 if 검사가 끝없이 이어집니다
    return payload
```

필드가 4개뿐인데도 검사 코드가 핸들러보다 길어집니다. 필드가 늘고 중첩 구조(주문 안의 여러 품목)가 들어오면 이 방식은 곧 무너집니다. 누락·타입 오류·범위 위반을 일관되게 보고하기도 어렵습니다.

Pydantic 모델을 쓰면 같은 일을 **선언만으로** 끝낼 수 있습니다. "이 본문은 이런 구조다"라고 클래스로 한 번 적으면, FastAPI가 들어온 JSON을 그 구조에 맞춰 파싱·검증하고, 어긋나면 어떤 필드가 왜 틀렸는지 담은 422 응답을 자동으로 돌려줍니다. 핸들러는 검증이 이미 끝난 깨끗한 객체만 받습니다.

이 장에서 만나는 키워드

- `BaseModel`: Pydantic 모델의 부모 클래스. 상속해 본문 구조를 선언합니다.
- `Field()`: 필드에 제약(`gt`·`ge`·`max_length`)·기본값·예시 메타데이터를 붙이는 함수입니다.
- `field_validator`: 한 필드의 값을 추가로 검사·가공하는 Pydantic v2 데코레이터입니다.
- 422 Unprocessable Entity: 본문이 스키마를 어겼을 때 FastAPI가 자동으로 돌려주는 상태 코드입니다.

2. 정의: Pydantic v2 BaseModel과 요청 본문

이 절부터 다루는 개념은 **Pydantic v2 모델과 요청 본문 검증**입니다. **Pydantic 모델(Pydantic model)**은 `BaseModel`을 상속해 필드 이름과 타입을 선언한 클래스로, 들어온 데이터를 그 선언에 맞춰 **파싱(parsing)** 하고 **검증(validation)** 해 타입 안전한 객체로 만들어 주는 데이터 구조입니다.

FastAPI에서는 경로 동작 함수의 인자에 **Pydantic 모델 타입**을 선언하면, FastAPI가 그 인자를 "요청 본문"으로 해석합니다.

```
from pydantic import BaseModel

class ProductCreate(BaseModel):
    name: str
    category: str
    price: int
    stock: int
```

이렇게 선언한 뒤 핸들러에서 `product: ProductCreate` 라고 받으면, FastAPI는 다음을 모두 자동으로 합니다.

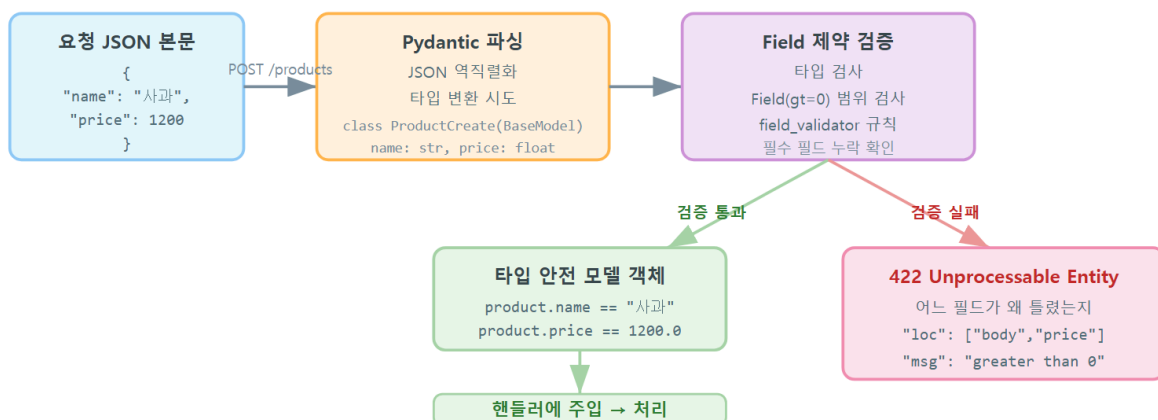
1. 요청 본문의 JSON을 읽어 들입니다.
2. `ProductCreate` 의 필드 타입에 맞춰 변환·검증합니다.
3. 통과하면 `product` 는 `ProductCreate` 의 인스턴스(객체)가 되어 `product.name` 처럼 점 표기로 접근됩니다.
4. 실패하면 핸들러는 호출되지 않고, FastAPI가 422 응답을 돌려줍니다.

여기서 중요한 구분이 하나 있습니다. 함수 인자의 타입이 **단순 타입**(`int`·`str`) 이면 FastAPI는 그것을 쿼리/경로 파라미터로 보고, **Pydantic 모델**이면 요청 본문으로 봅니다. 즉, 모델로 감싸는 행위 자체가 "이건 본문이다"라는 신호입니다.

Pydantic v2를 쓰고 있는지 확인: 이 교재는 Pydantic v2를 전제로 합니다. `pip show pydantic` 의 버전이 2.x인지 확인하십시오. v1과 v2는 검증기 데코레이터 이름(`@validator` → `field_validator`)과 설정 방식(`class Config / orm_mode` → `model_config / from_attributes`)이 다릅니다. 본문은 모두 v2 표기를 씁니다.

3. 동작 원리: JSON에서 모델 객체 또는 422까지

요청 본문 검증: JSON → Pydantic → 모델 객체 or 422



요청 본문이 핸들러에 닿기까지 FastAPI와 Pydantic이 거치는 단계를 따라가 보겠습니다.

단계	하는 일	통과 시	실패 시
1. 수신	요청 본문의 바이트를 읽어 JSON으로 역직렬화	파이썬 <code>dict</code>	깨진 JSON이면 422
2. 타입 변환	각 필드를 모델 타입으로 강제 변환 (coerce)	변환된 값	변환 불가면 오류 수집
3. 제약 검사	<code>Field()</code> · <code>field_validator</code> 의 규칙 적용	통과	위반 시 오류 수집
4. 결과	오류가 하나도 없으면 모델 인스턴스 생성	핸들러에 객체 전달	모인 오류를 422로 반환

핵심은 **검증이 핸들러보다 먼저, 그리고 한꺼번에** 일어난다는 점입니다. Pydantic은 첫 오류에서 멈추지 않고 가능한 모든 필드의 오류를 **모아서** 한 번에 보고합니다. 그래서 클라이언트는 "name이 없고 price가 음수다"를 한 응답으로 알 수 있습니다.

검증에 실패하면 FastAPI는 다음과 같은 구조의 422 응답을 만듭니다(Pydantic v2 형식).

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "price"],
      "msg": "Input should be greater than 0",
      "input": -5000
    }
  ]
}
```

`loc`은 어디서(`body`의 `price`), `type`·`msg`는 무엇이 왜 틀렸는지, `input`은 들어온 실제 값을 알려 줍니다. 이 일관된 오류 포맷을 우리가 손으로 만들 필요 없이 공짜로 얻는 것이 Pydantic의 가장 큰 실무 가치입니다.

4. worked example: ProductCreate로 POST 본문 검증

모범 예제 1 — 상품 등록 본문 검증

상품 등록 본문을 `ProductCreate` 모델로 검증하는 완전한 예제입니다. 타입 검사만으로도 상당한 방어가 이루어지는 것을 봅니다.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ProductCreate(BaseModel):
    name: str
    category: str
    price: int
    stock: int

@app.post("/products")
async def create_product(product: ProductCreate):
    # 여기 도달했다는 것은 검증이 이미 통과했다는 뜻입니다.
    total_value = product.price * product.stock
    return {
        "message": f"{product.name} 등록 접수",
        "inventory_value": total_value,
    }
```

- `class ProductCreate(BaseModel)`: 상품 등록 본문의 구조를 선언합니다. 네 필드 모두 기본값이 없으므로 **전부 필수**입니다.
- `async def create_product(product: ProductCreate)`: 인자 타입이 Pydantic 모델이므로 FastAPI가 이 인자를 요청 본문으로 해석합니다.
- `product.price * product.stock`: 핸들러 안에서는 `product`가 이미 검증된 객체이므로, `price`가 정수임이 보장됩니다. `if isinstance(...)` 같은 방어 코드가 필요 없습니다.
- 반환한 `dict`는 FastAPI가 JSON으로 직렬화해 200 응답으로 내보냅니다.

올바른 본문 `{"name": "기계식 키보드", "category": "주변기기", "price": 89000, "stock": 25}`을 보내면 실행 결과는 다음과 같습니다.

```
HTTP/1.1 200 OK

{
  "message": "기계식 키보드 등록 접수",
  "inventory_value": 2225000
}
```

이번에는 `price`에 문자열을, `stock`을 누락한 잘못된 본문 `{"name": "마우스", "category": "주변기기", "price": "비쌈"}`을 보내 보겠습니다.

HTTP/1.1 422 Unprocessable Entity

```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": ["body", "price"],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "비쌘"
    },
    {
      "type": "missing",
      "loc": ["body", "stock"],
      "msg": "Field required",
      "input": {"name": "마우스", "category": "주변기기", "price": "비쌘"}
    }
  ]
}
```

핸들러 코드는 한 줄도 검증을 적지 않았는데, 잘못된 타입(`price`)과 누락 필드(`stock`)가 둘 다 잡혀 한 응답으로 보고되었습니다. 이것이 선언적 검증의 힘입니다.

5. Field()로 제약·기본값·예시

타입 검사만으로는 "가격은 0보다 커야 한다", "이름은 120자 이내"처럼 **값의 범위**까지 막을 수 없습니다. 이때 `Field()`로 제약과 메타데이터를 붙입니다.

자주 쓰는 제약은 다음과 같습니다.

제약	의미	적용 타입
<code>gt / ge</code>	초과(greater than) / 이상(greater or equal)	숫자
<code>lt / le</code>	미만 / 이하	숫자
<code>min_length / max_length</code>	최소/최대 길이	문자열·리스트
<code>pattern</code>	정규식 패턴 일치	문자열
<code>default</code>	기본값(주면 선택 필드가 됨)	모든 타입
<code>examples</code>	OpenAPI 문서에 보일 예시 값 목록	모든 타입

`Field()`를 적용한 `ProductCreate`는 다음과 같습니다.

```

from decimal import Decimal
from pydantic import BaseModel, Field

class ProductCreate(BaseModel):
    name: str = Field(min_length=1, max_length=120, examples=["기계식 키보드"])
    category: str = Field(min_length=1, max_length=40)
    price: Decimal = Field(gt=0, max_digits=10, decimal_places=2, examples=[89000])
    stock: int = Field(ge=0, examples=[25])
    is_active: bool = Field(default=True)

```

- `name: str = Field(min_length=1, max_length=120, ...)`: 빈 문자열("")과 120자 초과를 막습니다. `examples` 는 검증에는 영향이 없고 `/docs` 에 예시 값으로 표시됩니다.
- `price: Decimal = Field(gt=0, max_digits=10, decimal_places=2)`: 가격은 0보다 커야 하고(음수·0 거부), 소수 둘째 자리까지만 허용합니다. 돈은 부동소수점(`float`) 오차를 피하려고 `Decimal` 을 쓰는 것이 실무 관행입니다.
- `stock: int = Field(ge=0)`: 재고는 0 이상(음수 거부). 매진 상태인 0은 허용합니다.
- `is_active: bool = Field(default=True)`: 기본값이 있으므로 선택 필드입니다. 본문에서 빠지면 `True` 가 됩니다.

`price` 에 `-5000` 을 담아 보내면 `gt=0` 제약에 걸려 다음 422가 나옵니다.

```

HTTP/1.1 422 Unprocessable Entity

{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "price"],
      "msg": "Input should be greater than 0",
      "input": "-5000"
    }
  ]
}

```

기본값이 곧 필수/선택을 가른다: 필드에 기본값이 없으면 필수, 있으면 선택입니다. 선택 필드를 만들려면 `is_active: bool = Field(default=True)` 처럼 기본값을 주거나, 값이 올 수도 안 올 수도 있으면 `description: str | None = None` 처럼 `| None` 과 기본값 `None` 을 함께 줍니다.

6. 중첩 모델과 `list[Model]`: `OrderCreate(items)`

실무의 본문은 평평하지 않습니다. 주문 한 건에는 여러 **주문 항목(order item)** 이 들어갑니다. Pydantic 모델은 다른 모델을 필드 타입으로 가질 수 있어서, 이런 중첩 구조를 그대로 표현합니다.


```
POST /orders
{
  "customer_email": "buyer@shop.com",
  "items": [
    {"product_id": 1, "quantity": 2},
    {"product_id": 7, "quantity": 1}
  ]
}
```

이 본문을 받는 모델은 다음과 같이 작은 모델을 큰 모델 안에 끼워 선언합니다.

```
from pydantic import BaseModel, EmailStr, Field

class OrderItemCreate(BaseModel):
    product_id: int = Field(gt=0)
    quantity: int = Field(gt=0, le=99)

class OrderCreate(BaseModel):
    customer_email: EmailStr
    items: list[OrderItemCreate] = Field(min_length=1)
```

- `OrderItemCreate`: 주문 항목 하나의 구조입니다. `product_id`는 양수, `quantity`는 1 이상 99 이하로 제약했습니다.
- `items: list[OrderItemCreate]`: 주문 항목 여러 개를 리스트로 받습니다. FastAPI는 리스트 안의 각 원소도 `OrderItemCreate`로 검증합니다.
- `Field(min_length=1)`: 빈 주문(항목 0개)을 막습니다. 리스트에도 `min_length`가 적용됩니다.
- `customer_email: EmailStr`: 이메일 형식을 검증하는 Pydantic 특수 타입입니다(`pip install "pydantic[email]"` 필요). 형식이 아니면 422가 납니다.

핸들러에서는 중첩 객체를 자연스럽게 순회합니다.

```
@app.post("/orders")
async def create_order(order: OrderCreate):
    total_quantity = sum(item.quantity for item in order.items)
    return {
        "customer": order.customer_email,
        "line_count": len(order.items),
        "total_quantity": total_quantity,
    }
```

`items` 중 하나에 `quantity: 0`이 들어오면, 오류 위치가 정확히 그 항목을 가리킵니다.

HTTP/1.1 422 Unprocessable Entity

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": ["body", "items", 1, "quantity"],
      "msg": "Input should be greater than 0",
      "input": 0
    }
  ]
}
```

loc 의 ["body", "items", 1, "quantity"] 는 "본문의 items 리스트 중 **인덱스 1번** 항목의 quantity"라는 뜻입니다. 중첩이 깊어져도 오류 위치를 정확히 짚어 주는 점이 디버깅을 크게 줄여 줍니다.

7. worked example: field_validator로 가격 규칙 강제

모범 예제 2 — 도메인 규칙을 코드로

Field() 의 제약만으로는 표현하기 어려운 규칙도 있습니다. 예컨대 "상품명 앞뒤 공백을 제거하고 비어 있으면 거부", "카테고리는 정해진 목록 중 하나"처럼 **값을 가공하거나 교차 검증**하려면 field_validator (Pydantic v2)를 씁니다.

```

from decimal import Decimal
from pydantic import BaseModel, Field, field_validator

ALLOWED_CATEGORIES = {"의류", "주변기기", "식품", "도서"}

class ProductCreate(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    category: str
    price: Decimal = Field(gt=0, max_digits=10, decimal_places=2)
    stock: int = Field(ge=0)

    @field_validator("name")
    @classmethod
    def strip_and_check_name(cls, v: str) -> str:
        cleaned = v.strip()
        if not cleaned:
            raise ValueError("상품명은 공백만으로 채울 수 없습니다")
        return cleaned

    @field_validator("category")
    @classmethod
    def category_in_allowed(cls, v: str) -> str:
        if v not in ALLOWED_CATEGORIES:
            allowed = ", ".join(sorted(ALLOWED_CATEGORIES))
            raise ValueError(f"category는 다음 중 하나여야 합니다: {allowed}")
        return v

```

- `@field_validator("name")` + `@classmethod`: Pydantic v2에서는 검증기를 **클래스 메서드**로 선언합니다. 데코레이터 인자(`"name"`)가 어떤 필드를 검사할지 지정합니다. v1의 `@validator`가 아니라 `field_validator`를 쓰는 점이 핵심입니다.
- `return cleaned`: 검증기는 검사만 하는 것이 아니라 **가공한 값을 반환**할 수 있습니다. 여기서는 앞뒤 공백을 제거한 이름으로 바꿔 저장합니다.
- `raise ValueError(...)`: 규칙 위반 시 `ValueError`를 던지면, FastAPI가 이를 422 오류 항목으로 자동 변환합니다. `HTTPException`을 직접 던지지 않습니다.
- 검증기는 `Field()` 제약(`min_length` 등)을 **통과한 뒤** 실행됩니다. 즉 타입·기본 제약 → 검증기 순서입니다.

`{"name": " ", "category": "가전", "price": 1000, "stock": 5}`을 보내면, 공백뿐인 이름과 허용되지 않은 카테고리가 모두 잡힙니다.

HTTP/1.1 422 Unprocessable Entity

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": ["body", "name"],
      "msg": "Value error, 상품명은 공백만으로 채울 수 없습니다",
      "input": "  "
    },
    {
      "type": "value_error",
      "loc": ["body", "category"],
      "msg": "Value error, category는 다음 중 하나여야 합니다: 도서, 식품, 의류, 주변기기",
      "input": "가전"
    }
  ]
}
```

여러 필드를 한꺼번에: model_validator

한 필드가 아니라 **여러 필드의 관계**를 검사하려면 `model_validator` 를 씁니다. 예를 들어 "할인가는 정가보다 작아야 한다"는 규칙입니다.

```
from decimal import Decimal
from pydantic import BaseModel, Field, model_validator

class PricedProduct(BaseModel):
    list_price: Decimal = Field(gt=0)
    sale_price: Decimal = Field(gt=0)

    @model_validator(mode="after")
    def sale_not_above_list(self) -> "PricedProduct":
        if self.sale_price > self.list_price:
            raise ValueError("sale_price는 list_price를 넘을 수 없습니다")
        return self
```

- `@model_validator(mode="after")`: 모든 필드가 개별 검증을 마친 **뒤(after)** 모델 전체를 받아 검사합니다. `mode="after"` 이면 메서드는 인스턴스(`self`)를 받습니다.
- `self.sale_price > self.list_price`: 두 필드를 함께 비교하는 교차 검증입니다. 한 필드만 보는 `field_validator` 로는 불가능한 검사입니다.
- 위반 시 `ValueError` 를 던지고, 통과하면 `self` 를 반환합니다.

`{"list_price": 10000, "sale_price": 12000}` 을 보내면 `value_error` 가 나며 `loc` 은 본문 전체를 가리킵니다(특정 필드가 아니라 모델 차원의 규칙이기 때문입니다).

8. 즉시 연습 (2종 1세트)

직접 작성 1 — 리뷰 등록 모델 (빈칸 채우기)

상품 리뷰를 등록하는 `ReviewCreate` 모델을 완성하십시오. 요구사항은 다음과 같습니다.

- `rating` (평점): 1 이상 5 이하의 정수
- `title` (제목): 1자 이상 80자 이하
- `comment` (본문): 선택 필드이며 기본값은 `None`
- 제목은 앞뒤 공백을 제거하고, 비어 있으면 거부

```
from pydantic import BaseModel, Field, field_validator

class ReviewCreate(BaseModel):
    rating: int = Field(____, _____) # 1 이상 5 이하 제약 두 개
    title: str = Field(min_length=1, max_length=80)
    comment: str | None = _____ # 선택 필드 기본값

    @____("title") # 제목 검증기 데코레이터(v2)
    @classmethod
    def strip_title(cls, v: str) -> str:
        cleaned = v.strip()
        if not cleaned:
            raise ____("제목은 공백만으로 채울 수 없습니다") # 422로 변환되는 예외
        return cleaned
```

자가체크 체크리스트

- [] 첫 빈칸에 `ge=1, le=5` 를 넣어 평점 범위를 강제했는가
- [] `comment` 의 기본값을 `None` 으로 주어 선택 필드로 만들었는가
- [] 검증기 데코레이터로 v1의 `@validator` 가 아니라 `field_validator` 를 썼는가
- [] 규칙 위반에 `HTTPException` 이 아니라 `ValueError` 를 던졌는가
- [] `rating` 에 6을 넣으면 422가, 1~5면 통과가 나오는가

직접 작성 2 — 중첩 장바구니 모델 (빈칸 채우기)

장바구니 담기 요청을 받는 `CartCreate` 모델을 완성하십시오. 장바구니에는 여러 품목이 들어가며, 빈 장바구니는 거부합니다.

```

from pydantic import BaseModel, Field

class CartItem(BaseModel):
    product_id: int = Field(gt=0)
    quantity: int = Field(gt=0, le=50)

class CartCreate(BaseModel):
    items: list[____] = Field(____)      # 항목 모델 타입 + 최소 1개 제약

```

자가체크 체크리스트

- [] items 의 원소 타입을 CartItem 으로 지정해 중첩 검증되게 했는가
- [] Field(min_length=1) 로 빈 장바구니(0개)를 막았는가
- [] 항목 중 하나에 quantity: 0 을 넣으면 loc 이 해당 인덱스를 가리키는가
- [] 검증 통과 시 핸들러에서 cart.items[0].product_id 로 접근되는가

9. 흔한 오류와 오개념

오개념 1 — "Pydantic v1의 @validator-orm_mode를 그대로 쓴다"

Pydantic v2에서는 검증기와 설정의 이름·방식이 바뀌었습니다. v1 코드를 복사해 오면 동작하지 않거나 경고가 납니다.

```

# Pydantic v1 (낡은 방식 — v2에서 쓰지 마십시오)
from pydantic import BaseModel, validator

class ProductCreate(BaseModel):
    price: int

    @validator("price")          # v2에서는 field_validator
    def check(cls, v):
        ...

class Config:                  # v2에서는 model_config = ConfigDict(...)
    orm_mode = True            # v2에서는 from_attributes = True

```

교정: v2에서는 field_validator (+ @classmethod), 여러 필드 교차 검증은 model_validator(mode="after"), ORM 객체 직렬화 설정은 model_config = ConfigDict(from_attributes=True) 를 씁니다.

오개념 2 — "검증된 모델인데 핸들러에서 또 if로 검사한다"

타입 힌트와 `Field`·`field_validator` 로 이미 검증이 끝났으므로, 핸들러 안에서 같은 검사를 다시 짜는 것은 중복입니다.

```
@app.post("/products")
async def create_product(product: ProductCreate):
    if product.price <= 0:
        # 불필요 — Field(gt=0)가 이미 막았습니다
        raise HTTPException(422, "가격 오류")
    if not product.name:
        # 불필요 — min_length=10이 이미 막았습니다
        raise HTTPException(422, "이름 오류")
    ...
```

교정: 입력 형식·범위 검증은 모델에 맡기고, 핸들러에는 **모델로 표현할 수 없는 비즈니스 로직**(예: DB에 같은 이름이 이미 있는지)만 남깁니다. 핸들러에 도달했다는 것은 형식 검증이 끝났다는 뜻입니다.

오개념 3 — "필드에 기본값이 없어도 어떻게든 채워지겠지"

본문 모델 필드에 기본값이 없으면 그 필드는 **필수**입니다. 누락하면 `type: "missing"` 의 422가 납니다. 종종 "선택이려니" 하고 빠뜨려 422를 만나는 일이 있습니다.

```
class ProductCreate(BaseModel):
    name: str
    is_active: bool = True
    # 필수 — 빠지면 422
    # 선택 — 빠지면 True
```

교정: 선택으로 두려면 기본값을 명시하십시오(`is_active: bool = True`). 값이 올 수도, 안 올 수도, null 일 수도 있으면 `description: str | None = None` 처럼 `| None` + 기본값 `None` 을 줍니다.

주의 — `field_validator` 에서 `HTTPException` 을 던지지 마십시오: 검증기 안에서는 `ValueError` (또는 `AssertionError`)를 던져야 FastAPI가 일관된 422 형식으로 변환합니다. `HTTPException` 을 던지면 검증 오류 묶음에 합쳐지지 않고 흐름이 달라집니다.

10. 단원 미니 프로젝트: 상품 등록 검증 API (2종 1세트)

이 단원에서 배운 `BaseModel`·`Field`·중첩 모델·`field_validator` 를 모아 **상품 등록 검증 API**를 완성합니다. 마일스톤을 증분으로 나눠 진행합니다.

- **M1:** `ProductCreate` 모델을 정의하고 `POST /products` 본문을 자동 검증합니다.
- **M2:** `field_validator` 로 가격을 양수·소수 2자리로 제한하고 카테고리를 허용 목록으로 제한합니다.
- **M3:** 중첩 `OrderCreate(items: list[OrderItemCreate])` 로 여러 품목 주문 본문을 받습니다.

모범 산출물


```

from decimal import Decimal

from fastapi import FastAPI
from pydantic import BaseModel, EmailStr, Field, field_validator

app = FastAPI(title="상품 등록 검증 API")

ALLOWED_CATEGORIES = {"의류", "주변기기", "식품", "도서"}

# --- M1·M2: 상품 등록 모델 ---
class ProductCreate(BaseModel):
    name: str = Field(min_length=1, max_length=120, examples=["기계식 키보드"])
    category: str = Field(examples=["주변기기"])
    price: Decimal = Field(gt=0, max_digits=10, decimal_places=2, examples=[89000])
    stock: int = Field(ge=0, examples=[25])
    is_active: bool = Field(default=True)

    @field_validator("name")
    @classmethod
    def strip_name(cls, v: str) -> str:
        cleaned = v.strip()
        if not cleaned:
            raise ValueError("상품명은 공백만으로 채울 수 없습니다")
        return cleaned

    @field_validator("category")
    @classmethod
    def category_allowed(cls, v: str) -> str:
        if v not in ALLOWED_CATEGORIES:
            allowed = ", ".join(sorted(ALLOWED_CATEGORIES))
            raise ValueError(f"category는 다음 중 하나여야 합니다: {allowed}")
        return v

# --- M3: 중첩 주문 모델 ---
class OrderItemCreate(BaseModel):
    product_id: int = Field(gt=0)
    quantity: int = Field(gt=0, le=99)

class OrderCreate(BaseModel):
    customer_email: EmailStr
    items: list[OrderItemCreate] = Field(min_length=1)

@app.post("/products")
async def create_product(product: ProductCreate):
    return {"message": f"{product.name} 등록 접수", "stock": product.stock}

@app.post("/orders")
async def create_order(order: OrderCreate):
    total_quantity = sum(item.quantity for item in order.items)
    return {
        "customer": order.customer_email,
        "line_count": len(order.items),
    }

```

```
    "total_quantity": total_quantity,
}
```

올바른 주문 본문을 보내면 다음과 같이 응답합니다.

```
POST /orders
{"customer_email": "buyer@shop.com",
 "items": [{"product_id": 1, "quantity": 2}, {"product_id": 7, "quantity": 1}]}

HTTP/1.1 200 OK
{
  "customer": "buyer@shop.com",
  "line_count": 2,
  "total_quantity": 3
}
```

학생 작성형 빈 스타터

직접 작성 — 상품 등록 검증 API 완성하기

아래 빈칸을 채워 미니 프로젝트를 완성하십시오. 모범 산출물을 그대로 베끼지 말고, 자가체크를 통과하도록 스스로 채워 보십시오.

```

from decimal import Decimal

from fastapi import FastAPI
from pydantic import BaseModel, Field, field_validator

app = FastAPI(title="상품 등록 검증 API")

ALLOWED_CATEGORIES = {"의류", "주변기기", "식품", "도서"}

class ProductCreate(BaseModel):
    name: str = Field(____, _____) # 1~120자 제약
    category: str
    price: Decimal = Field(_____) # 0 초과 + 소수 2자리
    stock: int = Field(_____) # 0 이상

    @field_validator("category")
    @classmethod
    def category_allowed(cls, v: str) -> str:
        if v not in ALLOWED_CATEGORIES:
            raise ____("허용되지 않은 카테고리입니다") # 422로 변환
        return v

class OrderItemCreate(BaseModel):
    product_id: int = Field(gt=0)
    quantity: int = Field(gt=0, le=99)

class OrderCreate(BaseModel):
    items: list[____] = Field(_____) # 항목 타입 + 최소 1개

# TODO: POST /products 와 POST /orders 핸들러를 직접 작성하십시오.

```

자가체크 체크리스트

- [] name 에 min_length=1, max_length=120 을 넣었는가
- [] price 에 gt=0, max_digits=10, decimal_places=2 를 넣어 음수·세 자리 소수를 막았는가
- [] stock 에 ge=0 을 넣어 음수 재고를 막았는가
- [] 카테고리 검증기에서 ValueError 를 던졌는가
- [] OrderCreate.items 가 list[OrderItemCreate] 이고 min_length=1 로 빈 주문을 막는가
- [] 잘못된 본문에 422가, 올바른 본문에 정상 응답이 나오는지 /docs 에서 확인했는가

11. 자주 묻는 질문(FAQ)

Q. BaseModel 로 본문을 받는데도 422 대신 400이 나옵니다. **A.** 본문 자체가 깨진 JSON이거나 Content-Type: application/json 이 아니면 다른 오류가 날 수 있습니다. 스키마 위반(타입·제약)은

422, 본문 파싱 자체가 불가하면 400 계열이 납니다. 요청 헤더와 JSON 형식을 먼저 확인하십시오.

Q. float 로 가격을 받아도 되지 않나요? A. 동작은 합니다. 다만 `0.1 + 0.2` 가 정확히 `0.3` 이 되지 않는 부동소수점 오차 때문에, 금액은 `Decimal` 을 쓰고 `max_digits` · `decimal_places` 로 자릿수를 통제하는 것이 실무 관행입니다.

Q. field_validator 와 model_validator 는 언제 갈라 쓰나요? A. 한 필드만 보고 검사·가공하면 `field_validator`, 여러 필드의 관계(할인가 ≤ 정가 등)를 봐야 하면 `model_validator(mode="after")` 를 씁니다.

Q. 검증된 모델을 dict 로 바꾸려면? A. Pydantic v2에서는 `product.model_dump()` (딕셔너리) · `product.model_dump_json()` (JSON 문자열)을 씁니다. v1의 `.dict()` · `.json()` 은 v2에서 사용 중단(deprecated)되었습니다.

Q. EmailStr 을 썼더니 import 오류가 납니다. A. 이메일 검증에는 추가 의존성이 필요합니다. `pip install "pydantic[email]"` 로 `email-validator` 를 설치하면 됩니다.

이 장에서는 요청 본문을 Pydantic v2 모델로 선언해 **들어오는 데이터**를 안전하게 검증하는 법을 익혔습니다. 다음 장에서는 방향을 뒤집어, **나가는 데이터**를 `response_model` 과 상태 코드로 통제하는 법 — 입력 모델과 출력 모델을 분리해 민감 필드를 숨기는 법을 배웁니다.

응답 모델·상태 코드: 출력을 통제하기

학습 목표

- `response_model` 로 출력 스키마를 지정해 응답을 필터링할 수 있습니다.
- `status_code` 로 201(생성)·204(본문 없음) 등 정확한 HTTP 상태를 돌려줄 수 있습니다.
- 입력(Create) 모델과 출력(Read) 모델을 분리해 민감 필드(예: `password_hash`)를 응답에서 제외할 수 있습니다.
- `response_model_exclude_unset` 와 `responses=` 로 부분 갱신 응답을 표현하고 여러 상태를 문서화할 수 있습니다.

앞 장에서는 클라이언트가 보내는 **요청 본문**을 Pydantic 모델로 검증했습니다. 이번 장은 방향을 뒤집습니다. 우리가 클라이언트에게 **내보내는 응답**을, 어떤 필드를 담고 어떤 상태 코드로 보낼지까지 모델로 통제합니다. 핵심 통찰은 하나입니다 — **들어오는 모델과 나가는 모델은 다르다**.

선수 개념 상기: 앞 단원(c03)에서 `ProductCreate` 로 본문을 받아 검증했습니다. 그 모델에는 클라이언트가 **보낼** 필드(이름·가격·재고)만 있었습니다. 그런데 응답에는 서버가 부여하는 `id`·`created_at` 이 더해지고, 반대로 내부 원가나 비밀번호 해시처럼 **절대 내보내면 안 되는** 필드는 빠져야 합니다. 그래서 입력용 `Create` 모델과 출력용 `Read` 모델을 나누게 됩니다.

1. 도입: 들어온 모델과 내보낼 모델은 다르다

상품을 등록하면 서버는 DB에 저장된 완성된 상품 객체를 다룹니다. 이 내부 객체는 보통 클라이언트가 보낸 것보다 **더 많은 필드**를 가집니다.

클라이언트가 보낸 것(ProductCreate) <pre>{ "name": "기계식 키보드", "category": "주변기기", "price": 89000, "stock": 25 }</pre>	==>	서버 내부 객체(DB 행) <pre>{ "id": 42, "name": "기계식 키보드", "category": "주변기기", "price": 89000, "stock": 25, "cost": 41000, "created_at": "..." }</pre>
--	-----	--

<- 서버가 부여
 <- 내부 원가(비밀)
 <- 서버가 부여

이 내부 객체를 그대로 응답으로 돌려주면 내부 원가(`cost`)가 외부에 새어 나갑니다. 사용자 객체라면 `password_hash`가 노출되는 더 심각한 사고가 됩니다. 그렇다고 핸들러마다 손으로 `del result["cost"]` 같은 코드를 적는 것은 잊어버리기 쉽고 위험합니다.

FastAPI의 해법은 선언적입니다. "이 엔드포인트의 응답은 이런 모양이다"라고 **출력 모델(response model)**을 지정하면, FastAPI가 반환 객체에서 그 모델에 있는 필드만 남기고 나머지는 **자동으로 걸러 냅니다**. 모델에 `cost`가 없으면, 반환 객체에 `cost`가 있어도 응답에는 나가지 않습니다.

이 장에서 만나는 키워드

- `response_model`: 응답의 출력 스키마를 지정하는 경로 동작 인자입니다. 반환값을 이 모델로 필터링·직렬화합니다.
- `status_code`: 응답의 HTTP 상태 코드(201·204 등)를 지정합니다.
- 입력/출력 모델 분리: 받는 모델(`ProductCreate`)과 내보내는 모델(`ProductRead`)을 나누는 설계입니다.
- `response_model_exclude_unset`: 설정되지 않은(기본값인) 필드를 응답에서 빼는 옵션입니다.

2. 정의: `response_model`과 `status_code`

이 절부터 다루는 개념은 **응답 모델·상태 코드·직렬화**입니다. **응답 모델(response model)**은 경로 동작이 내보낼 응답의 구조를 선언한 Pydantic 모델로, FastAPI가 반환 객체를 이 모델에 맞춰 **필터링·직렬화**하고 OpenAPI 문서에 응답 스키마로 기술합니다. **상태 코드(status code)**는 그 응답의 의미를 나타내는 HTTP 숫자 코드입니다.

지정 방법은 데코레이터의 인자입니다.

```
@app.post("/products", response_model=ProductRead, status_code=201)
async def create_product(product: ProductCreate):
    ...
```

- `response_model=ProductRead`: 반환값이 무엇이든 `ProductRead`의 필드만 남겨 응답합니다.
- `status_code=201`: 기본값 200 대신 201 Created를 돌려줍니다.

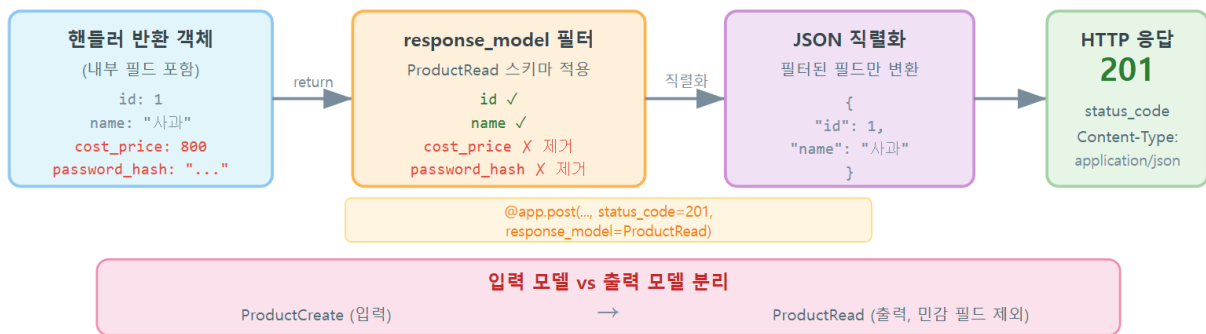
자주 쓰는 상태 코드는 다음과 같습니다.

코드	이름	언제 쓰나	응답 본문
200	OK	조회·일반 성공(GET, 갱신 결과 반환)	있음
201	Created	자원 생성 성공(POST로 새로 만들)	보통 생성된 자원
204	No Content	성공했고 돌려줄 본문이 없음(DELETE 등)	없음
404	Not Found	요청한 자원이 없음	오류 detail
409	Conflict	상태 충돌(중복·재고 부족 등)	오류 detail

숫자 대신 상수로 의도를 드러내기: `status_code=201` 도 되지만, FastAPI가 제공하는 상수 `status.HTTP_201_CREATED` 를 쓰면 코드의 의도가 분명해집니다. `from fastapi import status` 후 `status_code=status.HTTP_201_CREATED` 처럼 씁니다.

3. 동작 원리: 반환 객체에서 필터·직렬화·상태 코드까지

응답 처리: 반환 객체 → `response_model` 필터 → 직렬화 → 응답



핸들러가 객체를 반환한 뒤 응답이 만들어지는 과정을 단계로 따라가 보겠습니다.

단계	하는 일	결과
1. 반환	핸들러가 객체(또는 dict-ORM 객체)를 반환	내부·민감 필드 포함 가능
2. 필터	<code>response_model</code> 에 있는 필드만 추출, 나머지 제거	모델에 없는 필드 탈락
3. 검증	추출한 값을 응답 모델로 다시 검증	출력도 스키마 보장
4. 직렬화	모델을 JSON으로 변환	응답 본문
5. 상태	<code>status_code</code> 를 헤더에 실어 전송	201-204 등

핵심은 필터링이 "모델에 있는 것만 통과"하는 화이트리스트 방식이라는 점입니다. 응답 모델에 `cost`가 없으면, 반환 객체가 `cost`를 갖고 있어도 응답에는 절대 나가지 않습니다. 그래서 "빠야 할 것을 일일이 제거"하는 대신 "내보낼 것만 모델에 적는" 안전한 방향으로 설계가 뒤집힙니다.

또 한 가지, `response_model`은 단순 필터가 아니라 **응답의 계약(contract)**입니다. 반환 객체에 모델이 요구하는 필드가 없으면 응답 검증에서 오류가 나므로, "약속한 모양대로 응답이 나간다"가 보장됩니다. 동시에 그 모양이 `/docs`의 응답 스키마로 자동 기술됩니다.

이 "출력도 검증한다"는 성질은 실무에서 생각보다 큰 안전망입니다. 입력만 검증하고 출력은 손으로 만들면, 시간이 지나 내부 객체에 새 필드(예: 마진율·공급처)가 추가될 때 아무도 모르게 그 필드가 응답으로 새어 나갈 수 있습니다. 그러나 출력 모델을 두면, 모델에 명시하지 않은 새 필드는 **자동으로 차단**되므로 "추가했더니 실수로 노출"되는 사고가 구조적으로 막힙니다. 즉 응답 모델은 한 번 정해 두면 이후의 변경으로부터 응답을 지켜 주는 방어선 역할을 합니다. 입력 검증(c03)과 출력 통제(c04)를 한 쌍으로 다루는 이유가 여기에 있습니다 — 데이터가 **들어올 때와 나갈 때 모두** 스키마라는 관문을 통과하게 만드는 것입니다.

4. worked example: 201과 ProductRead로 내부 필드 숨기기

모범 예제 1 — 생성 응답에서 원가 숨기기

상품을 등록하면 201과 함께 생성된 상품을 돌려주되, 내부 원가(`cost`)는 숨기는 예제입니다.

```

from decimal import Decimal

from fastapi import FastAPI, status
from pydantic import BaseModel, Field

app = FastAPI()

# 입력: 클라이언트가 보내는 필드만
class ProductCreate(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    category: str
    price: Decimal = Field(gt=0)
    cost: Decimal = Field(gt=0)          # 내부 원가 — 등록 시 받지만 노출 금지

# 출력: 클라이언트에게 내보낼 필드만(cost 없음)
class ProductRead(BaseModel):
    id: int
    name: str
    category: str
    price: Decimal

_DB: dict[int, dict] = {}
_NEXT_ID = 1

@app.post(
    "/products",
    response_model=ProductRead,
    status_code=status.HTTP_201_CREATED,
)
async def create_product(product: ProductCreate):
    global _NEXT_ID
    record = {"id": _NEXT_ID, **product.model_dump()}
    _DB[_NEXT_ID] = record
    _NEXT_ID += 1
    # record에는 cost가 들어 있지만, response_model이 ProductRead라 응답에서 빠집니다.
    return record

```

- ProductCreate 에는 cost 가 있고 ProductRead 에는 없습니다. 즉 원가는 받기는 하되 내보내지 않는 필드입니다.
- response_model=ProductRead : 반환한 record 에 cost 가 들어 있어도, 출력 모델에 없으므로 응답에서 자동으로 걸러집니다.
- status_code=status.HTTP_201_CREATED : POST 생성이므로 200이 아니라 201을 돌려줍니다.
- product.model_dump() : Pydantic v2에서 모델을 딕셔너리로 바꾸는 메서드입니다(v1의 .dict() 아님).

{"name": "기계식 키보드", "category": "주변기기", "price": 89000, "cost": 41000} 을 보내면 실행 결과는 다음과 같습니다.

```
HTTP/1.1 201 Created
```

```
{
  "id": 1,
  "name": "기계식 키보드",
  "category": "주변기기",
  "price": "89000"
}
```

반환 객체에는 분명 `cost: 41000` 이 있었지만, 응답에는 **나타나지 않습니다**. 상태 코드도 201로 정확히 나갔습니다. 만약 `response_model` 을 빼고 `record` 를 그대로 반환했다면 원가가 그대로 노출되었을 것입니다.

5. 입력·출력 모델 분리(Create vs Read)

위 예제에서 본 것처럼, 한 자원에 대해 보통 **두 개 이상의 모델**을 둡니다. 이 분리가 실무 API 설계의 기본기입니다.

모델	용도	특징
<code>ProductCreate</code>	생성 요청 본문	클라이언트가 정할 수 있는 필드만(id·created_at 없음)
<code>ProductRead</code>	응답 출력	서버가 부여한 id 포함, 내부·민감 필드 제외
<code>ProductUpdate</code>	부분 갱신 본문	모든 필드가 선택(보낸 것만 갱신)

왜 굳이 나눌까요. 한 모델로 모든 것을 처리하려 하면 모순이 생깁니다.

- 생성 시점에는 `id` 가 아직 없는데, 응답에는 `id` 가 있어야 합니다.
- 클라이언트가 `id` 나 `created_at` 을 마음대로 정하게 두면 안 됩니다(서버가 부여해야 함).
- 내부 원가·비밀번호 해시는 입력으로 받더라도 출력에는 절대 나가면 안 됩니다.

이 모순을 모델 분리가 깔끔히 해결합니다. 각 모델은 자기 방향(입력/출력)에 필요한 필드만 가집니다.

```

class ProductBase(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    category: str
    price: Decimal = Field(gt=0)

class ProductCreate(ProductBase):
    cost: Decimal = Field(gt=0)      # 입력에만 있는 필드

class ProductRead(ProductBase):
    id: int                          # 출력에만 있는 필드

```

- `ProductBase`: 입력·출력이 공유하는 공통 필드를 모은 부모 모델입니다. 중복 선언을 줄입니다.
- `ProductCreate(ProductBase)`: 공통 필드에 입력 전용 `cost` 를 더합니다.
- `ProductRead(ProductBase)`: 공통 필드에 출력 전용 `id` 를 더합니다. `cost` 는 상속하지 않으므로 응답에 나가지 않습니다.
- 상속으로 공통부를 묶으면, 필드를 추가할 때 한 곳만 고치면 됩니다.

6. worked example: UserRead로 password_hash 제외

모범 예제 2 — 비밀번호 해시 숨기기

응답 모델 분리가 가장 빛나는 사례는 사용자(user)입니다. 내부 저장 객체에는 비밀번호 해시가 있지만, 어떤 응답에도 이것이 나가서는 안 됩니다.

```

from fastapi import FastAPI, status
from pydantic import BaseModel, EmailStr, Field

app = FastAPI()

# 입력: 가입 시 평문 비밀번호를 받음
class UserCreate(BaseModel):
    email: EmailStr
    password: str = Field(min_length=8)

# 출력: 해시도 평문 비밀번호도 절대 포함하지 않음
class UserRead(BaseModel):
    id: int
    email: EmailStr
    is_admin: bool

def fake_hash(password: str) -> str:
    # 실제로는 bcrypt/argon2를 씁니다(9장). 여기서는 형태만 보여 줍니다.
    return "hashed:" + password[::-1]

_USERS: dict[int, dict] = {}
_NEXT_ID = 1

@app.post(
    "/users",
    response_model=UserRead,
    status_code=status.HTTP_201_CREATED,
)
async def register_user(payload: UserCreate):
    global _NEXT_ID
    record = {
        "id": _NEXT_ID,
        "email": payload.email,
        "password_hash": fake_hash(payload.password), # 내부 저장
        "is_admin": False,
    }
    _USERS[_NEXT_ID] = record
    _NEXT_ID += 1
    return record # password_hash가 있지만 UserRead가 걸러 냅니다.

```

- UserCreate 는 평문 password 를 받지만, UserRead 에는 password 도 password_hash 도 없습니다. 두 모델이 완전히 다른 필드 집합을 가집니다.
- record 에는 password_hash 가 저장되지만, response_model=UserRead 가 응답에서 그것을 제거합니다.
- 만약 출력 모델 없이 record 를 그대로 반환했다면, 비밀번호 해시가 외부로 노출되는 보안 사고가 됩니다. 출력 모델 분리는 보안 장치입니다.

{"email": "user@shop.com", "password": "secret123"} 로 가입하면 실행 결과는 다음과 같습니다.

```
HTTP/1.1 201 Created
```

```
{
  "id": 1,
  "email": "user@shop.com",
  "is_admin": false
}
```

응답 어디에도 비밀번호나 해시가 없습니다. 내부 `record`에는 분명 `password_hash`가 있었지만 `UserRead`의 화이트리스트를 통과하지 못한 것입니다.

주의 — 출력 모델은 "내보내도 되는 것만" 담는다: 새 필드를 출력 모델에 추가할 때는 "이 필드가 외부에 나가도 안전한가"를 항상 자문하십시오. 출력 모델은 의도적으로 작게 유지하는 편이 안전합니다.

7. `exclude_unset·responses=`로 응답 가공·문서화

부분 갱신과 `response_model_exclude_unset`

PATCH로 일부 필드만 갱신할 때, 응답에 "실제로 설정된 필드만" 담고 싶을 수 있습니다. 이때 `response_model_exclude_unset=True`를 씁니다.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ProductRead(BaseModel):
    id: int
    name: str | None = None
    price: float | None = None
    stock: int | None = None

@app.get("/products/{product_id}", response_model=ProductRead,
        response_model_exclude_unset=True)
async def get_product(product_id: int):
    # name·stock만 채워 반환하면, 설정 안 된 price는 응답에서 빠집니다.
    return {"id": product_id, "name": "마우스", "stock": 12}
```

- `response_model_exclude_unset=True`: 반환 객체에서 실제로 값이 설정된 필드만 응답에 포함합니다. 기본값으로만 남은 필드(`price`)는 제외됩니다.

- 이 옵션이 없으면 `price` 가 기본값 `null` 로 응답에 나타납니다. 부분 응답을 표현할 때 유용합니다.

GET `/products/5` 의 실행 결과는 다음과 같습니다(설정하지 않은 `price` 가 빠짐).

```
HTTP/1.1 200 OK

{
  "id": 5,
  "name": "마우스",
  "stock": 12
}
```

204 No Content — 돌려줄 본문이 없을 때

삭제처럼 성공했지만 돌려줄 내용이 없을 때는 204를 씁니다. 204에는 **본문을 넣지 않으며**, `response_model` 도 지정하지 않습니다.

```
from fastapi import FastAPI, status

app = FastAPI()

@app.delete("/products/{product_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_product(product_id: int):
    # 삭제 처리만 하고 아무것도 반환하지 않습니다(None).
    return None
```

responses=로 여러 상태 문서화

엔드포인트가 성공 외에 404-409 같은 오류도 낼 수 있다면, `responses=` 로 그 가능성을 OpenAPI 문서에 명시할 수 있습니다.

```
@app.get(
    "/products/{product_id}",
    response_model=ProductRead,
    responses={404: {"description": "상품을 찾을 수 없음"}},
)
async def get_product(product_id: int):
    ...
```

`responses={404: {...}}` 는 실제 동작을 바꾸지 않고, `/docs` 에 "이 엔드포인트는 404도 낼 수 있다"는 설명을 추가합니다. 문서를 읽는 프론트엔드 개발자가 오류 케이스를 미리 알 수 있게 합니다.

from_attributes — ORM 객체를 그대로 직렬화

7장에서 DB(SQLModel/SQLAlchemy)를 붙이면, 핸들러는 `dict`가 아니라 **ORM 객체**를 반환하게 됩니다. 응답 모델이 ORM 객체의 속성(attribute)을 읽어 직렬화하려면 `from_attributes=True` 설정이 필요합니다(Pydantic v2).

```
from pydantic import BaseModel, ConfigDict

class ProductRead(BaseModel):
    model_config = ConfigDict(from_attributes=True)  # v1의 orm_mode 아님

    id: int
    name: str
    price: float
```

- `model_config = ConfigDict(from_attributes=True)`: 응답 모델이 딕셔너리뿐 아니라 **객체의 속성**에서도 값을 읽도록 허용합니다. ORM 객체를 그대로 반환할 때 필요합니다.
- Pydantic v1의 `class Config: orm_mode = True`가 v2에서 이 형태로 바뀌었습니다. 이름을 혼동하지 마십시오.

8. 즉시 연습 (2종 1세트)

직접 작성 1 — 주문 응답 모델 분리 (빈칸 채우기)

주문 생성 엔드포인트를 완성하십시오. 요구사항은 다음과 같습니다.

- 입력 `OrderCreate`: `customer_email`, `items` (품목 리스트)
- 출력 `OrderRead`: `id`, `customer_email`, `total` (서버가 계산), `status`
- 내부 결제 토큰(`payment_token`)은 저장하되 응답에는 절대 포함하지 않음
- 생성이므로 201을 반환


```

from fastapi import FastAPI, status
from pydantic import BaseModel

class OrderCreate(BaseModel):
    customer_email: str
    items: list[int]

class OrderRead(BaseModel):
    id: int
    customer_email: str
    total: int
    status: str
    # payment_token을 여기에 넣지 마십시오(노출 금지 필드)

app = FastAPI()

@app.post("/orders", response_model=____, status_code=____) # 출력 모델 + 201
async def create_order(order: OrderCreate):
    record = {
        "id": 1,
        "customer_email": order.customer_email,
        "total": len(order.items) * 10000,
        "status": "pending",
        "payment_token": "tok_secret_abc", # 내부 보관
    }
    return ____ # record 그대로 반환

```

자가체크 체크리스트

- [] `response_model=OrderRead` 로 출력 모델을 지정했는가
- [] `status_code=status.HTTP_201_CREATED` (또는 201)로 생성 상태를 돌려주는가
- [] `OrderRead` 에 `payment_token` 을 넣지 않아 응답에서 빠지게 했는가
- [] `record` 를 그대로 반환해도 토큰이 응답에 나타나지 않는지 `/docs` 에서 확인했는가

직접 작성 2 — 삭제와 204 (빈칸 채우기)

상품 삭제 엔드포인트를 완성하십시오. 성공 시 돌려줄 본문이 없으므로 204를 씁니다.

```

from fastapi import FastAPI, status

app = FastAPI()

@app.delete("/products/{product_id}", status_code=____) # 본문 없는 성공 코드
async def delete_product(product_id: int):
    # 삭제 처리 가정
    return ____ # 본문 없음

```

자가체크 체크리스트

- [] `status_code=status.HTTP_204_NO_CONTENT` (또는 204)를 지정했는가
- [] 204에는 `response_model` 을 지정하지 않았는가
- [] 반환값으로 `None` 을 주어 본문을 비웠는가
- [] 응답 상태가 204이고 본문이 비어 있는지 확인했는가

9. 혼한 오류와 오개념

오개념 1 — "response_model을 안 써도 필요한 것만 반환하면 된다"

`response_model` 없이 내부 객체를 그대로 반환하면, 모델이 걸러 주지 않으므로 **모든 필드가 그대로 나갑니다**. 원가·비밀번호 해시 같은 민감 필드가 노출됩니다.

```
@app.post("/users")
async def register(payload: UserCreate):
    record = {"id": 1, "email": payload.email,
              "password_hash": "hashed:..."} # 내부 저장
    return record # response_model이 없으면 password_hash가 그대로 노출!
```

교정: 출력에 내보낼 필드만 가진 `UserRead` 를 만들고 `response_model=UserRead` 를 지정하십시오. 그러면 모델에 없는 필드는 화이트리스트 방식으로 자동 제거됩니다.

오개념 2 — "POST면 상태 코드가 알아서 201이 된다"

`status_code` 를 지정하지 않으면 POST도 **기본 200**입니다. 201은 명시해야 합니다. 자원 생성에 200을 돌려주면 의미가 흐려집니다.

```
@app.post("/products") # 상태 코드 미지정 → 기본 200
async def create_product(...):
    ...
```

교정: 생성 엔드포인트에는 `status_code=status.HTTP_201_CREATED` 를, 본문 없는 성공(삭제 등)에는 `status_code=status.HTTP_204_NO_CONTENT` 를 명시합니다.

오개념 3 — "ORM 객체를 반환하면 자동으로 직렬화된다"

DB ORM 객체는 딕셔너리가 아니라 속성을 가진 객체입니다. 응답 모델에 `from_attributes=True` 가 없으면 직렬화에서 오류가 나거나 의도대로 동작하지 않습니다.

```
class ProductRead(BaseModel):
    id: int
    name: str
    # from_attributes 설정이 없으면 ORM 객체 반환 시 문제가 생길 수 있습니다.
```

교정: Pydantic v2에서는 응답 모델에 `model_config = ConfigDict(from_attributes=True)` 를 추가합니다. v1의 `class Config: orm_mode = True` 가 아니라 이 형태입니다.

주의 — `response_model_exclude_unset` 은 "설정 여부"로 판단합니다. 이 옵션은 값이 기본값과 같은지가 아니라, 명시적으로 설정되었는지로 제외를 판단합니다. 부분 응답을 만들 때 의도와 다르게 빠지는 필드가 없는지 확인하십시오.

10. 단원 미니 프로젝트: 안전한 응답 모델 API (2종 1세트)

이 단원에서 배운 `response_model · status_code · 입출력 모델 분리 · 민감 필드 제외` 를 모아 **안전한 응답 모델 API**를 완성합니다. 마일스톤을 증분으로 나눕니다.

- **M1:** `ProductRead` 출력 모델을 만들고 `response_model` 로 응답을 필터링합니다.
- **M2:** 생성에 `status_code=201` 을 적용합니다.
- **M3:** `UserRead` 로 `password_hash` 를 제외하고, 조회에 `responses={404: ...}` 를 문서화합니다.

모범 산출물

```

from decimal import Decimal

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, ConfigDict, EmailStr, Field

app = FastAPI(title="안전한 응답 모델 API")

# --- 상품 입출력 모델 ---
class ProductBase(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    category: str
    price: Decimal = Field(gt=0)

class ProductCreate(ProductBase):
    cost: Decimal = Field(gt=0) # 입력 전용(원가)

class ProductRead(ProductBase):
    model_config = ConfigDict(from_attributes=True)
    id: int # 출력 전용(cost 없음)

# --- 사용자 입출력 모델 ---
class UserCreate(BaseModel):
    email: EmailStr
    password: str = Field(min_length=8)

class UserRead(BaseModel):
    id: int
    email: EmailStr
    is_admin: bool # password_hash 없음

_PRODUCTS: dict[int, dict] = {}
_NEXT_PID = 1

@app.post("/products", response_model=ProductRead,
          status_code=status.HTTP_201_CREATED)
async def create_product(product: ProductCreate):
    global _NEXT_PID
    record = {"id": _NEXT_PID, **product.model_dump()}
    _PRODUCTS[_NEXT_PID] = record
    _NEXT_PID += 1
    return record # cost는 ProductRead가 걸러냄

@app.get("/products/{product_id}", response_model=ProductRead,
         responses={404: {"description": "상품을 찾을 수 없음"}})
async def get_product(product_id: int):
    record = _PRODUCTS.get(product_id)
    if record is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "상품을 찾을 수 없음")
    return record

```

```
@app.post("/users", response_model=UserRead,
          status_code=status.HTTP_201_CREATED)
async def register_user(payload: UserCreate):
    record = {
        "id": 1,
        "email": payload.email,
        "password_hash": "hashed:" + payload.password[:-1],
        "is_admin": False,
    }
    return record  # password_hash는 UserRead가 걸러 냄
```

상품을 등록하면 원가가 빠진 201 응답이 나옵니다.

```
POST /products
{"name": "마우스", "category": "주변기기", "price": 25000, "cost": 9000}

HTTP/1.1 201 Created
{
  "id": 1,
  "name": "마우스",
  "category": "주변기기",
  "price": "25000"
}
```

학생 작성형 빈 스타터

직접 작성 — 안전한 응답 모델 API 완성하기

아래 빈칸을 채워 미니 프로젝트를 완성하십시오. 모범 산출물을 베끼지 말고 자가체크를 통과하도록 직접 채워 보십시오.

```

from decimal import Decimal

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, ConfigDict, EmailStr, Field

app = FastAPI(title="안전한 응답 모델 API")

class ProductBase(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    category: str
    price: Decimal = Field(gt=0)

class ProductCreate(ProductBase):
    cost: Decimal = Field(gt=0)

class ProductRead(ProductBase):
    model_config = ConfigDict(____)      # ORM 객체 직렬화 허용(v2)
    id: int

class UserCreate(BaseModel):
    email: EmailStr
    password: str = Field(min_length=8)

class UserRead(BaseModel):
    id: int
    email: EmailStr
    is_admin: bool
    # password_hash를 넣지 마십시오

@app.post("/products", response_model=____, status_code=____) # 출력 모델 + 201
async def create_product(product: ProductCreate):
    record = {"id": 1, **product.model_dump()}
    return record

# TODO: GET /products/{id}에 responses={404: ...}와 404 처리를 추가하고,
#       POST /users에 response_model=UserRead·201을 적용하십시오.

```

자가체크 체크리스트

- [] `ProductRead.model_config`에 `from_attributes=True`를 넣었는가
- [] `POST /products`에 `response_model=ProductRead`와 201을 지정했는가
- [] 상품 응답에 `cost`가 빠지는지 확인했는가
- [] `UserRead`에 `password_hash`를 넣지 않아 사용자 응답에서 빠지는가
- [] 없는 상품 조회에 404가, `/docs`에 404 응답이 문서화되는가
- [] 삭제 엔드포인트를 추가한다면 204·본문 없음으로 처리할 수 있는가

11. 자주 묻는 질문(FAQ)

Q. `response_model` 과 함수의 반환 타입 힌트 (`-> ProductRead`)는 무엇이 다른가요? A. 최신 FastAPI에서는 `-> ProductRead` 반환 타입 힌트만으로도 응답 모델로 인식됩니다. `response_model=` 은 더 명시적이고, 반환 타입과 다른 출력 스키마를 쓰고 싶을 때(또는 `response_model_exclude_unset` 등 옵션을 줄 때) 유용합니다. 둘 다 지정하면 `response_model=` 이 우선합니다.

Q. 목록 응답은 어떻게 모델을 지정하나요? A. `response_model=list[ProductRead]` 처럼 리스트로 감쌉니다. 그러면 리스트 안의 각 원소가 `ProductRead` 로 필터링·직렬화됩니다.

Q. 201을 썼는데 본문도 같이 나가는데 정상인가요? A. 정상입니다. 201 Created는 보통 생성된 자원을 본문에 담아 돌려줍니다. 본문이 없는 성공은 204를 씁니다.

Q. 응답에서 `Decimal` 이 문자열 ("`89000`")로 나옵니다. A. JSON에는 `Decimal` 타입이 없어 FastAPI가 문자열로 직렬화합니다. 정밀도를 지키기 위한 동작이며, 클라이언트는 이를 숫자로 파싱해 쓰면 됩니다.

Q. `from_attributes` 는 항상 켜 두면 안 되나요? A. ORM 객체를 직렬화하는 출력 모델에만 켜면 됩니다. 순수 입력 모델(`Create`)에는 보통 필요 없습니다. 필요한 모델에만 명시적으로 두는 편이 의도가 분명합니다.

이 장에서는 `response_model` 로 출력을 필터링하고, `status_code` 로 정확한 상태를 돌려주며, 입력·출력 모델을 분리해 민감 필드를 숨기는 법을 익혔습니다. 여기까지로 "들어오는 검증(c03)"과 "나가는 통제(c04)"의 한 쌍이 완성되었습니다. 다음 단원에서는 `async / await` 로 외부 호출을 기다리는 동안에도 서버가 다른 요청을 처리하게 하는 비동기 동시성을 배웁니다.

async/await: 비동기 동시성

이 챕터에서 배우는 것

- 동기·비동기와 이벤트 루프(event loop)의 동작을 설명할 수 있습니다.
- `async def` 경로 동작에서 `await` 로 비동기 I/O를 호출할 수 있습니다.
- `httpx.AsyncClient` 로 외부 API를 비동기로 호출하고, `asyncio.gather` 로 여러 호출을 동시에 실행할 수 있습니다.
- 블로킹(blocking) 코드가 이벤트 루프를 막는 문제를 진단·회피할 수 있습니다.
- `async def` 와 일반 `def` 핸들러를 상황에 맞게 선택할 수 있습니다.

이번 챕터에서 가르치는 개념은 **async/await 비동기 처리(async and await)** 한 가지입니다. 선수 개념은 ch_02의 **경로·쿼리 파라미터**입니다. 그때 만든 핸들러는 모두 값을 곧바로 계산해 돌려주는 함수였습니다. 그런데 실무 백엔드는 그렇게 단순하지 않습니다. 결제 게이트웨이, 배송 추정 서비스, 창고 재고 API처럼 **네트워크 너머의 느린 외부 호출**이 끊임없이 끼어듭니다. 이번 챕터에서는 그 "기다림"을 낭비하지 않고 동시에 여러 일을 처리하는 비동기 모델을 배웁니다. 종합 프로젝트(ch_15 M4)의 결제·배송 호출이 바로 이 개념 위에서 동작합니다.

5.1 도입: I/O를 기다리는 동안 다른 일 하기

이커머스 주문 API를 생각해 봅시다. 사용자가 주문을 넣으면 서버는 외부 결제 게이트웨이에 결제를 요청하고, 배송 추정 서비스에 도착 예정일을 묻고, 창고 시스템에 재고를 확인합니다. 이 세 호출은 각각 200ms씩 걸린다고 해 봅시다. 코드를 순서대로 늘어놓으면 한 요청을 처리하는 데 600ms가 걸립니다.

여기서 중요한 사실이 있습니다. 그 600ms 동안 **CPU는 거의 놀고 있습니다**. 서버는 그저 외부 서버의 응답이 네트워크를 타고 돌아오기를 기다릴 뿐입니다. 이 "기다리는 시간"이 바로 **I/O 대기**입니다. 데이터베이스 조회, 파일 읽기, HTTP 호출은 모두 I/O 대기를 동반합니다.

전통적인 동기(synchronous) 방식에서는 한 요청이 외부 응답을 기다리는 동안 그 작업 단위(스레드)가 통째로 멈춰 있습니다. 동시 접속자가 100명이면 스레드 100개가 각자 멍하니 기다려야 하고, 스레드는 메모리와 전환 비용이 비쌉니다. 비동기(asynchronous) 방식은 이 문제를 뒤집습니다. **하나의 작업이 외부 응답을 기다리기 시작하는 순간, 그 대기 시간을 다른 요청을 처리하는 데 양보합니다**. 기다림이 곧 다른 일을 할 기회가 되는 것입니다.

한눈에 보는 `async/await`

`async def` 로 선언한 함수(코루틴)는 `await` 지점에서 실행을 잠시 멈추고 제어권을 **이벤트 루프**에 돌려줍니다. 이벤트 루프는 그 틈에 대기 중이던 다른 코루틴을 실행합니다. 외부 응답이 도착하면 멈췄던 코루틴이 그 자리에서 이어 실행됩니다. 이렇게 단일 스레드 안에서 여러 작업을 번갈아(인터리브) 진행해, I/O 대기가 많은 서버의 처리량을 크게 높입니다. 핵심은 "병렬로 CPU를 더 쓰는 것"이 아니라 "노는 대기 시간을 재활용하는 것"입니다.

FastAPI가 다른 프레임워크와 차별화되는 큰 이유 중 하나가 바로 이 비동기 지원입니다. FastAPI는 Starlette(ASGI) 위에 서 있고, ASGI는 애초에 비동기 처리를 전제로 설계된 규격입니다. 그래서 핸들러를 `async def` 로 선언하기만 하면 별도 설정 없이 비동기 모델을 그대로 활용할 수 있습니다.

5.2 정의: 이벤트 루프와 `async/await`

먼저 핵심 용어를 한 문장씩 정의하겠습니다.

- **이벤트 루프(event loop)**: 비동기 작업들을 관리하며, 실행 가능한 코루틴을 골라 돌리고 `await` 로 멈춘 코루틴을 대기열에 보관했다가 준비되면 다시 깨우는 단일 스레드 스케줄러입니다.
- **코루틴(coroutine)**: `async def` 로 정의한 함수를 호출하면 만들어지는 "일시 정지·재개가 가능한 함수 실행"입니다. 일반 함수처럼 호출한다고 바로 실행되지 않고, `await` 되거나 이벤트 루프에 등록되어야 진행됩니다.
- **`await`**: 코루틴이나 awaitable(기다릴 수 있는 객체)의 완료를 기다리는 표현식입니다. `await` 지점에서 현재 코루틴은 멈추고 이벤트 루프에 제어권을 넘깁니다.

여기서 가장 자주 오해하는 지점을 짚겠습니다. `await` 는 "기다리느라 막는다"는 뜻이 **아닙니다**. 오히려 정반대로 "내가 기다리는 동안 너(이벤트 루프)는 다른 일을 해도 좋다"고 **양보**하는 지점입니다. 동기 함수가 외부 응답을 기다릴 때는 스레드 전체가 멈추지만, `await` 로 기다릴 때는 이벤트 루프가 자유로워집니다.

구분	동기(<code>def</code> 로 직접 블로킹 호출)	비동기(<code>async def</code> + <code>await</code>)
대기 동안	작업 단위(스레드)가 통째로 멈춤	이벤트 루프가 다른 코루틴 실행
동시성 모델	스레드 여러 개	단일 스레드 + 협력적 양보
비용	스레드 메모리·전환 비용 큼	코루틴 전환 비용 작음
적합한 작업	CPU 계산, 블로킹 라이브러리만 있을 때	네트워크·DB 등 I/O 대기가 많을 때

FastAPI에서 경로 동작 함수를 선언하는 방법은 두 가지입니다.

```
@app.get("/a")
async def handler_a():
    # 이벤트 루프에서 실행됨. 내부에서 await로 비동기 I/O를 호출.
    ...

@app.get("/b")
def handler_b():
    # FastAPI가 별도 스레드풀에서 실행함. 블로킹 코드를 써도 루프를 막지 않음.
    ...
```

두 방식은 우열이 아니라 **상황에 따른 선택**입니다. 5.6에서 그 판단 기준을 자세히 다룹니다. 우선은 "비동기 라이브러리로 `await` 할 수 있으면 `async def`, 어쩔 수 없이 블로킹 라이브러리만 있으면 `def`"라고 기억해 두십시오.

5.3 동작 원리: 동기 직렬 vs 비동기 인터리브

동기 직렬 처리 vs 비동기 인터리브 처리



세 개의 외부 호출(결제·배송·재고)이 각각 200ms 걸린다고 할 때, 동기 직렬 처리와 비동기 인터리브 처리가 어떻게 다른지 단계로 따라가 보겠습니다.

동기 직렬 처리라면 한 줄씩 순서대로 진행합니다.

1. 결제 요청을 보내고 응답을 기다립니다. (0ms ~ 200ms, 스레드 멈춤)
2. 응답이 오면 배송 요청을 보내고 기다립니다. (200ms ~ 400ms, 스레드 멈춤)
3. 응답이 오면 재고 요청을 보내고 기다립니다. (400ms ~ 600ms, 스레드 멈춤)
4. 합계 약 600ms 소요.

세 작업이 서로 결과를 기다릴 필요가 없는데도, 코드를 순서대로 적었다는 이유만으로 600ms가 걸립니다.

비동기 인터리브 처리(특히 `asyncio.gather` 로 동시에 시작)라면 다릅니다.

1. 결제·배송·재고 세 요청을 거의 동시에 모두 내보냅니다. (0ms 부근)
2. 세 응답을 한꺼번에 기다립니다. 이벤트 루프는 이 대기 시간에 다른 요청들도 처리합니다.
3. 가장 늦은 응답이 도착하면 끝납니다. (약 200ms)
4. 합계 약 200ms 소요. 600ms가 200ms로 줄어듭니다.

여기서 비동기가 처리량을 높이는 두 가지 효과가 동시에 나타납니다. 첫째, **한 요청 내부**에서 독립적인 외부 호출을 동시에 진행해 응답 시간을 줄입니다(600ms → 200ms). 둘째, **여러 요청 사이**에서 한 요청이 `await` 로 대기하는 동안 이벤트 루프가 다른 요청을 처리해 전체 처리량을 높입니다.

비동기는 마법이 아닙니다 — 조건이 있습니다

비동기의 이점은 "기다리는 동안 다른 일을 할 수 있을 때"만 발생합니다. 즉 코드가 `await` 지점에서 제대로 제어권을 양보해야 합니다. 만약 `await` 없이 CPU만 200ms 동안 빙빙 도는 계산(예: 큰 수의 소인수분해)을 한다면, 이벤트 루프는 그동안 다른 일을 전혀 하지 못합니다. 비동기는 **I/O 대기**가 많은 작업에 효과적이지, CPU 계산을 빠르게 해 주지는 않습니다.

5.4 worked example: `httpx.AsyncClient`로 배송 추정 `await`

이제 실제로 외부 배송 추정 API를 비동기로 호출해 응답에 합쳐 보겠습니다. HTTP 호출에는 비동기 클라이언트인 `httpx.AsyncClient` 를 씁니다. `requests` 라이브러리는 동기(블로킹)라 `async def` 안에서 쓰면 이벤트 루프를 막으므로 적합하지 않습니다.

아래 예제는 상품 상세를 조회하면서 외부 배송 서비스에 도착 예정일을 물어 함께 돌려주는 핸들러입니다.

```

import httpx
from fastapi import FastAPI, HTTPException

app = FastAPI()

# 외부 배송 추정 서비스의 기준 URL (실제로는 설정에서 주입)
SHIPPING_API = "https://shipping.example.com"

async def estimate_delivery(product_id: int) -> str:
    """외부 배송 서비스에 도착 예정일을 비동기로 물어본다."""
    async with httpx.AsyncClient(timeout=5.0) as client:
        resp = await client.get(f"{SHIPPING_API}/eta/{product_id}")
        resp.raise_for_status()
        data = resp.json()
        return data["eta"] # 예: "2026-06-25"

@app.get("/products/{product_id}/detail")
async def product_detail(product_id: int):
    """상품 상세에 외부 배송 추정 결과를 합쳐서 반환한다."""
    try:
        eta = await estimate_delivery(product_id)
    except httpx.HTTPError:
        # 외부 서비스 실패는 502(Bad Gateway)로 변환
        raise HTTPException(status_code=502, detail="배송 추정 서비스 호출 실패")
    return {"product_id": product_id, "name": "무선 키보드", "delivery_eta": eta}

```

```

GET /products/42/detail
200 OK
{
  "product_id": 42,
  "name": "무선 키보드",
  "delivery_eta": "2026-06-25"
}

```

- `async def estimate_delivery(...)` 는 코루틴 함수입니다. 내부에서 `await` 로 비동기 I/O를 호출하므로 반드시 `async def` 로 선언했습니다.
- `async with httpx.AsyncClient(...) as client:` 는 비동기 컨텍스트 매니저입니다. `with` 블록을 벗어날 때 커넥션 풀을 정리하며, 이 정리 자체도 비동기로 처리됩니다. `timeout=5.0` 으로 외부 서비스가 응답하지 않을 때 무한정 매달리지 않도록 했습니다.
- `await client.get(...)` 이 핵심입니다. 네트워크 응답을 기다리는 이 지점에서 코루틴은 멈추고 이벤트 루프에 제어권을 넘깁니다. 그래서 이 요청이 배송 응답을 기다리는 동안 서버는 다른 요청을 처리할 수 있습니다.
- `resp.raise_for_status()` 는 4xx-5xx 응답을 예외로 바꿉니다. 외부 호출 실패를 잡아 `HTTPException(502)` 로 변환해, 우리 API가 외부 장애를 일관된 형태로 사용자에게 알리게 했습니다.

- 경로 동작 `product_detail` 도 `async def` 이며 `await estimate_delivery(...)` 로 코루틴을 기다립니다. 코루틴은 호출만으로는 실행되지 않고 `await` 되어야 진행된다는 점이 일반 함수와 다른 결정적 차이입니다.

여기서 한 가지를 강조하겠습니다. `estimate_delivery(product_id)` 라고만 쓰고 `await` 를 빠뜨리면, 함수가 실행되는 게 아니라 코루틴 객체만 만들어지고 끝납니다. 실제 호출은 일어나지 않고 `eta` 에는 코루틴 객체가 들어가 버립니다. `await` 를 빠뜨리는 실수는 5.9에서 다시 다룹니다.

5.5 asyncio.gather로 여러 외부 호출 동시 실행

앞 예제는 외부 호출이 하나뿐이었습니다. 그런데 주문 화면에서는 여러 상품의 재고를 한꺼번에 확인해야 할 때가 많습니다. 상품이 3개라면 외부 창고 API를 3번 호출해야 합니다.

이때 `await` 를 순서대로 세 번 쓰면 어떻게 될까요? 첫 호출을 끝까지 기다린 뒤 두 번째를 시작하므로, 5.3에서 본 "동기 직렬"과 똑같이 $200\text{ms} \times 3 = 600\text{ms}$ 가 걸립니다. 비록 `async def` 안이라도, `await` 를 직렬로 늘어놓으면 동시 실행이 아닙니다.

여러 코루틴을 진짜로 동시에 진행시키려면 `asyncio.gather` 를 씁니다. `gather` 는 여러 awaitable을 한꺼번에 이벤트 루프에 등록하고, 모두 끝날 때까지 기다린 뒤 결과를 리스트로 돌려줍니다.

```
import asyncio
import httpx
from fastapi import FastAPI

app = FastAPI()
WAREHOUSE_API = "https://warehouse.example.com"

async def fetch_stock(client: httpx.AsyncClient, product_id: int) -> dict:
    """외부 창고 API에서 한 상품의 재고를 비동기로 조회한다."""
    resp = await client.get(f"{WAREHOUSE_API}/stock/{product_id}")
    resp.raise_for_status()
    return {"product_id": product_id, "stock": resp.json()["stock"]}

@app.get("/cart/stock")
async def cart_stock(product_ids: list[int]):
    """장바구니 여러 상품의 재고를 동시에 조회한다."""
    async with httpx.AsyncClient(timeout=5.0) as client:
        # 각 상품에 대한 코루틴을 만들어 리스트로 모은다(아직 실행 전).
        tasks = [fetch_stock(client, pid) for pid in product_ids]
        # gather로 한꺼번에 등록·동시 실행하고 모두 끝나길 기다린다.
        results = await asyncio.gather(*tasks)
    return {"items": results}
```

```
GET /cart/stock?product_ids=42&product_ids=43&product_ids=44
200 OK
{
  "items": [
    {"product_id": 42, "stock": 17},
    {"product_id": 43, "stock": 0},
    {"product_id": 44, "stock": 5}
  ]
}
```

- `tasks = [fetch_stock(client, pid) for pid in product_ids]` 는 코루틴 객체들의 리스트입니다. 이 줄에서는 아직 어떤 호출도 일어나지 않습니다(코루틴은 `await` 되어야 실행됨).
- `await asyncio.gather(*tasks)` 가 세 코루틴을 동시에 이벤트 루프에 올립니다. 세 HTTP 요청이 거의 동시에 나가고, 셋 다 응답을 기다리는 동안 이벤트 루프는 자유롭게 돌아갑니다. 그래서 전체 소요 시간은 합(600ms)이 아니라 가장 느린 하나(약 200ms)에 가깝습니다.
- `gather` 는 입력 순서대로 결과를 돌려줍니다. 즉 `results[0]` 은 항상 `product_ids[0]` 의 결과입니다. 동시에 실행했어도 결과 순서는 보장됩니다.
- 하나의 `client` 를 모든 호출이 공유합니다. `AsyncClient` 는 커넥션 풀을 재사용하므로, 매 호출마다 새 클라이언트를 만드는 것보다 효율적입니다.
- 쿼리 파라미터 `product_ids: list[int]` 는 ch_02에서 배운 리스트 쿼리 파라미터입니다. `?product_ids=42&product_ids=43` 처럼 같은 키를 여러 번 보내면 리스트로 모입니다.

gather와 예외 처리

`asyncio.gather` 는 기본적으로 하나의 코루틴에서 예외가 나면 전체가 그 예외를 던집니다. 일부가 실패해도 나머지 결과는 받고 싶다면 `asyncio.gather(*tasks, return_exceptions=True)` 를 쓰십시오. 이 경우 실패한 자리에는 예외 객체가 결과로 들어오므로, 결과를 순회하며 `isinstance(r, Exception)` 로 걸러 처리하면 됩니다. 외부 서비스 일부 장애가 전체 주문을 막지 않게 할 때 유용합니다.

5.6 일반 def 핸들러와 스레드풀·블로킹 회피

지금까지는 `async def` 만 봤지만, FastAPI는 일반 `def` 핸들러도 완전히 지원합니다. 그리고 이 둘이 실행되는 방식은 근본적으로 다릅니다.

- **async def 핸들러**: 이벤트 루프에서 직접 실행됩니다. 그래서 내부에서 블로킹 호출을 하면 이 이벤트 루프 전체가 멈추고, 그 순간 서버가 처리하던 모든 요청이 함께 멈춥니다.
- **일반 def 핸들러**: FastAPI가 자동으로 별도의 **스레드풀(threadpool)** 에서 실행합니다. 그래서 내부에서 블로킹 호출을 해도 이벤트 루프는 막히지 않고 다른 요청을 계속 처리합니다.

이것이 왜 중요한지 봅시다. 어떤 라이브러리는 동기(블로킹) 버전만 제공합니다. 예를 들어 일부 결제 SDK, 이미지 처리 라이브러리, 또는 오래된 DB 드라이버가 그렇습니다. 이런 블로킹 코드를 `async def` 안에 넣으면 재앙입니다.

```
import time
from fastapi import FastAPI

app = FastAPI()

# 나쁜 예: 블로킹 호출을 async def에 넣음
@app.get("/bad")
async def bad_handler():
    time.sleep(3) # 블로킹! 3초 동안 이벤트 루프 전체가 멈춘다.
    return {"done": True}

# 좋은 예: 어쩔 수 없이 블로킹 라이브러리만 있다면 def로 선언
@app.get("/good")
def good_handler():
    time.sleep(3) # def 핸들러는 스레드풀에서 돌아 루프를 막지 않는다.
    return {"done": True}
```

/bad 를 호출하는 동안 다른 모든 요청이 최대 3초까지 멈춘다.
/good 을 호출하면 그 요청만 3초 걸리고, 다른 요청은 정상 처리된다.

- `bad_handler` 는 `async def` 인데 그 안에서 `time.sleep(3)` 이라는 동기 블로킹 호출을 합니다. `time.sleep` 은 `await` 로 양보하지 않고 스레드를 통째로 3초 멈춥니다. 이 핸들러가 이벤트 루프에서 실행되므로, 그 3초 동안 다른 요청들도 처리되지 못합니다.
- `good_handler` 는 똑같이 `time.sleep(3)` 을 하지만 `def` 로 선언했습니다. FastAPI가 이 함수를 스레드풀에서 실행하므로, 멈추는 것은 그 스레드 하나뿐이고 이벤트 루프는 계속 다른 요청을 처리합니다.
- 판단 규칙: 비동기 라이브러리로 `await` 할 수 있으면 `async def`, 비동기 버전이 없어 블로킹 호출이 불가피하면 `def` 를 선택합니다. 절대로 `async def` 안에서 블로킹 호출을 직접 하지 않습니다.

비동기 이벤트 루프에서 어쩔 수 없이 블로킹 함수를 호출해야 한다면, `def` 핸들러로 빼는 것 외에 `await anyio.to_thread.run_sync(블로킹함수, 인자)` 로 명시적으로 스레드에 떠넘기는 방법도 있습니다. 다만 입문 단계에서는 "블로킹만 있으면 `def` 핸들러"라는 기본 규칙으로 충분합니다.

언제 `async`, 언제 `def` 인가 — 결정 기준

- 외부 호출·DB를 비동기 라이브러리(`httpx.AsyncClient`, 비동기 DB 드라이버)로 할 수 있다 → `async def` + `await`
- 사용할 라이브러리가 블로킹 버전만 있다(동기 SDK·드라이버) → 일반 `def` (스레드풀)

- 외부 I/O 없이 순수 계산·간단한 응답뿐이다 → 둘 다 무방하나 `def` 도 충분
- 절대 금지: `async def` 안에서 `time.sleep`, 동기 `requests.get`, 블로킹 DB 호출 직접 사용

5.7 worked example: 블로킹 vs 비동기 응답시간 비교

이번에는 "직렬로 `await`" vs "gather로 동시 `await`"의 차이를 실측 코드로 비교해 보겠습니다. 외부 호출을 흉내 내기 위해 `await asyncio.sleep(0.2)` (비동기 대기)를 한 번 호출하는 가짜 작업을 만들고, 이를 3번 수행하는 두 방식을 측정합니다.

`asyncio.sleep` 은 `time.sleep` 과 달리 **비동기 대기**입니다. `await asyncio.sleep(0.2)` 는 0.2초 동안 이벤트 루프에 제어권을 양보하므로, 그 사이 다른 코루틴이 실행될 수 있습니다.

```

import asyncio
import time
from fastapi import FastAPI

app = FastAPI()

async def fake_call(label: str) -> str:
    """외부 호출 0.2초를 흉내 내는 비동기 작업."""
    await asyncio.sleep(0.2) # 비동기 대기: 여기서 이벤트 루프에 양보
    return f"{label} 완료"

@app.get("/compare")
async def compare():
    """직렬 await와 gather 동시 실행의 소요 시간을 비교한다."""
    # 1) 직렬: await를 하나씩 순서대로 -> 약 0.6초
    t0 = time.perf_counter()
    serial = [
        await fake_call("결제"),
        await fake_call("배송"),
        await fake_call("재고"),
    ]
    serial_sec = round(time.perf_counter() - t0, 2)

    # 2) 동시: gather로 한꺼번에 -> 약 0.2초
    t0 = time.perf_counter()
    concurrent = await asyncio.gather(
        fake_call("결제"),
        fake_call("배송"),
        fake_call("재고"),
    )
    concurrent_sec = round(time.perf_counter() - t0, 2)

    return {
        "serial": {"result": serial, "seconds": serial_sec},
        "concurrent": {"result": concurrent, "seconds": concurrent_sec},
    }

```

```

GET /compare
200 OK
{
  "serial": {"result": ["결제 완료", "배송 완료", "재고 완료"], "seconds": 0.6},
  "concurrent": {"result": ["결제 완료", "배송 완료", "재고 완료"], "seconds": 0.2}
}

```

- `fake_call` 은 `await asyncio.sleep(0.2)` 로 외부 호출의 대기 시간을 흉내 냅니다. `asyncio.sleep` 은 비동기이므로 이 0.2초 동안 다른 코루틴이 끼어들 수 있습니다.
- 직렬 방식은 `await fake_call(...)` 을 세 번 차례로 적었습니다. 첫 호출이 0.2초를 다 채운 뒤에야 다음 호출이 시작되므로 총 0.6초가 걸립니다. `async def` 안이라도 직렬 `await` 는 동시 실행이 아니라는 점이 드러납니다.

- 동시 방식은 세 코루틴을 `asyncio.gather` 로 한꺼번에 등록합니다. 셋이 동시에 0.2초를 대기하므로 총 시간은 약 0.2초입니다. 결과는 동일하지만 시간은 3분의 1입니다.
- `time.perf_counter()` 는 경과 시간 측정용 고해상도 타이머입니다. 여기서는 짧은 동기 호출이라 이벤트 루프를 의미 있게 막지 않으므로 측정에 사용했습니다.

이 한 화면이 비동기 동시성의 핵심을 압축합니다. 독립적인 I/O 작업은 직렬 `await` 가 아니라 `gather` 로 묶어야 한다는 것입니다. 종합 프로젝트의 주문 처리에서 결제·배송·재고 확인이 서로 독립적이라면, 바로 이 패턴으로 응답 시간을 줄일 수 있습니다.

5.8 즉시 연습(2종 1세트): gather 동시 호출 채우기

이제 직접 작성할 차례입니다. 모범 답안을 먼저 확인한 뒤, 빈칸 코드를 덮어 두고 도전해 보십시오. 여러 주문의 결제 상태를 외부 결제 게이트웨이에서 동시에 조회하는 핸들러를 완성하는 과제입니다.

모범 답안

```
import asyncio
import httpx
from fastapi import FastAPI

app = FastAPI()
PAYMENT_API = "https://pay.example.com"

async def fetch_payment(client: httpx.AsyncClient, order_id: int) -> dict:
    """한 주문의 결제 상태를 비동기로 조회한다."""
    resp = await client.get(f"{PAYMENT_API}/status/{order_id}")
    resp.raise_for_status()
    return {"order_id": order_id, "status": resp.json()["status"]}

@app.get("/orders/payments")
async def order_payments(order_ids: list[int]):
    async with httpx.AsyncClient(timeout=5.0) as client:
        tasks = [fetch_payment(client, oid) for oid in order_ids]
        results = await asyncio.gather(*tasks)
    return {"payments": results}
```

직접 작성: 동시 결제 상태 조회 (빈칸 코드)

아래 빈칸(____)을 채워 동시 조회 핸들러를 완성해 보십시오. 핵심은 코루틴 함수 선언, `await`, `gather` 입니다.

```

import asyncio
import httpx
from fastapi import FastAPI

app = FastAPI()
PAYMENT_API = "https://pay.example.com"

---- def fetch_payment(client: httpx.AsyncClient, order_id: int) -> dict:
    # 외부 호출을 비동기로 await 하는 함수
    resp = client.get(f"{PAYMENT_API}/status/{order_id}") # await 키워드는?
    resp.raise_for_status()
    return {"order_id": order_id, "status": resp.json()["status"]}

@app.get("/orders/payments")
async def order_payments(order_ids: list[int]):
    async with httpx.AsyncClient(timeout=5.0) as client:
        tasks = [fetch_payment(client, oid) for oid in order_ids]
        results = await asyncio.gather(*tasks) # 동시 실행 함수는?
    return {"payments": results}

```

확장 과제: 일부 주문 조회가 실패해도 나머지는 받아오도록 `gather`에 `return_exceptions=True`를 주고, 결과를 순회하며 예외인 항목을 골라 `{"order_id": ..., "error": "조회 실패"}` 형태로 바꿔 보십시오.

자가체크 체크리스트

스스로 점검해 보십시오.

- [] 외부 호출 함수를 `async def`로 선언했는가
- [] 외부 호출 앞에 `await`를 빠뜨리지 않았는가(빠뜨리면 코루틴 객체만 생김)
- [] 직렬 `await`가 아니라 `asyncio.gather`로 동시 실행했는가
- [] `httpx.AsyncClient`(비동기)를 썼는가(`requests` 같은 블로킹 클라이언트가 아님)
- [] (확장) `return_exceptions=True`로 일부 실패를 격리했는가

5.9 흔한 오류와 오개념: 무조건 `async-await` 누락·스레드풀

비동기에서 실무자가 자주 부딪히는 함정들입니다.

오개념 1: 모든 핸들러를 `async def`로 바꾸면 빨라진다

`async def`는 그 자체로 성능을 올려 주지 않습니다. 이점은 내부에서 `await`로 I/O 대기를 양보할 때만 생깁니다. 더 위험한 것은, 비동기 버전이 없는 블로킹 코드를 `async def` 안에 넣는 경우입니다.

```
import time

# 잘못: async 안의 블로킹 호출 -> 이벤트 루프 전체가 멈춤
@app.get("/slow")
async def slow():
    time.sleep(2)          # 블로킹! 2초간 모든 요청 정지
    return {"ok": True}
```

이렇게 하면 오히려 동기 방식보다 나쁩니다. 블로킹만 있다면 `async` 를 떼고 일반 `def` 로 선언해 스레드풀에서 돌게 해야 합니다.

오개념 2: `await`를 빠뜨려도 호출은 된다

코루틴 함수는 `await` 없이 호출하면 실행되지 않고 코루틴 객체만 만들어집니다. 외부 호출이 실제로 일어나지 않습니다.

```
async def estimate_delivery(product_id: int) -> str:
    ...

# 잘못: await 누락 -> eta는 응답이 아니라 코루틴 객체
eta = estimate_delivery(42)
print(eta)          # <coroutine object ...> (경고도 뚝)

# 올바른
eta = await estimate_delivery(42)
```

`await` 를 빠뜨리면 보통 파이썬이 "coroutine was never awaited" 경고를 띄웁니다. 비동기 함수 호출 앞에는 항상 `await` 가 붙는지 확인하십시오.

오개념 3: `async def` 안에서는 어떤 라이브러리든 써도 된다

`async def` 안에서는 **비동기(awaitable) 라이브러리**만 안전합니다. 동기 `requests.get`, `time.sleep`, 블로킹 DB 드라이버를 직접 호출하면 이벤트 루프를 막습니다.

막는 것(블로킹)	대신 쓸 것(비동기)
<code>requests.get(...)</code>	<code>await client.get(...)</code> (<code>httpx.AsyncClient</code>)
<code>time.sleep(2)</code>	<code>await asyncio.sleep(2)</code>
블로킹 DB 호출	비동기 드라이버로 <code>await</code> , 또는 <code>def</code> 핸들러

FastAPI가 일반 `def` 핸들러를 스레드풀에서 자동 실행한다는 사실은 안전망입니다. 비동기 라이브러리가 없을 때는 억지로 `async def` 를 쓰지 말고 `def` 를 선택하는 것이 더 안전합니다.

그 밖의 주의점

- `asyncio.gather` 로 묶을 작업은 **서로 독립적**이어야 합니다. 한 호출의 결과가 다음 호출의 입력이라면 동시 실행이 불가능하므로 순서대로 `await` 해야 합니다.
- `httpx.AsyncClient` 는 매 요청마다 새로 만들기보다 컨텍스트 매니저나 `lifespan(ch_12)`에서 재 사용하는 편이 커넥션 풀 측면에서 효율적입니다.
- 외부 호출에는 항상 `timeout` 을 지정하십시오. 타임아웃이 없으면 외부 서비스 지연이 우리 서버의 코루틴을 무한정 묶어 둘 수 있습니다.

5.10 단원 미니 프로젝트: 비동기 외부 조회 엔드포인트

이 단원에서 배운 `async/await`, `httpx.AsyncClient`, `asyncio.gather`, 블로킹 회피를 한데 모아 작은 산출물을 완성합니다. 외부 호출은 결정론적 실행을 위해 스텝(가짜) 함수로 모킹합니다.

목표: `async def` 핸들러에서 여러 상품의 배송 추정을 동시에 조회하고, 블로킹 호출과 비동기 호출의 응답 시간 차이를 보여 주는 엔드포인트를 만듭니다.

마일스톤(증분)으로 나눠 진행합니다.

- **M1:** `async def` 핸들러에서 `await` 로 단일 외부 호출(모킹) 수행
- **M2:** `asyncio.gather` 로 여러 상품의 배송 추정을 동시 조회
- **M3:** 블로킹(`time.sleep`) vs 비동기(`await asyncio.sleep`) 응답 시간 비교 엔드포인트 추가

모범 답안 (main.py)

```

import asyncio
import time
from fastapi import FastAPI

app = FastAPI(title="비동기 외부 조회 데모")

# 외부 배송 서비스 모킹: 0.3초 걸리는 비동기 호출을 흉내 낸다.
async def mock_shipping_eta(product_id: int) -> dict:
    await asyncio.sleep(0.3) # 외부 호출 대기를 비동기로 흉내
    return {"product_id": product_id, "eta_days": (product_id % 3) + 2}

# M1: 단일 비동기 외부 호출
@app.get("/eta/{product_id}")
async def single_eta(product_id: int):
    result = await mock_shipping_eta(product_id)
    return result

# M2: 여러 상품 배송 추정치를 동시에 조회
@app.get("/eta")
async def many_eta(product_ids: list[int]):
    t0 = time.perf_counter()
    tasks = [mock_shipping_eta(pid) for pid in product_ids]
    results = await asyncio.gather(*tasks)
    elapsed = round(time.perf_counter() - t0, 2)
    return {"items": results, "seconds": elapsed}

# M3: 블로킹 def 핸들러(스레드풀)와 비동기 비교용 엔드포인트
@app.get("/blocking")
def blocking_demo():
    time.sleep(0.3) # def 핸들러라 스레드풀에서 실행 -> 루프를 막지 않음
    return {"mode": "blocking-in-threadpool"}

```

```

GET /eta/42          -> {"product_id": 42, "eta_days": 2}
GET /eta?product_ids=1&product_ids=2&product_ids=3
  -> {"items": [...3개...], "seconds": 0.3} # 직렬이면 0.9초
GET /blocking        -> {"mode": "blocking-in-threadpool"}

```

직접 작성: 비동기 외부 조회 (빈 스타터)

아래 빈칸을 채워 직접 완성해 보십시오. 모킹 함수는 제공되며, 핸들러의 비동기 처리를 채우는 것이 과제입니다.

```

import asyncio
import time
from fastapi import FastAPI

app = FastAPI(title="비동기 외부 조회 데모")

async def mock_shipping_eta(product_id: int) -> dict:
    await asyncio.sleep(0.3)
    return {"product_id": product_id, "eta_days": (product_id % 3) + 2}

# M1: 단일 비동기 호출
@app.get("/eta/{product_id}")
---- def single_eta(product_id: int):          # 코루틴 핸들러 선언 키워드는?
    result = ---- mock_shipping_eta(product_id) # 비동기 호출을 기다리는 키워드는?
    return result

# M2: 여러 상품 동시 조회
@app.get("/eta")
async def many_eta(product_ids: list[int]):
    tasks = [mock_shipping_eta(pid) for pid in product_ids]
    results = await asyncio.____(*tasks)      # 동시 실행 함수는?
    return {"items": results}

# M3: 블로킹 작업은 어떤 키워드로 선언해야 루프를 안 막을까?
@app.get("/blocking")
---- blocking_demo():                          # def 인가 async def 인가?
    time.sleep(0.3)
    return {"mode": "blocking-in-threadpool"}

```

자가체크 체크리스트

- [] 동시 호출(/eta)이 직렬 합계(0.9초)보다 빠른 약 0.3초로 끝나는가
- [] 비동기 호출 앞에 await 를 빠뜨리지 않았는가
- [] asyncio.gather 로 독립 호출들을 동시에 실행했는가
- [] 블로킹 호출(time.sleep)이 들어간 핸들러를 일반 def 로 선언했는가(async def 가 아님)
- [] 외부 호출을 모킹으로 두어 실행이 결정론적인가

5.11 자주 묻는 질문(FAQ)

Q. 비동기로 만들면 CPU 계산도 빨라지나요?

아니요. 비동기는 I/O 대기 시간을 재활용하는 기법이지 CPU를 더 빠르게 쓰는 것이 아닙니다. 무거운 계산(이미지 변환, 암호 해시 반복 등)은 오히려 이벤트 루프를 막으므로 def 핸들러(스레드풀)나

별도 프로세스로 빼야 합니다.

Q. `async def` 안에서 동기 DB 라이브러리를 꼭 써야 한다면요?

두 가지 선택지가 있습니다. 핸들러를 일반 `def` 로 선언해 스레드풀에서 돌리거나, `await anyio.to_thread.run_sync`(블로킹함수, 인자) 로 명시적으로 스레드에 떠넘기는 것입니다. 절대로 `async def` 안에서 동기 호출을 직접 하지 마십시오.

Q. `gather`와 그냥 `await`를 여러 번 하는 것의 차이가 헛갈립니다.

`await a(); await b()` 는 `a`를 끝까지 기다린 뒤 `b`를 시작합니다(직렬). `await asyncio.gather(a(), b())` 는 `a`와 `b`를 동시에 시작하고 둘 다 끝나길 기다립니다(동시). `a`와 `b`가 서로 독립적일 때만 `gather`가 의미가 있습니다.

Q. `httpx` 말고 `requests`를 쓰면 안 되나요?

`requests` 는 동기(블로킹)라 `async def` 안에서 쓰면 이벤트 루프를 막습니다. 비동기 핸들러에서는 `httpx.AsyncClient` 를 쓰십시오. 참고로 `httpx`는 동기 `httpx.Client` 도 제공하므로 `def` 핸들러에서는 그쪽을 쓸 수 있습니다.

Q. FastAPI는 `async def`와 `def` 중 어떤 걸 권장하나요?

상황에 맞게 고르는 것을 권장합니다. 비동기 라이브러리로 I/O를 처리할 수 있으면 `async def` 가 동시성에 유리하고, 블로킹 라이브러리만 있으면 `def` 가 안전합니다. "무조건 `async`"가 정답은 아닙니다.

Q. 이벤트 루프는 몇 개나 도나요?

기본적으로 워커 프로세스 하나당 이벤트 루프 하나입니다. 처리량을 더 높이려면 `ch_12`에서 다루는 `uvicorn` 워커 수를 늘려 여러 프로세스가 각자 루프를 돌게 합니다. 비동기는 한 루프 안에서 I/O 대기기를 재활용하고, 멀티 워커는 CPU 코어를 더 활용하는 보완 관계입니다.

정리

이번 챕터에서는 **`async/await` 비동기 처리** 개념을 깊이 익혔습니다.

- 이벤트 루프가 `await` 지점에서 양보된 대기 시간을 재활용해 동시성을 높이는 원리를 이해했습니다.
- `httpx.AsyncClient` 로 외부 배송·재고·결제 API를 비동기로 `await` 호출했습니다.
- `asyncio.gather` 로 독립적인 외부 호출들을 동시에 실행해 응답 시간을 줄였습니다.

- 블로킹 코드가 이벤트 루프를 막는 문제와, 일반 `def` 핸들러가 스레드풀에서 실행되는 안전망을 배웠습니다.
- "무조건 async", "await 누락", "async 안의 블로킹"이라는 3대 오개념을 점검했습니다.

여러 핸들러가 외부 호출·DB 세션 같은 **공통 자원을 반복해서 준비**하는 것이 눈에 띄었을 것입니다. 다음 챕터(ch_06)에서는 그 공통 로직을 함수로 분리해 핸들러에 깔끔하게 주입하는 **의존성 주입 (Depends)** 을 배웁니다. 비동기 의존성도 자연스럽게 이어집니다.

의존성 주입: Depends로 공통 로직 분리

이 챕터에서 배우는 것

- `Depends` 로 공통 로직(페이지네이션·세션·인증)을 의존성 함수로 분리해 핸들러에 주입할 수 있습니다.
- `Annotated[T, Depends()]` 권장 형식과 타입 별칭으로 핸들러 시그니처를 단순화할 수 있습니다.
- `yield` 의존성으로 자원의 `setup/teardown`(정리)을 보장할 수 있습니다.
- 하위 의존성과 클래스 의존성으로 의존성 그래프를 합성할 수 있습니다.
- 의존성 캐시가 한 요청 안에서 어떻게 작동하는지 설명할 수 있습니다.

이번 챕터에서 가르치는 개념은 **의존성 주입(dependency injection)** 한 가지입니다. 선수 개념은 ch_05의 **async/await**입니다. 앞 챕터에서 여러 핸들러가 `httpx.AsyncClient` 를 매번 만들고, 페이지네이션 파라미터(`limit`, `offset`)를 매번 똑같이 선언하는 모습을 보았습니다. 이렇게 반복되는 공통 준비 작업을 한 곳에 모아 핸들러에 깔끔하게 "주입"하는 것이 의존성 주입입니다. 종합 프로젝트(ch_15)의 DB 세션·현재 사용자·페이지네이션은 모두 이 메커니즘으로 공급됩니다.

6.1 도입: 반복되는 공통 로직을 주입으로

목록 API를 여러 개 만든다고 해 봅시다. 상품 목록, 주문 목록, 사용자 목록 모두 페이지네이션이 필요합니다. ch_05까지 배운 방식이라면 핸들러마다 똑같은 코드를 반복해야 합니다.

```
@app.get("/products")
async def list_products(limit: int = 20, offset: int = 0):
    ...

@app.get("/orders")
async def list_orders(limit: int = 20, offset: int = 0):
    ...

@app.get("/users")
async def list_users(limit: int = 20, offset: int = 0):
    ...
```

`limit: int = 20, offset: int = 0` 이 세 곳에 똑같이 반복됩니다. 만약 "limit 최대값을 100으로 제한하자"는 정책이 생기면 세 곳을 모두 고쳐야 하고, 빠뜨리면 일관성이 깨집니다. DB 세션을 여는 코

드, 토큰에서 현재 사용자를 꺼내는 코드도 마찬가지로 모든 핸들러에서 반복됩니다.

여기서 떠오르는 자연스러운 해법은 "공통 부분을 함수로 빼자"입니다. 하지만 일반 함수로 빼면 핸들러 안에서 직접 호출해야 하고, 그 함수가 또 다른 자원(세션 등)을 필요로 하면 그것까지 핸들러가 챙겨 넘겨야 합니다. FastAPI의 **의존성 주입**은 이 문제를 우아하게 푼다. 공통 로직을 함수로 만들어 두고 핸들러에 "이게 필요하다"고 선언만 하면, FastAPI가 그 함수를 알아서 실행해 결과를 핸들러에 넣어 줍니다.

한눈에 보는 의존성 주입

의존성 주입은 핸들러가 필요로 하는 값(페이지네이션 파라미터·DB 세션·현재 사용자)을 **직접 만들지 않고, 선언만 하면 FastAPI가 공급해 주는 메커니즘**입니다. `Depends(의존성함수)` 로 "이 함수의 결과가 필요하다"고 표시하면, FastAPI가 요청마다 그 함수를 실행해 반환값을 핸들러 인자로 넣어 줍니다. 덕분에 공통 로직을 한 곳에 모아 재사용하고, 핸들러는 비즈니스 로직에만 집중하며, 테스트 때는 의존성을 가짜로 교체(ch_11)할 수 있습니다.

의존성 주입이 주는 이득은 세 가지로 정리됩니다. **재사용**(공통 로직을 한 번 정의해 여러 핸들러가 공유), **관심사 분리**(핸들러는 비즈니스 로직만, 준비 작업은 의존성이 담당), **테스트 용이성**(의존성을 테스트용으로 갈아끼움). 이 세 가지가 FastAPI를 실무에서 견고하게 만드는 핵심 도구입니다.

6.2 정의: Depends와 의존성 함수

핵심 용어를 한 문장씩 정의하겠습니다.

- **의존성 함수(dependency)**: 핸들러가 필요로 하는 값을 만들어 반환하는 보통의 함수(또는 코루틴)입니다. 일반 경로 동작처럼 파라미터를 선언할 수 있어, 쿼리 파라미터·다른 의존성도 받을 수 있습니다.
- **Depends**: 핸들러(또는 다른 의존성)의 인자에 "이 자리에는 이 의존성 함수의 실행 결과를 넣어 달라"고 표시하는 마커입니다. `Depends(get_pagination)` 처럼 **함수 자체**를 인자로 넘깁니다 (괄호로 호출하지 않음).

한 문장 정의는 이렇습니다. **의존성 주입**은 `Depends(의존성함수)` 로 **공통 로직 함수**를 핸들러에 주입해, FastAPI가 요청마다 그 함수를 실행하고 **결과**를 핸들러 인자에 채워 주는 메커니즘입니다.

가장 단순한 예부터 봅시다.

```

from fastapi import Depends, FastAPI

app = FastAPI()

# 의존성 함수: 공통 페이지네이션 파라미터를 dict로 반환
def get_pagination(limit: int = 20, offset: int = 0) -> dict:
    return {"limit": limit, "offset": offset}

@app.get("/products")
async def list_products(page: dict = Depends(get_pagination)):
    return {"page": page, "items": ["...상품 목록..."]}

```

```

GET /products?limit=10&offset=5
200 OK
{"page": {"limit": 10, "offset": 5}, "items": ["...상품 목록..."]}

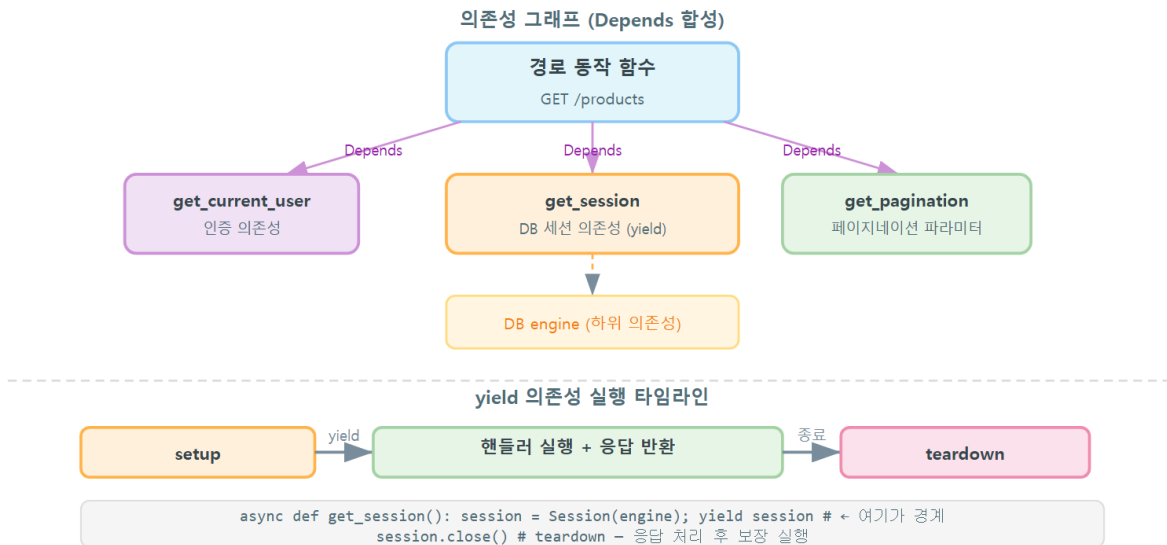
```

여기서 결정적으로 중요한 점이 하나 있습니다. `Depends(get_pagination)` 에서 `get_pagination` 은 **콜 호 없이** 함수 자체를 넘깁니다. `Depends(get_pagination())` 처럼 호출하면 안 됩니다. FastAPI가 요청마다 적절한 시점에 직접 호출하기 위해, 우리는 "이 함수를 써라"고 함수만 건네는 것입니다. 이 실수는 매우 흔하므로 6.9에서 다시 강조합니다.

또 하나 주목할 점은, 의존성 함수 `get_pagination` 이 `limit`, `offset` 이라는 쿼리 파라미터를 받는다는 것입니다. FastAPI는 의존성 함수의 파라미터도 일반 경로 동작처럼 해석합니다. 그래서 ?
`limit=10&offset=5` 같은 쿼리 파라미터가 의존성으로 자동 전달되고, OpenAPI 문서(/docs)에도 이 파라미터들이 그대로 나타납니다.

6.3 동작 원리: 의존성 그래프 합성과 yield 정리

의존성 주입: 그래프 합성과 yield 정리 타임라인



요청 하나가 들어왔을 때 FastAPI가 의존성을 어떻게 처리하는지 단계로 따라가 보겠습니다.

- 의존성 수집:** FastAPI는 핸들러 시그니처를 보고 `Depends` 로 표시된 의존성들을 찾습니다. 의존성이 또 다른 의존성을 요구하면(하위 의존성) 그래프를 따라 끝까지 펼칩니다.
- 순서대로 해결:** 가장 깊은(선행) 의존성부터 실행합니다. 각 의존성의 파라미터(쿼리·다른 의존성)도 이때 함께 해석합니다.
- 결과 주입:** 각 의존성의 반환값을 의존성·핸들러의 해당 인자에 채웁니다.
- 핸들러 실행:** 모든 의존성이 준비되면 핸들러 본문이 실행됩니다.
- 정리(teardown):** `yield` 의존성이 있으면, 응답이 만들어진 뒤 `yield` 다음 코드가 실행되어 자원을 정리합니다.

특히 5번이 중요합니다. 일반 의존성은 값을 `return` 하고 끝나지만, **yield 의존성**은 자원을 만들어 `yield` 로 건넨 뒤, 핸들러와 응답 처리가 끝나면 `yield` 다음 줄(정리 코드)이 실행됩니다. 마치 `try ... finally` 처럼 동작해, DB 세션 닫기·파일 닫기 같은 정리를 빠짐없이 보장합니다.

시점	일반 의존성(return)	yield 의존성(yield)
요청 시작	함수 실행 → 값 반환	yield 앞(setup) 실행 → 자원 yield
핸들러 실행 중	반환값 사용	yield된 자원 사용
응답 후	(정리 없음)	yield 뒤(teardown) 실행 → 자원 정리

여러 의존성이 모이면 **의존성 그래프**가 됩니다. 예를 들어 "현재 사용자" 의존성이 "토큰 추출" 의존성과 "DB 세션" 의존성을 동시에 필요로 할 수 있습니다. FastAPI는 이 그래프를 위상 순서로 해결하고, 같은 요청 안에서 같은 의존성이 여러 번 쓰이면 **한 번만 실행하고 결과를 재사용**합니다(의존성 캐시, 6.7).

6.4 worked example: get_pagination 공통 쿼리 의존성

이제 페이지네이션 의존성을 제대로 만들어 여러 목록 API에 주입해 보겠습니다. 단순히 dict를 반환 하던 6.2를 발전시켜, 제약 검증(ch_02의 Query)까지 의존성 안에 넣겠습니다.

```
from fastapi import Depends, FastAPI, Query

app = FastAPI()

# 도메인 데이터(예시): 실제로는 DB에서 조회
PRODUCTS = [{"id": i, "name": f"상품{i}"} for i in range(1, 101)]
ORDERS = [{"id": i, "total": i * 1000} for i in range(1, 51)]

def get_pagination(
    limit: int = Query(20, ge=1, le=100, description="한 페이지 항목 수"),
    offset: int = Query(0, ge=0, description="건너뛴 항목 수"),
) -> dict:
    """공통 페이지네이션 파라미터를 검증해 반환하는 의존성."""
    return {"limit": limit, "offset": offset}

@app.get("/products")
async def list_products(page: dict = Depends(get_pagination)):
    start = page["offset"]
    items = PRODUCTS[start : start + page["limit"]]
    return {"page": page, "total": len(PRODUCTS), "items": items}

@app.get("/orders")
async def list_orders(page: dict = Depends(get_pagination)):
    start = page["offset"]
    items = ORDERS[start : start + page["limit"]]
    return {"page": page, "total": len(ORDERS), "items": items}
```

```
GET /products?limit=3&offset=0
200 OK
{"page": {"limit": 3, "offset": 0}, "total": 100,
 "items": [{"id":1,"name":"상품1"}, {"id":2,"name":"상품2"}, {"id":3,"name":"상품3"}]}
```

```
GET /products?limit=200
422 Unprocessable Entity # le=100 제약 위반
```

- `get_pagination` 은 `limit`, `offset` 을 `Query` 제약과 함께 선언한 의존성 함수입니다. `ge=1`, `le=100` 으로 한 페이지 크기를 1~100으로 강제하므로, `limit=200` 을 보내면 핸들러에 닿기도 전에 422가 납니다. 제약 검증이 의존성 한 곳에 모였습니다.
- `list_products` 와 `list_orders` 둘 다 `Depends(get_pagination)` 로 같은 의존성을 주입받습니다. 페이지네이션 정책이 바뀌면 `get_pagination` 한 곳만 고치면 두 핸들러에 동시에 반영됩니다.

이것이 재사용의 이득입니다.

- 의존성의 쿼리 파라미터(`limit`, `offset`)는 핸들러의 쿼리 파라미터처럼 OpenAPI 문서에 그대로 나타납니다. `/docs` 에서 두 목록 API 모두 동일한 페이지네이션 입력란을 갖습니다.
- 핸들러 본문은 `page["offset"]`, `page["limit"]` 만 꺼내 슬라이싱에 씁니다. 핸들러는 "어떻게 페이지 파라미터를 받고 검증하는가"를 신경 쓰지 않고 비즈니스 로직(목록 자르기)에만 집중합니다.

이 예제만으로도 의존성 주입의 가치가 분명한겁니다. 같은 입력 검증·준비 로직이 여러 핸들러에 흩어지지 않고 한 곳에 모여, 변경에 강하고 읽기 쉬운 구조가 됩니다.

6.5 Annotated[T, Depends()]와 타입 별칭(PaginationDep)

6.4의 `page: dict = Depends(get_pagination)` 는 잘 동작하지만 두 가지 아쉬움이 있습니다. 첫째, 반환 타입이 `dict` 라 어떤 키가 있는지 코드만 봐서는 알 수 없습니다. 둘째, 여러 핸들러에서 `= Depends(get_pagination)` 를 반복해 써야 합니다.

현대 FastAPI는 `Annotated[T, Depends()]` 형식을 권장합니다. `Annotated` 는 "이 값의 타입은 T이고, 추가로 이런 메타데이터(여기서는 `Depends`)가 붙어 있다"를 한 자리에 적는 표준 타입 표기입니다.

```

from typing import Annotated
from fastapi import Depends, FastAPI, Query
from pydantic import BaseModel

app = FastAPI()

class Pagination(BaseModel):
    """페이지네이션 파라미터를 담는 타입 안전한 모델."""
    limit: int
    offset: int

def get_pagination(
    limit: int = Query(20, ge=1, le=100),
    offset: int = Query(0, ge=0),
) -> Pagination:
    return Pagination(limit=limit, offset=offset)

# 타입 별칭: 여러 핸들러에서 재사용할 주입 타입을 한 번만 정의
PaginationDep = Annotated[Pagination, Depends(get_pagination)]

@app.get("/products")
async def list_products(page: PaginationDep):
    return {"limit": page.limit, "offset": page.offset}

@app.get("/orders")
async def list_orders(page: PaginationDep):
    return {"limit": page.limit, "offset": page.offset}

```

```

GET /orders?limit=5&offset=10
200 OK
{"limit": 5, "offset": 10}

```

- `PaginationDep = Annotated[Pagination, Depends(get_pagination)]` 가 핵심입니다. "타입은 `Pagination` 이고, 값은 `get_pagination` 의존성으로 주입한다"를 하나의 타입 별칭으로 정의했습니다.
- 핸들러는 `page: PaginationDep` 라고만 적으면 됩니다. `= Depends(...)` 를 반복하지 않아 시그니처가 깔끔하고, 타입이 `Pagination` 이라 `page.limit` 처럼 속성 접근에 자동완성·타입 검사가 동작합니다.
- 의존성이 `Pagination` 모델 객체를 반환하므로, 핸들러에서 `page["limit"]` (딕셔너리 접근)가 아니라 `page.limit` (속성 접근)을 씁니다. dict보다 타입 안전합니다.
- `Annotated` 형식은 같은 의존성을 핸들러뿐 아니라 다른 의존성에서도 재사용하기 좋고, FastAPI 공식 문서가 권장하는 현대적 작성법입니다. `def f(x: int = Depends(...))` 같은 기본 값 형식도 동작하지만, 새 코드는 `Annotated` 로 통일하는 것을 권장합니다.

Annotated를 권장하는 이유

`x: int = Depends(...)` 형식은 의존성을 "기본값" 자리에 두기 때문에, 같은 의존성을 여러 곳에서 쓰면 매번 반복해야 하고 실제 기본값과 헷갈릴 수 있습니다. `Annotated[T, Depends(...)]` 는 타입 정보와 주입 정보를 한 자리에 모아 **타입 별칭으로 재사용**할 수 있고(`PaginationDep`), 에디터의 타입 추론도 더 정확합니다. 이 교재의 이후 챕터(세션·현재 사용자 의존성)는 모두 `Annotated` 형식을 씁니다.

6.6 worked example: yield로 세션 setup/teardown

지금까지의 의존성은 값을 `return` 하고 끝났습니다. 그런데 DB 세션처럼 **다 쓴 뒤 반드시 닫아야 하는 자원**은 어떻게 할까요? 핸들러가 끝날 때 세션을 닫는 코드를 매번 적으면 또 반복이고, 예외가 나면 닫기를 빠뜨릴 위험도 있습니다.

`yield` 의존성이 이 문제를 해결합니다. `return` 대신 `yield` 로 자원을 건네면, FastAPI가 핸들러와 응답 처리를 마친 뒤 `yield` 다음 코드를 실행해 정리를 보장합니다. `try ... finally` 로 감싸면 예외가 나도 정리가 됩니다.

아래는 가상의 세션을 예로 든 것입니다. 실제 `SQLModel` 세션은 ch_07에서 다루지만, 정리 패턴은 동일합니다.

```

from typing import Annotated
from fastapi import Depends, FastAPI

app = FastAPI()

class FakeSession:
    """DB 세션을 흉내 내는 예시 객체."""
    def __init__(self) -> None:
        self.log: list[str] = []
        print("세션 열림 (setup)")

    def add(self, item: str) -> None:
        self.log.append(item)

    def commit(self) -> None:
        print(f"커밋: {self.log}")

    def close(self) -> None:
        print("세션 닫힘 (teardown)")

def get_session():
    """yield 의존성: 세션을 열어 주고, 요청이 끝나면 반드시 닫는다."""
    session = FakeSession()      # setup: yield 앞에서 자원 생성
    try:
        yield session            # 핸들러에 세션을 건넨다
    finally:
        session.close()         # teardown: 응답 후 정리(예외에도 보장)

SessionDep = Annotated[FakeSession, Depends(get_session)]

@app.post("/products")
async def create_product(name: str, session: SessionDep):
    session.add(name)
    session.commit()
    return {"created": name}

```

```

POST /products?name=keyboard
콘솔:
세션 열림 (setup)
커밋: ['keyboard']
세션 닫힘 (teardown)
응답: {"created": "keyboard"}

```

- `get_session` 은 `return` 이 아니라 `yield` 를 씁니다. `yield session` 까지가 `setup`이고, 이 시점에 세션이 핸들러로 주입됩니다.
- `try ... finally` 로 감쌌습니다. `finally` 블록의 `session.close()` 는 핸들러가 정상 종료하든 예외로 끝나든 **항상** 실행되어, 세션이 새는 일을 막습니다.

- 콘솔 출력 순서가 동작 원리(6.3)를 그대로 보여 줍니다. "세션 열림"(setup) → 핸들러의 "커밋" → "세션 닫힘"(teardown). teardown은 응답이 처리된 뒤에 실행됩니다.
- 핸들러 `create_product` 는 `session: SessionDep` 로 세션을 주입받아 `add · commit` 만 합니다. 세션을 언제 열고 닫는지는 전혀 신경 쓰지 않습니다. 관심사가 깔끔하게 분리됐습니다.

이 패턴이 실무에서 가장 자주 쓰이는 의존성 형태입니다. ch_07의 DB 세션, 외부 클라이언트 풀, 파일 핸들 등 "열고 반드시 닫아야 하는" 모든 자원이 `yield` 의존성으로 관리됩니다.

6.7 하위 의존성·클래스 의존성·의존성 캐시

의존성은 또 다른 의존성을 가질 수 있습니다. 이를 **하위 의존성(sub-dependency)** 이라 합니다. 그리고 함수뿐 아니라 **클래스**도 의존성으로 쓸 수 있으며, 같은 요청 안에서 같은 의존성은 **캐시**되어 한 번만 실행됩니다. 셋을 차례로 보겠습니다.

하위 의존성: "현재 사용자" 의존성이 "세션" 의존성을 필요로 하는 상황입니다.

```

from typing import Annotated
from fastapi import Depends, FastAPI, Header, HTTPException

app = FastAPI()

def get_session():
    session = {"db": "connected"}
    try:
        yield session
    finally:
        pass # 정리

SessionDep = Annotated[dict, Depends(get_session)]

def get_current_user(
    session: SessionDep, # 하위 의존성: 세션을 요구
    x_user_id: Annotated[str | None, Header()] = None, # 헤더에서 사용자 식별
) -> dict:
    if x_user_id is None:
        raise HTTPException(status_code=401, detail="인증 필요")
    # 실제로는 session으로 DB 조회. 여기서는 예시.
    return {"id": x_user_id, "name": f"user-{x_user_id}"}

CurrentUser = Annotated[dict, Depends(get_current_user)]

@app.get("/orders/me")
async def my_orders(user: CurrentUser):
    return {"owner": user["id"], "orders": ["...본인 주문..."]}

```

```

GET /orders/me          (X-User-Id 헤더 없음) -> 401
GET /orders/me with X-User-Id: 7          -> {"owner": "7", "orders": [...]}

```

- `get_current_user` 는 그 자체가 의존성이면서, 내부에서 `session: SessionDep` 로 다른 의존성을 요구합니다. 이것이 하위 의존성입니다. FastAPI는 `get_current_user` 를 해결하기 전에 `get_session` 을 먼저 해결합니다.
- 핸들러 `my_orders` 는 `user: CurrentUser` 하나만 선언했는데, 그 뒤로 세션·헤더 검증·401 처리가 의존성 그래프로 모두 엮여 실행됩니다. 핸들러 시그니처는 단순하지만 그 뒤에 합성된 로직이 풍부합니다.
- `x_user_id: Annotated[str | None, Header()]` 는 HTTP 헤더 `X-User-Id` 를 받습니다(이름의 밑줄이 하이픈으로 매핑). 실제 인증은 ch_09에서 JWT로 구현하며, 여기서는 의존성 합성 구조에 집중하기 위해 헤더로 단순화했습니다.

클래스 의존성: 호출 가능한(callable) 것은 의존성이 될 수 있으므로, 클래스도 의존성으로 쓸 수 있습니다. 클래스를 의존성으로 쓰면 `__init__` 파라미터가 쿼리 파라미터로 해석되고, 주입받는 객체

의 타입이 그 클래스라 속성 접근이 명확해집니다.

```
from typing import Annotated
from fastapi import Depends, FastAPI, Query

app = FastAPI()

class ProductFilter:
    """상품 목록 필터를 묶은 클래스 의존성."""
    def __init__(
        self,
        category: str | None = None,
        min_price: int = Query(0, ge=0),
        limit: int = Query(20, ge=1, le=100),
    ) -> None:
        self.category = category
        self.min_price = min_price
        self.limit = limit

FilterDep = Annotated[ProductFilter, Depends(ProductFilter)]

@app.get("/products")
async def list_products(f: FilterDep):
    return {"category": f.category, "min_price": f.min_price, "limit": f.limit}
```

```
GET /products?category=book&min_price=1000&limit=10
200 OK
{"category": "book", "min_price": 1000, "limit": 10}
```

- `Depends(ProductFilter)` 는 클래스를 의존성으로 넘깁니다. FastAPI가 요청마다 `ProductFilter(...)` 를 생성하며, `__init__` 의 파라미터(`category` · `min_price` · `limit`)를 쿼리 파라미터로 해석합니다.
- 주입받은 `f` 는 `ProductFilter` 인스턴스이므로 `f.category` 처럼 속성으로 접근합니다. 관련된 여러 필터 파라미터를 하나의 객체로 묶어 핸들러 시그니처를 단정하게 만듭니다.
- 클래스 의존성은 필터·검색 조건처럼 "여러 파라미터를 한 덩어리로 다루고 싶을 때" 특히 깔끔합니다.

의존성 캐시: FastAPI는 한 요청 안에서 같은 의존성이 여러 번 쓰이면 **처음 한 번만 실행하고 결과를 재사용**합니다. 예를 들어 `get_current_user` 와 다른 의존성이 모두 `get_session` 을 요구해도, `get_session` 은 그 요청에서 한 번만 실행되어 같은 세션을 공유합니다. 이것이 의존성 캐시이며, 같은 요청에서 DB 세션을 하나로 유지하는 데 핵심적입니다. 매번 새로 실행하고 싶다면 `Depends(get_session, use_cache=False)` 로 캐시를 끌 수 있습니다.

6.8 즉시 연습(2종 1세트): yield 의존성 채우기

직접 작성할 차례입니다. 모범 답안을 먼저 보고, 빈칸 코드를 덮어 두고 도전해 보십시오. 외부 API 클라이언트를 열고 닫는 `yield` 의존성을 완성하는 과제입니다.

모범 답안

```
from typing import Annotated
from fastapi import Depends, FastAPI

app = FastAPI()

class ApiClient:
    """외부 API 클라이언트를 흉내 내는 예시."""
    def __init__(self) -> None:
        print("클라이언트 연결")

    def get(self, path: str) -> dict:
        return {"path": path, "ok": True}

    def close(self) -> None:
        print("클라이언트 종료")

def get_client():
    client = ApiClient()          # setup
    try:
        yield client              # 핸들러에 주입
    finally:
        client.close()           # teardown: 항상 종료

ClientDep = Annotated[ApiClient, Depends(get_client)]

@app.get("/ping")
async def ping(client: ClientDep):
    return client.get("/health")
```

직접 작성: yield 클라이언트 의존성 (빈칸 코드)

아래 빈칸(____)을 채워 setup/teardown이 보장되는 의존성을 완성해 보십시오.


```

from typing import Annotated
from fastapi import Depends, FastAPI

app = FastAPI()

class ApiClient:
    def __init__(self) -> None:
        print("클라이언트 연결")

    def get(self, path: str) -> dict:
        return {"path": path, "ok": True}

    def close(self) -> None:
        print("클라이언트 종료")

def get_client():
    client = ApiClient()          # setup
    try:
        ____ client              # 핸들러에 자원을 건네는 키워드는?(return 아님)
    ____:                         # 예외에도 정리를 보장하는 블록은?
        client.____()            # 종료 메서드 호출

```

```

# 타입 별칭으로 주입 형식을 정의하고 핸들러에 주입하세요.
ClientDep = Annotated[ApiClient, Depends(____)]    # 괄호 없이 함수 자체!

@app.get("/ping")
async def ping(client: ClientDep):
    return client.get("/health")

```

확장 과제: `get_client` 가 `get_pagination` 같은 다른 의존성을 하위 의존성으로 받도록 바꿔, 두 의존성이 한 핸들러에 함께 주입되게 만들어 보십시오.

자가체크 체크리스트

- [] `return` 이 아니라 `yield` 로 자원을 건넸는가
- [] `try ... finally` 로 예외에도 `teardown`(정리)이 실행되게 했는가
- [] `Depends` 에 함수를 **괄호 없이** 넘겼는가(`Depends(get_client)`)
- [] `Annotated[T, Depends()]` 형식과 타입 별칭으로 시그니처를 단순화했는가
- [] 핸들러가 자원을 받아 쓰기만 하고 열고 닫는 책임을 지지 않는가

6.9 혼한 오류와 오개념: 괄호 없이 넘기기·캐시·정리 시점

의존성 주입에서 실무자가 자주 부딪히는 함정들입니다.

오개념 1: Depends에 함수를 호출해서 넘긴다

Depends 에는 의존성 함수를 **괄호 없이** 넘겨야 합니다. FastAPI가 요청마다 직접 호출하기 때문입니다. 호출해서 넘기면 의도대로 동작하지 않습니다.

```
# 잘못: 함수를 호출해서 넘김 -> 실행 결과를 넘기게 되어 오동작
page: dict = Depends(get_pagination())

# 올바른: 함수 자체를 넘김 (괄호 없이)
page: dict = Depends(get_pagination)
```

Annotated[T, Depends(get_pagination)] 에서도 마찬가지로 괄호 없이 함수만 넣습니다. 단, 클래스 의존성 Depends(ProductFilter) 도 클래스 자체를 넘기는 것이므로 괄호가 없습니다.

오개념 2: yield 다음 코드가 핸들러보다 먼저 실행된다

yield 의존성에서 yield 다음 코드(정리)는 핸들러 실행과 응답 처리가 끝난 **뒤에** 실행됩니다. 정리가 핸들러보다 먼저 돈다고 착각하면 안 됩니다.

```
def get_session():
    session = open_session()
    try:
        yield session      # (1) 여기서 핸들러로 제어가 넘어감
    finally:
        session.close()   # (3) 응답이 끝난 뒤 실행 (핸들러 (2) 다음)
```

실행 순서는 setup(yield 앞) → 핸들러 → teardown(yield 뒤)입니다. 그래서 핸들러가 세션을 다 쓴 다음에 안전하게 닫힙니다.

오개념 3: 의존성은 매번 새로 실행되니 세션도 매번 새로 열린다

한 요청 안에서 같은 의존성은 **캐시되어 한 번만 실행**됩니다. 그래서 여러 의존성이 get_session 을 요구해도 그 요청에서는 세션이 하나만 열려 공유됩니다.

```
# get_current_user 와 다른 의존성이 모두 get_session 을 요구해도,
# 한 요청에서 get_session 은 한 번만 실행되어 같은 세션을 공유한다.
```

이 덕분에 한 요청의 모든 작업이 같은 트랜잭션 세션에서 일관되게 처리됩니다. 의도적으로 매번 새 인스턴스가 필요하면 Depends(dep, use_cache=False) 로 캐시를 끕니다. 단, 캐시는 **같은 요청 안에서**만 유효하며 요청이 끝나면 사라집니다(요청 간 공유 아님).

그 밖의 주의점

- 의존성 함수도 `async def` 로 만들 수 있습니다. 내부에서 비동기 I/O(`await`)를 한다면 `async def` 의존성으로 선언하십시오(ch_05 연계).
- 반환값이 필요 없고 부수 효과(검증·권한 확인)만 필요한 의존성은 경로 동작 데코레이터의 `dependencies=[Depends(...)]` 에 넣어, 핸들러 인자 없이 실행만 시킬 수 있습니다.
- 의존성은 라우터·앱 레벨에도 걸 수 있습니다(`APIRouter(dependencies=[...])`, ch_08). 그러면 그 라우터의 모든 경로에 공통 적용됩니다.

6.10 단원 미니 프로젝트: 공통 의존성 모듈

이 단원에서 배운 `Depends`, `Annotated` 타입 별칭, `yield` 의존성, 하위 의존성을 한데 모아 **재사용 가능한 공통 의존성 모듈**(`deps.py`) 을 만듭니다. 이 모듈은 종합 프로젝트에서 그대로 발전시켜 쓰게 됩니다.

목표: 페이지네이션 의존성과 `yield` 기반 세션 의존성을 정의하고, `Annotated` 타입 별칭으로 묶어 여러 핸들러에 주입합니다.

마일스톤(증분)으로 진행합니다.

- **M1:** `get_pagination` 공통 쿼리 의존성을 정의하고 목록 핸들러에 주입
- **M2:** `Annotated` 타입 별칭(`PaginationDep`)으로 시그니처 단순화
- **M3:** `yield` 의존성 `get_session` 으로 세션 `setup/teardown` 구현

모범 답안 (`deps.py` + `main.py`)

```
# deps.py — 공통 의존성 모듈
from typing import Annotated
from fastapi import Depends, Query
from pydantic import BaseModel

class Pagination(BaseModel):
    limit: int
    offset: int

def get_pagination(
    limit: int = Query(20, ge=1, le=100),
    offset: int = Query(0, ge=0),
) -> Pagination:
    return Pagination(limit=limit, offset=offset)

class FakeSession:
    def __init__(self) -> None:
        self.items: list[str] = []

    def close(self) -> None:
        print("세션 닫힘")

def get_session():
    session = FakeSession()      # setup
    try:
        yield session
    finally:
        session.close()         # teardown

# 타입 별칭: 핸들러에서 재사용
PaginationDep = Annotated[Pagination, Depends(get_pagination)]
SessionDep = Annotated[FakeSession, Depends(get_session)]
```

```
# main.py — 의존성을 주입받는 앱
from fastapi import FastAPI
from deps import PaginationDep, SessionDep

app = FastAPI(title="공통 의존성 모듈 데모")

PRODUCTS = [{"id": i, "name": f"상품{i}"} for i in range(1, 51)]

@app.get("/products")
async def list_products(page: PaginationDep, session: SessionDep):
    start = page.offset
    items = PRODUCTS[start : start + page.limit]
    return {"limit": page.limit, "offset": page.offset, "items": items}
```

```
GET /products?limit=3&offset=0
200 OK
{"limit": 3, "offset": 0, "items": [{"id":1,...}, {"id":2,...}, {"id":3,...}]}
(콘솔) 세션 닫힘      # 응답 후 teardown
```

직접 작성: 공통 의존성 모듈 (빈 스타터)

deps.py의 빈칸을 채워 모듈을 완성해 보십시오. 핵심은 `yield` 의존성의 정리(teardown)와 타입 별칭입니다.

```
# deps.py
from typing import Annotated
from fastapi import Depends, Query
from pydantic import BaseModel

class Pagination(BaseModel):
    limit: int
    offset: int

def get_pagination(
    limit: int = Query(20, ge=1, le=100),
    offset: int = Query(0, ge=0),
) -> Pagination:
    return Pagination(limit=limit, offset=offset)

class FakeSession:
    def close(self) -> None:
        print("세션 닫힘")

def get_session():
    session = FakeSession()
    try:
        ____ session          # 자원을 건네는 키워드는?
        ____:                  # 정리를 보장하는 블록은?
        session.____()         # 정리 메서드 호출

# 타입 별칭 두 개를 완성하세요(괄호 없이 함수를 넘길 것).
PaginationDep = Annotated[Pagination, Depends(____)]
SessionDep = Annotated[FakeSession, Depends(____)]
```

확장 과제: `get_current_user` 의존성을 추가하되 내부에서 `SessionDep`를 하위 의존성으로 받게 만들고, 그것을 `CurrentUser` 타입 별칭으로 묶어 보십시오.

자가체크 체크리스트

- [] `get_pagination`이 여러 핸들러에서 재사용되는가

- [] `Annotated` 타입 별칭(`PaginationDep · SessionDep`)으로 시그니처를 단순화했는가
- [] `yield` 의존성에서 `try ... finally`로 `teardown(close)`이 항상 실행되는가
- [] `Depends`에 함수를 괄호 없이 넘겼는가
- [] 응답 후 콘솔에 "세션 닫힘"이 찍혀 정리 시점을 확인했는가

6.11 자주 묻는 질문(FAQ)

Q. Depends와 일반 함수 호출은 무엇이 다른가요?

일반 함수 호출은 핸들러가 직접 함수를 부르고 인자를 챙겨 넘겨야 합니다. `Depends`는 핸들러가 "필요하다"고 선언만 하면 FastAPI가 의존성의 쿼리·헤더·하위 의존성까지 알아서 해석해 실행하고, 결과를 주입하며, OpenAPI 문서에도 반영합니다. 단순 재사용을 넘어 입력 해석·정리·문서화가 함께 따라옵니다.

Q. Annotated 형식과 = Depends() 형식 중 무엇을 써야 하나요?

새 코드는 `Annotated[T, Depends(...)]`를 권장합니다. 타입 별칭으로 재사용하기 쉽고 에디터 타입 추론이 정확합니다. `x: T = Depends(...)` 형식도 여전히 동작하므로 기존 코드는 그대로 두어도 됩니다.

Q. yield 의존성과 return 의존성은 언제 구분해 쓰나요?

다 쓴 뒤 정리(close·해제)가 필요한 자원(DB 세션·파일·외부 클라이언트)은 `yield`를 씁니다. 단순히 값을 계산해 넘기기만 하면 되는 경우(페이지네이션 파라미터 등)는 `return`이면 충분합니다.

Q. 의존성 캐시는 요청 간에도 공유되나요?

아니요. 캐시는 **하나의 요청 안에서만** 유효합니다. 한 요청에서 같은 의존성을 여러 번 쓰면 한 번만 실행해 공유하지만, 다음 요청에서는 다시 새로 실행됩니다. 요청 간에 무언가를 공유하려면 앱 상태나 `lifespan(ch_12)`을 씁니다.

Q. 인증·권한 검사처럼 반환값이 필요 없는 의존성은요?

경로 동작 데코레이터의 `dependencies=[Depends(verify_token)]`에 넣으면 핸들러 인자 없이 실행만 시킬 수 있습니다. 검사를 통과하지 못하면 의존성이 예외를 던져 핸들러에 도달하지 못합니다. `ch_09`의 권한 검사에서 이 형태를 씁니다.

Q. 의존성을 테스트에서 가짜로 바꿀 수 있나요?

네. `app.dependency_overrides[get_session] = fake_get_session`처럼 의존성을 테스트용으로 교체할 수 있습니다. 이것이 의존성 주입이 테스트를 쉽게 만드는 핵심 이유이며, `ch_11`에서 인메모리 DB로

세션을 교체하는 데 그대로 활용합니다.

정리

이번 챗터에서는 **의존성 주입(Depends)** 개념을 깊이 익혔습니다.

- `Depends` 로 공통 로직(페이지네이션)을 의존성 함수로 분리해 여러 핸들러에 주입했습니다.
- `Annotated[T, Depends()]` 권장 형식과 타입 별칭(`PaginationDep`)으로 시그니처를 단순화했습니다.
- `yield` 의존성과 `try ... finally` 로 세션의 `setup-teardown`을 보장했습니다.
- 하위 의존성·클래스 의존성으로 의존성 그래프를 합성하고, 한 요청 안의 의존성 캐시를 이해했습니다.
- "괄호 없이 넘기기", "정리 시점", "요청 단위 캐시"라는 3대 오개념을 점검했습니다.

`yield` 의존성으로 세션을 열고 닫는 패턴이 무엇을 위한 것인지 궁금했을 것입니다. 다음 챗터(ch_07)에서는 바로 그 자리에 진짜 데이터베이스를 연결해, **`SQLModel`로 데이터를 영속화**합니다. 이번에 만든 `get_session` 이 실제 DB 세션으로 자라납니다.

데이터베이스 연동: SQLAlchemy로 영속화

학습 목표

- `engine` 을 만들고 테이블을 생성·관리할 수 있습니다.
- SQLAlchemy의 `table=True` 모델로 엔티티를 선언할 수 있습니다.
- `Session` 으로 `add / commit / refresh` 와 `select()` 조회를 수행할 수 있습니다.
- 세션 의존성으로 요청 범위 트랜잭션 경계를 관리할 수 있습니다.
- 테이블 모델과 입력·출력(Read) 모델을 분리해 내부 필드 노출을 막을 수 있습니다.

바로 앞 6장에서는 **의존성 주입(Depends)** 으로 공통 로직을 함수로 떼어내 경로 동작에 주입하는 법을 배웠습니다. 페이지네이션 파라미터를 `get_pagination` 으로 모으고, `yield` 의존성으로 자원을 열고 닫는 `setup → 핸들러 → teardown` 흐름을 만들었습니다. 그런데 지금까지 우리가 만든 모든 API에는 치명적인 한계가 하나 있었습니다. 상품 목록이 **파이썬 리스트나 딕셔너리에 담긴 채 메모리에만 존재**했다는 점입니다. 서버를 한 번 재시작하면 그동안 등록한 상품이 모두 사라졌습니다. 이 커머스 백엔드라면 있을 수 없는 일입니다.

이 장에서 배우는 **데이터베이스 연동(SQLAlchemy)** 은 바로 그 빈자리를 채웁니다. 상품·주문·사용자 데이터를 디스크에 있는 데이터베이스(database)에 **영속(persist)** 시켜, 서버를 꺾다 켜도 남아 있게 만듭니다. 동시에 6장에서 익힌 `yield` 의존성을 그대로 활용해 "요청마다 세션을 열고, 응답이 끝나면 닫는" 트랜잭션 경계를 자연스럽게 세웁니다. 이 장을 마치면 주문·이커머스 API의 **데이터 계층**이 완성됩니다.

선수 개념 상기 (c06 → c07) 이 장은 6장의 **의존성 주입(c06)** 위에 세워집니다. `Depends` 에 함수를 괄호 없이 넘기던 방식, `Annotated[T, Depends(...)]` 형식, 그리고 무엇보다 `yield` 의존성의 `setup → 핸들러 → teardown` 흐름을 떠올리며 읽으면 "세션 의존성"이 왜 자연스러운지 또렷해집니다. 또 3장의 Pydantic 모델(c03)과 4장의 입출력 모델 분리(c04)도 그대로 이어집니다.

도입: 메모리가 아니라 DB에 남기기

메모리 저장의 한계

지금까지의 상품 목록을 떠올려 봅시다. 보통 이런 모양이었습니.

```
# 지금까지: 메모리에만 사는 상품 목록
products = [
    {"id": 1, "name": "기계식 키보드", "price": 89000},
    {"id": 2, "name": "무선 마우스", "price": 32000},
]
```

이 리스트는 파이썬 프로세스가 살아 있는 동안에만 존재합니다. `uvicorn` 을 멈추면 통째로 사라지고, 워커(worker) 프로세스를 여러 개 띄우면 프로세스마다 **서로 다른 리스트**를 들고 따로 놓입니다. 한 워커에서 등록한 상품이 다른 워커에는 보이지 않습니다. 게다가 "가격이 5만 원 이상인 상품"을 찾으려면 매번 전체를 순회해야 하고, 동시에 두 요청이 같은 리스트를 수정하면 데이터가 깨질 위험도 있습니다.

데이터베이스는 이 모든 문제를 정공법으로 해결합니다. 데이터를 **프로세스 밖의 영속 저장소**에 두고, 조회·필터·정렬을 SQL로 위임하며, 트랜잭션(transaction)으로 동시 변경의 일관성을 지킵니다. 우리가 할 일은 파이썬 객체와 데이터베이스 테이블 사이를 이어 주는 다리, 곧 **ORM(Object-Relational Mapping)** 을 쓰는 것입니다.

왜 SQLAlchemy인가

FastAPI 생태계에는 ORM 선택지가 여럿 있지만, 이 교재는 **SQLModel** 을 씁니다. SQLModel은 FastAPI를 만든 같은 저자가 만든 라이브러리로, 우리가 이미 아는 두 가지를 하나로 합쳐 줍니다.

우리가 이미 아는 것	SQLModel이 합치는 방식
Pydantic 모델(c03-c04) — 검증·직렬화	모델 클래스가 곧 Pydantic 모델이라 그대로 검증·문서화
SQLAlchemy — 테이블·세션·SQL 생성	<code>table=True</code> 한 줄로 같은 클래스가 DB 테이블이 됨

즉 SQLModel을 쓰면 **모델을 한 번만 선언**해도 검증(Pydantic)과 영속화(SQLAlchemy)를 동시에 얻습니다. 3·4장에서 만든 `ProductCreate` · `ProductRead` 의 지식이 그대로 이어진다는 뜻입니다.

정의: SQLModel(SQLAlchemy+Pydantic)과 engine·Session

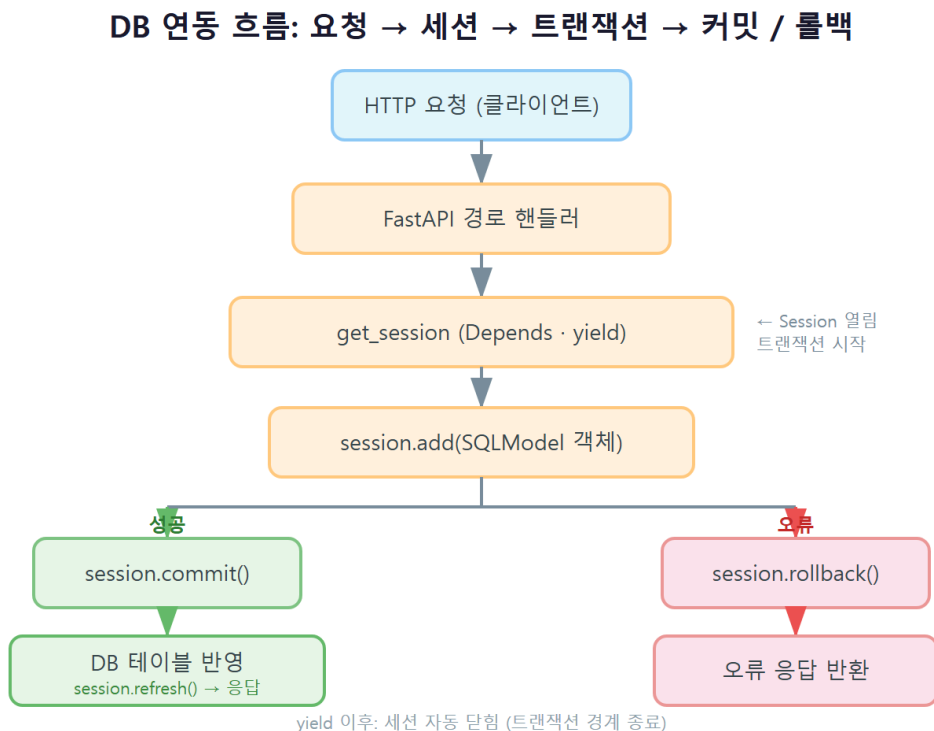
한 문장 정의 데이터베이스 연동(SQLModel) 이란, SQLAlchemy와 Pydantic을 결합한 ORM으로 `table=True` 모델로 테이블을 선언하고 `engine` · `Session` 을 통해 파이썬 객체를 데이터베이스 행(row)으로 영속화·조회하는 방식입니다.

핵심 용어를 한글(영문)으로 정리합니다.

- **ORM(Object-Relational Mapping, 객체-관계 매핑)**: 파이썬 객체와 관계형 DB의 테이블·행을 서로 변환해 주는 계층입니다. SQL 문자열을 손으로 쓰지 않고 파이썬 코드로 DB를 다루게 합니다.
- **엔진(engine)**: 데이터베이스로 가는 연결 풀(connection pool)이자 진입점입니다.
`create_engine(데이터베이스_URL)` 로 한 번 만들어 앱 전역에서 공유합니다.
- **테이블 모델(table model)**: `class Product(SQLModel, table=True)` 처럼 `table=True` 를 준 클래스로, 하나의 테이블에 대응합니다. 각 인스턴스가 한 행(row)입니다.
- **세션(Session)**: 트랜잭션 단위의 작업 공간입니다. `add` 로 객체를 등록하고 `commit` 으로 실제 DB에 반영하며, 요청마다 새로 만들어 쓰고 닫습니다.
- **select()**: 조회용 쿼리를 파이썬으로 표현하는 함수입니다. `select(Product).where(...)` 로 조건·정렬을 붙여 `session.exec(...)` 로 실행합니다.

이 다섯 가지가 데이터 계층의 전부입니다. 엔진은 "DB로 가는 길", 테이블 모델은 "데이터의 모양", 세션은 "이번 요청의 작업 공간", `select` 는 "조회 질문"이라고 기억하면 됩니다.

동작 원리: 요청→세션→트랜잭션→테이블 커밋/롤백



한 번의 HTTP 요청이 데이터베이스에 도달했다가 응답으로 돌아오기까지의 흐름을 단계로 따라가 봅시다.

1. **요청 도착**: 클라이언트가 `POST /products` 로 새 상품을 보냅니다. FastAPI는 본문을 `ProductCreate` 로 검증합니다(c03).
2. **세션 열기**: 경로 동작이 `get_session` 의존성을 요구하면, FastAPI가 그 `yield` 의존성을 실행해 새 `Session` 하나를 만들어 주입합니다(c06). 이 세션이 이번 요청의 **트랜잭션 경계**입니다.
3. **객체 등록**: 핸들러가 `session.add(product)` 로 새 행을 세션에 등록합니다. 아직 DB에는 아무것도 쓰이지 않았습니다. 세션의 메모리에만 "보류 중(pending)" 상태로 담깁니다.
4. **커밋(commit)**: `session.commit()` 을 부르면 그동안 보류된 변경이 **하나의 트랜잭션**으로 DB에 확정됩니다. 도중에 오류가 나면 트랜잭션이 통째로 **롤백(rollback)** 되어 절반만 반영되는 사고를 막습니다.
5. **새로고침(refresh)**: 커밋 직후 객체에는 DB가 자동으로 채운 값(자동 증가 `id`, 생성 시각 등)이 아직 비어 있습니다. `session.refresh(product)` 로 DB의 최신 값을 다시 읽어 와 채웁니다.
6. **응답·세션 닫기**: 핸들러가 객체를 반환하면 `response_model` (c04)이 `ProductRead` 로 필터링해 직렬화합니다. 응답이 끝나면 `yield` 이후 코드가 실행되어 세션이 닫힙니다(teardown).

핵심은 **세션의 수명이 요청의 수명과 같다**는 점입니다. 요청이 시작될 때 열리고 끝날 때 닫히므로, 각 요청은 자기만의 깨끗한 작업 공간에서 일합니다. 이것이 6장 `yield` 의존성을 DB에 적용한 모습입니다.

메서드	하는 일	빠뜨리면
<code>session.add(obj)</code>	객체를 세션에 "보류" 등록	DB에 들어가지 않음
<code>session.commit()</code>	보류된 변경을 트랜잭션으로 확정	변경이 저장되지 않음
<code>session.refresh(obj)</code>	DB가 채운 값(자동 id 등)을 다시 읽음	새 <code>id</code> 가 <code>None</code> 인 채로 남음
<code>session.exec(select(...))</code>	조회 쿼리를 실행	(조회 없음)

worked example: Product 테이블과 상품 저장

이제 실제로 동작하는 코드를 끝까지 만들어 봅시다. 이커머스 도메인의 상품(Product)을 SQLite 데이터베이스에 저장하는 최소 앱입니다. SQLite를 쓰는 이유는 별도 서버 설치 없이 파일 하나로 동작해 실습에 적합하기 때문입니다(운영에서는 같은 코드로 PostgreSQL 등으로 바꿀 수 있습니다).

먼저 모델과 엔진을 정의합니다.

```

# db.py — 엔진과 모델 정의
from sqlmodel import SQLModel, Field, create_engine

class Product(SQLModel, table=True):
    """상품 테이블 — 각 인스턴스가 한 행입니다."""
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    category: str = Field(index=True)
    price: int
    stock: int = 0
    is_active: bool = True

# SQLite 파일 데이터베이스로 가는 엔진(앱 전역에서 한 번만 생성)
DATABASE_URL = "sqlite:///shop.db"
connect_args = {"check_same_thread": False}
engine = create_engine(DATABASE_URL, echo=True, connect_args=connect_args)

def create_db_and_tables() -> None:
    """등록된 모든 table=True 모델에 대응하는 테이블을 만듭니다."""
    SQLModel.metadata.create_all(engine)

```

이제 이 모델로 상품을 저장하고 조회하는 앱을 작성합니다.

```
# main.py — 상품 저장과 조회
from contextlib import asynccontextmanager

from fastapi import FastAPI
from sqlmodel import Session, select

from db import Product, engine, create_db_and_tables

@asynccontextmanager
async def lifespan(app: FastAPI):
    # 앱 시작 시 테이블을 생성합니다(없으면 만들고, 있으면 그대로 둡니다).
    create_db_and_tables()
    yield

app = FastAPI(lifespan=lifespan)

@app.post("/products")
def create_product(name: str, category: str, price: int) -> Product:
    product = Product(name=name, category=category, price=price)
    with Session(engine) as session:
        session.add(product)      # (1) 세션에 보류 등록
        session.commit()         # (2) 트랜잭션 확정
        session.refresh(product)  # (3) DB가 채운 id를 다시 읽음
    return product

@app.get("/products")
def list_products() -> list[Product]:
    with Session(engine) as session:
        products = session.exec(select(Product)).all()
    return list(products)
```

실행하고 상품 하나를 등록한 뒤 목록을 조회하면 다음과 같습니다.

```
$ uvicorn main:app --reload
INFO: Application startup complete.

# POST /products?name=기계식 키보드&category=keyboard&price=89000
{"id":1,"name":"기계식 키보드","category":"keyboard","price":89000,"stock":0,"is_active":true}

# GET /products
[{"id":1,"name":"기계식 키보드","category":"keyboard","price":89000,"stock":0,"is_active":true}]
```

id 가 1로 채워진 점에 주목하십시오. 우리는 id 를 보내지 않았지만 DB가 자동 증가시킨 값을 refresh 가 읽어 왔습니다. 이제 서버를 재시작해도 shop.db 파일에 상품이 그대로 남아 있습니다. 메모리 저장과 결정적으로 다른 점입니다.

줄별 해설

코드	하는 일
<code>class Product(SQLModel, table=True)</code>	이 클래스를 DB 테이블로 등록합니다. <code>table=True</code> 가 핵심입니다.
<code>id: int \ None = Field(default=None, primary_key=True)</code>	기본키(primary key)이며, DB가 자동 증가시키므로 등록 시엔 <code>None</code> 입니다.
<code>name: str = Field(index=True)</code>	이름으로 자주 조회하므로 인덱스(index)를 겁니다.
<code>create_engine(DATABASE_URL, ...)</code>	DB로 가는 엔진(연결 풀)을 한 번 만듭니다.
<code>check_same_thread: False</code>	SQLite를 여러 요청에서 쓰기 위한 옵션입니다(다른 DB는 불필요).
<code>SQLModel.metadata.create_all(engine)</code>	선언된 모든 테이블을 실제로 생성합니다.
<code>session.add(product)</code>	새 객체를 세션에 보류 등록합니다(아직 미저장).
<code>session.commit()</code>	보류된 변경을 트랜잭션으로 DB에 확정합니다.
<code>session.refresh(product)</code>	DB가 채운 자동 <code>id</code> 를 객체에 다시 채웁니다.
<code>session.exec(select(Product)).all()</code>	전체 상품을 조회해 리스트로 받습니다.

`echo=True`로 **SQL 들여다보기** 엔진을 `create_engine(..., echo=True)`로 만들면 `SQLModel`이 생성하는 실제 SQL이 콘솔에 찍힙니다. `INSERT INTO product ...`, `SELECT product.id ...` 같은 문장을 직접 보면 ORM이 무엇을 하는지 한눈에 이해됩니다. 운영에서는 로그가 너무 많아지므로 `echo=False`로 둡니다.

select()로 필터·정렬 조회

전체 조회만으로는 실무를 감당할 수 없습니다. `select()`에 조건을 붙여 "특정 카테고리", "가격 순 정렬", "활성 상품만" 같은 질의를 표현합니다. `select`는 우리가 3장에서 다룬 모델의 속성을 그대로 비교 대상으로 씁니다.

```

from sqlmodel import Session, select
from db import Product, engine

with Session(engine) as session:
    # (1) where: 카테고리 필터
    stmt = select(Product).where(Product.category == "keyboard")
    keyboards = session.exec(stmt).all()

    # (2) 여러 조건(AND): 활성 + 가격 하한
    stmt = select(Product).where(
        Product.is_active == True,
        Product.price >= 50000,
    )
    pricey = session.exec(stmt).all()

    # (3) 정렬 + 개수 제한
    stmt = (
        select(Product)
        .order_by(Product.price.desc())
        .limit(5)
    )
    top5 = session.exec(stmt).all()

    # (4) 단건 조회: 기본키로 바로 찾기
    one = session.get(Product, 1)  # id=1인 상품(없으면 None)

```

`where`에 인자를 여러 개 넘기면 모두 만족하는 조건(AND)이 됩니다. `order_by(...desc())`로 내림차순 정렬하고, `limit`으로 개수를 자릅니다. 특히 **기본키 단건 조회**는 `select`보다 `session.get(Model, 키)`가 간결합니다. 이 패턴들은 다음 8장에서 페이지네이션·필터 라우터의 토대가 됩니다.

exec vs execute SQLAlchemy은 `session.exec(...)`를 쓰며, 결과가 모델 객체 그대로 나오도록 타입을 보존합니다. SQLAlchemy의 `session.execute(...)`와 이름이 비슷하지만 반환 형태가 다르니, SQLAlchemy 코드에서는 `exec`를 쓴다고 기억하십시오.

worked example: get_session 의존성으로 트랜잭션 관리

앞의 예제는 핸들러마다 `with Session(engine) as session:`을 직접 열었습니다. 동작은 하지만 반복적이고, 테스트에서 세션을 교체하기도 어렵습니다. 6장에서 배운 `yield` 의존성을 쓰면 세션 생성·정리를 한곳에 모으고 핸들러는 깔끔해집니다. 동시에 4장의 **입력·출력 모델 분리(c04)**도 함께 적용해, 테이블 모델을 직접 노출하지 않습니다.


```

# 입력·출력 모델 분리 + 세션 의존성
from typing import Annotated

from fastapi import Depends, FastAPI, HTTPException
from sqlmodel import Session, SQLModel, Field, create_engine, select

engine = create_engine(
    "sqlite:///shop.db",
    connect_args={"check_same_thread": False},
)

# (A) 공통 필드를 가진 베이스 — 입력·출력 모델이 함께 상속합니다.
class ProductBase(SQLModel):
    name: str
    category: str
    price: int
    stock: int = 0

# (B) 테이블 모델 — DB에 저장되는 실체(내부 필드 포함)
class Product(ProductBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    cost: int = 0          # 원가: 내부 전용, 외부에 노출 금지

# (C) 입력 모델 — 클라이언트가 보내는 본문
class ProductCreate(ProductBase):
    cost: int = 0

# (D) 출력 모델 — 응답에 내보낼 필드만(cost 제외)
class ProductRead(ProductBase):
    id: int

# (E) 세션 의존성: 요청마다 세션을 열고 끝나면 닫습니다(c06 yield).
def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]

app = FastAPI()

@app.post("/products", response_model=ProductRead, status_code=201)
def create_product(payload: ProductCreate, session: SessionDep):
    product = Product.model_validate(payload) # 입력 모델 → 테이블 모델
    session.add(product)
    session.commit()
    session.refresh(product)
    return product

@app.get("/products/{product_id}", response_model=ProductRead)

```

```
def get_product(product_id: int, session: SessionDep):
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    return product
```

실행 결과는 다음과 같습니다. 응답에 `cost` (원가)가 빠진 점을 확인하십시오.

```
# POST /products (본문: {"name":"무선 마우스","category":"mouse","price":32000,"cost":18000})
HTTP 201 Created
{"name":"무선 마우스","category":"mouse","price":32000,"stock":0,"id":2}

# GET /products/2
HTTP 200 OK
{"name":"무선 마우스","category":"mouse","price":32000,"stock":0,"id":2}

# GET /products/999
HTTP 404 Not Found
{"detail":"상품을 찾을 수 없습니다"}
```

줄별 해설

코드	하는 일
<code>class ProductBase(SQLModel)</code>	입력·출력·테이블이 공유하는 공통 필드 묶음입니다.
<code>class Product(ProductBase, table=True)</code>	베이스 + 내부 필드(<code>cost</code>)를 더한 실제 테이블 모델입니다.
<code>class ProductCreate(ProductBase)</code>	클라이언트가 보내는 입력 모델입니다.
<code>class ProductRead(ProductBase)</code>	응답 전용 출력 모델로 <code>cost</code> 가 없어 노출되지 않습니다.
<code>def get_session(): ... yield session</code>	<code>yield</code> 의존성으로 세션을 열고 응답 후 닫습니다(c06).
<code>SessionDep = Annotated[Session, Depends(get_session)]</code>	타입 별칭으로 핸들러 시그니처를 단순화합니다.
<code>Product.model_validate(payload)</code>	입력 모델을 테이블 모델로 변환합니다.
<code>response_model=ProductRead</code>	반환 객체를 출력 모델로 필터링해 직렬화합니다(c04).
<code>session.get(Product, product_id)</code>	기본키로 단건 조회, 없으면 <code>None</code> 입니다.
<code>raise HTTPException(status_code=404, ...)</code>	없는 자원에 404를 반환합니다.

세 가지 모델(테이블·입력·출력)을 분리한 덕분에, **DB에는 원가를 저장하되 응답에는 절대 내보내지 않는** 구조가 자연스럽게 만들어졌습니다. 이것이 4장 입출력 분리와 7장 영속화가 만나는 지점입니다.

테이블/입력/출력 모델 분리와 마이그레이션 개요

위 예제에서 본 상속 구조를 한 번 더 정리합니다. `SQLModel`에서 권장하는 표준 패턴입니다.

모델	역할	table=True	포함 필드
ProductBase	공통 필드 정의(상속용)	아니오	name·category·price·stock
Product	DB 테이블(실체)	예	베이스 + id + cost(내부)
ProductCreate	요청 본문(입력)	아니오	베이스 + cost(등록 시 입력)
ProductRead	응답(출력)	아니오	베이스 + id (cost 제외)

이 분리가 주는 이점은 분명합니다. 테이블 모델을 그대로 `response_model` 로 쓰면 관계 필드나 내부 컬럼이 응답으로 새어 나갈 수 있는데, `Read` 모델을 따로 두면 **무엇을 내보낼지 명시적으로 통제**됩니다.

마이그레이션 개요

`SQLModel.metadata.create_all` 은 "테이블이 없으면 만든다"만 할 뿐, **이미 있는 테이블의 구조를 바꾸지는 못합니다**. 개발 초기에는 충분하지만, 운영 중 컬럼을 추가·변경하려면 **마이그레이션 (migration)** 도구가 필요합니다. `SQLModel/SQLAlchemy` 생태계에서는 **Alembic** 이 표준입니다. 이 교재의 범위에서는 `create_all` 로 시작하되, "스키마가 바뀌면 Alembic으로 버전 관리한다"는 사실만 알아 두면 됩니다.

`create_all` 은 **스키마 변경을 못 합니다** 모델에 새 필드를 추가하고 `create_all` 만 다시 불러도, **이미 존재하는 테이블은 그대로**입니다. 새 컬럼이 반영되지 않아 조회 시 오류가 날 수 있습니다. 실습 중 모델을 크게 바꿨다면 개발용 DB 파일(`shop.db`)을 지우고 다시 만들거나, 운영이라면 Alembic 마이그레이션을 작성해야 합니다.

즉시 연습(2종 1세트): 저장·조회 채우기

직접 작성 1: 상품 저장·조회 빈칸 채우기

아래 빈칸(____)을 채워, `Order` 테이블에 주문을 저장하고 조회하는 코드를 완성하십시오. 7장의 `add / commit / refresh` 와 `select` 를 그대로 적용하면 됩니다.

```

from typing import Annotated
from fastapi import Depends, FastAPI
from sqlmodel import SQLModel, Field, Session, create_engine, select

engine = create_engine(
    "sqlite:///shop.db",
    connect_args={"check_same_thread": False},
)

class Order(SQLModel, table=True):
    id: int | None = Field(default=None, index=True)  # (1) 기본키 옵션
    user_id: int
    total: int
    status: str = "pending"

def get_session():
    with Session(engine) as session:
        ____ session  # (2) yield? return?

SessionDep = Annotated[Session, Depends(get_session)]
app = FastAPI()

@app.post("/orders")
def create_order(user_id: int, total: int, session: SessionDep) -> Order:
    order = Order(user_id=user_id, total=total)
    session.____(order)  # (3) 세션에 보류 등록
    session.____()  # (4) 트랜잭션 확정
    session.____(order)  # (5) 자동 id 다시 읽기
    return order

@app.get("/orders")
def list_orders(session: SessionDep) -> list[Order]:
    orders = session.exec(____(Order)).all()  # (6) 조회 쿼리 함수
    return list(orders)

```

기대 동작:

```

# POST /orders?user_id=7&total=121000
{"id":1,"user_id":7,"total":121000,"status":"pending"}

# GET /orders
[{"id":1,"user_id":7,"total":121000,"status":"pending"}]

```

직접 작성 2: 필터 조회를 백지에서 작성

요구사항만 보고 `select` 를 직접 작성해 보십시오.

1. `get_session` 의존성을 주입받는 `GET /orders/search` 핸들러를 만든다.
2. 쿼리 파라미터 `user_id` (필수)와 `status` (선택, 기본 `None`)를 받는다.
3. `select(Order).where(Order.user_id == user_id)` 로 사용자별 주문을 조회한다.
4. `status` 가 주어지면 `.where(Order.status == status)` 조건을 추가한다.
5. `Order.id` 내림차순(`desc()`)으로 정렬해 최신 주문이 먼저 오게 한다.
6. 결과를 리스트로 반환한다.

```
@app.get("/orders/search")
def search_orders(
    user_id: int,
    session: SessionDep,
    status: str | None = None,
) -> list[Order]:
    stmt = ---- # 여기서부터 직접 작성
    # status가 있으면 조건 추가
    # id 내림차순 정렬
    # 실행해 리스트로 반환
```

자가체크 (스스로 점검)

- [] `commit` 후 서버를 재시작해도 데이터가 남아 있습니까?
- [] `refresh` 로 자동 생성된 `id` 를 응답에서 확인했습니까?
- [] 테이블 모델을 직접 노출하지 않고 `Read` 모델로 응답을 분리했습니까?
- [] `get_session` 을 `yield` 의존성으로 만들어 요청마다 세션이 열리고 닫힙니까?
- [] `where` 를 여러 번 이어 붙여 조건을 합성할 수 있었습니까?

흔한 오류와 오개념: commit/refresh·모델 노출·세션 공유

오개념 1: "add 만 하면 저장된다" `session.add(obj)` 는 객체를 세션에 보류 등록할 뿐, DB에는 아무 것도 쓰지 않습니다. 반드시 `session.commit()` 을 불러야 트랜잭션이 확정되어 실제로 저장됩니다. "추가했는데 DB에 없다"는 사고의 대부분은 `commit` 누락입니다.

```
session.add(product)
# session.commit() <- 빠뜨리면 저장 안 됨!
```

오개념 2: "commit 후 객체의 id 를 바로 쓸 수 있다" 자동 증가 `id` 는 DB가 채우는 값이라, `commit` 직후 객체에는 아직 반영돼 있지 않을 수 있습니다. `session.refresh(obj)` 로 DB의 최신 값을 다시 읽어야 `obj.id` 가 채워집니다. `refresh` 를 빠뜨리면 응답의 `id` 가 `None` 으로 나가거나 접근 시 오류가 납니다.

오개념 3: "테이블 모델을 그대로 `response_model` 로 써도 된다" `table=True` 모델을 응답에 직접 쓰면 원가(`cost`)나 비밀번호 해시 같은 내부 필드가 새어 나갑니다. 또 관계(`relationship`) 필드가 의도치 않게 직렬화되며 성능 문제를 부르기도 합니다. 반드시 `ProductRead` 같은 출력 모델을 분리해 `response_model` 로 지정하십시오.

오개념 4: "세션은 하나 만들어 전역에서 공유하면 효율적이다" `Session` 은 여러 요청·스레드가 공유하면 안 됩니다. 한 세션을 동시에 여러 요청이 건드리면 트랜잭션이 뒤섞이고 스레드 안전성 문제가 생깁니다. 공유하는 것은 `engine` (연결 풀)이고, 세션은 `get_session` 의존성으로 요청마다 새로 만들어야 합니다.

단원 미니 프로젝트: 상품 DB 영속화 (2종 1세트)

이 단원에서 배운 `SQLModel` 개념만으로 상품을 DB에 영속화하는 최소 앱을 완성합니다. 8장에서 이 앱을 라우터로 모듈화하므로, 여기서 데이터 계층을 탄탄히 다져 두면 다음 장이 수월합니다.

목표: `models.py` + `db.py` + `main.py` 로 나누어, `Product` 테이블 모델·엔진·`get_session` 의존성을 갖추고 상품을 `add/commit/refresh` 로 저장하고 `select` 로 조회하는 앱을 만든다.

마일스톤 (증분으로 쌓기)

- **M1:** `Product` 테이블 모델과 `engine.create_all` 을 구성하고, 앱 시작(`lifespan`) 시 테이블을 만든다.
- **M2:** `get_session(yield)` 의존성으로 상품을 저장한다(`add/commit/refresh`), 입력은 `ProductCreate`, 응답은 `ProductRead`.
- **M3:** `select(Product).where(...)` 로 카테고리 필터 조회를 추가한다.

모범 골격 (M1~M3 완성 형태)

```

# db.py
from sqlalchemy import SQLModel, Field, Session, create_engine

engine = create_engine(
    "sqlite:///shop.db",
    connect_args={"check_same_thread": False},
)

class ProductBase(SQLModel):
    name: str
    category: str
    price: int
    stock: int = 0

class Product(ProductBase, table=True):
    id: int | None = Field(default=None, primary_key=True)

class ProductCreate(ProductBase):
    pass

class ProductRead(ProductBase):
    id: int

def create_db_and_tables() -> None:
    SQLModel.metadata.create_all(engine)

def get_session():
    with Session(engine) as session:
        yield session

```



```

# main.py
from contextlib import asynccontextmanager
from typing import Annotated

from fastapi import Depends, FastAPI
from sqlmodel import Session, select

from db import (Product, ProductCreate, ProductRead,
                create_db_and_tables, get_session)

@asynccontextmanager
async def lifespan(app: FastAPI):
    create_db_and_tables()
    yield

app = FastAPI(lifespan=lifespan)
SessionDep = Annotated[Session, Depends(get_session)]

@app.post("/products", response_model=ProductRead, status_code=201)
def create_product(payload: ProductCreate, session: SessionDep):
    product = Product.model_validate(payload)
    session.add(product)
    session.commit()
    session.refresh(product)
    return product

@app.get("/products", response_model=list[ProductRead])
def list_products(session: SessionDep, category: str | None = None):
    stmt = select(Product)
    if category is not None:
        stmt = stmt.where(Product.category == category)
    return list(session.exec(stmt).all())

```

직접 작성용 빈 스타터

아래 빈칸을 채워 같은 앱을 완성하십시오. 모범 골격을 보지 않고 먼저 시도해 보길 권합니다.

```
# db.py
from sqlmodel import SQLModel, Field, Session, create_engine

engine = create_engine("sqlite:///shop.db",
                       connect_args={"check_same_thread": False})

class ProductBase(SQLModel):
    name: str
    category: str
    price: int
    stock: int = 0

class Product(ProductBase, __slots__=True):  # (1) 테이블 선언
    id: int | None = Field(default=None, __slots__=True)  # (2) 기본키

class ProductCreate(ProductBase):
    pass

class ProductRead(ProductBase):
    id: int

def create_db_and_tables() -> None:
    SQLModel.metadata.create_all(engine)  # (3) 테이블 생성

def get_session():
    with Session(engine) as session:
        ____ session  # (4) 의존성 산출
```

작성 후 `uvicorn main:app --reload` 로 실행하고, `/docs` 에서 `POST /products` 로 상품을 등록한 뒤 `GET /products` 로 조회·카테고리 필터를 확인하십시오.

자가체크 (스스로 점검)

- [] 앱 시작 시 테이블이 생성되고 `shop.db` 파일이 만들어집니까?
- [] `POST /products` 가 201과 함께 자동 `id` 가 채워진 응답을 돌려줍니까?
- [] 서버를 재시작해도 등록된 상품이 `GET /products` 에 남아 있습니까?
- [] `?category=keyboard` 필터가 해당 카테고리만 반환합니까?
- [] 응답이 `ProductRead` 로 직렬화되어 내부 필드가 노출되지 않습니까?

자주 묻는 질문(FAQ)

Q. SQLite로 배우면 운영 DB(PostgreSQL)로 못 옮기나요? A. 옮길 수 있습니다. SQLAlchemy은 SQLAlchemy 위에 있어, DATABASE_URL 만 postgresql+psycopg://... 로 바꾸면 같은 모델·세션 코드가 그대로 동작합니다. check_same_thread 는 SQLite 전용 옵션이라 PostgreSQL에선 빼면 됩니다.

Q. Session(engine) 을 직접 열 때와 get_session 의존성, 무엇을 써야 하나요? A. 실제 엔드포인트에서는 get_session 의존성을 쓰는 것이 표준입니다. 세션 생성·정리를 한곳에 모으고, 11장 테스트에서 dependency_overrides 로 인메모리 DB로 교체하기도 쉽습니다. 직접 여는 방식은 스크립트나 시드(seed) 데이터 작성 같은 일회성 작업에 적합합니다.

Q. commit 이 실패하면 어떻게 되나요? A. 트랜잭션이 롤백되어 그 세션에서 보류 중이던 모든 변경이 취소됩니다. 절반만 저장되는 일은 없습니다. 10장에서 배울 예외 처리와 결합하면, 재고 차감 같은 여러 단계 작업을 "전부 성공하거나 전부 취소"로 안전하게 묶을 수 있습니다.

Q. index=True 는 꼭 필요한가요? A. 필수는 아니지만, 자주 조회·필터하는 컬럼(상품 이름·카테고리)에 인덱스를 두면 조회 성능이 크게 좋아집니다. 반대로 모든 컬럼에 인덱스를 달면 쓰기가 느려지므로, 조회 패턴을 보고 선택적으로 답니다.

마무리

이 장에서는 **데이터베이스 연동(SQLModel)** 으로 데이터를 메모리에서 디스크로 옮겨 영속화했습니다. engine 으로 DB에 연결하고, table=True 모델로 테이블을 선언하며, Session 의 add/commit/refresh 로 저장하고 select 로 조회했습니다. 6장의 yield 의존성을 get_session 으로 적용해 요청 범위 트랜잭션을 세웠고, 4장의 입출력 모델 분리로 내부 필드를 안전하게 숨겼습니다.

다음 8장에서는 이 데이터 계층 위에 **APIRouter로 자원별 CRUD 라우터**를 올려, 상품 모듈을 분리하고 페이지네이션·필터·404·PATCH 부분 갱신까지 갖춘 실무 수준의 API로 완성합니다.

CRUD 라우터: APIRouter·페이지네이션·필터

학습 목표

- APIRouter 로 자원별 라우트를 모듈로 분리하고 앱에 포함할 수 있습니다.
- 표준 CRUD(생성·목록·단건·수정·삭제)를 일관되게 구현할 수 있습니다.
- limit / offset 페이지네이션과 쿼리 기반 필터·정렬을 적용할 수 있습니다.
- 없는 자원에 404, 부분 갱신에 PATCH (exclude_unset) 를 올바르게 처리할 수 있습니다.

바로 앞 7장에서는 **데이터베이스 연동(SQLModel)** 으로 상품을 DB에 영속화하고, `get_session` 의존성으로 요청 범위 트랜잭션을 세웠습니다. 이제 상품을 저장·조회할 수 있게 됐지만, 모든 엔드포인트가 `main.py` 하나에 뒤엉켜 있다는 문제가 남았습니다. 상품·주문·사용자·인증이 한 파일에 쌓이면 수백 줄짜리 거대 파일이 되어 협업도, 테스트도 어려워집니다. 실무 백엔드라면 반드시 마주치는 규모의 문제입니다.

이 장에서 배우는 **CRUD 라우터·페이지네이션·필터** 는 그 문제를 정면으로 해결합니다. APIRouter 로 "상품에 관한 모든 라우트"를 한 모듈(`routers/products.py`)로 묶고, 앱은 그 모듈을 `include_router` 로 합치기만 합니다. 동시에 목록 API를 실무 수준으로 다듬습니다. 데이터가 수만 건이 되어도 안전하도록 `limit / offset` 페이지네이션을 넣고, 카테고리·가격으로 필터하고, 없는 상품에 404를, 부분 수정에 PATCH 를 표준 방식으로 처리합니다. 이 장을 마치면 종합 프로젝트의 **상품 모듈**이 완성되어, 9장 인증과 결합할 준비가 끝납니다.

선수 개념 상기 (c07 → c08) 이 장은 7장의 **SQLModel(c07)** 위에 세워집니다. `get_session` 의존성, `add/commit/refresh`, `select(...).where(...)`, 그리고 입력·출력 모델 분리 (`ProductCreate` · `ProductRead`)를 그대로 가져다 라우터에 담습니다. 6장 의존성 주입(c06)과 2장 쿼리 파라미터·`Query()` 제약(c02)도 페이지네이션·필터에서 다시 등장합니다.

도입: 큰 앱을 자원별 모듈로

한 파일에 모든 라우트를 두면

지금까지 우리는 `@app.post(...)`, `@app.get(...)` 을 모두 `main.py` 의 `app` 객체에 직접 달았습니다. 상품만 다룰 때는 괜찮았지만, 종합 프로젝트는 상품·주문·사용자·인증까지 다룹니다. 한 파일에 전부 넣으면 다음이 생깁니다.

- **거대 파일:** 한 파일이 수백 줄로 불어나 어디에 무엇이 있는지 찾기 어렵습니다.
- **충돌:** 여러 명이 같은 파일을 동시에 고치면 병합 충돌이 발생합니다.
- **재사용 곤란:** "상품 라우트만 떼어 다른 서비스에 쓰기"가 불가능합니다.

해법은 자원(resource)별로 파일을 나누는 것입니다. 상품 라우트는 `routers/products.py`, 주문은 `routers/orders.py` 로 두고, 각 파일은 자기 라우트만 담은 **작은 라우터 객체**를 노출합니다. `main.py` 는 그 라우터들을 한데 모으는 **조립 지점**이 됩니다.

구분	한 파일(<code>@app...</code>)	라우터 모듈(<code>APIRouter</code>)
파일 구조	<code>main.py</code> 하나에 전부	자원별 파일로 분리
공통 접두어	라우트마다 <code>/products</code> 반복	<code>prefix="/products"</code> 로 한 번
문서 그룹	평면적	<code>tags</code> 로 자원별 묶음
협업·테스트	충돌·곤란	모듈 단위로 독립

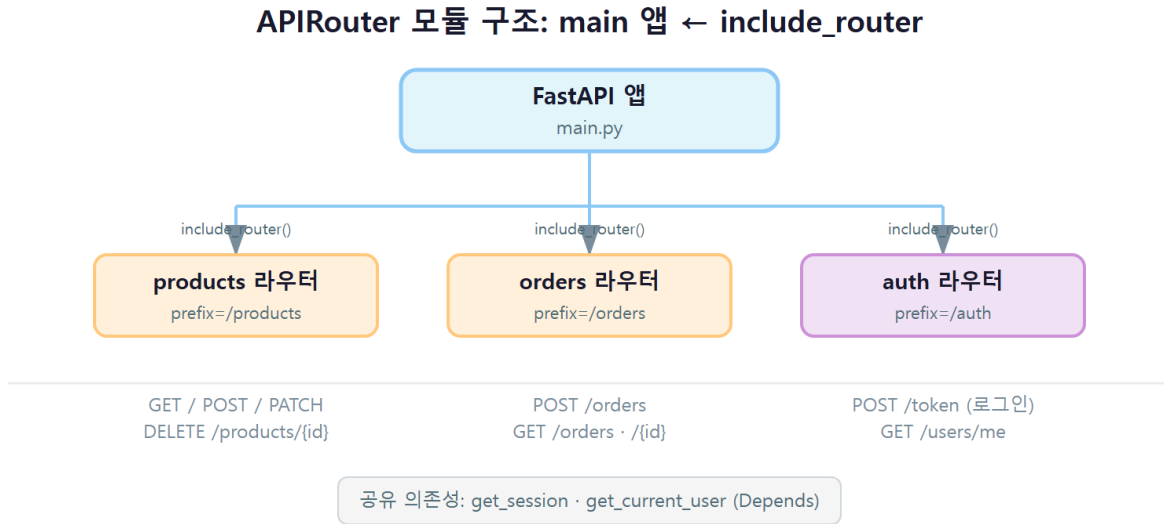
정의: `APIRouter`와 `prefix`·`tags`·`include_router`

한 문장 정의 **CRUD 라우터·페이지네이션·필터**란, `APIRouter` 로 자원별 CRUD 엔드포인트를 모듈로 분리해 `prefix`·`tags` 로 묶고 `include_router` 로 앱에 합치며, `limit` / `offset` 페이지네이션과 필터·정렬로 목록을 실무 수준으로 다듬는 구성 방식입니다.

핵심 용어를 한글(영문)으로 정리합니다.

- **`APIRouter`(API 라우터):** `app` 처럼 경로 동작을 등록할 수 있는 "미니 앱"입니다. `router = APIRouter()` 를 만들고 `@router.get(...)` 으로 라우트를 답니다. 단, 라우터는 스스로 실행되지 않고 앱에 포함되어야 동작합니다.
- **`prefix`(접두어):** `APIRouter(prefix="/products")` 처럼 그 라우터의 모든 경로 앞에 자동으로 붙는 공통 경로입니다. 라우트마다 `/products` 를 반복하지 않게 합니다.
- **`tags`(태그):** `tags=["products"]` 로 지정하면 `/docs` 문서에서 해당 라우트들이 한 그룹으로 묶여 보입니다.
- **`include_router`(라우터 포함):** `app.include_router(products.router)` 로 라우터를 앱에 합칩니다. 이때 비로소 라우터의 경로들이 실제 엔드포인트가 됩니다.
- **CRUD:** Create(생성)·Read(조회)·Update(수정)·Delete(삭제)의 머리글자로, 자원에 대한 표준 4대 연산입니다. HTTP 메서드 `POST`·`GET`·`PATCH` / `PUT`·`DELETE` 에 각각 대응합니다.

동작 원리: main ← include_router 모듈 구조



라우터로 나눈 앱이 조립되고 요청이 처리되는 흐름을 단계로 봅시다.

- 라우터 정의:** routers/products.py 에서 `router = APIRouter(prefix="/products", tags=["products"])` 를 만들고, `@router.post("") · @router.get("/{product_id}")` 등으로 라우트를 합니다. 이 시점에는 아직 앱과 연결되지 않았습니다.
- 앱이 포함:** main.py 에서 `from routers import products` 로 모듈을 불러와 `app.include_router(products.router)` 를 부릅니다. 이 순간 라우터의 경로들이 `prefix` 와 합쳐져 실제 경로(`/products` , `/products/{product_id}`)로 앱에 등록됩니다.
- 요청 라우팅:** 클라이언트가 `GET /products/3` 을 보내면, FastAPI는 등록된 경로 표에서 일치하는 라우트를 찾아 해당 핸들러를 실행합니다. 핸들러는 7장의 `get_session` 의존성으로 세션을 받아 DB를 조회합니다.
- 응답·문서:** 핸들러가 반환한 객체는 `response_model` 로 필터링되어 응답이 되고, 같은 정보가 `/docs` 에서 `tags` 그룹으로 자동 문서화됩니다.

핵심은 **라우터는 "부품", 앱은 "조립 라인"** 이라는 점입니다. 부품을 자원별로 만들어 두고, 앱은 그것들을 포함만 합니다. 표준 CRUD 라우트와 HTTP 메서드의 대응은 다음과 같습니다.

연산	메서드·경로	의미	성공 상태코드
Create	POST /products	새 상품 생성	201
Read(목록)	GET /products	상품 목록(페이지네이션·필터)	200
Read(단건)	GET /products/{id}	한 상품 조회	200 / 404
Update(부분)	PATCH /products/{id}	받은 필드만 수정	200 / 404
Delete	DELETE /products/{id}	상품 삭제	204 / 404

worked example: products 라우터로 표준 CRUD

7장에서 만든 데이터 계층(db.py 의 `Product` · `ProductCreate` · `ProductRead` · `get_session`)을 그대로 재사용해, 상품 CRUD를 별도 라우터 모듈로 구현합니다. 먼저 라우터 파일입니다.


```

# routers/products.py — 상품 자원 라우터
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, status
from sqlmodel import Session, select

from db import Product, ProductCreate, ProductRead, get_session

router = APIRouter(prefix="/products", tags=["products"])
SessionDep = Annotated[Session, Depends(get_session)]

@router.post("", response_model=ProductRead, status_code=status.HTTP_201_CREATED)
def create_product(payload: ProductCreate, session: SessionDep):
    product = Product.model_validate(payload)
    session.add(product)
    session.commit()
    session.refresh(product)
    return product

@router.get("", response_model=list[ProductRead])
def list_products(session: SessionDep):
    return list(session.exec(select(Product)).all())

@router.get("/{product_id}", response_model=ProductRead)
def get_product(product_id: int, session: SessionDep):
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    return product

@router.delete("/{product_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_product(product_id: int, session: SessionDep):
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    session.delete(product)
    session.commit()
    # 204는 본문이 없으므로 아무것도 반환하지 않습니다.

```

이제 `main.py` 는 이 라우터를 포함하기만 합니다.

```
# main.py — 라우터 조립 지점
from contextlib import asynccontextmanager

from fastapi import FastAPI

from db import create_db_and_tables
from routers import products

@asynccontextmanager
async def lifespan(app: FastAPI):
    create_db_and_tables()
    yield

app = FastAPI(lifespan=lifespan)
app.include_router(products.router)
```

실행 결과는 다음과 같습니다.

```
# POST /products (본문: {"name":"기계식 키보드","category":"keyboard","price":89000})
HTTP 201 Created
{"name":"기계식 키보드","category":"keyboard","price":89000,"stock":0,"id":1}

# GET /products/1
HTTP 200 OK
{"name":"기계식 키보드","category":"keyboard","price":89000,"stock":0,"id":1}

# GET /products/999
HTTP 404 Not Found
{"detail":"상품을 찾을 수 없습니다"}

# DELETE /products/1
HTTP 204 No Content
```

줄별 해설

코드	하는 일
<code>router = APIRouter(prefix="/products", tags=["products"])</code>	상품 라우터를 만들고 공통 접두어·문서 태그를 지정합니다.
<code>@router.post("")</code>	경로는 <code>prefix</code> 와 합쳐져 <code>POST /products</code> 가 됩니다(접두어 중복 금지).
<code>status_code=status.HTTP_201_CREATED</code>	생성 성공에 201을 명시합니다(c04).
<code>@router.get("/{product_id}")</code>	단건 조회 경로는 <code>/products/{product_id}</code> 가 됩니다.
<code>session.get(Product, product_id)</code>	기본키로 단건 조회, 없으면 <code>None</code> 입니다(c07).
<code>raise HTTPException(status_code=404, ...)</code>	없는 상품에 404를 던집니다.
<code>@router.delete(..., status_code=204)</code>	삭제 성공은 본문 없는 204를 반환합니다.
<code>session.delete(product); session.commit()</code>	행을 삭제하고 트랜잭션을 확정합니다.
<code>app.include_router(products.router)</code>	라우터를 앱에 합쳐 경로를 실제로 활성화합니다.

`@router.post("")` 의 경로가 빈 문자열(`""`)인 점에 주목하십시오. `prefix="/products"` 가 이미 있으니 여기에 다시 `/products` 를 쓰면 경로가 `/products/products` 로 깨집니다. 이 실수는 아래 오개념 절에서 다시 다룹니다.

limit/offset 페이지네이션

목록 API에서 상품을 **전부** 반환하면, 데이터가 수만 건일 때 응답이 거대해지고 DB·네트워크가 위험해집니다. 실무에서는 한 번에 일부만 잘라 주는 **페이지네이션(pagination)** 이 필수입니다. 가장 표준적인 방식이 `limit` (한 페이지 개수)·`offset` (건너뛴 개수)입니다.

```

from typing import Annotated
from fastapi import Query
from sqlmodel import select
from db import Product

# 2장의 Query() 제약으로 안전한 범위를 강제합니다(c02).
@router.get("", response_model=list[ProductRead])
def list_products(
    session: SessionDep,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
):
    stmt = select(Product).offset(offset).limit(limit)
    return list(session.exec(stmt).all())

```

limit=20, offset=0 이면 1~20번째, limit=20, offset=20 이면 21~40번째를 돌려줍니다. Query(ge=1, le=100) 로 limit 상한을 100으로 강제하므로, 클라이언트가 limit=100000 을 보내도 422로 막힙니다. 페이지 번호로 환산하면 offset = (page - 1) * limit 입니다.

요청	의미
GET /products?limit=20&offset=0	1페이지(1~20번째)
GET /products?limit=20&offset=20	2페이지(21~40번째)
GET /products?limit=100000	422 — 상한(100) 초과 거부

6장 의존성으로 페이지네이션 재사용 limit / offset 은 상품·주문·사용자 목록에서 똑같이 쓰입니다. 6장에서 배운 대로 get_pagination 의존성으로 묶어 PaginationDep 타입 별칭을 만들면, 여러 라우터가 같은 파라미터-제약을 재사용합니다. 라우터마다 같은 Query(...) 를 반복하지 않게 됩니다.

worked example: 필터·정렬·PATCH 부분 갱신

이제 목록에 **필터·정렬** 을 더하고, 상품의 일부 필드만 바꾸는 **PATCH 부분 갱신** 을 구현합니다. PATCH 는 받은 필드만 수정해야 하므로, 7장의 입력 모델과 다른 "전부 선택 필드" 모델이 필요합니다.

```

# routers/products.py 에 추가 — 필터·정렬 목록과 PATCH
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlmodel import Session, SQLModel, select

from db import Product, ProductRead, get_session

router = APIRouter(prefix="/products", tags=["products"])
SessionDep = Annotated[Session, Depends(get_session)]

# PATCH 전용 입력 모델 — 모든 필드가 선택(None 기본)입니다.
class ProductUpdate(SQLModel):
    name: str | None = None
    category: str | None = None
    price: int | None = None
    stock: int | None = None

@router.get("", response_model=list[ProductRead])
def list_products(
    session: SessionDep,
    category: str | None = None,
    min_price: Annotated[int | None, Query(ge=0)] = None,
    sort: str = "id",
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
):
    stmt = select(Product)
    if category is not None:
        # (1) 카테고리 필터
        stmt = stmt.where(Product.category == category)
    if min_price is not None:
        # (2) 가격 하한 필터
        stmt = stmt.where(Product.price >= min_price)
    order_col = Product.price if sort == "price" else Product.id
    stmt = stmt.order_by(order_col)
    # (3) 정렬
    stmt = stmt.offset(offset).limit(limit)
    # (4) 페이지네이션
    return list(session.exec(stmt).all())

@router.patch("/{product_id}", response_model=ProductRead)
def update_product(
    product_id: int,
    payload: ProductUpdate,
    session: SessionDep,
):
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    # exclude_unset: 클라이언트가 실제로 보낸 필드만 추립니다(c04).
    changes = payload.model_dump(exclude_unset=True)
    for key, value in changes.items():
        setattr(product, key, value)
        # 받은 필드만 갱신
    session.add(product)
    session.commit()

```

```
session.refresh(product)
return product
```

실행 결과는 다음과 같습니다. PATCH가 보낸 필드(`price`)만 바꾸고 나머지(`name`·`stock`)는 그대로 둔 점을 확인하십시오.

```
# 사전 상태: {"id":1,"name":"기계식 키보드","category":"keyboard","price":89000,"stock":10}

# GET /products?category=keyboard&min_price=50000&sort=price&limit=10
HTTP 200 OK
[{"name":"기계식 키보드","category":"keyboard","price":89000,"stock":10,"id":1}]

# PATCH /products/1 (본문: {"price":79000})
HTTP 200 OK
{"name":"기계식 키보드","category":"keyboard","price":79000,"stock":10,"id":1}

# PATCH /products/999 (본문: {"price":1000})
HTTP 404 Not Found
{"detail":"상품을 찾을 수 없습니다"}
```

줄별 해설

코드	하는 일
<code>class ProductUpdate(SQLModel)</code>	모든 필드가 <code>None</code> 기본인 PATCH 전용 입력 모델입니다.
<code>if category is not None: stmt = stmt.where(...)</code>	값이 주어진 필터만 조건으로 합성합니다.
<code>Product.price if sort == "price" else Product.id</code>	정렬 기준 컬럼을 안전하게 선택합니다.
<code>stmt.offset(offset).limit(limit)</code>	페이지네이션을 조회에 적용합니다.
<code>payload.model_dump(exclude_unset=True)</code>	클라이언트가 실제 보낸 필드만 딕셔너리로 추립니다.
<code>setattr(product, key, value)</code>	보낸 필드만 객체에 반영합니다(나머지 유지).
<code>session.add/commit/refresh</code>	변경을 트랜잭션으로 저장하고 최신 값을 읽습니다(c07).

`exclude_unset=True`가 PATCH의 핵심입니다. 이 옵션이 없으면 `ProductUpdate`의 모든 필드(`None` 포함)가 딕셔너리에 들어가, 보내지 않은 필드까지 `None`으로 덮어써 데이터가 지워집니다. "실제로 보낸 것만 골라낸다"는 점에서 PATCH(부분 갱신)와 PUT(전체 교체)이 갈립니다.

404 처리와 일관된 자원 응답

자원이 없을 때의 처리는 모든 단건·수정·삭제 라우트에서 반복됩니다. "get으로 찾고, `None`이면 404를 던진다"는 패턴을 작은 헬퍼로 묶으면 일관성과 가독성이 함께 좋아집니다.

```
from fastapi import HTTPException
from sqlmodel import Session
from db import Product

def get_product_or_404(session: Session, product_id: int) -> Product:
    """상품을 찾고, 없으면 404를 던지는 공통 헬퍼."""
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(
            status_code=404,
            detail=f"상품 {product_id}을(를) 찾을 수 없습니다",
        )
    return product
```

이제 단건·PATCH·DELETE 핸들러는 `product = get_product_or_404(session, product_id)` 한 줄로 시작하면 됩니다. 404 메시지 형식이 자원 전체에서 통일되고, 같은 if 검사를 반복하지 않게 됩니다. 10장에서 배울 예외 핸들러와 결합하면 이 일관성을 앱 전체로 넓힐 수 있습니다.

즉시 연습(2종 1세트): 페이지네이션·PATCH 채우기

직접 작성 1: 페이지네이션·필터 빈칸 채우기

아래 빈칸(____)을 채워, 카테고리 필터와 페이지네이션을 갖춘 상품 목록 라우트를 완성하십시오. 2장의 `Query()` 제약과 7장의 `select`를 그대로 적용합니다.

```

from typing import Annotated
from fastapi import APIRouter, Depends, Query
from sqlmodel import Session, select
from db import Product, ProductRead, get_session

router = APIRouter(prefix="/products", tags=["products"])
SessionDep = Annotated[Session, Depends(get_session)]

@router.get("", response_model=list[ProductRead])
def list_products(
    session: SessionDep,
    category: str | None = None,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,  # (1) 상한 제약
    offset: Annotated[int, Query(ge=0)] = 0,
):
    stmt = select(Product)
    if category is not None:
        stmt = stmt.where(Product.category == category)  # (2) 조건 추가
    stmt = stmt.offset(offset).limit(limit)  # (3) 페이지네이션
    return list(session.exec(stmt).all())

```

기대 동작:

```

# GET /products?category=mouse&limit=2&offset=0 -> 마우스 최대 2건
# GET /products?limit=99999 -> 422 (상한 100 초과)

```

직접 작성 2: PATCH 부분 갱신을 백지에서 작성

요구사항만 보고 PATCH 핸들러를 직접 작성해 보십시오.

1. ProductUpdate (모든 필드 None 기본) 입력 모델을 만든다.
2. PATCH /products/{product_id} 핸들러를 만들고 get_session 을 주입한다.
3. session.get 으로 상품을 찾고, 없으면 404를 던진다.
4. payload.model_dump(exclude_unset=True) 로 보낸 필드만 추린다.
5. 추린 필드를 setattr 로 반영하고 add/commit/refresh 로 저장한다.
6. response_model=ProductRead 로 갱신된 상품을 반환한다.


```
class ProductUpdate(SQLModel):
    name: str | None = None
    # ... 나머지 선택 필드

@router.patch("/{product_id}", response_model=ProductRead)
def update_product(product_id: int, payload: ProductUpdate,
                   session: SessionDep):
    # 여기서부터 직접 작성
    # (3) 없으면 404
    # (4) exclude_unset로 보낸 필드만
    # (5) setattr + add/commit/refresh
    ...
```

자가체크 (스스로 점검)

- [] prefix 와 경로에 /products 를 중복해 //products 로 깨지지 않았습니까?
- [] 없는 id 에 404가, 잘못된 limit 에 422가 납니까?
- [] PATCH가 보낸 필드만 바꾸고 나머지는 그대로 둡니까?
- [] limit 상한(le=100)이 실제로 강제됩니까?
- [] 라우터를 include_router 로 포함해야 경로가 동작함을 확인했습니까?

흔한 오류와 오개념: 접두어 중복·PATCH vs PUT·전체 반환

오개념 1: "라우터에도 전체 경로를 적어야 한다" APIRouter(prefix="/products") 를 줬는데 라우트에도 @router.get("/products") 를 적으면, 둘이 합쳐져 경로가 /products/products 로 **중복**됩니다. prefix 가 있으면 라우트에는 **나머지 경로만** 적습니다. 목록·생성은 빈 문자열(""), 단건은 "{product_id}" 만 씁니다.

```
router = APIRouter(prefix="/products")
@router.get("/products") # 잘못 -> /products/products
@router.get("")          # 옳바름 -> /products
```

오개념 2: "PATCH도 PUT처럼 전체를 받아 교체하면 된다" PATCH 는 **부분 갱신**입니다. 받은 필드만 바꾸고 나머지는 유지해야 합니다. exclude_unset=True 없이 입력 모델 전체를 반영하면, 클라이언트가 보내지 않은 필드까지 None (또는 기본값)으로 덮여써 데이터가 지워집니다. 전체 교체가 필요하다면 PUT 을 따로 두고, 부분 수정은 반드시 exclude_unset 으로 보낸 필드만 추립니다.

오개념 3: "목록은 전체를 반환하면 편하다" 페이지네이션 없이 select(Product) 전체를 반환하면, 상품이 1건일 때는 멀쩡해 보여도 수만 건이 되는 순간 응답이 거대해지고 DB·메모리·네트워크가 한꺼번에 위험해집니다. 목록 API는 처음부터 limit / offset 을 기본값과 상한(le=100)과 함께 두어, 데이터가 커져도 안전하도록 설계해야 합니다.

오개념 4: "라우터만 정의하면 동작한다" APIRouter 에 라우트를 아무리 많이 달아도, `main.py` 에서 `app.include_router(...)` 로 포함하지 않으면 경로가 존재하지 않습니다(404). 라우터는 부품일 뿐이고, 앱에 조립되어야 비로소 엔드포인트가 됩니다. "분명히 만들었는데 404"의 흔한 원인입니다.

단원 미니 프로젝트: 상품 CRUD 라우터 (2종 1세트)

이 단원에서 배운 개념만으로 **상품 자원의 전체 CRUD와 페이지네이션·필터를 갖춘 라우터 모듈**을 완성합니다. 이것이 종합 프로젝트의 상품 모듈 그대로이며, 다음 9장에서 여기에 관리자 인증(상품 생성·삭제 보호)을 입힙니다.

목표: `routers/products.py` + `main.py` 로 나누어, `APIRouter(prefix="/products", tags=["products"])` 에 생성·목록(페이지네이션·필터)·단건·PATCH·DELETE를 구현하고 `include_router` 로 합친다.

마일스톤 (증분으로 쌓기)

- **M1:** APIRouter 로 products 모듈을 분리하고 `main.py` 에서 `include_router` 로 포함한다(생성·목록 기본형).
- **M2:** 단건 조회·DELETE와 404 처리를 추가한다(`get_product_or_404` 헬퍼).
- **M3:** 목록에 `limit / offset` 페이지네이션·카테고리/가격 필터·정렬과 PATCH 부분 갱신을 더한다.

모범 골격 (M1~M3 완성 형태)

```

# routers/products.py
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlmodel import Session, SQLModel, select

from db import Product, ProductCreate, ProductRead, get_session

router = APIRouter(prefix="/products", tags=["products"])
SessionDep = Annotated[Session, Depends(get_session)]

class ProductUpdate(SQLModel):
    name: str | None = None
    category: str | None = None
    price: int | None = None
    stock: int | None = None

def get_product_or_404(session: Session, product_id: int) -> Product:
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    return product

@router.post("", response_model=ProductRead,
              status_code=status.HTTP_201_CREATED)
def create_product(payload: ProductCreate, session: SessionDep):
    product = Product.model_validate(payload)
    session.add(product)
    session.commit()
    session.refresh(product)
    return product

@router.get("", response_model=list[ProductRead])
def list_products(
    session: SessionDep,
    category: str | None = None,
    min_price: Annotated[int | None, Query(ge=0)] = None,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
):
    stmt = select(Product)
    if category is not None:
        stmt = stmt.where(Product.category == category)
    if min_price is not None:
        stmt = stmt.where(Product.price >= min_price)
    stmt = stmt.order_by(Product.id).offset(offset).limit(limit)
    return list(session.exec(stmt).all())

@router.get("/{product_id}", response_model=ProductRead)
def get_product(product_id: int, session: SessionDep):
    return get_product_or_404(session, product_id)

```

```

@router.patch("/{product_id}", response_model=ProductRead)
def update_product(product_id: int, payload: ProductUpdate,
                  session: SessionDep):
    product = get_product_or_404(session, product_id)
    for key, value in payload.model_dump(exclude_unset=True).items():
        setattr(product, key, value)
    session.add(product)
    session.commit()
    session.refresh(product)
    return product

@router.delete("/{product_id}",
              status_code=status.HTTP_204_NO_CONTENT)
def delete_product(product_id: int, session: SessionDep):
    product = get_product_or_404(session, product_id)
    session.delete(product)
    session.commit()

```

```

# main.py
from contextlib import asynccontextmanager

from fastapi import FastAPI

from db import create_db_and_tables
from routers import products

@asynccontextmanager
async def lifespan(app: FastAPI):
    create_db_and_tables()
    yield

app = FastAPI(lifespan=lifespan)
app.include_router(products.router)

```

직접 작성용 빈 스타터

아래 빈칸을 채워 같은 라우터를 완성하십시오. 모범 골격을 보지 않고 먼저 시도해 보길 권합니다.

```
# routers/products.py
from typing import Annotated
from fastapi import APIRouter, Depends, HTTPException, Query, status
from sqlmodel import Session, SQLModel, select
from db import Product, ProductCreate, ProductRead, get_session

router = APIRouter(____="/products", tags=["products"]) # (1) 접두어 키워드
SessionDep = Annotated[Session, Depends(get_session)]

@router.post("", response_model=ProductRead,
             status_code=status.HTTP_201_CREATED)
def create_product(payload: ProductCreate, session: SessionDep):
    product = Product.model_validate(payload)
    session.add(product); session.commit(); session.refresh(product)
    return product

@router.get("", response_model=list[ProductRead])
def list_products(
    session: SessionDep,
    category: str | None = None,
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
):
    stmt = select(Product)
    if category is not None:
        stmt = stmt.____(Product.category == category) # (2) 필터 조건
    stmt = stmt.offset(offset).limit(limit)
    return list(session.exec(stmt).all())

@router.get("/{product_id}", response_model=ProductRead)
def get_product(product_id: int, session: SessionDep):
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=____, detail="없음") # (3) 상태코드
    return product
```

작성 후 `main.py` 에서 `app.____(products.router)` 로 포함((4) 포함 메서드)하고, `/docs` 에서 생성·목록·단건·필터·페이지네이션을 차례로 시험하십시오.

자가체크 (스스로 점검)

- [] 라우터를 별도 파일로 분리하고 `include_router` 로 포함했습니까?
- [] 경로가 중복 접두어로 깨지지 않습니까?
- [] 생성이 201, 삭제가 204, 없는 자원이 404를 반환합니까?
- [] 목록이 `limit / offset` 과 카테고리·가격 필터를 지원합니까?
- [] PATCH가 보낸 필드만 부분 갱신합니까?

자주 묻는 질문(FAQ)

Q. prefix 를 라우터에 두지 않고 include_router 에서 줘도 되나요? A. 됩니다.

`app.include_router(products.router, prefix="/products", tags=["products"])` 처럼 포함 시점에 줄 수도 있습니다. 라우터 정의부에 두면 라우터가 자기 경로를 알아 독립적이고, 포함 시점에 두면 앱이 배치를 통제합니다. 한쪽에만 두어 중복을 피하면 됩니다.

Q. PATCH와 PUT은 언제 무엇을 쓰나요? A. 받은 필드만 바꾸는 **부분 수정**은 `PATCH`, 자원 전체를 새 표현으로 **교체**하는 것은 `PUT` 입니다. 이커머스 상품처럼 "가격만 수정", "재고만 조정"이 잦은 도메인에서는 `PATCH`가 더 자연스럽습니다.

Q. limit / offset 말고 다른 페이지네이션은 없나요? A. 커서(cursor) 기반 페이지네이션도 널리 쓰입니다. 데이터가 매우 크거나 자주 바뀔 때 `offset` 은 깊은 페이지에서 느려질 수 있어, 마지막으로 본 키를 기준으로 다음 페이지를 가져오는 커서 방식이 유리합니다. 이 교재 범위에서는 표준적이고 단순한 `limit / offset` 을 씁니다.

Q. 삭제는 정말 행을 지워야 하나요? A. 실무에서는 `is_active=False` 로 표시만 하는 **소프트 삭제 (soft delete)** 를 많이 씁니다. 주문 이력 등 참조 무결성과 복구를 위해서입니다. 이 장에서는 개념 학습을 위해 `session.delete` 로 실제 삭제하지만, 종합 프로젝트에서는 상품 비활성화(`is_active`)도 함께 고려합니다.

마무리

이 장에서는 **CRUD 라우터·페이지네이션·필터** 로 앱을 자원별 모듈로 구조화했습니다. `APIRouter` 의 `prefix · tags` 로 상품 라우트를 한 모듈에 묶고 `include_router` 로 앱에 합쳤으며, 표준 CRUD(201 생성·204 삭제·404 처리)를 일관되게 구현했습니다. 목록에는 `limit / offset` 페이지네이션과 카테고리·가격 필터·정렬을 더했고, `PATCH` 는 `exclude_unset` 으로 보낸 필드만 부분 갱신했습니다. 7장 데이터 계층과 결합해 상품 모듈이 실무 수준으로 완성됐습니다.

다음 9장에서는 **인증·인가(OAuth2-JWT)** 로 이 상품 라우터에 보안을 입혀, "관리자만 상품을 생성·삭제"하고 "인증된 사용자만 주문"하도록 보호 라우트를 만듭니다.

9장. 인증·인가: OAuth2와 JWT

이 장에서 다루는 개념은 **인증·인가(OAuth2·JWT)** 하나이지만, 실무 API의 보안을 책임지는 가장 무게 있는 주제입니다. 앞 8장에서 우리는 `APIRouter` 로 상품 자원의 CRUD를 모듈로 분리하고, 없는 자원에 404를 돌려주는 일관된 라우터를 만들었습니다. 그런데 거기에는 큰 구멍이 하나 있었습니다. 바로 **누구든 상품을 생성·삭제할 수 있었다**는 점입니다. 이 장에서는 그 구멍을 막습니다. 요청을 보낸 사람이 **누구인지**(인증, authentication) 확인하고, 그 사람이 그 동작을 **할 권한이 있는지**(인가, authorization) 판단하는 메커니즘을 FastAPI 위에서 표준 방식으로 구현합니다.

이 장의 학습 목표는 다음과 같습니다.

- `OAuth2PasswordBearer` 와 `/token` 로그인 흐름을 구성할 수 있다.
- 비밀번호를 솔트(salt) 포함 해시로 저장·검증할 수 있다.
- JWT를 만료(exp)·서명과 함께 발급하고 검증할 수 있다.
- `get_current_user` 의존성으로 보호·권한 라우트를 구현할 수 있다.

9.1 도입: 누구의 요청인가, 권한이 있는가

8장의 상품 라우터를 떠올려 봅시다. `POST /products` 와 `DELETE /products/{id}` 는 모듈로 잘 분리됐지만, 인터넷에 그대로 배포하면 **익명의 누구라도** 상품을 등록하고 삭제할 수 있습니다. 이커머스 백엔드에서 이는 곧 사고입니다. 일반 구매자가 상품 재고를 마음대로 바꾸거나, 남의 주문을 조회하는 일이 막히지 않기 때문입니다.

이 문제를 풀려면 모든 보호 대상 요청에 대해 두 가지 질문에 답해야 합니다.

1. **인증(authentication)**: 이 요청을 보낸 사람은 누구인가? 정말 본인이 맞는가?
2. **인가(authorization)**: 그 사람은 이 동작을 할 권한이 있는가?

선수 개념 상기: 이 장은 6장의 **의존성 주입(Depends)** 위에 통째로 올라섭니다. "현재 로그인한 사용자"를 알아내는 로직을 `get_current_user` 라는 **의존성 함수** 하나로 만들고, 보호하고 싶은 모든 라우트에 `Depends` 로 주입합니다. 또 8장의 **라우터·404 처리**와 4장의 **입출력 모델 분리**(`UserRead` 로 `password_hash` 를 숨김)도 그대로 활용합니다.

매 요청마다 아이디·비밀번호를 다시 보내는 방식은 위험하고 비효율적입니다. 그래서 실무에서는 **한 번 로그인하면 토큰(token)을 발급**하고, 이후 요청은 그 토큰만 들고 오게 합니다. 이 장은 그 표준 흐름인 **OAuth2 비밀번호 플로우(password flow)**와, 토큰 형식인 **JWT(JSON Web Token)**를 FastAPI로 구현합니다.

9.2 정의: OAuth2PasswordBearer · JWT · 해시

핵심 개념인 **인증·인가(OAuth2-JWT)**를 한 문장으로 정의하면 다음과 같습니다.

정의 — OAuth2PasswordBearer 로 요청의 `Authorization: Bearer <토큰>` 헤더를 받고, `/token` 로그인에서 저장된 **비밀번호 해시(password hash)**를 검증해 서명된 **JWT(PyJWT)**를 발급합니다.

`get_current_user` 의존성으로 그 토큰을 검증해 보호 엔드포인트의 **인증(누구인가, 401)**과 **인가(권한이 있는가, 403)**를 강제합니다.

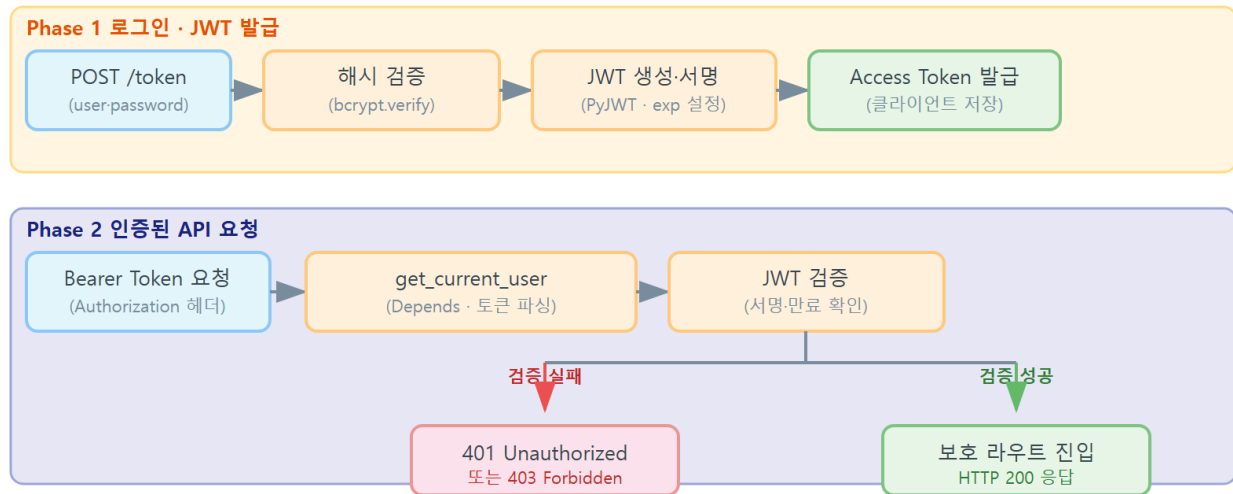
낮선 용어를 하나씩 풀어봅니다.

용어	한글(영문)	의미
인증	認證 (authentication)	요청자가 누구인지 신원을 확인하는 단계. 실패 시 401 Unauthorized .
인가	認可 (authorization)	인증된 사용자가 그 동작을 할 권한 이 있는지 판단. 실패 시 403 Forbidden .
해시	해시(hash)	비밀번호를 되돌릴 수 없는 고정 길이 값으로 변환한 것. 원문 저장 대신 해시를 저장합니다.
솔트	솔트(salt)	해시 전에 섞는 임의 값. 같은 비밀번호라도 매번 다른 해시가 나와 무지개표(rainbow table) 공격을 막습니다.
JWT	JWT(JSON Web Token)	<code>헤더.페이로드.서명</code> 세 부분을 점으로 이은 문자열 토큰. 서버가 서명해 위변조를 검출합니다.
베어러 토큰	베어러(Bearer)	"이 토큰을 가진(bear) 자에게 권한을 준다"는 의미. <code>Authorization: Bearer <토큰></code> 헤더로 전송합니다.
OAuth2 비밀번호 플로우	OAuth2 password flow	아이디·비밀번호를 <code>/token</code> 에 보내 액세스 토큰을 받는 표준 인증 흐름.

`OAuth2PasswordBearer` 는 FastAPI(정확히는 그 토대인 `Starlette: fastapi.security`)가 제공하는 의존성 클래스로, 두 가지 일을 합니다. 첫째, 들어온 요청에서 `Authorization: Bearer ...` 헤더를 꺼내 토큰 문자열만 돌려줍니다. 둘째, OpenAPI 스키마에 "이 API는 `tokenUrl` 에서 토큰을 발급받는다"는 정보를 심어 `/docs` 의 **Authorize** 버튼을 자동으로 만들어 줍니다.

9.3 동작 원리: 로그인에서 검증까지의 시퀀스

인증·인가 흐름: 로그인 → JWT 발급 → Bearer → 검증 → 보호 라우트



인증·인가는 시간 순서가 핵심입니다. 크게 **발급 단계**와 **검증 단계** 둘로 나뉩니다.

발급 단계 (로그인, 한 번만)

- 클라이언트가 `POST /token` 에 `username · password` 를 폼(form) 형식으로 보냅니다.
- 서버가 DB에서 그 사용자를 찾고, 저장된 **비밀번호 해시**와 입력 비밀번호를 비교합니다(해시 검증).
- 일치하면, 사용자 식별자(보통 이메일)를 `sub` 클레임(claim)에 담고 **만료 시각 `exp`**를 더해 **JWT를 서명·발급**합니다.
- 클라이언트는 응답으로 받은 `access_token` 을 저장합니다.

검증 단계 (이후 모든 보호 요청마다)

- 클라이언트가 `Authorization: Bearer <토큰>` 헤더를 붙여 보호 라우트를 호출합니다.
- `OAuth2PasswordBearer` 가 헤더에서 토큰을 꺼냅니다(없으면 401).
- `get_current_user` 의존성이 토큰의 **서명**을 검증하고 **만료(`exp`)**를 확인합니다(위변조·만료면 401).
- 토큰의 `sub` 로 DB에서 사용자를 조회해 "현재 사용자" 객체를 핸들러에 주입합니다.

5. 라우트가 **인가**를 추가로 검사합니다(예: 관리자만 허용, 아니면 403).

이 흐름의 단계와 실패 시 상태 코드를 표로 정리하면 다음과 같습니다.

단계	하는 일	실패 시 응답
헤더 추출	<code>Authorization: Bearer</code> 토큰 꺼내기	401 (토큰 없음)
서명 검증	비밀키로 서명이 위변조되지 않았는지 확인	401 (<code>InvalidTokenError</code>)
만료 확인	<code>exp</code> 가 현재 시각보다 미래인지	401 (<code>ExpiredSignatureError</code>)
사용자 조회	<code>sub</code> 로 DB 사용자 찾기	401 (없는 사용자)
권한 검사	관리자/소유자 여부	403 (권한 없음)

여기서 가장 중요한 직관은 이것입니다. **JWT는 암호화가 아니라 서명입니다**. 페이로드를 누구나 디코드해 읽을 수 있고, 서버는 비밀키로 만든 서명을 보고 "이 토큰을 내가 발급했고 변조되지 않았다"만 보장합니다. 그래서 토큰에 비밀번호 같은 민감 정보를 담으면 안 됩니다.

9.4 worked example: 비밀번호 해시와 `/token` 발급

먼저 비밀번호를 안전하게 저장·검증하고, 로그인에 성공하면 JWT를 발급하는 `/token` 엔드포인트를 완성해 봅시다. 비밀번호 해시는 **pwdlib**(권장)를 사용합니다. `PasswordHash.recommended()` 는 메모리 하드 알고리즘인 **Argon2**를 기본으로 설정하며, 내부적으로 솔트를 포함합니다(`passlib`의 `bcrypt`를 써도 됩니다 — 9.9에서 비교합니다).

설치는 `pip install "pwdlib[argon2]" pyjwt` 입니다.

```

# auth.py - 비밀번호 해시와 토큰 발급
from datetime import datetime, timedelta, timezone

import jwt # PyJWT (python-jose 아님)
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from pwdlib import PasswordHash
from pydantic import BaseModel

# 운영에서는 환경변수로 주입합니다(12장 pydantic-settings). 여기서는 설명용 상수입니다.
SECRET_KEY = "change-me-to-a-long-random-secret"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

# Argon2 기반 권장 설정. 솔트는 해시 안에 포함됩니다.
password_hash = PasswordHash.recommended()

# 학습용 인메모리 사용자 저장소(다음 장에서 SQLAlchemy로 대체).
fake_users = {
    "buyer@shop.com": {
        "email": "buyer@shop.com",
        "password_hash": password_hash.hash("buyer-pw"),
        "is_admin": False,
    },
    "admin@shop.com": {
        "email": "admin@shop.com",
        "password_hash": password_hash.hash("admin-pw"),
        "is_admin": True,
    },
}

class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"

def verify_password(plain: str, hashed: str) -> bool:
    # 평문과 저장된 해시를 상수 시간(constant-time)으로 비교합니다.
    return password_hash.verify(plain, hashed)

def create_access_token(subject: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=ACCESS_TOKEN_EXPIRE_MINUTES
    )
    payload = {"sub": subject, "exp": expire}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

router = APIRouter(tags=["auth"])

@router.post("/token", response_model=Token)
def login(form_data: OAuth2PasswordRequestForm = Depends()) -> Token:
    user = fake_users.get(form_data.username)
    if user is None or not verify_password(

```

```

        form_data.password, user["password_hash"]
    ):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="이메일 또는 비밀번호가 올바르지 않습니다",
            headers={"WWW-Authenticate": "Bearer"},
        )
    token = create_access_token(subject=user["email"])
    return Token(access_token=token)

```

- `import jwt` 는 **PyJWT** 패키지입니다. 과거 자료에 자주 보이는 `python-jose` 가 아니라 `PyJWT` 를 사용합니다(임포트 이름이 `jwt` 인 점에 주의).
- `password_hash = PasswordHash.recommended()` 는 Argon2 기반 권장 해서를 만듭니다. `password_hash.hash("buyer-pw")` 가 솔트를 포함한 해시 문자열을 반환하므로, 같은 비밀번호라도 호출마다 다른 결과가 나옵니다.
- `verify_password` 는 평문과 저장 해시를 비교합니다. `pwdlib`의 `verify(plain, hashed)` 는 인자 순서가 (평문, 해시)이며 상수 시간 비교로 타이밍 공격을 줄입니다.
- `create_access_token` 은 `sub` (식별자)와 `exp` (만료)를 담아 `jwt.encode` 로 서명합니다. `exp` 는 `datetime` 이면 `PyJWT`가 자동으로 UNIX 타임스탬프로 변환합니다. **시간대를 명시** (`timezone.utc`)해 서버 로컬 시간 의존을 없앱니다.
- `/token` 핸들러는 `OAuth2PasswordRequestForm` 을 `Depends()` 로 받습니다. 이 의존성은 `username · password` 를 **JSON이 아니라 `application/x-www-form-urlencoded` 폼**으로 받습니다 (`OAuth2` 사양). 그래서 `form_data.username`, `form_data.password` 로 접근합니다.
- 로그인 실패에는 사용자 존재 여부를 구분하지 않는 **동일한 401 메시지를 돌려줍니다**(계정 존재 여부 노출 방지). `WWW-Authenticate: Bearer` 헤더는 `OAuth2` 표준 권고입니다.

`curl` 로 토큰을 받아 보면 다음과 같습니다.

```

$ curl -X POST http://127.0.0.1:8000/token \
  -d "username=buyer@shop.com&password=buyer-pw"
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI...","token_type":"bearer"}

$ curl -X POST http://127.0.0.1:8000/token \
  -d "username=buyer@shop.com&password=wrong"
{"detail":"이메일 또는 비밀번호가 올바르지 않습니다"}

```

발급된 토큰의 가운데(페이로드) 부분을 Base64 디코드하면

`{"sub":"buyer@shop.com","exp":1735660800}` 처럼 보입니다. **누구나 읽을 수 있다**는 사실을 직접 확인하는 것이 중요합니다 — 그래서 비밀을 담지 않습니다.

9.5 JWT 생성·검증 (PyJWT: `exp` · 서명 · 알고리즘)

앞 예제에서 토큰을 **만드는** 쪽(`jwt.encode`)을 봤으니, 이제 **검증하는** 쪽(`jwt.decode`)을 자세히 봅니다. 검증의 핵심은 세 가지입니다.

1. **서명(signature)**: 같은 `SECRET_KEY · ALGORITHM`으로 서명이 맞는지. 누군가 페이로드를 바꿨다면 서명이 깨집니다.
2. **만료(exp)**: 토큰의 `exp`가 현재 시각보다 미래인지. 지났으면 거부합니다.
3. **알고리즘(algorithm)**: 허용 알고리즘을 **명시적 리스트**로 못 박습니다. 이걸 빠뜨리면 `alg=None` 우회 같은 공격에 노출됩니다.

```
# auth.py 이어서 - 토큰 검증 핵심
def decode_token(token: str) -> str:
    """JWT를 검증하고 sub(사용자 식별자)를 돌려줍니다. 실패 시 예외."""
    try:
        payload = jwt.decode(
            token,
            SECRET_KEY,
            algorithms=[ALGORITHM], # 허용 알고리즘을 리스트로 못 박습니다
        )
    except jwt.ExpiredSignatureError as exc:
        raise ValueError("토큰이 만료되었습니다") from exc
    except jwt.InvalidTokenError as exc:
        # 서명 불일치·형식 오류 등 모든 검증 실패를 포괄합니다
        raise ValueError("토큰이 유효하지 않습니다") from exc

    subject = payload.get("sub")
    if subject is None:
        raise ValueError("토큰에 sub가 없습니다")
    return subject
```

- `jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])`는 **서명 검증·만료 검사·알고리즘 제한**을 한 번에 수행합니다. `algorithms`는 반드시 리스트로 주며, `encode`에서 쓴 알고리즘이 포함돼야 합니다.
- `exp`는 PyJWT가 자동으로 검사합니다. 만료됐으면 `jwt.ExpiredSignatureError`를 던집니다 — 별도 시간 비교 코드를 짤 필요가 없습니다.
- `jwt.InvalidTokenError`는 PyJWT 예외의 **상위 클래스**입니다. 서명 오류(`InvalidSignatureError`)·형식 오류(`DecodeError`) 등을 모두 잡으려면 이 상위 예외를 잡으면 됩니다. `ExpiredSignatureError`도 그 하위지만, 만료는 메시지를 구분하려고 먼저 잡았습니다.
- 여기서는 검증 실패를 일단 `ValueError`로 바꿔 던졌습니다. 다음 절의 `get_current_user`에서 이를 HTTP 401로 변환합니다(도메인 로직과 HTTP 응답을 분리하는 습관).

9.6 worked example: `get_current_user`로 보호 라우트

이제 토큰 추출(`OAuth2PasswordBearer`)과 검증(`decode_token`)을 묶어, **현재 로그인한 사용자**를 돌려주는 의존성 `get_current_user`를 만듭니다. 이 하나를 `Depends`로 주입하면 어떤 라우트든 보호됩니다.

```
# auth.py 이어서 - 현재 사용자 의존성과 보호 라우트
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from pydantic import BaseModel

# tokenUrl은 토큰을 발급하는 경로. /docs의 Authorize 버튼이 이 경로를 씁니다.
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

class UserRead(BaseModel):
    email: str
    is_admin: bool

def get_current_user(
    token: Annotated[str, Depends(oauth2_scheme)],
) -> UserRead:
    credentials_exc = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="자격 증명을 확인할 수 없습니다",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        email = decode_token(token) # 9.5에서 만든 검증 함수
    except ValueError as exc:
        raise credentials_exc from exc

    user = fake_users.get(email)
    if user is None:
        raise credentials_exc
    # 응답·주입에는 password_hash를 제외한 안전한 모델만 노출합니다(4장 입출력 분리).
    return UserRead(email=user["email"], is_admin=user["is_admin"])

# 타입 별칭으로 시그니처를 단순화합니다(6장 Annotated 패턴).
CurrentUser = Annotated[UserRead, Depends(get_current_user)]

protected = APIRouter(tags=["me"])

@protected.get("/users/me", response_model=UserRead)
def read_me(current_user: CurrentUser) -> UserRead:
    # 토큰만 유효하면 여기 도달합니다. current_user는 검증된 사용자입니다.
    return current_user
```

- `oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")`는 요청에서 `Authorization: Bearer` 토큰을 꺼내는 의존성입니다. 헤더가 없으면 FastAPI가 자동으로 **401**을 돌려줍니다(우리가 코

드를 짜지 않아도 됩니다).

- `get_current_user` 는 `token` 을 의존성으로 받아 `decode_token` 으로 검증하고, `sub` (이메일)로 사용자를 조회합니다. 검증 실패·없는 사용자 모두 같은 401(`credentials_exc`)로 통일합니다.
- 반환을 `UserRead` (이메일·관리자 여부만)로 제한해 `password_hash` 가 절대 핸들러로 새지 않게 합니다 — 4장에서 배운 **입력·출력 모델 분리**를 인증에도 적용한 것입니다.
- `CurrentUser = Annotated[UserRead, Depends(get_current_user)]` 는 6장에서 배운 **타입 별칭** 패턴입니다. 이 별칭 하나를 핸들러 인자 타입으로 쓰면 보호 라우트가 한 줄로 깔끔해집니다.
- `/users/me` 는 토큰이 유효한 사용자만 자기 정보를 봅니다. 같은 패턴으로 어떤 라우트든 인자에 `current_user: CurrentUser` 만 추가하면 즉시 보호됩니다.

실행 결과는 다음과 같습니다. 토큰 없이 호출하면 401, 유효한 토큰이면 200입니다.

```
$ curl http://127.0.0.1:8000/users/me
{"detail":"Not authenticated"}

$ TOKEN=$(curl -s -X POST http://127.0.0.1:8000/token \
    -d "username=buyer@shop.com&password=buyer-pw" \
    | python -c "import sys,json;print(json.load(sys.stdin)['access_token'])")
$ curl -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8000/users/me
{"email":"buyer@shop.com","is_admin":false}
```

`/docs` 를 열면 우측 상단에 **Authorize** 버튼이 생깁니다. 거기에 아이디·비밀번호를 넣으면 Swagger UI가 자동으로 `/token` 을 호출하고, 이후 보호 라우트 요청에 `Bearer` 헤더를 붙여 줍니다. 이는 `OAuth2PasswordBearer` 가 OpenAPI 스키마를 채워 준 덕분입니다.

9.7 권한(관리자 vs 일반)과 401 vs 403

여기까지가 **인증**입니다. 이제 **인가**를 더합니다. 이커머스에서 상품 등록·삭제는 **관리자만** 할 수 있어야 합니다. 일반 사용자가 시도하면 막아야 하는데, 이때 401이 아니라 **403**을 돌려주는 것이 핵심입니다.

401 vs 403을 가르는 한 문장 — 401은 "너가 누구인지 모르겠다"(인증 실패: 토큰 없음·만료·위조)이고, 403은 "네가 누구인지는 알지만 이 동작을 **할 권한이 없다**"(인가 실패)입니다. 로그인만 관리자만 할 수 있는 경우엔 403입니다.

관리자 검사도 의존성으로 만들면, 인증 의존성 위에 자연스럽게 **하위 의존성**으로 쌓입니다(6장 의존성 합성).


```
# auth.py 이어서 - 관리자 인가 의존성
def require_admin(current_user: CurrentUser) -> UserRead:
    if not current_user.is_admin:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="관리자 권한이 필요합니다",
        )
    return current_user

AdminUser = Annotated[UserRead, Depends(require_admin)]

admin_router = APIRouter(prefix="/products", tags=["products"])

@admin_router.delete("/{product_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_product(product_id: int, admin: AdminUser) -> None:
    # 여기 도달했다는 것은 (1) 토큰이 유효하고 (2) 관리자라는 뜻입니다.
    # 실제 삭제 로직은 7~8장의 DB·라우터와 결합합니다.
    return None
```

- `require_admin` 은 `get_current_user` 를 하위 의존성으로 받습니다(`CurrentUser` 별칭 안에 `Depends(get_current_user)` 가 들어 있음). 즉 FastAPI가 먼저 토큰을 검증해 사용자를 만든 뒤, 그 사용자를 `require_admin` 에 넘깁니다. 인증→인가가 자동으로 순서대로 일어납니다.
- 관리자가 아니면 **403**을 던집니다. 토큰 자체가 없거나 잘못됐다면 그보다 앞 단계 (`get_current_user`)에서 이미 **401**이 납니다. 두 실패가 자연스럽게 다른 코드로 갈립니다.
- `AdminUser` 별칭을 핸들러 인자에 쓰는 것만으로 `DELETE /products/{id}` 가 관리자 전용이 됩니다. 라우트 코드에는 권한 검사 if문이 보이지 않아 관심사가 깔끔히 분리됩니다.

```
# 일반 사용자(buyer) 토큰으로 관리자 라우트 호출
$ curl -X DELETE -H "Authorization: Bearer $BUYER_TOKEN" \
  http://127.0.0.1:8000/products/42
{"detail": "관리자 권한이 필요합니다"}      # 403 Forbidden

# 관리자(admin) 토큰으로 호출 -> 204 No Content (본문 없음)
$ curl -i -X DELETE -H "Authorization: Bearer $ADMIN_TOKEN" \
  http://127.0.0.1:8000/products/42
HTTP/1.1 204 No Content
```

9.8 즉시 연습 (2종 1세트): 토큰 검증 채우기

배운 검증 흐름을 손으로 완성해 봅니다. 아래는 빈칸 코드입니다. 주석의 지시에 따라 빈칸을 직접 작성하세요.

연습 목표 — `decode_token` 과 `get_current_user` 의 핵심 검증 부분을 빈칸으로 비웠습니다. 토큰을 검증해 현재 사용자를 돌려주도록 채웁니다.

```
# 빈칸 코드 - 직접 작성해 완성하세요
import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from typing import Annotated

SECRET_KEY = "change-me"
ALGORITHM = "HS256"
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

def decode_token(token: str) -> str:
    try:
        # 빈칸 1: 서명·만료·알고리즘을 검증해 payload를 얻으세요
        payload = jwt.decode(_____, _____, algorithms=_____)
    except jwt.ExpiredSignatureError as exc:
        raise ValueError("토큰이 만료되었습니다") from exc
    except jwt.InvalidTokenError as exc:
        raise ValueError("토큰이 유효하지 않습니다") from exc
    # 빈칸 2: payload에서 sub를 꺼내 없으면 ValueError를 던지세요
    subject = _____
    if subject is None:
        _____
    return subject

def get_current_user(token: Annotated[str, Depends(oauth2_scheme)]) -> str:
    credentials_exc = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="자격 증명을 확인할 수 없습니다",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        # 빈칸 3: decode_token으로 이메일을 얻으세요
        email = _____
    except ValueError as exc:
        # 빈칸 4: 검증 실패를 401로 바꿔 던지세요
        raise _____ from exc
    return email
```

자가 체크리스트로 스스로 점검하세요.

- [] `jwt.decode` 에 `algorithms=[ALGORITHM]` 처럼 **리스트**로 알고리즘을 넘겼는가?
- [] 토큰이 없을 때(헤더 누락) `OAuth2PasswordBearer` 가 자동으로 401을 내는 것을 확인했는가?
- [] 만료된 토큰으로 호출하면 401이 나는가? (`exp` 를 과거로 만든 토큰으로 시험)
- [] 위조된 토큰(가운데 페이로드 한 글자 변경)에 401이 나는가?
- [] 검증 실패를 `ValueError` 로 잡아 **401 HTTPException**으로 변환했는가?

두 번째 연습은 인가(권한)까지 확장합니다.

연습 목표(심화) — "본인 주문만 조회" 인가를 구현합니다. `get_current_user` 로 현재 사용자를 받고, 주문의 소유자가 본인인 아니면 **403**을 내는 의존성을 직접 작성하세요.

```
# 빈칸 코드 - 본인 소유 검증
from fastapi import Depends, HTTPException, status

# orders_db[order_id] = {"id": ..., "owner_email": ...}
orders_db = {1: {"id": 1, "owner_email": "buyer@shop.com"}}

def get_owned_order(order_id: int, current_user=Depends(get_current_user)):
    order = orders_db.get(order_id)
    if order is None:
        # 빈칸 1: 없는 주문은 어떤 상태코드가 적절한가?
        raise HTTPException(status_code=_____, detail="주문을 찾을 수 없습니다")
    # 빈칸 2: 소유자가 현재 사용자와 다르면 무슨 상태코드로 막아야 하는가?
    if order["owner_email"] != current_user:
        raise HTTPException(status_code=_____, detail="본인 주문만 접근할 수 있습니다")
    return order
```

자가 체크리스트.

- [] 없는 주문에 **404**, 남의 주문 접근에 **403**을 구분해 돌려주는가?
- [] 토큰이 아예 없으면(인증 실패) 이 의존성에 도달하기 전에 401이 나는가?
- [] `get_owned_order` 를 `Depends` 로 주입한 라우트가 소유자 검증을 통과한 주문만 받는가?

9.9 흔한 오류와 오개념

오개념 1 — "JWT는 암호화라 안에 비밀을 담아도 안전하다" 틀립니다. JWT는 암호화가 아니라 서명입니다. 페이로드는 Base64로만 인코딩돼 누구나 디코드해 읽습니다. 서명은 "내가 발급했고 변조되지 않았다"만 보장합니다. 비밀번호·카드번호 같은 민감 정보를 페이로드에 넣지 마세요.

오개념 2 — "비밀번호를 그냥 해시하면 충분하다" 단순 `sha256(password)` 는 무지개표 공격에 취약합니다. 반드시 **솔트 포함 해시**(Argon2-bcrypt)를 쓰세요. `pwdlib PasswordHash.recommended()` 나 `passlib CryptContext(schemes=["bcrypt"])` 가 솔트를 자동 처리합니다. 평문 저장은 절대 금물입니다.

오개념 3 — "인증과 인가는 같은 것이다" 다릅니다. 인증(누구인가, 401)과 인가(권한이 있는가, 403)는 별개 단계입니다. 로그인은 됐지만 관리자가 아니어서 막히는 경우는 401이 아니라 403입니다. 둘을 401로 뭉뚱그리면 클라이언트가 "재로그인하면 되는지(401)"와 "권한 자체가 없는지(403)"를 구분하지 못합니다.

흔한 오류 4 — `algorithms` 를 빠뜨리거나 문자열로 전달 `jwt.decode(token, key)` 처럼 `algorithms` 를 빠뜨리면 최신 PyJWT는 오류를 내거나 위험합니다. 반드시 `algorithms=["HS256"]` 처럼 **리스트로 명**

시하세요. `alg=none` 우회를 막는 핵심 방어입니다.

흔한 오류 5 — `/token` 을 **JSON 본문으로 받기** OAuth2 비밀번호 플로우 는 `application/x-www-form-urlencoded` 폼을 씁니다. `OAuth2PasswordRequestForm` 을 `Depends()` 로 받아야 하며, Pydantic 본문 모델로 받으면 `/docs` 의 Authorize 버튼과 호환되지 않습니다.

흔한 오류 6 — `SECRET_KEY` 를 코드에 하드코딩 서명 키가 노출되면 누구나 토큰을 위조할 수 있습니다. 키는 충분히 길고 무작위여야 하며(`openssl rand -hex 32`), **환경변수로 주입합니다**(12장 pydantic-settings). 학습 예제의 상수는 운영에 그대로 쓰지 마세요.

9.10 단원 미니 프로젝트: 로그인·보호 라우트 (2종 1세트)

이 단원에서 배운 **인증·인가(OAuth2·JWT)** 개념만으로 작은 인증 모듈을 완성합니다. 산출물은 `auth.py` 와 이를 포함한 `main.py` 이며, 로그인→JWT→보호 라우트→관리자 권한(403)까지 동작합니다.

마일스톤(중분).

단계	목표	사용 개념
M1	비밀번호 해시 저장·검증과 <code>/token</code> 로그인	해시· <code>OAuth2PasswordRequestForm</code>
M2	JWT 발급(exp·서명)과 <code>get_current_user</code> 검증	PyJWT· <code>OAuth2PasswordBearer</code> ·의존성
M3	보호 라우트와 관리자 전용(403) 의존성	인가·하위 의존성·401 vs 403

모범 산출물

```
# main.py - 로그인·보호 라우트 통합 (모범)
from fastapi import FastAPI
from auth import router as auth_router, protected, admin_router

app = FastAPI(title="shop-auth")
app.include_router(auth_router)           # POST /token
app.include_router(protected)            # GET /users/me (인증 필요)
app.include_router(admin_router)         # DELETE /products/{id} (관리자 필요)

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}
```

정상 흐름 실행 결과 요약

- 1) POST /token (buyer) -> 200 {"access_token":"...", "token_type":"bearer"}
- 2) GET /users/me (토큰 O) -> 200 {"email":"buyer@shop.com", "is_admin":false}
- 3) GET /users/me (토큰 X) -> 401 {"detail":"Not authenticated"}
- 4) DELETE /products/42 (buyer)-> 403 {"detail":"관리자 권한이 필요합니다"}
- 5) DELETE /products/42 (admin)-> 204 No Content

학생 작성형 빈 스타터

과제 — 아래 빈 스타터를 채워 `auth.py` 를 완성하고 `main.py` 에 포함하세요. 위 모범의 5가지 실행 결과를 모두 재현하면 성공입니다.

```

# auth.py - 빈 스타터 (직접 작성)
from datetime import datetime, timedelta, timezone
from typing import Annotated

import jwt
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from pwdlib import PasswordHash
from pydantic import BaseModel

SECRET_KEY = "change-me"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
password_hash = PasswordHash.recommended()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

fake_users = {
    # TODO: buyer(is_admin=False)와 admin(is_admin=True)을 password_hash.hash로 등록
}

class Token(BaseModel):
    access_token: str
    token_type: str = "bearer"

class UserRead(BaseModel):
    email: str
    is_admin: bool

def create_access_token(subject: str) -> str:
    # TODO: sub와 exp(만료)를 담아 jwt.encode로 서명해 반환
    ...

def get_current_user(token: Annotated[str, Depends(oauth2_scheme)]) -> UserRead:
    # TODO: 토큰 검증(서명·만료·알고리즘) -> sub로 사용자 조회 -> UserRead 반환
    # 실패는 모두 401(credentials_exc)로 통일
    ...

def require_admin(current_user: Annotated[UserRead, Depends(get_current_user)]) -> UserRead:
    # TODO: is_admin이 아니면 403, 맞으면 current_user 반환
    ...

router = APIRouter(tags=["auth"])
# TODO: POST /token, GET /users/me, DELETE /products/{id}(관리자) 라우트 작성

```

자가 체크리스트로 스스로 점검하세요.

- [] 무토큰 요청이 보호 라우트에서 **401**이 되는가?
- [] 권한 없는(일반) 사용자가 관리자 라우트에서 **403**이 되는가?

- [] 비밀번호를 평문이 아니라 **해시**로 저장했는가? (`fake_users` 에 평문이 없는지 확인)
- [] JWT 디코드 시 `algorithms` 를 리스트로 명시했는가?
- [] `/docs` 의 Authorize 버튼으로 로그인해 보호 라우트를 호출할 수 있는가?

9.11 자주 묻는 질문 (FAQ)

Q. 토큰 만료 시간은 얼마가 적당합니까? 액세스 토큰은 보통 15~30분으로 짧게 둡니다. 더 긴 세션이 필요하면 **리프레시 토큰(refresh token)**을 별도로 발급해 액세스 토큰을 재발급하는 패턴을 씁니다(이 장 범위 밖, 확장 주제).

Q. python-jose와 PyJWT 중 무엇을 써야 합니까? 이 교재는 **PyJWT**를 사용합니다. 임포트 이름이 `jwt` 이고 유지보수가 활발합니다. 과거 FastAPI 문서가 `python-jose`를 쓰던 시절의 코드를 그대로 베끼지 않도록 주의하세요.

Q. pwdlib과 passlib 중 무엇을 권장합니까? 신규 프로젝트는 **pwdlib**(`PasswordHash.recommended()` → `Argon2`)를 권장합니다. 기존 코드가 `passlib bcrypt`를 쓰고 있다면 `CryptContext(schemes=["bcrypt"], deprecated="auto")` 도 유효합니다. 둘 다 솔트를 자동 처리하므로 직접 솔트를 다루지 마세요.

Q. 토큰을 어디에 저장합니까? 모바일·서버 간 통신은 보통 메모리나 보안 저장소에 둡니다. 브라우저는 `localStorage` 보다 `HttpOnly` 쿠키가 XSS에 안전하지만, CSRF 대비가 필요합니다(트레이드오프). 저장 위치는 클라이언트 보안 정책의 문제입니다.

10장. 에러 처리·미들웨어·CORS: 견고한 API

이 장의 개념은 **에러 처리·예외·미들웨어·CORS** 하나로 묶이지만, "동작하는 API"와 "운영에서 버티는 견고한 API"를 가르는 결정적 차이를 담고 있습니다. 9장에서 우리는 인증·인가를 붙여 잘못된 요청을 401·403으로 막았습니다. 그런데 실제 서비스에서는 그 외에도 수많은 "정상적이지 않은 상황"이 발생합니다. 재고가 부족해 주문을 받을 수 없거나, 잘못된 형식의 입력이 들어오거나, 예상치 못한 예외가 터집니다. 이 장은 그 모든 상황을 **일관된 오류 응답**으로 변환하고, 모든 요청에 **공통 처리**(로깅·처리 시간)를 적용하며, 브라우저의 **교차 출처(CORS)** 요청을 안전하게 허용하는 방법을 가르칩니다.

이 장의 학습 목표는 다음과 같습니다.

- `HTTPException` 으로 상태코드·detail을 담은 오류를 던질 수 있다.
- 사용자 정의 예외 핸들러로 도메인 예외를 HTTP 응답으로 변환할 수 있다.
- 미들웨어로 모든 요청에 공통 처리를 적용할 수 있다.
- `CORSMiddleware` 로 허용 출처·메서드를 설정할 수 있다.

10.1 도입: 오류를 일관된 응답으로

8장에서 만든 상품 라우터를 다시 떠올려 봅시다. 없는 상품을 조회하면 우리는 `HTTPException(404)` 을 던졌습니다. 거기까지는 좋았습니다. 그런데 주문 도메인으로 넘어오면 오류의 종류가 폭발적으로 늘어납니다.

- 주문하려는 상품의 **재고가 부족하다** → 클라이언트 잘못이 아니라 "지금은 불가능"한 상태입니다(409).
- 결제 게이트웨이가 **응답하지 않는다** → 서버 측 일시 장애입니다(502/503).
- 본문이 **JSON 형식 오류**거나 필수 필드가 빠졌다 → 422입니다.
- 코드 어딘가에서 **예상치 못한 예외**가 터진다 → 그냥 두면 500과 함께 스택 트레이스가 노출됩니다.

이 오류들을 라우트마다 제각각 처리하면, 클라이언트는 어떤 형식의 오류가 올지 예측할 수 없습니다. 어떤 곳은 `{"detail": "..."}` , 어떤 곳은 `{"error": "..."}` , 어떤 곳은 평문이 옵니다. **견고한 API는 오류조차 일관된 계약**입니다.

선수 개념 상기: 이 장은 8장의 **라우터·404 처리**와 4장의 **응답 모델·상태 코드**를 직접 잊습니다. 8장에서 `HTTPException(404)` 을 던져 봤다면, 이 장은 그것을 **도메인 예외 → HTTP 변환**이라는 한 단계 위 패턴으로 일반화합니다. 또 모든 요청을 감싸는 미들웨어는 9장에서 `OAuth2PasswordBearer` 가 요청을 가로채던 것과 같은 "요청 파이프라인" 위에서 동작합니다.

10.2 정의: HTTPException · 예외 핸들러 · 미들웨어 · CORS

핵심 개념인 **에러 처리·예외·미들웨어·CORS**를 한 문장으로 정의하면 다음과 같습니다.

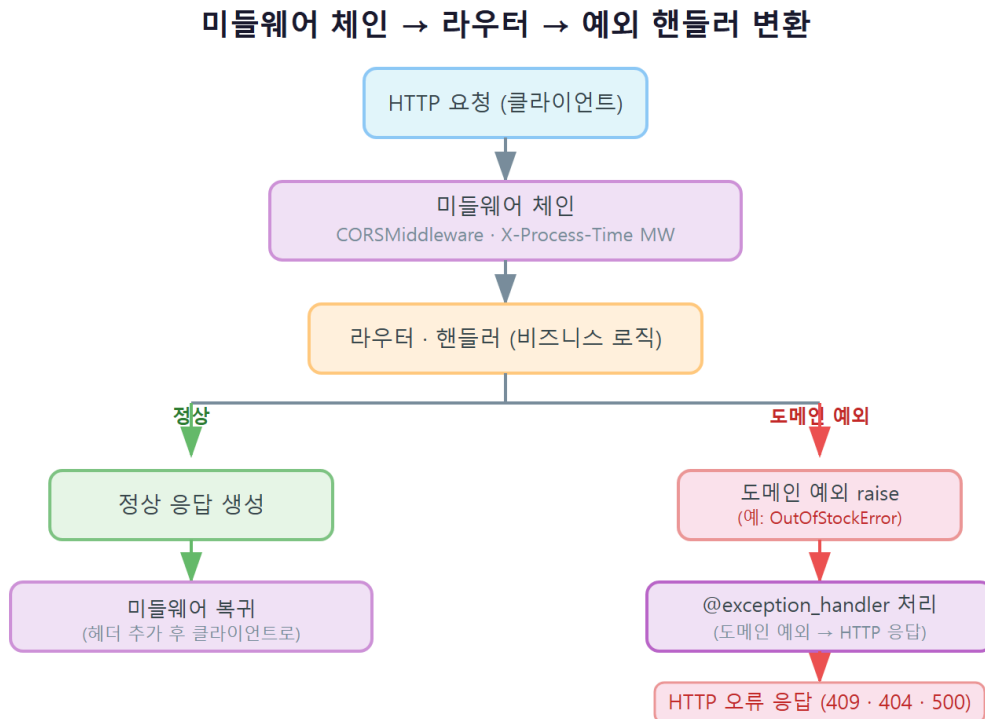
정의 — `HTTPException` 과 사용자 정의 예외 핸들러(`@app.exception_handler`)로 일관된 오류 응답을 만들고, **미들웨어(middleware)**로 모든 요청에 공통 처리(로깅·처리 시간 측정)를 적용하며, `CORSMiddleware` 로 브라우저의 **교차 출처(CORS, Cross-Origin Resource Sharing)** 요청을 허용합니다. 재고 부족 같은 **도메인 예외(domain exception)**를 적절한 HTTP 상태로 변환합니다.

용어를 하나씩 정리합니다.

용어	한글(영문)	의미
HTTP 예외	<code>HTTPException</code>	상태코드·detail을 담아 <code>raise</code> 로 던지면 FastAPI가 JSON 오류 응답으로 변환하는 내장 예외.
예외 핸들러	예외 핸들러(exception handler)	특정 예외 타입이 터졌을 때 어떤 응답으로 바꿀지 등록하는 함수(<code>@app.exception_handler</code>).
도메인 예외	도메인 예외(domain exception)	<code>OutOfStockError</code> 처럼 비즈니스 규칙 위반을 나타내는 사용자 정의 예외. HTTP를 모릅니다.
미들웨어	미들웨어(middleware)	모든 요청·응답을 가로채 공통 처리를 적용하는 계층. 라우트 앞뒤를 감쌉니다.
검증 오류	<code>RequestValidationError</code>	Pydantic 검증 실패 시 FastAPI가 던지는 예외. 기본은 422로 변환됩니다.
교차 출처 자원 공유	CORS	브라우저가 다른 출처(origin)의 API 호출을 허용할지 결정하는 정책. 서버가 헤더로 허용 범위를 알려 줍니다.

여기서 가장 중요한 설계 원칙은 **도메인 로직과 HTTP를 분리**하는 것입니다. 재고를 차감하는 서비스 함수는 HTTP 상태 코드를 알 필요가 없습니다. 재고가 부족하면 그냥 `OutOfStockError` 를 던지면 됩니다. 그 예외를 **409**라는 HTTP 응답으로 바꾸는 일은 예외 핸들러가 한 곳에서 책임집니다.

10.3 동작 원리: 미들웨어 체인에서 예외 변환까지



요청 하나가 들어와 응답이 나가기까지의 경로를 따라가 봅니다.

- 미들웨어 체인 진입:** 요청이 등록된 미들웨어들을 **바깥에서 안쪽 순서**로 통과합니다(CORS → 처리시간 측정 → ...). 각 미들웨어는 다음 단계를 호출하기 전후로 코드를 실행할 수 있습니다.
- 라우팅·의존성:** 경로에 맞는 라우트를 찾고, 의존성(9장의 인증 등)을 평가한 뒤 핸들러를 실행합니다.
- 예외 발생:** 핸들러나 서비스에서 예외가 터지면, 그 타입에 맞는 **예외 핸들러**가 잡아 적절한 HTTP 응답으로 변환합니다(예: `OutOfStockError` → 409). `HTTPException` 이면 FastAPI 기본 핸들러가 처리합니다.
- 응답 복귀:** 변환된 응답이 들어왔던 미들웨어 체인을 **역순**으로 거슬러 나갑니다. 이때 각 미들웨어가 응답에 헤더를 더하거나(예: `X-Process-Time`) 로그를 남깁니다.

미들웨어의 실행 순서를 표로 보면 직관이 분명해집니다. 미들웨어를 A → B 순으로 등록하면, **나중**에 추가한 것이 **바깥**을 감쌉니다.

등록 순서	요청 진입 순서	응답 복귀 순서
1번째 추가: 처리시간	두 번째(안쪽)	첫 번째
2번째 추가: CORS	첫 번째(바깥)	두 번째

이 "바깥에서 안쪽, 다시 바깥으로" 구조 때문에 미들웨어는 흔히 **양파(onion) 모델**이라 부릅니다. 그래서 등록 순서가 동작에 영향을 줍니다(10.9의 혼한 오류 참고).

예외 변환의 갈래는 다음과 같습니다.

발생한 예외	처리 주체	결과 응답
<code>HTTPException(404, ...)</code>	FastAPI 기본 핸들러	<code>{"detail": "..."} + 지정 상태코드</code>
<code>RequestValidationError</code>	FastAPI 기본(또는 커스텀) 핸들러	422 + 어떤 필드가 왜 틀렸는지
<code>OutOfStockError</code> (도메인)	<code>@app.exception_handler</code> 로 등록한 핸들러	409 + 일관된 오류 본문
처리 안 된 일반 예외	서버 기본	500(상세는 숨김)

10.4 worked example: 재고 부족 도메인 예외 → 409

이커머스 주문의 대표 오류인 "재고 부족"을 다룹니다. 핵심은 **서비스 함수는 HTTP를 모르고**, 도메인 예외만 던진다는 점입니다.

```

# main.py - 도메인 예외를 409로 변환
from fastapi import FastAPI, Request, status
from fastapi.responses import JSONResponse
from pydantic import BaseModel

app = FastAPI(title="shop-orders")

# 1) 도메인 예외 - HTTP를 전혀 모릅니다. 비즈니스 규칙만 표현합니다.
class OutOfStockError(Exception):
    def __init__(self, product_id: int, requested: int, available: int) -> None:
        self.product_id = product_id
        self.requested = requested
        self.available = available
        super().__init__(
            f"상품 {product_id} 재고 부족: 요청 {requested}, 가용 {available}"
        )

# 2) 예외 핸들러 - 도메인 예외를 409 JSON 응답으로 변환합니다(한 곳에서).
@app.exception_handler(OutOfStockError)
async def out_of_stock_handler(
    request: Request, exc: OutOfStockError
) -> JSONResponse:
    return JSONResponse(
        status_code=status.HTTP_409_CONFLICT,
        content={
            "error": "out_of_stock",
            "product_id": exc.product_id,
            "requested": exc.requested,
            "available": exc.available,
            "message": str(exc),
        },
    )

# 3) 서비스 - 재고가 모자라면 그냥 도메인 예외를 던집니다.
stock_db = {42: 3} # 상품 42의 재고는 3개

def reserve_stock(product_id: int, quantity: int) -> int:
    available = stock_db.get(product_id, 0)
    if quantity > available:
        raise OutOfStockError(product_id, quantity, available)
    stock_db[product_id] = available - quantity
    return stock_db[product_id]

class OrderLine(BaseModel):
    product_id: int
    quantity: int

@app.post("/orders", status_code=status.HTTP_201_CREATED)
def create_order(line: OrderLine) -> dict:

```

```
remaining = reserve_stock(line.product_id, line.quantity)
return {"product_id": line.product_id, "remaining_stock": remaining}
```

- `OutOfStockError` 는 평범한 `Exception` 하위 클래스입니다. **HTTP 상태 코드를 전혀 알지 못합니다** — 오직 "어떤 상품이, 얼마를 요청했는데, 얼마만 있다"는 도메인 정보만 담습니다. 이렇게 두면 같은 서비스를 CLI·배치에서 재사용해도 HTTP에 묶이지 않습니다.
- `@app.exception_handler(OutOfStockError)` 는 그 예외가 앱 어디서든 터지면 이 함수가 잡도록 등록합니다. 여기서 단 한 번 **409 Conflict**로 변환하므로, 재고를 다루는 모든 라우트가 자동으로 일관된 응답을 갖습니다.
- 핸들러는 `Request` 와 예외 객체를 받아 `JSONResponse` 를 돌려줍니다. 본문 형식(`error · message` 등)을 우리가 정한 표준으로 통일합니다.
- `reserve_stock` 서비스는 재고가 모자라면 그냥 `raise OutOfStockError(...)` 합니다. `create_order` 라우트에는 `try/except`가 한 줄도 없습니다 — 예외 핸들러가 가로채기 때문입니다. 관심사가 깔끔히 분리됩니다.

```
# 재고(3개)보다 적게 주문 -> 201, 재고 차감
$ curl -X POST http://127.0.0.1:8000/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id": 42, "quantity": 2}'
{"product_id":42,"remaining_stock":1}

# 남은 재고(1개)보다 많이 주문 -> 409, 일관된 오류 본문
$ curl -i -X POST http://127.0.0.1:8000/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id": 42, "quantity": 5}'
HTTP/1.1 409 Conflict
{"error":"out_of_stock","product_id":42,"requested":5,"available":1,"message":"상품 42 재고 부족: 요
```

10.5 RequestValidationError 가공과 일관 오류 포맷

Pydantic 검증 실패(3장)는 기본적으로 **422**로 변환되고, FastAPI가 어떤 필드가 왜 틀렸는지 상세 배열을 돌려줍니다. 그 기본 형식은 친절하지만, 우리 API의 다른 오류(`out_of_stock` 등)와 **형식이 다릅니다**. 클라이언트가 모든 오류를 동일한 모양으로 받게 하려면 `RequestValidationError` 핸들러를 덮어쓰면 됩니다.

```
# main.py 이어서 - 검증 오류를 우리 표준 포맷으로 가공
from fastapi.exceptions import RequestValidationError
from fastapi import Request, status
from fastapi.responses import JSONResponse

@app.exception_handler(RequestValidationError)
async def validation_handler(
    request: Request, exc: RequestValidationError
) -> JSONResponse:
    # exc.errors()는 [{loc, msg, type}, ...] 형식입니다.
    fields = [
        {
            "field": ".".join(str(p) for p in err["loc"] if p != "body"),
            "message": err["msg"],
        }
        for err in exc.errors()
    ]
    return JSONResponse(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        content={"error": "validation_error", "fields": fields},
    )
```

- `@app.exception_handler(RequestValidationError)` 로 FastAPI의 기본 422 핸들러를 우리 것으로 교체합니다. 검증이 실패하면 이 함수가 대신 호출됩니다.
- `exc.errors()` 는 Pydantic이 만든 오류 목록입니다. 각 항목의 `loc` (위치)·`msg` (메시지)·`type` (오류 종류)에서 우리가 필요한 `field`·`message` 만 뽑아 재구성합니다.
- `loc` 의 첫 요소가 보통 `"body"` 이므로 그것을 빼고 점으로 이어 `items.0.quantity` 같은 읽기 좋은 경로를 만듭니다.
- 결과는 `out_of_stock` 과 동일하게 `{"error": ..., ...}` 형태입니다. 이로써 모든 오류가 같은 계약을 따릅니다.

```
# quantity에 음수(검증 위반) -> 422, 우리 표준 포맷
$ curl -X POST http://127.0.0.1:8000/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id": 42, "quantity": -1}'
{"error": "validation_error", "fields": [{"field": "quantity", "message": "Input should be greater than 0"}]
```

참고: 위 결과를 내려면 `OrderLine.quantity` 에 `Field(gt=0)` 같은 제약이 있어야 합니다(3장). 제약이 검증을 일으키고, 그 검증 실패를 이 핸들러가 우리 포맷으로 가공합니다.

10.6 worked example: X-Process-Time 미들웨어

이제 **모든 요청**에 공통 처리를 적용하는 미들웨어를 만듭니다. 대표 예로, 각 요청의 처리 시간을 측정해 응답 헤더 `X-Process-Time`에 담습니다. 성능 관측의 기본 도구입니다.

```
# main.py 이어서 - 처리 시간 측정 미들웨어
import time

from fastapi import Request

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start = time.perf_counter()
    # call_next가 라우트(와 안쪽 미들웨어)를 실행하고 응답을 돌려줍니다.
    response = await call_next(request)
    elapsed = time.perf_counter() - start
    response.headers["X-Process-Time"] = f"{elapsed:.4f}"
    return response
```

- `@app.middleware("http")` 데코레이터로 HTTP 미들웨어를 등록합니다. 함수는 `request`와 `call_next`를 받습니다.
- `call_next(request)`는 다음 단계(안쪽 미들웨어와 라우트)를 실행하고 응답을 돌려주는 함수입니다. 반드시 `await`해야 합니다(미들웨어는 `async`).
- `call_next` 앞의 코드는 요청 진입 시, 뒤의 코드는 응답 복귀 시 실행됩니다. 이 "감싸기" 구조가 10.3에서 본 양파 모델입니다. 시작 시각을 재고, 응답이 돌아온 뒤 경과를 계산합니다.
- `response.headers[...]`에 측정값을 더해 모든 응답에 `X-Process-Time`이 붙습니다. 라우트 코드는 전혀 건드리지 않고도 공통 관측이 적용됩니다.
- 무거운 작업(블로킹)을 미들웨어에 넣으면 모든 요청이 느려집니다(5장 비동기). 미들웨어는 가볍게 유지합니다.

```
$ curl -i http://127.0.0.1:8000/orders -X POST \
-H "Content-Type: application/json" \
-d '{"product_id": 42, "quantity": 1}'
HTTP/1.1 201 Created
x-process-time: 0.0013
content-type: application/json
{"product_id":42,"remaining_stock":2}
```

10.7 CORSMiddleware 설정 (출처·자격증명)

브라우저에서 동작하는 프론트엔드(예: `https://shop.example.com`)가 다른 출처의 API(`https://api.example.com`)를 호출하면, 브라우저는 **동일 출처 정책(same-origin policy)**에 따라 기본적으로 막습니다. 서버가 "이 출처는 허용한다"는 CORS 헤더를 응답에 실어야 브라우저가 통과 시킵니다. FastAPI는 이를 위한 `CORSMiddleware`를 제공합니다.


```
# main.py 이어서 - CORS 설정
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "https://shop.example.com", # 운영 프론트엔드
        "http://localhost:3000",    # 로컬 개발
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- `app.add_middleware(CORSMiddleware, ...)` 로 CORS를 등록합니다. `@app.middleware("http")` 와 달리 미리 만들어진 미들웨어 클래스를 옵션과 함께 추가합니다.
- `allow_origins` 에 **허용할 출처를 명시**합니다. 자격증명을 쓸 때는 와일드카드(`"*"`)를 쓸 수 없으므로 구체적 출처 목록을 둡니다.
- `allow_credentials=True` 는 쿠키·Authorization 헤더(9장 토큰) 같은 자격증명을 동반한 요청을 허용합니다. 이 옵션과 `allow_origins=["*"]` 는 **함께 쓸 수 없습니다**(브라우저가 거부).
- `allow_methods` · `allow_headers` 는 허용할 HTTP 메서드·요청 헤더입니다. 개발 초기엔 `["*"]` 로 두고, 운영에서는 필요한 것만 좁히는 것이 안전합니다.
- CORS는 **브라우저 정책**일 뿐 서버 인증·인가를 대체하지 않습니다. `curl` 이나 서버 간 호출에는 적용되지 않습니다(10.9 참고).

브라우저는 본 요청을 보내기 전에, `PUT` · `DELETE` 나 커스텀 헤더가 붙은 요청에 대해 **사전 요청 (preflight)**이라 부르는 `OPTIONS` 요청을 먼저 보냅니다. "이 출처에서 이 메서드·헤더로 호출해도 되느냐"를 묻는 것입니다. `CORSMiddleware` 는 이 `OPTIONS` 사전 요청에 자동으로 응답해 `allow_methods` · `allow_headers` 에 선언한 허용 범위를 알려 줍니다. 우리가 별도 라우트를 만들지 않아도 됩니다. CORS 협상에 쓰이는 주요 응답 헤더는 다음과 같습니다.

응답 헤더	의미	대응 옵션
<code>Access-Control-Allow-Origin</code>	허용된 출처	<code>allow_origins</code>
<code>Access-Control-Allow-Methods</code>	허용된 HTTP 메서드	<code>allow_methods</code>
<code>Access-Control-Allow-Headers</code>	허용된 요청 헤더	<code>allow_headers</code>
<code>Access-Control-Allow-Credentials</code>	자격증명 동반 허용 여부	<code>allow_credentials</code>

이 헤더들을 직접 손으로 다룰 일은 거의 없습니다. `CORSMiddleware` 에 정책만 선언하면 사전 요청 응답과 본 응답 헤더가 자동으로 채워집니다. 운영에서는 `allow_origins` 를 신뢰하는 프론트엔드 출처로 좁히고, 와일드카드는 자격증명을 쓰지 않는 공개 API에만 제한적으로 사용합니다.

10.8 즉시 연습 (2종 1세트): 예외 핸들러 채우기

배운 패턴을 손으로 완성합니다. 아래 빈칸 코드의 지시에 따라 직접 작성하세요.

연습 목표 — 결제 실패 도메인 예외(`PaymentFailedError`)를 **402 Payment Required**로 변환하는 핸들러를 완성합니다.

```
# 빈칸 코드 - 직접 작성해 완성하세요
from fastapi import FastAPI, Request, status
from fastapi.responses import JSONResponse

app = FastAPI()

class PaymentFailedError(Exception):
    def __init__(self, order_id: int, reason: str) -> None:
        self.order_id = order_id
        self.reason = reason
        super().__init__(f"주문 {order_id} 결제 실패: {reason}")

# 빈칸 1: PaymentFailedError를 잡는 예외 핸들러 데코레이터를 작성하세요
@app.__(_____)
async def payment_failed_handler(request: Request, exc: PaymentFailedError):
    # 빈칸 2: 402 상태와 일관된 오류 본문(error.order_id·message)을 반환하세요
    return JSONResponse(
        status_code=_____,
        content=_____,
    )

def charge(order_id: int, amount: int) -> None:
    if amount > 1000:
        # 빈칸 3: 결제 실패를 도메인 예외로 던지세요
        raise _____
```

자가 체크리스트로 스스로 점검하세요.

- [] 핸들러가 `@app.exception_handler(PaymentFailedError)` 로 등록됐는가?
- [] 응답 상태코드가 **402**이고, 본문 형식이 다른 오류와 **일관**되는가?
- [] 서비스 `charge` 함수에는 HTTP 코드가 등장하지 않고 도메인 예외만 던지는가?
- [] 라우트에 `try/except` 없이도 402가 정상적으로 나오는가?

두 번째 연습은 미들웨어입니다.

연습 목표(심화) — 모든 요청에 고유 요청 ID를 부여해 응답 헤더 `X-Request-ID` 에 담는 미들웨어를 직접 작성하세요(분산 추적의 기초).

```
# 빈칸 코드 - 요청 ID 미들웨어
import uuid
from fastapi import Request

# 빈칸 1: http 미들웨어 데코레이터를 작성하세요
@app._____("http")
async def add_request_id(request: Request, call_next):
    request_id = uuid.uuid4().hex
    # 빈칸 2: 다음 단계를 실행하고 응답을 받으세요(await 필수)
    response = _____
    # 빈칸 3: 응답 헤더에 X-Request-ID를 추가하세요
    _____
    return response
```

자가 체크리스트.

- [] `call_next(request)` 를 **await** 했는가? (빠뜨리면 코루틴 객체가 반환돼 깨짐)
- [] 모든 응답(성공·오류 포함)에 `X-Request-ID` 헤더가 붙는가?
- [] 미들웨어가 무거운 블로킹 작업을 하지 않아 응답이 느려지지 않는가?
- [] CORS 미들웨어와의 등록 순서를 의식했는가?

10.9 흔한 오류와 오개념

오개념 1 — "HTTPException은 return하면 된다" 틀립니다. `HTTPException` 은 반드시 `raise` 로 던져야 합니다. `return HTTPException(...)` 은 그냥 객체를 직렬화해 200으로 내보내려다 깨집니다. 도메인 예외도 마찬가지로 `raise` 해야 등록된 핸들러가 가로칩니다.

오개념 2 — "미들웨어 등록 순서는 상관없다" 순서가 중요합니다. 미들웨어는 **나중에 추가한 것이 바깥을 감쌉니다**(양파 모델). 예를 들어 CORS를 처리시간 미들웨어보다 나중에 추가하면 CORS가 가장 바깥에서 동작합니다. 인증·CORS·로깅의 상대 순서가 동작에 영향을 주므로 의도적으로 배치하세요.

오개념 3 — "CORS는 서버 보안 기능이다" 아닙니다. **CORS는 브라우저 정책입니다**. `curl` ·서버 간 호출·모바일 앱에는 적용되지 않습니다. 따라서 CORS를 인증·인가의 대체물로 믿으면 안 됩니다. 실제 접근 통제는 9장의 인증·인가가 담당합니다.

오개념 4 — `allow_origins=["*"]` 와 `allow_credentials=True` 를 함께 쓰기 브라우저가 이 조합을 거부합니다. 자격증명(쿠키·`Authorization` 헤더)을 보내려면 `allow_origins` 에 **구체적 출처**를 명시해야 합니다. 와일드카드와 자격증명은 양립하지 않습니다.

흔한 오류 5 — 미들웨어에서 `call_next` 를 **await하지 않음** `response = call_next(request)` 처럼 `await` 를 빠뜨리면 코루틴 객체가 응답 자리에 들어가 앱이 깨집니다. 반드시 `response = await`

`call_next(request)` 로 작성하세요.

흔한 오류 6 — 예외 핸들러를 라우트보다 늦게 등록해도 된다고 가정 `@app.exception_handler` 는 앱 시작 시 등록되므로 라우트 정의 위치와 무관하게 동작합니다. 다만 같은 예외 타입에 핸들러를 두 번 등록하면 마지막 것이 이깁니다 — 중복 등록을 피하세요.

10.10 단원 미니 프로젝트: 도메인 예외·CORS 설정 (2종 1세트)

이 단원의 **에러 처리·예외·미들웨어·CORS** 개념만으로 견고한 주문 앱의 횡단 관심사를 완성합니다. 산출물은 예외 핸들러·미들웨어·CORS가 갖춰진 `main.py` 입니다.

마일스톤(중분).

단계	목표	사용 개념
M1	<code>HTTPException</code> 과 도메인 예외(<code>OutOfStockError</code>) → 409 핸들러	예외 핸들러·도메인 분리
M2	처리시간 측정 미들웨어(<code>X-Process-Time</code>)	<code>@app.middleware("http") · call_next</code>
M3	<code>CORSMiddleware</code> 로 허용 출처 설정	CORS·자격증명 정책

모범 산출물

```

# main.py - 횡단 관심사 통합 (모범)
import time

from fastapi import FastAPI, Request, status
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field

app = FastAPI(title="shop-robust")

class OutOfStockError(Exception):
    def __init__(self, product_id: int, requested: int, available: int) -> None:
        self.product_id = product_id
        self.requested = requested
        self.available = available
        super().__init__("재고 부족")

@app.exception_handler(OutOfStockError)
async def out_of_stock_handler(request: Request, exc: OutOfStockError):
    return JSONResponse(
        status_code=status.HTTP_409_CONFLICT,
        content={
            "error": "out_of_stock",
            "product_id": exc.product_id,
            "requested": exc.requested,
            "available": exc.available,
        },
    )

@app.exception_handler(RequestValidationError)
async def validation_handler(request: Request, exc: RequestValidationError):
    fields = [
        {"field": ".".join(str(p) for p in e["loc"]) if p != "body",
         "message": e["msg"]}
        for e in exc.errors()
    ]
    return JSONResponse(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        content={"error": "validation_error", "fields": fields},
    )

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Process-Time"] = f"{time.perf_counter() - start:.4f}"
    return response

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://shop.example.com", "http://localhost:3000"],

```

```

        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

stock_db = {42: 3}

class OrderLine(BaseModel):
    product_id: int
    quantity: int = Field(gt=0)

@app.post("/orders", status_code=status.HTTP_201_CREATED)
def create_order(line: OrderLine) -> dict:
    available = stock_db.get(line.product_id, 0)
    if line.quantity > available:
        raise OutOfStockError(line.product_id, line.quantity, available)
    stock_db[line.product_id] = available - line.quantity
    return {"product_id": line.product_id, "remaining_stock": stock_db[line.product_id]}

```

```

# 통합 동작 결과 요약
POST /orders {quantity:2}    -> 201, X-Process-Time 헤더, remaining_stock:1
POST /orders {quantity:5}    -> 409 {"error":"out_of_stock",...}
POST /orders {quantity:-1}   -> 422 {"error":"validation_error","fields":[...]}
브라우저(localhost:3000)    -> CORS 허용, 그 외 출처 -> 브라우저가 차단

```

학생 작성형 빈 스타터

과제 — 아래 빈 스타터를 채워 `main.py` 를 완성하세요. 위 모범의 4가지 결과(201·409·422·CORS)를 모두 재현하면 성공입니다.

```
# main.py - 빈 스타터 (직접 작성)
import time
from fastapi import FastAPI, Request, status
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field

app = FastAPI()

class OutOfStockError(Exception):
    ... # TODO: product_id·requested·available를 담은 도메인 예외

# TODO: OutOfStockError -> 409 변환 핸들러
# TODO: RequestValidationError -> 422 우리 표준 포맷 핸들러
# TODO: @app.middleware("http") 처리시간 미들웨어(call_next await 주의)
# TODO: app.add_middleware(CORSMiddleware, ...) 허용 출처·credentials

class OrderLine(BaseModel):
    product_id: int
    quantity: int = Field(gt=0)

@app.post("/orders", status_code=201)
def create_order(line: OrderLine) -> dict:
    # TODO: 재고 확인 -> 부족하면 OutOfStockError raise, 충분하면 차감
    ...
```

자가 체크리스트로 스스로 점검하세요.

- [] 재고 부족 주문에 **409**가 나는가? (raise 로 던졌는가)
- [] 응답에 **X-Process-Time** 헤더가 있는가? (call_next 를 await했는가)
- [] 음수 수량에 **422**가 우리 표준 포맷으로 나오는가?
- [] 허용한 출처만 CORS를 통과하고, allow_credentials=True 와 와일드카드를 함께 쓰지 않았는가?
- [] 서비스/라우트 코드에 HTTP 상태 코드가 흩어지지 않고 핸들러 한 곳에 모였는가?

10.11 자주 묻는 질문 (FAQ)

Q. 재고 부족은 400입니까 409입니까? 재고 부족은 입력 형식 자체가 틀린 것이 아니라 "현재 자원 상태와 충돌"하는 상황이므로 **409 Conflict**가 가장 적절합니다. 400은 일반적 잘못된 요청, 422는 검증(스키마) 실패에 씁니다.

Q. 모든 예외를 잡는 전역 핸들러를 뒤편도 됩니까? `@app.exception_handler(Exception)` 로 최후의 그물을 둘 수 있지만, 그러면 예상치 못한 버그가 200/일관 응답으로 숨겨질 수 있습니다. 알려진 도메인 예외는 구체적으로 잡고, 미지의 예외는 500으로 두되 **상세 메시지는 로그에만 남기고 응답에는 노출하지 않는 것**이 안전합니다.

Q. `@app.middleware("http")` 와 `add_middleware` 의 차이는 무엇입니까? `@app.middleware("http")` 는 간단한 함수형 미들웨어를 빠르게 정의할 때, `add_middleware` 는 `CORSMiddleware` 처럼 미리 만들어진 클래스형 미들웨어를 옵션과 함께 등록할 때 씁니다. 둘 다 같은 요청 파이프라인에 쌓입니다.

Q. 미들웨어와 의존성(Depends) 중 무엇을 써야 합니까? 모든 경로에 무조건 적용할 횡단 관심사(로깅·처리시간·CORS)는 **미들웨어**가, 특정 라우트 그룹에만 적용하고 반환값을 핸들러로 넘겨야 하는 로직(인증·페이지네이션)은 **의존성(6장·9장)**이 적합합니다.

11장 테스트: pytest와 TestClient

이 장에서 배우는 것

- `TestClient` 로 실제 서버를 띄우지 않고 엔드포인트의 상태 코드와 본문을 검증할 수 있습니다.
- `pytest` 픽스처(fixture)로 앱·클라이언트·테스트 DB를 한 번에 구성하고 재사용할 수 있습니다.
- `app.dependency_overrides` 로 DB 세션·인증 같은 의존성을 테스트용(인메모리 SQLite 등)으로 교체할 수 있습니다.
- 토큰 발급이 필요한 보호 엔드포인트와 검증 오류(422)를 자동으로 검증하고, 테스트 간 격리를 보장할 수 있습니다.

이 장에서 가르치는 개념은 하나입니다. **테스트(pytest·TestClient)** 입니다. 단일 개념이지만 실무에서 가장 무겁게 쓰이는 주제이므로, 정의부터 동작 원리, 완전히 풀어 보이는 예제, 직접 채우는 연습, 그리고 전형적인 실수까지 차근차근 익히겠습니다.

11.1 도입: 손이 아니라 코드로 검증하기

앞 장(10장)에서 우리는 `HTTPException` 과 사용자 정의 예외 핸들러, 미들웨어, CORS로 **견고한 API**를 만들었습니다. 재고 부족이면 409, 권한이 없으면 403, 잘못된 입력이면 422가 일관되게 나가도록 다듬었습니다. 그런데 한 가지 질문이 남습니다. 그 모든 분기가 **정말로** 그렇게 동작한다고 어떻게 확신할 수 있습니까?

지금까지의 검증 방법을 떠올려 봅시다. `fastapi dev` 로 앱을 띄우고, 브라우저에서 `/docs` 를 열어, 폼에 값을 넣고 "Execute"를 눌러 응답을 눈으로 확인했습니다. 이것을 **수동 테스트(manual testing)** 라고 합니다. 입문 단계에서는 충분하지만, 이커머스 주문 API처럼 엔드포인트가 수십 개로 늘어나면 곧 한계에 부딪힙니다.

왜 수동 테스트로는 부족한가

- 상품 생성 코드를 한 줄 고쳤을 뿐인데, 그 변경이 주문 생성·재고 차감에 영향을 주지 않았는지 **모든 엔드포인트를 다시 손으로 눌러 봐야** 합니다.
- "무토큰 요청은 401" 같은 음성 경로(negative path)는 손으로 재현하기 번거로워 **자꾸 건너뛰게** 됩니다.

- 사람이 누르는 검증은 **재현 가능(reproducible)**하지 않습니다. 누가, 언제, 어떤 순서로 눌렀는지에 따라 결과가 달라집니다.

해결책은 **검증을 코드로 작성해 자동 실행**하는 것입니다. 이것이 자동화 테스트(automated testing)입니다. 한 번 작성해 두면, 코드를 고칠 때마다 `pytest` 한 줄로 수백 개의 검증을 몇 초 만에 돌릴 수 있습니다. FastAPI는 이를 위해 `TestClient` 라는 도구를 제공하며, 파이썬 표준 테스트 도구인 `pytest` 와 결합해 실무 수준의 테스트 스위트를 만듭니다.

선수 개념 상기: 이 장은 앞에서 배운 거의 모든 것을 검증 대상으로 삼습니다. 의존성 주입(6장 `Depends`)을 교체하고, DB 세션(7장 `SQLModel`)을 인메모리로 바꾸며, 인증(9장 JWT)으로 보호된 라우트를 토큰과 함께 호출하고, 예외 처리(10장)가 올바른 상태 코드를 내는지 단언합니다. 특히 **의존성 주입을 이해하고 있어야** `dependency_overrides`의 교체 메커니즘이 자연스럽게 이해됩니다.

11.2 정의: `TestClient`와 `pytest` 픽스처

먼저 핵심 용어를 한 문장으로 정의하겠습니다.

테스트(`pytest·TestClient`)란, `TestClient` (스타렛·httpx 기반)로 실제 서버를 띄우지 않고 FastAPI 앱을 직접 호출해 엔드포인트를 검증하고, `pytest` 픽스처로 앱·클라이언트·DB를 구성하며 `app.dependency_overrides`로 의존성을 테스트용으로 교체해 **격리되고 결정론적인** 자동화 검증을 만드는 기법입니다.

용어를 하나씩 풀어 보겠습니다.

용어	한글(영문)	의미
테스트 클라이언트	TestClient	서버를 네트워크로 띄우지 않고 앱 객체를 직접 호출해 요청·응답을 주고받는 가짜 클라이언트. 내부는 httpx를 사용합니다.
픽스처	fixture	각 테스트가 시작되기 전에 준비물(앱·클라이언트·DB 등)을 만들어 넘겨주고, 끝나면 정리하는 pytest의 준비 함수
의존성 교체	dependency override	<code>app.dependency_overrides</code> 딕셔너리에 "이 의존성 대신 저 함수를 써라"를 등록해, 운영용 의존성을 테스트용으로 바꾸는 기능
단언	assertion	<code>assert</code> 조건 으로 "이래야 한다"를 선언하는 검증 문장. 거짓이면 테스트가 실패합니다.
테스트 격리	test isolation	한 테스트의 데이터·상태가 다른 테스트에 새어 들어가지 않도록 분리하는 것

TestClient는 "서버 없는 클라이언트"입니다

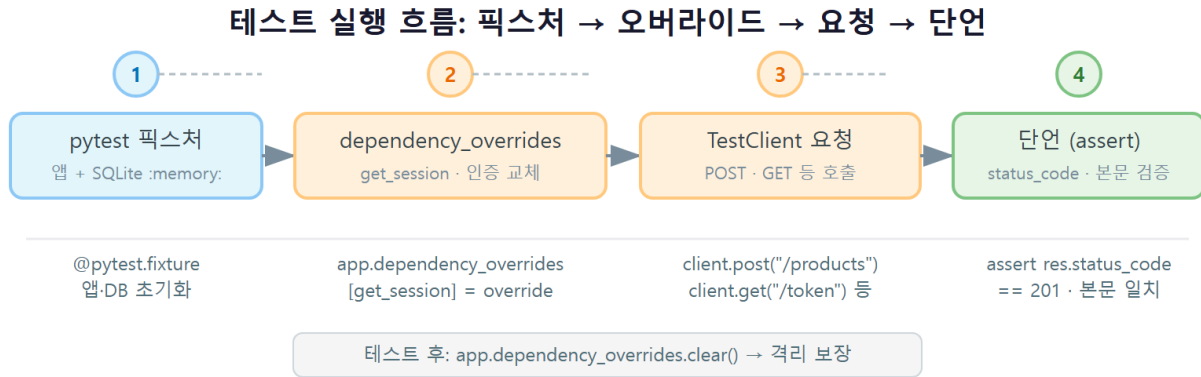
흔히 테스트라고 하면 `uvicorn`으로 서버를 띄우고 `requests`로 `http://localhost:8000`에 진짜 HTTP 요청을 보내는 모습을 떠올립니다. `TestClient`는 그렇지 않습니다. **앱 객체를 인자로 받아** 내부에서 ASGI 호출로 직접 핸들러를 실행합니다. 따라서 포트 충돌도, 네트워크 지연도, 서버를 띄우고 끄는 번거로움도 없습니다. 그러면서도 사용법은 `requests`와 거의 같아서 `client.get(...)`, `client.post(...)`처럼 쓸 수 있습니다.

설치는 다음과 같습니다. `TestClient`는 `httpx`에 의존하므로 함께 설치합니다.

```
pip install pytest httpx
```

`fastapi[standard]`를 설치했다면 `httpx`는 보통 이미 포함되어 있습니다. `pytest`만 추가하면 됩니다.

11.3 동작 원리: 픽스처에서 오버라이드, 요청, 단언까지



자동화 테스트 한 건이 실행될 때 내부에서 무슨 일이 벌어지는지 단계로 따라가 보겠습니다. 핵심은 **운영 의존성을 테스트용으로 갈아 끼운 뒤 호출한다**는 것입니다.

- 픽스처 준비(setup):** pytest 가 테스트 함수를 실행하기 직전, 필요한 픽스처를 먼저 실행합니다. 인메모리 SQLite 엔진을 만들고 테이블을 생성합니다.
- 의존성 교체(override):** app.dependency_overrides[get_session] 에 테스트용 세션 함수를 등록합니다. 이제 앱은 운영 DB가 아니라 인메모리 DB를 보게 됩니다. 인증이 필요하면 토큰을 발급받거나 get_current_user 도 교체합니다.
- 클라이언트 생성:** 교체가 끝난 앱으로 TestClient(app) 를 만듭니다. 이 클라이언트가 곧 우리의 손가락 역할을 합니다.
- 요청(request):** client.post("/products", json={...}) 처럼 요청을 보냅니다. TestClient 는 이를 ASGI 호출로 바꿔 실제 핸들러·검증·의존성·DB 로직을 그대로 통과시킵니다.
- 단언(assert):** 돌아온 response 의 status_code 와 json() 을 assert 로 검사합니다. 기대와 다르면 그 자리에서 테스트가 실패하고, 어디가 어떻게 틀렸는지 보고됩니다.
- 정리(teardown):** 테스트가 끝나면 픽스처의 yield 이후 코드가 실행되어 인메모리 DB와 오버라이드를 비웁니다. 그래야 다음 테스트가 깨끗한 상태에서 시작합니다.

왜 "인메모리 SQLite"를 쓰는가

테스트 DB는 빠르고, 매번 비어 있고, 흔적을 남기지 않아야 합니다. sqlite:// (파일 경로 없는 SQLite)는 메모리에만 존재하므로 테스트가 끝나면 사라집니다. 운영에서 PostgreSQL을 쓰더라도, 테스트에서는 인메모리 SQLite로 교체해 **빠르고 격리된** 검증을 얻는 것이 표준 전략입니다. 단, 메모리 DB를 여러 스레드가 공유하도록 StaticPool 과 check_same_thread=False 를 지정해야 합니다(곧 예제에서 봅니다).

11.4 worked example: 상품 생성에서 조회까지 일치 테스트

이제 완전히 풀어 보이는 예제를 작성하겠습니다. 검증 대상은 이커머스의 가장 기본 흐름입니다. **상품을 POST로 생성한 뒤, GET으로 다시 조회하면 같은 값이 나와야 한다**는 것입니다. 먼저 검증 대상이 될 작은 앱을 보겠습니다(앞 장들에서 배운 SQLAlchemy·세션 의존성·입출력 모델 분리를 그대로 사용합니다).

```

# app/main.py - 검증 대상이 되는 상품 API (발책)
from fastapi import Depends, FastAPI, HTTPException, status
from sqlmodel import Field, Session, SQLModel, create_engine, select

# 운영용 엔진 (실제로는 PostgreSQL 등을 사용)
engine = create_engine("sqlite:///./shop.db")

class Product(SQLModel, table=True):
    # 상품 테이블 모델
    id: int | None = Field(default=None, primary_key=True)
    name: str
    category: str
    price: float
    stock: int = 0

class ProductCreate(SQLModel):
    # 입력 모델 (id는 받지 않음)
    name: str
    category: str
    price: float
    stock: int = 0

def get_session():
    # 운영용 세션 의존성 (요청마다 세션 열고 닫음)
    with Session(engine) as session:
        yield session

app = FastAPI()

@app.post("/products", status_code=status.HTTP_201_CREATED)
def create_product(
    payload: ProductCreate,
    session: Session = Depends(get_session),
) -> Product:
    # 입력 모델을 테이블 모델로 변환해 저장
    product = Product.model_validate(payload)
    session.add(product)
    session.commit()
    session.refresh(product) # DB가 채운 id를 다시 읽음
    return product

@app.get("/products/{product_id}")
def read_product(
    product_id: int,
    session: Session = Depends(get_session),
) -> Product:
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    return product

```

위 앱 코드 줄별 해설

- `engine = create_engine("sqlite:///./shop.db")` : 운영용 엔진입니다. 테스트에서는 이 엔진을 쓰지 않고, 곧 인메모리 엔진으로 **교체**할 것입니다.
- `get_session()` : 요청마다 세션을 열고 `yield`로 넘긴 뒤 닫는 의존성입니다. 6장에서 배운 `yield` 의존성이며, 바로 이 함수가 `dependency_overrides`의 교체 대상이 됩니다.
- `create_product(...)` : 입력 `ProductCreate`를 `Product.model_validate`로 테이블 모델로 바꾸고 `add` → `commit` → `refresh`로 저장합니다. `status_code=201`로 생성을 명시했습니다.
- `read_product(...)` : `session.get`으로 단건 조회하고, 없으면 404를 던집니다.

이제 이 앱을 검증하는 테스트를 작성합니다. 핵심은 **운영 엔진 대신 인메모리 엔진을 쓰도록 `get_session`을 교체**하는 것입니다.


```

# tests/test_products.py - 상품 생성→조회 일치 테스트
import pytest
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from app.main import app, get_session

@pytest.fixture(name="session")
def session_fixture():
    # 매 테스트마다 새 인메모리 SQLite 엔진을 만든다
    engine = create_engine(
        "sqlite://", # 파일 경로 없음 = 메모리 DB
        connect_args={"check_same_thread": False},
        poolclass=StaticPool, # 단일 연결을 공유해 스레드 간 동일 메모리 보장
    )
    SQLModel.metadata.create_all(engine) # 테이블 생성
    with Session(engine) as session:
        yield session # 테스트에 세션을 넘김

@pytest.fixture(name="client")
def client_fixture(session: Session):
    # 운영 get_session 대신 테스트용 세션을 돌려주는 함수로 교체
    def get_session_override():
        yield session

    app.dependency_overrides[get_session] = get_session_override
    client = TestClient(app)
    yield client
    app.dependency_overrides.clear() # 교체를 원복해 다음 테스트 오염 방지

def test_create_then_read_product(client: TestClient):
    # 1) 상품을 생성한다
    response = client.post(
        "/products",
        json={"name": "기계식 키보드", "category": "주변기기",
            "price": 89000, "stock": 30},
    )
    assert response.status_code == 201
    created = response.json()
    assert created["name"] == "기계식 키보드"
    assert created["id"] is not None # DB가 id를 채워 줬다

    # 2) 방금 만든 상품을 다시 조회하면 같은 값이어야 한다
    product_id = created["id"]
    read = client.get(f"/products/{product_id}")
    assert read.status_code == 200
    assert read.json()["price"] == 89000

def test_read_missing_product_returns_404(client: TestClient):
    # 없는 상품 조회는 404
    response = client.get("/products/9999")

```

```
assert response.status_code == 404
assert response.json()["detail"] == "상품을 찾을 수 없습니다"
```

테스트 코드 줄별 해설

- `@pytest.fixture(name="session")`: 픽스처를 정의합니다. `name="session"` 이면 테스트 함수가 `session` 이라는 인자로 이 픽스처를 받습니다.
- `create_engine("sqlite://", ...)`: 파일 경로가 없는 인메모리 SQLite입니다. `connect_args={"check_same_thread": False}` 와 `poolclass=StaticPool` 은 같은 메모리 DB를 `TestClient` 의 다른 스레드에서도 보게 하는 표준 조합입니다.
- `SQLModel.metadata.create_all(engine)`: 인메모리에 테이블을 만듭니다. 매 테스트가 빈 테이블에서 시작합니다.
- `client_fixture(session)`: `session` 픽스처를 받아, `get_session` 을 그 세션을 돌려주는 함수로 교체합니다. `app.dependency_overrides[get_session] = get_session_override` 한 줄이 교체의 핵심입니다.
- `yield client` 뒤의 `app.dependency_overrides.clear()`: 테스트가 끝나면 교체를 원복합니다. 이걸 빠뜨리면 다음 테스트가 망가진 상태를 물려받습니다(테스트 격리 위반).
- `test_create_then_read_product`: `client` 를 인자로 받으면 픽스처가 자동 주입됩니다. POST로 생성(201)하고, 응답의 `id` 로 GET 조회해 값 일치를 단언합니다.
- `assert created["id"] is not None: refresh` 덕분에 DB가 채운 `id` 가 응답에 들어 있는지 확인합니다.
- `test_read_missing_product_returns_404`: 음성 경로입니다. 없는 `id` 에 404와 `detail` 메시지가 나오는지 단언합니다.

테스트는 다음과 같이 실행합니다.

```
pytest -v
```

실행 결과는 다음과 같습니다.

```
tests/test_products.py::test_create_then_read_product PASSED      [ 50%]
tests/test_products.py::test_read_missing_product_returns_404 PASSED [100%]

===== 2 passed in 0.34s =====
```

서버를 띄우지 않았는데도 두 테스트가 0.34초 만에 통과했습니다. 이것이 자동화 테스트의 힘입니다.

11.5 dependency_overrides로 인메모리 DB 교체

앞 예제에서 가장 중요한 한 줄은 다음이었습니다.

```
app.dependency_overrides[get_session] = get_session_override
```

이 줄을 더 깊이 이해해 봅시다. `app.dependency_overrides` 는 **딕셔너리**입니다. 키는 "원래 의존성 함수", 값은 "대신 쓸 함수"입니다. FastAPI는 요청을 처리할 때 의존성을 해석하기 전에 이 딕셔너리를 먼저 확인합니다. 키에 등록된 함수가 있으면, 원래 함수 대신 등록된 함수를 호출합니다.

왜 의존성 교체가 깔끔한가

핸들러 코드(`create_product` , `read_product`)는 **한 줄도 고치지 않았습니**다. 운영에서는 `get_session` 이 진짜 DB 세션을 주고, 테스트에서는 같은 자리에 인메모리 세션을 끼워 넣습니다. 즉 **테스트를 위해 운영 코드를 더럽히지 않습니다**. 이것이 6장에서 배운 의존성 주입의 진짜 보상입니다. 의존성으로 분리해 둔 모든 것(DB·인증·외부 서비스 클라이언트·설정)은 테스트에서 자유롭게 교체할 수 있습니다.

교체할 수 있는 대상은 DB만이 아닙니다. 인증 의존성(`get_current_user`)도 교체할 수 있습니다. 보호 라우트를 테스트할 때 매번 로그인해 토큰을 받는 대신, **가짜 사용자를 바로 돌려주는 함수**로 교체하면 인증 로직을 우회하면서 핸들러만 검증할 수 있습니다.

```
# 인증 의존성을 가짜 사용자로 교체하는 예
from app.auth import get_current_user
from app.models import User

def fake_current_user():
    # 항상 같은 테스트 사용자를 돌려준다
    return User(id=1, email="tester@shop.com", is_admin=False)

app.dependency_overrides[get_current_user] = fake_current_user
```

이렇게 하면 토큰 발급 과정을 건너뛰고도 "로그인한 사용자가 호출한 상황"을 흉내 낼 수 있습니다. 다만 인증 자체(토큰 검증·만료·401)를 검증할 때는 교체하지 말고 실제 토큰으로 호출해야 합니다. 다음 절에서 그 방법을 보겠습니다.

conftest.py로 픽스처 공유하기

여러 테스트 파일이 같은 `session·client` 픽스처를 쓴다면, 이 픽스처들을 `tests/conftest.py` 에 옮기면 됩니다. `pytest` 는 `conftest.py` 의 픽스처를 같은 디렉터리의 모든 테스트에 자동으로 제공하므로 `import` 가 필요 없습니다. 실무 테스트 스위트는 거의 항상 `conftest.py` 에 공통 픽스처를 둡니다.

11.6 worked example: 토큰 발급에서 보호 라우트 200, 무 토큰 401

이번에는 인증이 걸린 엔드포인트를 테스트하겠습니다. 9장에서 만든 `/token` 로그인과 `get_current_user` 로 보호되는 라우트를 검증합니다. 검증 시나리오는 세 가지입니다.

- 올바른 자격으로 로그인하면 토큰이 발급된다.
- 발급받은 토큰으로 보호 라우트를 호출하면 200이 나온다.
- 토큰 없이 보호 라우트를 호출하면 401이 나온다.

검증 대상 앱은 다음과 같다고 가정합니다(9장에서 배운 구조입니다).

```

# app/auth_app.py - 인증이 걸린 엔드포인트 (발체)
from datetime import datetime, timedelta, timezone

import jwt
from fastapi import Depends, FastAPI, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from pwdlib import PasswordHash

SECRET_KEY = "test-secret-do-not-use-in-prod"
ALGORITHM = "HS256"
pwd_hash = PasswordHash.recommended()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

# 데모용 사용자 저장소 (실제로는 DB)
fake_users = {
    "buyer@shop.com": {
        "email": "buyer@shop.com",
        "password_hash": pwd_hash.hash("secret123"),
    }
}

app = FastAPI()

@app.post("/token")
def login(form: OAuth2PasswordRequestForm = Depends()):
    # 사용자 조회 후 비밀번호 해시 검증
    user = fake_users.get(form.username)
    if user is None or not pwd_hash.verify(form.password, user["password_hash"]):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="이메일 또는 비밀번호가 올바르지 않습니다",
        )
    # 만료(exp)를 담은 JWT 발급
    expire = datetime.now(timezone.utc) + timedelta(minutes=30)
    token = jwt.encode(
        {"sub": user["email"], "exp": expire}, SECRET_KEY, algorithm=ALGORITHM
    )
    return {"access_token": token, "token_type": "bearer"}

def get_current_user(token: str = Depends(oauth2_scheme)) -> str:
    # 토큰 서명·만료를 검증해 사용자 이메일을 돌려줌
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    except jwt.PyJWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="유효하지 않은 토큰입니다",
        )
    return payload["sub"]

@app.get("/orders/me")
def read_my_orders(current_user: str = Depends(get_current_user)):

```

```
# 보호 라우트: 토큰이 있어야 접근 가능
return {"user": current_user, "orders": []}
```

인증 앱 코드 줄별 해설

- `OAuth2PasswordBearer(tokenUrl="token")`: 요청의 `Authorization: Bearer ...` 헤더에서 토큰을 꺼내는 의존성입니다. 토큰이 없으면 자동으로 401을 냅니다.
- `login(form: OAuth2PasswordRequestForm = Depends())`: 로그인은 JSON이 아니라 **폼 데이터** (`username · password`)로 받습니다. 비밀번호 해시를 `verify` 로 검증하고, 통과하면 만료가 담긴 JWT를 발급합니다.
- `get_current_user`: 토큰을 `jwt.decode` 로 검증합니다. 서명이 틀리거나 만료됐으면 `PyJWTError` 가 나고 401로 변환합니다.
- `read_my_orders`: `Depends(get_current_user)` 로 보호됩니다. 토큰이 없으면 의존성 단계에서 막혀 핸들러까지 도달하지 못합니다.

이제 이 인증 흐름을 검증하는 테스트입니다.

```

# tests/test_auth.py - 토큰 발급과 보호 라우트 테스트
import pytest
from fastapi.testclient import TestClient

from app.auth_app import app

@pytest.fixture(name="client")
def client_fixture():
    yield TestClient(app)

def test_login_returns_token(client: TestClient):
    # 폼 데이터(data=)로 로그인 -> 토큰 발급
    response = client.post(
        "/token",
        data={"username": "buyer@shop.com", "password": "secret123"},
    )
    assert response.status_code == 200
    body = response.json()
    assert body["token_type"] == "bearer"
    assert "access_token" in body

def test_protected_route_with_token_returns_200(client: TestClient):
    # 1) 먼저 로그인해 토큰을 얻는다
    login = client.post(
        "/token",
        data={"username": "buyer@shop.com", "password": "secret123"},
    )
    token = login.json()["access_token"]

    # 2) Authorization 헤더에 토큰을 실어 보호 라우트 호출
    headers = {"Authorization": f"Bearer {token}"}
    response = client.get("/orders/me", headers=headers)
    assert response.status_code == 200
    assert response.json()["user"] == "buyer@shop.com"

def test_protected_route_without_token_returns_401(client: TestClient):
    # 토큰 없이 호출하면 401
    response = client.get("/orders/me")
    assert response.status_code == 401

def test_login_with_wrong_password_returns_401(client: TestClient):
    # 잘못된 비밀번호는 401
    response = client.post(
        "/token",
        data={"username": "buyer@shop.com", "password": "wrong"},
    )
    assert response.status_code == 401

```

인증 테스트 출별 해설

- `client.post("/token", data={...})`: 로그인은 폼 데이터이므로 `json=` 이 아니라 `data=` 를 씁니다. 이 차이를 헛갈리면 422가 나니 주의합니다.
- `token = login.json()["access_token"]`: 응답에서 토큰을 꺼냅니다. 이 토큰을 다음 요청의 헤더에 씁니다.
- `headers = {"Authorization": f"Bearer {token}"}`: 보호 라우트는 이 헤더를 봅니다. `Bearer` 접두어를 빠뜨리면 안 됩니다.
- `test_protected_route_without_token_returns_401`: 토큰을 안 보내면 `OAuth2PasswordBearer` 가 401을 내는지 검증합니다. 음성 경로를 빠뜨리지 않는 것이 좋은 테스트의 핵심입니다.

실행 결과는 다음과 같습니다.

```
tests/test_auth.py::test_login_returns_token PASSED [ 25%]
tests/test_auth.py::test_protected_route_with_token_returns_200 PASSED [ 50%]
tests/test_auth.py::test_protected_route_without_token_returns_401 PASSED [ 75%]
tests/test_auth.py::test_login_with_wrong_password_returns_401 PASSED [100%]

===== 4 passed in 0.51s =====
```

로그인·보호·무토큰·실패까지 네 경로가 모두 자동으로 검증됐습니다.

11.7 검증 오류(422) 단언과 테스트 격리

좋은 테스트는 **정상 경로뿐 아니라 실패 경로**도 검증합니다. 3장에서 배운 Pydantic 검증이 정말로 422를 내는지 확인해 봅시다. 음수 가격으로 상품을 만들려 하면 어떻게 되는지 테스트합니다.


```
# tests/test_validation.py - 검증 오류(422) 단언
def test_negative_price_returns_422(client):
    # price에 음수를 넣으면 Pydantic 검증이 422를 낸다고 가정
    response = client.post(
        "/products",
        json={"name": "불량 상품", "category": "기타",
              "price": -100, "stock": 5},
    )
    assert response.status_code == 422
    # 어떤 필드가 왜 틀렸는지 detail에 담겨 온다
    detail = response.json()["detail"]
    assert detail[0]["loc"][-1] == "price"

def test_missing_required_field_returns_422(client):
    # 필수 필드 name을 빼면 422
    response = client.post(
        "/products",
        json={"category": "기타", "price": 100, "stock": 5},
    )
    assert response.status_code == 422
```

422 테스트 줄별 해설

- `assert response.status_code == 422`: FastAPI는 Pydantic 검증 실패를 자동으로 422로 바꿉니다. 핸들러 안에서 직접 검사하지 않아도 됩니다.
- `detail[0]["loc"][-1] == "price"`: 422 응답의 `detail`은 어떤 필드(`loc`)가 왜(`msg`) 틀렸는지를 담은 리스트입니다. `loc`의 마지막이 문제 필드명이므로, 어떤 필드에서 실패했는지까지 단언할 수 있습니다.
- `test_missing_required_field_returns_422`: 필수 필드를 누락하면 422가 나는지 확인합니다.

테스트 격리: 왜 매번 새 DB인가

11.4의 `session` 픽스처는 **매 테스트마다** 새 인메모리 엔진을 만들고 테이블을 새로 생성했습니다. 이것이 **테스트 격리**입니다. 만약 모든 테스트가 같은 DB를 공유하면, 앞 테스트가 만든 상품이 뒤 테스트에 남아 `assert len(products) == 1` 같은 단언이 실행 순서에 따라 들쭉날쭉 실패합니다.

테스트는 서로 독립이어야 합니다

좋은 테스트 스위트의 철칙은 "어떤 순서로 돌려도, 하나만 돌려도, 전부 돌려도 결과가 같다"입니다. 이를 위해 각 테스트는 자기 데이터를 스스로 준비하고(픽스처), 끝나면 정리해야 합니다(`yield` 이후 또는 새 엔진). 전역 변수에 상태를 쌓거나, 앞 테스트의 부수효과에 기대는 순간 테스트는 신뢰를 잃습니다.

11.8 즉시 연습(2종 1세트): 오버라이드와 단언 채우기

이제 직접 작성할 차례입니다. 아래는 **상품 목록 페이지네이션** 엔드포인트를 검증하는 테스트의 빈 칸 코드입니다. 모범 답안의 형태를 떠올리며 빈칸을 채워 보십시오.

연습 11-A. 페이지네이션 테스트 빈칸 채우기

대상 엔드포인트는 `GET /products?limit=&offset=`이며, `limit` 기본값은 10입니다. 아래 빈칸(____)을 채워 "상품 3개를 만든 뒤 limit=2로 조회하면 2개만 온다"를 검증하세요.

```

import pytest
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from app.main import app, get_session

@pytest.fixture(name="session")
def session_fixture():
    engine = create_engine(
        "sqlite://",
        connect_args={"check_same_thread": False},
        poolclass=StaticPool,
    )
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session

@pytest.fixture(name="client")
def client_fixture(session: Session):
    def get_session_override():
        yield session

    # (1) 운영 세션 의존성을 테스트 세션으로 교체하세요
    app.dependency_overrides[get_session] = get_session_override
    client = TestClient(app)
    yield client
    # (2) 교체를 원복해 격리를 보장하세요
    app.dependency_overrides.clear()

def test_pagination_limits_results(client: TestClient):
    # 상품 3개 생성
    for i in range(3):
        client.post(
            "/products",
            json={"name": f"상품{i}", "category": "테스트",
                  "price": 1000, "stock": 1},
        )
    # limit=2로 조회
    response = client.get("/products", params={"limit": 2})
    # (3) 상태 코드와 개수를 단언하세요
    assert response.status_code == 200
    assert len(response.json()) == 2

```

자가 체크리스트 (스스로 점검하세요)

- [] (1) 교체의 키는 운영 의존성 함수 `get_session`, 값은 `get_session_override` 로 넣었습니까?
- [] (2) 정리 단계에서 `app.dependency_overrides.clear()` 를 호출했습니까?
- [] (3) 정상 목록 조회의 상태 코드는 200, 개수는 limit과 같은 2로 단언했습니까?
- [] 픽스처가 매 테스트마다 새 인메모리 DB를 만들어 격리됩니까?

- [] `pytest -v` 로 실행했을 때 통과합니까?

연습 11-A 정답 힌트

(1) `app.dependency_overrides[get_session] = get_session_override`, (2) `app.dependency_overrides.clear()`, (3) `200`, 2. 직접 채운 뒤 힌트와 대조해 보십시오.

11.9 흔한 오류와 오개념

전형적인 실수와 교정

오개념 1: 테스트를 운영 DB에서 돌린다. 테스트가 운영 DB에 직접 쓰면, 검증을 돌릴 때마다 실제 데이터가 더럽혀지고 결과가 환경에 따라 달라집니다. 반드시 `app.dependency_overrides` 로 세션 의존성을 격리된 테스트 DB(인메모리 SQLite 등) 로 교체해야 합니다. 이를 위해서라도 DB 접근은 항상 의존성으로 분리해 두어야 합니다.

오개념 2: async 핸들러는 비동기 테스트 도구가 있어야 한다고 생각한다. `TestClient` 는 내부에서 이벤트 루프를 직접 관리하므로, `async def` 핸들러도 동기적으로 호출할 수 있습니다. 즉 `await client.get(...)` 이 아니라 그냥 `client.get(...)` 으로 호출합니다. 테스트 함수도 `async def` 일 필요가 없습니다(`httpx.AsyncClient` 로 비동기 테스트를 쓸 수도 있지만, 입문 단계에서는 동기 `TestClient` 로 충분합니다).

오개념 3: 픽스처에서 DB를 초기화하지 않아 테스트가 서로 오염된다. 모든 테스트가 한 DB를 공유하면 앞 테스트가 남긴 데이터가 뒤 테스트를 망칩니다. 매 테스트마다 새 엔진·새 테이블을 만들거나, `yield` 이후 정리로 깨끗한 상태를 보장해야 합니다.

실수 4: 로그인 요청에 json= 을 쓴다. `OAuth2PasswordRequestForm` 은 폼 데이터를 받으므로 `client.post("/token", data={...})` 처럼 `data=` 를 써야 합니다. `json=` 을 쓰면 422가 납니다.

실수 5: 오버라이드를 원복하지 않는다. `app.dependency_overrides` 에 등록만 하고 `clear()` 로 비우지 않으면, 그 교체가 다른 테스트 파일까지 새어 들어가 엉뚱한 실패를 부릅니다. 픽스처의 `yield` 뒤에서 반드시 정리합니다.

11.10 단원 미니 프로젝트: 엔드포인트 테스트 스위트

이 단원의 개념(테스트·`TestClient`·`dependency_overrides`)만으로 작은 산출물을 완성하겠습니다. 목표는 상품 CRUD와 인증 흐름을 검증하는 테스트 스위트를 인메모리 DB 위에서 작성하는 것입니다. 마일스톤(증분)으로 나눠 진행합니다.

- **M1:** `conftest.py`에 인메모리 DB `session` 픽스처와 `dependency_overrides`로 교체한 `client` 픽스처를 만든다.
- **M2:** 상품을 생성(POST 201)한 뒤 조회(GET 200)해 값이 일치하는지 단언한다.
- **M3:** 토큰 발급(폼 로그인) → 보호 라우트 200, 무토큰 401, 검증 오류 422를 단언한다.

모범: 공통 픽스처(`conftest.py`)

```
# tests/conftest.py - 스위트 전체가 공유하는 픽스처
import pytest
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from app.main import app, get_session

@pytest.fixture(name="session")
def session_fixture():
    # 매 테스트마다 빈 인메모리 DB
    engine = create_engine(
        "sqlite://",
        connect_args={"check_same_thread": False},
        poolclass=StaticPool,
    )
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session

@pytest.fixture(name="client")
def client_fixture(session: Session):
    def get_session_override():
        yield session

    app.dependency_overrides[get_session] = get_session_override
    yield TestClient(app)
    app.dependency_overrides.clear()
```

모범 `conftest.py` 해설

- 이 파일은 `import` 하지 않아도 같은 디렉터리의 모든 테스트에서 `session`·`client` 픽스처를 자동으로 쓸 수 있게 합니다.
- `session`은 매 테스트마다 새 인메모리 DB를 만들어 격리를 보장합니다.
- `client`는 `get_session`을 교체한 뒤 `TestClient`를 넘기고, 끝나면 `clear()`로 원복합니다. 이 두 픽스처가 스위트의 토대입니다.

테스트 본문(`test_products.py`)은 11.4의 `test_create_then_read_product`를 그대로 쓰되, 픽스처가 `conftest.py`로 옮겨졌으므로 픽스처 정의는 지웁니다. 실행 결과는 다음과 같습니다.

```
tests/test_products.py::test_create_then_read_product PASSED
tests/test_auth.py::test_protected_route_with_token_returns_200 PASSED
tests/test_auth.py::test_protected_route_without_token_returns_401 PASSED
tests/test_validation.py::test_negative_price_returns_422 PASSED

===== 4 passed in 0.62s =====
```

작성형 빈 스타터(직접 작성)

미니 프로젝트 스타터 — 직접 작성

아래 골격을 채워 `tests/test_orders.py` 를 완성하세요. 검증할 흐름은 "로그인한 사용자가 주문을 생성(POST /orders, 201)하면 본인 주문 목록(GET /orders/me)에 그 주문이 보인다"입니다.

```
# tests/test_orders.py - 직접 작성
def _login(client) -> dict:
    # TODO: /token에 폼 데이터로 로그인하고
    #       Authorization 헤더 딕셔너리를 만들어 반환하세요
    login = client.post("/token", data={"username": ____, "password": ____})
    token = ____
    return {"Authorization": f"Bearer {token}"}

def test_create_order_then_list_mine(client):
    headers = _login(client)
    # TODO: 주문 생성 (201)
    created = client.post("/orders", json={"items": [____]}, headers=headers)
    assert created.status_code == ____

    # TODO: 본인 주문 목록 조회 (200) 후 1건인지 단언
    mine = client.get("/orders/me", headers=headers)
    assert mine.status_code == ____
    assert len(mine.json()) == ____

def test_create_order_without_token_returns_401(client):
    # TODO: 토큰 없이 주문 생성 시 401을 단언하세요
    response = client.post("/orders", json={"items": []})
    assert response.status_code == ____
```

자가 체크리스트 (스스로 점검하세요)

- [] `_login` 이 폼 데이터(`data=`)로 로그인하고 `Bearer` 헤더를 만들었습니까?
- [] 주문 생성이 201, 본인 목록 조회가 200으로 단언됩니까?
- [] 무토큰 주문 생성이 401로 막히는 음성 경로를 검증했습니까?
- [] 모든 테스트가 인메모리 DB 위에서 서로 격리되어 있습니까?
- [] `pytest -v` 로 전체가 통과합니까?

11.11 자주 묻는 질문(FAQ)

Q. `TestClient` 와 `httpx.AsyncClient` 중 무엇을 써야 합니까? A. 입문·대부분의 실무에서는 동기 `TestClient` 로 충분합니다. `async def` 핸들러도 동기적으로 호출됩니다. 비동기 의존성을 진짜 비동기로 검증해야 하거나, 동시성 자체를 테스트할 때만 `httpx.AsyncClient` + `pytest-asyncio` 를 고려합니다.

Q. 매 테스트마다 DB를 새로 만들면 느리지 않습니까? A. 인메모리 SQLite는 생성·삭제가 매우 빨라 수백 개 테스트도 몇 초 안에 끝납니다. 속도보다 **격리**가 훨씬 중요합니다.

Q. 실제 PostgreSQL로도 테스트해야 합니까? A. 단위·통합 테스트는 인메모리 SQLite로 빠르게 돌리고, 배포 전 일부 흐름만 실제 DB로 검증하는 이중 전략이 흔합니다. 다만 SQLite와 PostgreSQL의 SQL 차이(예: 특정 타입)가 문제 될 수 있으니, 운영과 동일한 DB로 도는 통합 테스트도 일부 두면 좋습니다.

Q. 외부 결제·메일 호출은 테스트에서 어떻게 합니까? A. 그 호출을 의존성이나 서비스 객체로 분리해 두고 `dependency_overrides` 나 모킹으로 교체합니다. 다음 장(12장)에서 백그라운드 작업과 함께 모킹 전략을 더 다룹니다.

11.12 정리

이 장에서는 **테스트(pytest·TestClient)** 개념 하나를 깊이 익혔습니다.

- `TestClient` 는 서버 없이 앱을 직접 호출해 상태 코드·본문을 검증합니다.
- `pytest` 픽스처로 인메모리 DB·클라이언트를 구성하고 매 테스트마다 격리합니다.
- `app.dependency_overrides` 로 DB 세션·인증 의존성을 테스트용으로 교체해, 운영 코드를 더럽히지 않고 검증합니다.
- 정상 경로(201·200)뿐 아니라 음성 경로(401·404·422)까지 단언해야 좋은 테스트입니다.

다음 장에서는 응답을 보낸 뒤 실행하는 백그라운드 작업과, 설정·배포로 앱을 프로덕션에 올리는 마지막 개념을 배웁니다. 그리고 종합 프로젝트(주문·이커머스 API)에서 이 테스트 기법으로 전체 흐름을 검증하게 됩니다.

12장 프로덕션: 백그라운드 작업·설정·배포

이 장에서 배우는 것

- `BackgroundTasks` 로 응답을 먼저 돌려준 뒤 후속 작업(확인 메일·로그 적재)을 실행할 수 있습니다.
- `lifespan` (`asynccontextmanager`)으로 앱 시작·종료 시 자원(DB 연결·외부 클라이언트)을 안전하게 열고 닫을 수 있습니다.
- `pydantic-settings` (`BaseSettings`)로 환경변수와 `.env` 에서 설정·시크릿을 형 검증과 함께 로드할 수 있습니다.
- `uvicorn` 워커와 멀티스테이지 `Dockerfile` 로 앱을 컨테이너화해 프로덕션에 배포할 수 있습니다.

이 장에서 가르치는 개념은 하나입니다. **백그라운드 작업·설정·배포(프로덕션)** 입니다. 개념 단원의 마지막 장으로, 지금까지 만든 API를 실제 운영 환경에 올리기 위한 마무리 도구들을 익힙니다.

12.1 도입: 응답 후 작업과 운영 준비

앞 장(11장)에서 우리는 `TestClient` 와 `pytest` 로 API를 자동 검증하는 법을 배웠습니다. 이제 검증된 앱을 **실제로 운영에 올릴** 차례입니다. 운영에서는 개발할 때 신경 쓰지 않던 세 가지 문제가 새로 등장합니다.

운영에서 새로 생기는 세 가지 문제

1. **응답을 빨리 돌려줘야 하는데 후속 작업이 무겁다.** 주문이 성공하면 확인 메일을 보내야 합니다. 그런데 메일 발송을 응답 안에서 기다리면 사용자는 메일 서버가 느린 만큼 응답을 늦게 받습니다. **응답은 먼저 보내고, 메일은 그 뒤에 보내야** 합니다.
2. **시작·종료 시 준비·정리가 필요하다.** 앱이 뜰 때 DB 연결 풀을 만들고, 내려갈 때 깨끗이 닫아야 합니다. 매 요청마다 하면 비효율이고, 안 하면 자원이 썩습니다.
3. **설정과 시크릿을 코드에 박으면 안 된다.** DB 비밀번호·JWT 키를 소스 코드에 하드코딩하면 보안 사고로 직결됩니다. 환경마다(개발·스테이징·운영) 다른 값을 **환경변수**로 주입해야 합니다.

이 세 문제를 FastAPI는 각각 `BackgroundTasks`, `lifespan`, `pydantic-settings` 로 풀고, 마지막으로 `Dockerfile` 과 `uvicorn` 워커로 어디서든 동일하게 실행되도록 배포합니다. 이 네 가지를 묶어 "프로

덕션 준비"라고 부릅니다.

선수 개념 상기: 이 장은 11장(테스트)을 선수로 합니다. 백그라운드 작업과 설정은 테스트로 검증할 대상이며, `BackgroundTasks` 도 의존성(6장)처럼 핸들러 인자로 주입됩니다. `lifespan` 에서 만든 DB 엔진은 7장에서 배운 `SQLModel engine` 이고, 결제·메일 모킹은 5장(비동기)·10장(예외)에서 다룬 외부 호출 패턴을 잇습니다.

12.2 정의: BackgroundTasks·lifespan·pydantic-settings

핵심 도구를 한 문장씩 정의하겠습니다.

백그라운드 작업·설정·배포란, `BackgroundTasks` 로 응답을 보낸 뒤 후속 작업을 같은 프로세스에서 실행하고, `lifespan` 컨텍스트로 시작·종료 자원을, `pydantic-settings` 로 환경변수 기반 설정을 관리하며, `uvicorn` 워커와 `Dockerfile` 로 컨테이너화해 프로덕션에 배포하는 일련의 운영 기법입니다.

용어	한글(영문)	의미
백그라운드 작업	<code>BackgroundTasks</code>	응답을 반환한 뒤 같은 프로세스에서 실행되는 후속 작업(메일·로그 등)을 등록하는 도구
수명주기	<code>lifespan</code>	앱이 시작될 때 한 번, 종료될 때 한 번 실행되는 코드를 담은 비동기 컨텍스트 매니저
설정	<code>pydantic-settings</code>	환경변수 <code>.env</code> 의 값을 형 검증과 함께 읽어 오는 <code>BaseSettings</code> 기반 설정 클래스
워커	<code>worker</code>	요청을 처리하는 프로세스. 여러 개를 띄워 동시 처리량을 늘립니다.
컨테이너화	<code>containerization</code>	앱과 의존성을 이미지 하나로 묶어 어디서든 동일하게 실행하는 것(Docker)

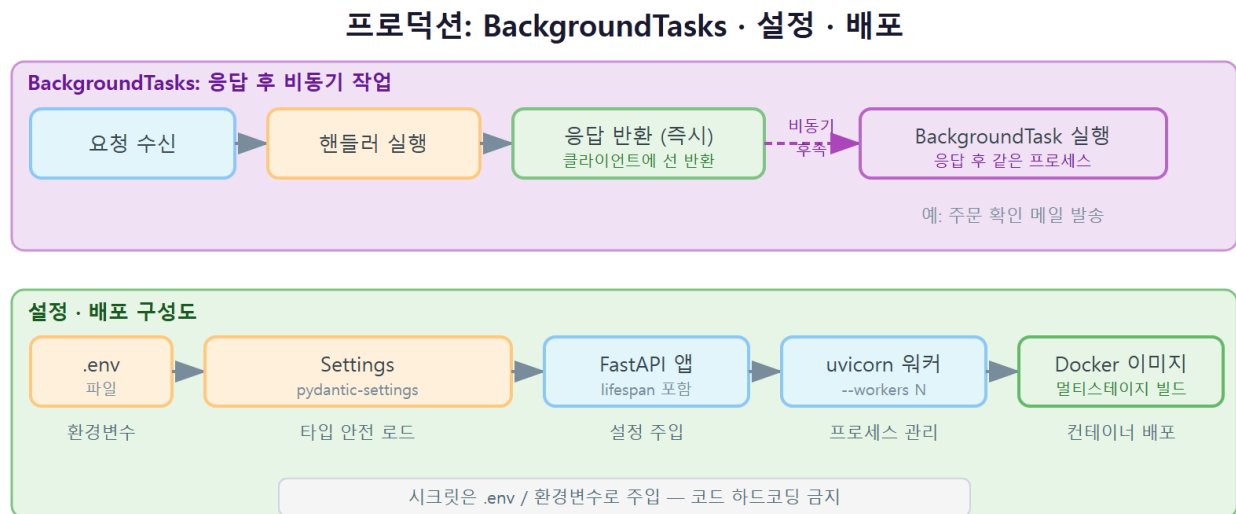
BackgroundTasks 는 "응답 후"이지 "다른 서버"가 아닙니다

`BackgroundTasks` 는 응답을 먼저 보내고 **같은 파이썬 프로세스**에서 후속 작업을 이어 실행합니다. 즉 진짜 별도 워커 큐(Celery 등)가 아닙니다. 가볍고 빠른 작업(확인 메일 한 통, 로그 한 줄)에는 완벽하지만, 수 분이 걸리거나 분산·재시도가 필요한 무거운 작업에는 메시지 큐가 따로 필요합니다. 이 경계를 아는 것이 중요합니다.

설치는 다음과 같습니다. `pydantic-settings` 는 별도 패키지입니다.

```
pip install pydantic-settings
```

12.3 동작 원리: 응답에서 백그라운드, `.env`에서 Settings, Docker 배포



이 장의 네 도구가 운영 흐름에서 어떻게 맞물리는지 단계로 보겠습니다.

- 요청 처리와 응답:** 클라이언트가 `POST /orders` 를 보내면 핸들러가 주문을 저장하고 **즉시 201 응답**을 돌려줍니다.
- 백그라운드 실행:** 응답이 나간 직후, 핸들러가 `background_tasks.add_task(...)` 로 등록해 둔 확인 메일 발송이 같은 프로세스에서 실행됩니다. 사용자는 메일을 기다리지 않습니다.
- 시작 자원(lifespan):** 위 모든 처리에 앞서, 앱이 뜰 때 `lifespan` 이 DB 테이블을 만들고 외부 클라이언트를 준비합니다. 앱이 내려갈 때는 같은 `lifespan` 이 자원을 정리합니다.
- 설정 주입(.env → Settings):** DB URL·JWT 키·메일 서버 주소 같은 값은 코드가 아니라 `.env` 나 환경변수에 있습니다. `Settings` 가 앱 시작 시 이를 읽어 형 검증된 설정 객체로 만듭니다.
- 배포(Docker → uvicorn):** 앱·의존성을 멀티스테이지 `Dockerfile` 로 이미지화하고, 컨테이너 안에서 `uvicorn` 이 워커 프로세스로 앱을 실행합니다. 시크릿은 이미지가 아니라 **실행 시 환경**

변수로 주입합니다.

`@app.on_event` 대신 `lifespan` 을 쓰세요

예전 FastAPI 코드에서 `@app.on_event("startup") / @app.on_event("shutdown")` 를 본 적이 있을 것입니다. 이 방식은 현재 **권장되지 않습니다(deprecated)**. 시작·종료 로직을 하나의 `asynccontextmanager (lifespan)`에 모으는 것이 현재의 표준입니다. `yield` 앞은 시작, `yield` 뒤는 종료입니다. 6장에서 배운 `yield` 의존성의 `setup/teardown`과 똑같은 모양이라 익숙할 것입니다.

12.4 worked example: 주문 확인 메일 백그라운드(모킹)

완전히 풀어 보이는 첫 예제입니다. 주문이 성공하면 **응답을 먼저 보내고**, 그 뒤에 확인 메일을 백그라운드로 발송합니다. 실제 SMTP 대신 콘솔에 출력하는 모킹으로 대체해 결정론을 확보합니다(11장에서 배운 모킹 전략의 연장입니다).

```
# app/orders.py - 주문 확인 메일을 백그라운드로 발송
import time

from fastapi import BackgroundTasks, FastAPI
from pydantic import BaseModel

app = FastAPI()

class OrderCreate(BaseModel):
    # 주문 입력 모델 (간략화)
    product_name: str
    quantity: int
    buyer_email: str

def send_confirmation_email(email: str, summary: str) -> None:
    # 실제 메일 발송을 흉내 내는 모킹 함수
    time.sleep(2) # 메일 서버가 느리다고 가정 (2초)
    print(f"[메일발송] 받는사람={email} / 내용={summary}")

@app.post("/orders", status_code=201)
def create_order(order: OrderCreate, background_tasks: BackgroundTasks):
    # 1) 주문 처리 (여기서는 요약 문자열만 생성)
    summary = f"{order.product_name} x {order.quantity}"

    # 2) 메일 발송을 백그라운드 작업으로 등록 (아직 실행 안 됨)
    background_tasks.add_task(
        send_confirmation_email, order.buyer_email, summary
    )

    # 3) 응답을 즉시 반환 -> 사용자는 메일 발송(2초)을 기다리지 않음
    return {"status": "created", "summary": summary}
```

백그라운드 예제 줄별 해설

- `def create_order(order: OrderCreate, background_tasks: BackgroundTasks):` 핸들러 인자에 `BackgroundTasks` 를 선언하면 FastAPI가 자동으로 주입합니다. 의존성처럼 그냥 받기만 하면 됩니다.
- `background_tasks.add_task(send_confirmation_email, order.buyer_email, summary):` 실행할 함수와 그 인자를 **등록만** 합니다. 이 줄에서 메일이 바로 나가지 않습니다. 함수는 괄호 없이 넘기고(`send_confirmation_email`), 인자는 뒤에 나열합니다.
- `return {...}`: 응답이 먼저 나갑니다. **응답이 클라이언트에 전송된 직후**, 등록된 `send_confirmation_email` 이 실행됩니다. `time.sleep(2)` 가 있어도 사용자는 즉시 201을 받습니다.

실행해 호출하면 다음과 같이 동작합니다. 응답은 곧바로 오고, 2초 뒤 서버 콘솔에 메일 로그가 찍힙니다.

```
$ curl -X POST localhost:8000/orders \
  -H "Content-Type: application/json" \
  -d '{"product_name":"키보드","quantity":2,"buyer_email":"buyer@shop.com"}'

{"status":"created","summary":"키보드 x 2"}

# (응답은 즉시, 2초 후 서버 콘솔에)
[메일발송] 받는사람=buyer@shop.com / 내용=키보드 x 2
```

백그라운드 작업은 같은 프로세스에서 돕니다

`send_confirmation_email` 이 2초가 아니라 5분 걸린다면, 그 5분 동안 이 워커 프로세스의 자원을 점유합니다. 또 프로세스가 중간에 죽으면 작업은 사라지고 재시도되지 않습니다. 그래서 **가볍고 실패해도 큰일 아닌 작업**(확인 메일·접속 로그)에만 `BackgroundTasks` 를 쓰고, 무겁거나 반드시 완료돼야 하는 작업은 Celery 같은 분산 큐로 분리합니다.

12.5 lifespan으로 시작/종료 자원 관리

앱이 뜰 때 DB 테이블을 만들고, 내려갈 때 정리하는 코드를 `lifespan` 에 담아 보겠습니다. `lifespan` 은 `@asynccontextmanager` 로 감싼 비동기 함수이며, `yield` 앞이 시작, 뒤가 종료입니다.

```
# app/main.py - lifespan으로 시작/종료 자원 관리
from contextlib import asynccontextmanager

from fastapi import FastAPI
from sqlmodel import SQLModel, create_engine

engine = create_engine("sqlite:///./shop.db")

@asynccontextmanager
async def lifespan(app: FastAPI):
    # === 시작(startup) ===
    print("앱 시작: DB 테이블을 생성합니다")
    SQLModel.metadata.create_all(engine) # 테이블 준비
    yield # 이 지점에서 앱이 요청을 받기 시작
    # === 종료(shutdown) ===
    print("앱 종료: 연결을 정리합니다")
    engine.dispose() # 연결 풀 정리

app = FastAPI(lifespan=lifespan)

@app.get("/health")
def health():
    # 헬스체크: 배포 환경에서 앱이 살아 있는지 확인하는 용도
    return {"status": "ok"}
```

lifespan 예제 줄별 해설

- `@asynccontextmanager` + `async def lifespan(app: FastAPI):` 시작·종료 로직을 하나의 컨텍스트 매니저로 묶습니다. `app` 을 인자로 받아 앱 상태에 자원을 붙일 수도 있습니다.
- `yield` 앞: 앱이 요청을 받기 **전에** 한 번 실행됩니다. 여기서 `create_all` 로 테이블을 만듭니다.
- `yield` 뒤: 앱이 종료될 때 실행됩니다. `engine.dispose()` 로 연결 풀을 정리합니다. `yield` 의존성의 teardown과 똑같은 구조입니다.
- `app = FastAPI(lifespan=lifespan):` 만든 `lifespan` 을 앱 생성 시 넘깁니다. 이것으로 등록이 끝납니다.
- `/health`: 컨테이너 오케스트레이터가 앱 생존을 확인하는 헬스체크 엔드포인트입니다. 운영에서 거의 필수입니다.

실행하면 시작·종료 시점에 로그가 한 번씩 찍힙니다.

```
$ uvicorn app.main:app
앱 시작: DB 테이블을 생성합니다
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000

# (Ctrl+C 로 종료하면)
앱 종료: 연결을 정리합니다
INFO:      Application shutdown complete.
```

테스트에서 lifespan을 실행하려면

`TestClient` 를 `with TestClient(app) as client:` 처럼 `with` 블록으로 쓰면 `lifespan` 의 시작·종료가 함께 실행됩니다. 단순 `TestClient(app)` 만으로는 `lifespan` 이 실행되지 않을 수 있으니, `lifespan` 에서 만든 자원이 필요한 테스트는 `with` 형태를 쓰십시오. 11장의 픽스처에서 인메모리 DB를 직접 만든 이유도, 운영 `lifespan` 에 의존하지 않고 격리하기 위해서였습니다.

12.6 worked example: Settings로 환경변수·시크릿 로드

두 번째 완전한 예제입니다. DB URL·JWT 키·메일 발신자 같은 값을 코드에 박지 않고 `pydantic-settings` 로 환경변수에서 읽습니다.

```
# app/config.py - pydantic-settings로 환경변수/시크릿 로드
from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    # .env 파일과 환경변수에서 값을 읽도록 설정
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    # 기본값이 없는 필드는 필수: 없으면 시작 시 에러로 잡힘
    database_url: str
    jwt_secret: str
    jwt_algorithm: str = "HS256" # 기본값 있음 = 선택
    access_token_expire_minutes: int = 30
    mail_sender: str = "no-reply@shop.com"
    debug: bool = False

    @lru_cache
    def get_settings() -> Settings:
        # 설정을 한 번만 읽어 캐시 (의존성으로도 주입 가능)
        return Settings()
```


다음과 같은 `.env` 파일을 프로젝트 루트에 두면, 코드 수정 없이 환경마다 다른 값을 줄 수 있습니다.

```
# .env (운영에서는 비밀저장소/오케스트레이터가 주입)
DATABASE_URL=postgresql+psycopg://shop:s3cr3t@db:5432/shop
JWT_SECRET=please-change-this-very-long-random-string
ACCESS_TOKEN_EXPIRE_MINUTES=60
DEBUG=false
```

설정을 핸들러에서 의존성으로 주입해 쓰는 모습은 다음과 같습니다.

```
# app/main.py - Settings를 의존성으로 주입
from typing import Annotated

from fastapi import Depends, FastAPI

from app.config import Settings, get_settings

app = FastAPI()

SettingsDep = Annotated[Settings, Depends(get_settings)]

@app.get("/config-check")
def config_check(settings: SettingsDep):
    # 시크릿 자체는 절대 노출하지 않고, 안전한 메타데이터만 반환
    return {
        "debug": settings.debug,
        "token_expire_minutes": settings.access_token_expire_minutes,
        "db_configured": bool(settings.database_url),
    }
```

Settings 예제 줄별 해설

- `class Settings(BaseSettings):` `pydantic-settings`의 `BaseSettings`를 상속하면 필드가 자동으로 환경변수와 매핑됩니다. 필드명 `database_url`은 환경변수 `DATABASE_URL`(대문자)과 매칭됩니다.
- `model_config = SettingsConfigDict(env_file=".env"):` `.env` 파일도 읽도록 지정합니다. 환경변수가 `.env`보다 우선합니다.
- `database_url: str`(기본값 없음): 필수입니다. 환경변수도 `.env`도 없으면 앱 시작 시 검증 오류로 즉시 실패합니다. 이렇게 "빠르게 실패(fail fast)"해야 운영에서 설정 누락을 일찍 잡습니다.
- `jwt_algorithm: str = "HS256"`(기본값 있음): 선택입니다. 안 주면 기본값을 씁니다.
- `@lru_cache`로 감싼 `get_settings`: 설정을 매번 다시 읽지 않고 한 번만 읽어 재사용합니다. `Depends(get_settings)`로 핸들러에 주입할 수 있습니다.
- `config_check`: 시크릿 값 자체는 응답에 절대 담지 않습니다. `db_configured`처럼 "설정됐는지" 여부 같은 안전한 정보만 노출합니다.

실행 결과는 다음과 같습니다.

```
$ curl localhost:8000/config-check
{"debug":false,"token_expire_minutes":60,"db_configured":true}
```

JWT 키나 DB 비밀번호가 응답에 전혀 노출되지 않은 점에 주목하십시오.

12.7 uvicorn 워커와 멀티스테이지 Dockerfile

마지막은 배포입니다. 앱을 어디서든 동일하게 실행하기 위해 컨테이너 이미지로 만듭니다. **멀티스테이지(multi-stage)** 빌드는 의존성을 빌드하는 단계와 실제 실행 단계를 나눠 최종 이미지를 가볍게 만드는 기법입니다.

```
# Dockerfile - 멀티스테이지 빌드
# === 1단계: 빌드(의존성 설치) ===
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
# 가상환경에 의존성을 설치 (이 레이어는 캐시됨)
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir -r requirements.txt

# === 2단계: 실행(가벼운 최종 이미지) ===
FROM python:3.12-slim
WORKDIR /app
# 빌드 단계에서 만든 가상환경만 복사 (빌드 도구는 제외 -> 이미지 경량화)
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY ./app ./app
EXPOSE 8000
# 운영 실행: --reload 없이, 워커 수를 지정
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

Dockerfile 줄별 해설

- `FROM python:3.12-slim AS builder`: 1단계를 `builder` 로 명명합니다. 여기서 의존성을 설치합니다.
- `RUN python -m venv /opt/venv`: 가상환경에 의존성을 깔아 두고, 2단계에서 이 폴더만 복사합니다. 빌드에만 필요한 도구는 최종 이미지에 남지 않아 가벼워집니다.
- `FROM python:3.12-slim` (2단계): 실행 전용의 깨끗한 이미지로 다시 시작합니다.
- `COPY --from=builder /opt/venv /opt/venv`: 1단계의 가상환경만 가져옵니다. 이것이 멀티스테이지의 핵심입니다.
- `EXPOSE 8000`: 컨테이너가 8000 포트를 쓴다고 문서화합니다.

- CMD [... "--workers", "4"]: 운영 실행 명령입니다. `--reload` 는 **개발 전용**이라 넣지 않고, `--host 0.0.0.0` 으로 컨테이너 밖에서 접근 가능하게 하며, `--workers 4` 로 워커 4개를 띄워 동시 처리량을 늘립니다.

빌드와 실행은 다음과 같습니다. 시크릿은 **이미지에 굽지 않고 실행 시 주입**하는 점이 핵심입니다.

```
$ docker build -t shop-api .
$ docker run -p 8000:8000 \
  -e DATABASE_URL="postgresql+psycopg://shop:s3cr3t@db:5432/shop" \
  -e JWT_SECRET="prod-very-long-random-secret" \
  shop-api

INFO:      Uvicorn running on http://0.0.0.0:8000
INFO:      Started parent process [1]
INFO:      Started server process [8]
INFO:      Application startup complete.
```

워커 수와 프로세스 관리

워커 수는 보통 "CPU 코어 수 x 2 + 1" 정도에서 시작해 부하 테스트로 조정합니다. 더 정교한 운영에서는 `uvicorn` 을 직접 띄우는 대신 `gunicorn` 에 `uvicorn` 워커 클래스를 얹거나, 쿠버네티스가 여러 컨테이너(파드)로 수평 확장하기도 합니다. 핵심 원칙은 변하지 않습니다. `--reload` 는 **개발용, 운영은 다중 워커·외부 주입 시크릿·헬스체크**입니다.

12.8 즉시 연습(2종 1세트): BackgroundTasks와 Settings 채우기

직접 작성할 차례입니다. 아래 두 빈칸 코드를 모범의 형태를 떠올리며 채우십시오.

연습 12-A. 백그라운드 로그 적재 빈칸 채우기

주문 취소 시, 응답을 먼저 보내고 백그라운드로 감사 로그(audit log)를 적재하는 핸들러입니다. 빈칸(____)을 채우세요.

```

from fastapi import BackgroundTasks, FastAPI

app = FastAPI()

def write_audit_log(order_id: int, action: str) -> None:
    # 실제로는 파일/DB에 기록 (여기선 모킹)
    print(f"[감사로그] order={order_id} action={action}")

@app.delete("/orders/{order_id}", status_code=200)
def cancel_order(order_id: int, background_tasks: ____):
    # 1) 주문 취소 처리 (생략)
    # 2) 감사 로그 적재를 백그라운드로 등록하세요
    background_tasks.____(write_audit_log, order_id, "cancelled")
    # 3) 응답을 즉시 반환
    return {"status": "cancelled", "order_id": order_id}

```

연습 12-B. Settings 필수 필드 채우기

DB URL과 JWT 키는 필수, 디버그 플래그는 선택(기본 False)인 Settings 를 완성하세요.

```

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file="____")

    # 필수 (기본값 없음)
    database_url: ____
    jwt_secret: ____
    # 선택 (기본값 있음)
    debug: bool = ____

```

자가 체크리스트 (스스로 점검하세요)

- [] 12-A에서 핸들러가 `BackgroundTasks` 를 인자로 받고 `add_task` 로 등록했습니까?
- [] 응답이 백그라운드 작업보다 먼저 반환되도록 작성했습니까?
- [] 12-B에서 필수 필드는 기본값을 주지 않고, 선택 필드만 기본값을 주었습니까?
- [] `env_file` 을 `.env` 로 지정했습니까?
- [] 시크릿 값을 코드에 직접 적지 않았습니까?

연습 정답 힌트

12-A: `BackgroundTasks`, `add_task`. 12-B: `".env"`, `str`, `str`, `False`. 직접 채운 뒤 대조하십시오.

12.9 혼한 오류와 오개념

전형적인 실수와 교정

오개념 1: BackgroundTasks 가 별도 워커·큐라고 생각한다. BackgroundTasks 는 응답 후 같은 프로세스에서 실행됩니다. 별도 서버나 재시도가 아닙니다. 무겁거나(수 분), 반드시 완료돼야 하거나, 분산 처리가 필요한 작업은 Celery-RQ 같은 메시지 큐로 분리해야 합니다.

오개념 2: 시크릿을 코드나 이미지에 하드코딩한다. DB 비밀번호·JWT 키를 소스에 적거나 Dockerfile 에 `ENV JWT_SECRET=...` 로 굽는 것은 위험합니다. 시크릿은 **실행 시 환경변수**(`docker run -e`, 오케스트레이터의 시크릿 저장소)로 주입하고, 코드는 `Settings` 로 읽기만 합니다. `.env` 는 `.gitignore` 에 넣어 저장소에 올리지 않습니다.

오개념 3: 운영에서 --reload 를 켜다. `--reload` 는 코드 변경을 감지해 자동 재시작하는 **개발 전용** 옵션입니다. 운영에서 켜면 성능이 떨어지고 불안정합니다. 운영은 `--reload` 없이 워커 수를 지정해 띄웁니다.

실수 4: @app.on_event 를 계속 쓴다. `@app.on_event("startup"/"shutdown")` 은 deprecated입니다. 시작·종료를 하나의 `lifespan` (`asyncontextmanager`)으로 모으는 것이 현재 표준입니다.

실수 5: add_task 에 함수를 호출해서 넘긴다. `background_tasks.add_task(send_mail(email))` 처럼 **괄호로 호출**하면 그 자리에서 바로 실행되고 결과값이 등록됩니다. 의존성(`Depends`)과 똑같이, 함수는 괄호 없이(`send_mail`) 넘기고 인자는 뒤에 나열합니다.

12.10 단원 미니 프로젝트: 백그라운드 작업과 도커 배포

이 단원 개념(`BackgroundTasks`·`lifespan`·`pydantic-settings`·`Dockerfile`)만으로 작은 산출물을 완성하겠습니다. 목표는 **확인 메일 백그라운드 + 설정 로드 + lifespan + Dockerfile**을 갖춘 **최소 배포 구성**입니다. 마일스톤으로 나눕니다.

- **M1:** `BackgroundTasks` 로 주문 응답 후 확인 메일(모킹)을 발송한다.
- **M2:** `pydantic-settings` 로 DB URL·시크릿을 환경변수에서 로드하고, `lifespan` 으로 시작 시 테이블을 만든다.
- **M3:** 멀티스테이지 `Dockerfile` 로 이미지를 빌드하고 `uvicorn` 으로 실행한다.

모범: 통합 main.py

```

# app/main.py - 백그라운드 + 설정 + lifespan 통합
from contextlib import asynccontextmanager
from functools import lru_cache
from typing import Annotated

from fastapi import BackgroundTasks, Depends, FastAPI
from pydantic import BaseModel
from pydantic_settings import BaseSettings, SettingsConfigDict
from sqlmodel import SQLModel, create_engine

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env")
    database_url: str = "sqlite:///./shop.db"
    mail_sender: str = "no-reply@shop.com"

@lru_cache
def get_settings() -> Settings:
    return Settings()

# 시작 시 설정을 읽어 엔진 생성, lifespan에서 테이블 준비
settings = get_settings()
engine = create_engine(settings.database_url)

@asynccontextmanager
async def lifespan(app: FastAPI):
    print("앱 시작: 테이블 생성")
    SQLModel.metadata.create_all(engine)
    yield
    print("앱 종료: 엔진 정리")
    engine.dispose()

app = FastAPI(lifespan=lifespan)
SettingsDep = Annotated[Settings, Depends(get_settings)]

class OrderCreate(BaseModel):
    product_name: str
    buyer_email: str

def send_confirmation_email(sender: str, to: str, summary: str) -> None:
    # 모킹: 콘솔 출력
    print(f"[메일발송] from={sender} to={to} / {summary}")

@app.post("/orders", status_code=201)
def create_order(
    order: OrderCreate,
    background_tasks: BackgroundTasks,
    settings: SettingsDep,
):
    summary = f"{order.product_name} 주문 접수"

```

```
# 응답 후 확인 메일 발송 등록
background_tasks.add_task(
    send_confirmation_email, settings.mail_sender,
    order.buyer_email, summary,
)
return {"status": "created", "summary": summary}

@app.get("/health")
def health():
    return {"status": "ok"}
```

통합 main.py 해설

- Settings 가 .env 환경변수에서 database_url, mail_sender 를 읽습니다(기본값이 있어 로컬에서도 즉시 동작).
- lifespan 이 시작 시 테이블을 만들고 종료 시 엔진을 정리합니다.
- create_order 가 응답을 먼저 보내고 add_task 로 확인 메일을 등록합니다. settings 도 의존성으로 주입받아 발신자를 가져옵니다.
- /health 는 컨테이너 헬스체크용입니다. 네 도구(BackgroundTasks·lifespan·Settings·헬스체크)가 한 파일에 통합됐습니다.

실행 결과는 다음과 같습니다.

```
$ uvicorn app.main:app
앱 시작: 테이블 생성
INFO: Application startup complete.

# POST /orders 호출 시
{"status": "created", "summary": "키보드 주문 접수"}
[메일발송] from=no-reply@shop.com to=buyer@shop.com / 키보드 주문 접수
```

작성형 빈 스타터(직접 작성)

미니 프로젝트 스타터 — 직접 작성

아래 골격을 채워 배포 구성을 완성하세요. 위 모범을 참고하되 빈칸(____)을 직접 채웁니다.

```
# app/main.py - 직접 작성
from contextlib import asynccontextmanager
from functools import lru_cache

from fastapi import BackgroundTasks, FastAPI
from pydantic_settings import BaseSettings, SettingsConfigDict
from sqlmodel import SQLModel, create_engine

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file="____")
    database_url: str = "sqlite:///./shop.db"

@lru_cache
def get_settings() -> Settings:
    return Settings()

engine = create_engine(get_settings().database_url)

@asynccontextmanager
async def lifespan(app: FastAPI):
    # TODO: 시작 시 테이블 생성, 종료 시 엔진 정리
    ----
    yield
    ----

app = FastAPI(lifespan=____)

def send_mail(to: str, msg: str) -> None:
    print(f"[메일] {to}: {msg}")

@app.post("/orders", status_code=201)
def create_order(buyer_email: str, background_tasks: ____):
    # TODO: 응답 후 메일 발송 등록
    background_tasks.____(send_mail, buyer_email, "주문 접수")
    return {"status": "created"}
```

추가로, 위 앱을 컨테이너화하는 멀티스테이지 Dockerfile 의 마지막 실행 줄을 직접 작성하세요(운영, `--reload` 금지).

```
# Dockerfile 마지막 줄 - 직접 작성 (운영 실행)
CMD ["uvicorn", "____", "--host", "____", "--port", "8000", "--workers", "____"]
```

자가 체크리스트 (스스로 점검하세요)

- [] lifespan 의 yield 앞에서 create_all, 뒤에서 engine.dispose() 를 호출했습니까?

- [] `FastAPI(lifespan=lifespan)` 로 등록했습니까?
- [] 핸들러가 `BackgroundTasks` 를 받고 `add_task` 로 메일을 등록했습니까?
- [] `Dockerfile` 실행 줄에 `app.main:app`, `0.0.0.0`, 워커 수를 넣고 `--reload` 를 넣지 않았습니까?
- [] 시크릿을 코드가 아니라 환경변수로 주입하도록 설계했습니까?

12.11 자주 묻는 질문(FAQ)

Q. `BackgroundTasks` 와 `Celery`는 언제 갈라 써야 합니까? A. 가볍고(수 초 이내) 실패해도 치명적이지 않은 작업(확인 메일·로그)은 `BackgroundTasks` 로 충분합니다. 무겁거나(수 분), 반드시 완료·재시도돼야 하거나, 여러 서버로 분산해야 하는 작업은 `Celery-RQ` 같은 분산 큐가 필요합니다.

Q. `Settings()` 를 매번 `get_settings()` 로 받는데 성능 문제는 없습니까? A. `@lru_cache` 로 감싸면 처음 한 번만 환경변수를 읽고 이후에는 캐시된 객체를 돌려주므로 비용이 거의 없습니다. 의존성으로 주입하면 테스트에서 다른 설정으로 교체하기도 쉽습니다(11장의 `dependency_overrides`).

Q. `lifespan`에서 만든 자원을 핸들러에서 쓰려면 어떻게 합니까? A. `app.state` 에 붙이거나 (`app.state.client = ...`) 모듈 전역으로 두고 의존성으로 감싸 주입합니다. 요청마다 새로 만들 필요 없는 외부 클라이언트(HTTP·캐시 연결)는 `lifespan`에서 한 번 만들어 재사용하는 것이 효율적입니다.

Q. `.env` 파일을 깃에 올려도 됩니까? A. 안 됩니다. `.env` 에는 시크릿이 들어가므로 `.gitignore` 에 추가합니다. 대신 키 목록만 담은 `.env.example` 을 올려 어떤 변수가 필요한지 문서화합니다.

12.12 정리

이 장에서는 **백그라운드 작업·설정·배포(프로덕션)** 개념 하나를 깊이 익혔습니다.

- `BackgroundTasks` 로 응답을 먼저 보내고 후속 작업을 같은 프로세스에서 실행합니다(가벼운 작업 한정).
- `lifespan` (`asynccontextmanager`)으로 시작·종료 자원을 한곳에서 관리합니다(`@app.on_event` 대신).
- `pydantic-settings` 로 환경변수·`.env` 에서 설정·시크릿을 형 검증과 함께 읽고, 시크릿은 절대 코드에 박지 않습니다.
- 멀티스테이지 `Dockerfile` 과 `uvicorn` 워커로 가볍고 동일하게 실행되는 이미지를 만듭니다(`--reload` 는 개발용).

이로써 12개 개념 단원이 모두 끝났습니다. 다음 장(13장)부터는 지금까지 배운 모든 개념을 통합해 **주문·이커머스 API**를 분석·설계·구현·발표하는 종합 프로젝트로 들어갑니다. 이 장의 백그라운드 메일·설정·배포는 그 종합 프로젝트의 마지막 마일스톤에서 다시 씁니다.

종합 프로젝트: 주문·이커머스 API — 분석(요구사항·범위)

학습 목표

- 드라이빙 퀘스천을 측정 가능한 기능 요구사항(FR)·비기능 요구사항(NFR)으로 구체화할 수 있습니다.
- 사용자 스토리와 유스케이스로 주문 생성·조회·취소 흐름을 표현할 수 있습니다.
- 범위(In/Out)를 정해 외부 연동(결제·메일)을 모킹으로 통제할 수 있습니다.
- 각 요구사항이 1~12장의 어느 단원 개념으로 구현되는지 추적성 표로 매핑할 수 있습니다.

지금까지 우리는 1장부터 12장까지 FastAPI의 개념을 하나씩 익혔습니다. 첫 앱과 자동 문서(1장)에서 시작해, 파라미터·요청 본문 검증·응답 모델(2~4장), 비동기·의존성 주입(5~6장), 데이터베이스·CRUD 라우터(7~8장), 인증·예외·테스트·배포(9~12장)까지, 각 단원에서 작은 미니 프로젝트로 적용도 해 보았습니다. 이 종합 프로젝트는 그 흩어진 개념들을 **하나의 완성품으로 통합**하는 자리입니다. 우리가 만들 것은 `shop_api` — 실행 가능한 **주문·이커머스 REST API**입니다.

이번 장(분석)은 코드를 한 줄도 쓰지 않습니다. 대신 "무엇을 만들 것인가"를 명확히 합니다. 실무에서 가장 비싼 실수는 잘못 짠 코드가 아니라 **틀린 것을 정확히 만드는 일**입니다. 요구사항이 흐릿하면 구현은 표류합니다. 그래서 이 장에서는 드라이빙 퀘스천을 측정 가능한 요구사항으로 쪼개고, 사용자 관점의 스토리·유스케이스로 흐름을 그리고, 범위를 칼같이 그어 외부 연동을 모킹으로 통제하는 결정을 내립니다. 마지막으로 **각 요구사항이 어느 단원 개념으로 구현되는지** 추적성 표로 못박아, 커리큘럼과 프로젝트가 어긋나지 않게 정렬합니다.

이 장은 새로운 FastAPI 개념을 가르치지 않습니다. 1~12장에서 배운 개념(c01~c12)을 **분석 단계의 산출물로 통합**합니다. 모르는 개념이 나오면 해당 장으로 돌아가 복습하면 됩니다.

1. 드라이빙 퀘스천과 POV 다시 보기

종합 프로젝트의 출발점은 책 도입에서 제시한 **드라이빙 퀘스천(driving question)**입니다. 분석을 시작하기 전에 이 질문과 그 바탕이 된 관점(POV)을 다시 꺼내 봅니다. 분석이란 결국 이 한 문장을

"검증 가능한 요구사항의 묶음"으로 번역하는 작업이기 때문입니다.

드라이빙 퀘스천

실무 백엔드 개발자가 FastAPI로, 상품·재고·주문·인증·결제 흐름을 갖추고 테스트·배포까지 되는 **안전하고 견고한 이커머스 주문 API**를 어떻게 설계하고 구현할 수 있을까?

POV(관점 진술 — Point Of View)

- **사용자(User):** FastAPI를 새로 도입해 이커머스 백엔드를 구축하려는 실무 백엔드 개발자
- **필요(Need):** 상품·재고를 관리하고, 인증된 사용자의 주문을 받아 재고를 안전하게 차감하며, 결제(모킹)를 처리하고, 그 전 과정을 자동 테스트와 컨테이너 배포로 신뢰성 있게 운영하기
- **통찰(Insight):** 이커머스 주문 흐름은 검증·인증·트랜잭션·동시성·예외가 한 군데서 맞물리는 도메인이라, 흩어져 배운 FastAPI 개념들이 어떻게 하나의 견고한 서비스로 통합되는지를 가장 잘 드러낸다.

이 질문에는 분석으로 풀어야 할 키워드가 이미 들어 있습니다. "상품·재고·주문·인증·결제"는 **무엇을 하는가(기능)**에 대한 단서이고, "안전하고 견고한", "테스트·배포까지"는 **얼마나 잘 해야 하는가(품질)**에 대한 단서입니다. 분석의 다음 두 절은 정확히 이 둘을 나눠 다룹니다. 전자는 기능 요구사항(FR), 후자는 비기능 요구사항(NFR)입니다.

분석의 목표를 한 문장으로 정리하면 이렇습니다. **"드라이빙 퀘스천을, 누가 봐도 충족 여부를 판정할 수 있는 요구사항들로 분해하고, 각 요구사항을 우리가 배운 개념과 연결한다."** 이 정렬이 잘 되어 있으면 다음 장(설계)과 그다음 장(구현)이 흔들리지 않습니다.

2. 기능 요구사항(FR): 상품·주문·인증·결제

기능 요구사항(Functional Requirement, FR)은 시스템이 **무엇을 해야 하는지**를 기술합니다. 좋은 FR은 (1) 한 가지 동작만 담고, (2) 충족 여부를 객관적으로 판정할 수 있어야 합니다. 드라이빙 퀘스천의 "상품·재고·주문·인증·결제"를 9개의 FR로 분해했습니다. 각 FR에는 그것을 구현할 **단위(uses_units)**와 **개념(uses_concepts)**을 함께 적어, 분석 단계에서부터 추적성을 확보합니다.

ID	기능 요구사항	적용 단위	사용 개념
FR1	상품 등록·수정·삭제(관리자)와 목록·단건 조회를 제공한다	u3·u4·u8	c03·c04·c08
FR2	상품 목록을 페이지네이션하고 카테고리/가격으로 필터한다	u2·u8	c02·c08
FR3	사용자가 가입하고 로그인하면 JWT를 발급한다	u9	c09
FR4	인증된 사용자가 여러 품목을 담은 주문을 생성한다	u3·u7·u9	c03·c07·c09
FR5	주문 생성 시 재고를 차감하고 재고 부족은 409로 거부한다	u7·u10	c07·c10
FR6	결제를 모킹 게이트웨이로 비동기 호출해 처리한다	u5	c05
FR7	주문 확인 메일을 응답 후 백그라운드로 발송한다(모킹)	u12	c12
FR8	본인 주문만 조회·취소하도록 인가를 강제한다	u9	c09
FR9	잘못된 입력·없는 자원·권한 오류를 일관된 응답으로 처리한다	u4·u10	c04·c10

각 FR을 조금 더 풀어 설명하겠습니다.

- **FR1·FR2(상품)**: 상품은 이 API의 가장 기초적인 자원입니다. 관리자는 상품을 등록·수정·삭제하고, 누구나 목록·단건을 조회할 수 있어야 합니다. 목록은 데이터가 커질 수 있으므로 `limit / offset` 페이지네이션과 카테고리·가격 필터가 필요합니다. 이는 3장(요청 본문 검증)·4장(응답 모델)·8장(CRUD 라우터)에서 배운 내용을 그대로 씁니다.
- **FR3·FR8(인증·인가)**: 가입·로그인은 JWT를 발급하는 흐름이며, 주문 조회·취소는 "본인 것만" 접근할 수 있어야 합니다. 인증(누구인가, 401)과 인가(권한이 있는가, 403)의 구분은 9장의 핵심입니다.
- **FR4·FR5(주문·재고)**: 주문 생성은 이 프로젝트의 심장입니다. 여러 품목을 한 주문에 담고(3장 중첩 모델), DB 트랜잭션 안에서 재고를 확인·차감(7장)하며, 재고가 모자라면 도메인 예외를 409로 변환(10장)합니다.
- **FR6·FR7(결제·메일)**: 결제는 외부 게이트웨이를 비동기로 호출(5장)하고, 확인 메일은 응답을 보낸 뒤 백그라운드로 발송(12장)합니다. 둘 다 실제 외부 연동이 아니라 모킹으로 처리하는데, 그 이유는 5절 범위 결정에서 다룹니다.
- **FR9(오류 일관성)**: 어떤 오류든 클라이언트가 예측 가능한 형태로 받아야 합니다. 422(검증)·404(없음)·401/403(인증·인가)·409(재고)를 일관된 포맷으로 돌려줍니다.

FR을 잘 쓰는 요령

- "~할 수 있어야 한다"가 아니라 "~한다"로, 동작 하나만 담습니다.
- "빠르게", "잘"처럼 판정 불가능한 표현은 FR이 아니라 NFR로 보냅니다.
- 각 FR이 어떤 HTTP 엔드포인트·상태코드로 드러나는지 머릿속에 그려 둡니다(설계의 씨앗).

3. 비기능 요구사항(NFR): 검증·트랜잭션·비동기·테스트·배포

비기능 요구사항(Non-Functional Requirement, NFR)은 *얼마나 잘* 해야 하는지를 기술합니다. 같은 기능이라도 "검증되지 않은 입력을 그대로 받는 API"와 "모든 입력이 422로 걸러지는 API"는 품질이 다릅니다. 드라이빙 퀘스천의 "안전하고 견고한", "테스트·배포까지"를 6개의 NFR로 구체화했습니다.

ID	비기능 요구사항	사용 개념
NFR1	모든 입력이 Pydantic으로 검증되고 잘못된 입력은 422로 거부된다	c03
NFR2	DB 작업은 요청 단위 세션·트랜잭션으로 일관성을 유지한다	c07·c06
NFR3	외부 결제·배송 호출은 비동기로 이벤트 루프를 막지 않는다	c05
NFR4	핵심 엔드포인트가 자동 테스트로 검증되고 격리된 DB에서 결정론적으로 돈다	c11
NFR5	시크릿은 환경변수로 주입되고 Docker로 배포 가능하다	c12
NFR6	응답은 OpenAPI 문서로 자동 기술되고 민감 필드는 노출되지 않는다	c01·c04

NFR은 FR과 달리 시스템 전반에 걸쳐 작동하는 **품질 속성(quality attribute)**입니다. 예컨대 NFR1(검증)은 특정 엔드포인트만이 아니라 입력을 받는 모든 엔드포인트에 적용됩니다. NFR을 분석 단계에서 명시해 두면, 설계·구현에서 "이 부분은 검증이 빠졌네", "여기는 트랜잭션 경계가 모호하네" 같은 누락을 일찍 잡을 수 있습니다.

- **NFR1(검증)**: 3장의 Pydantic 모델로 본문을, 2장의 `Query()` · `Path()` 로 파라미터를 검증합니다. 핸들러 안에서 손으로 `if` 검사를 다시 짜지 않습니다.
- **NFR2(트랜잭션)**: 7장의 `get_session` yield 의존성으로 요청마다 세션을 열고, 6장의 의존성 주입으로 그 세션을 핸들러에 주입합니다. 주문처럼 여러 테이블을 건드리는 작업은 한 트랜잭션 안에서 처리해 일관성을 지킵니다.
- **NFR3(비동기)**: 결제·배송 같은 외부 I/O는 5장의 `async / await` 로 호출해, 한 요청이 외부 응답을 기다리는 동안 이벤트 루프가 다른 요청을 처리하게 합니다. 블로킹 호출을 `async` 안에 넣

어 루프를 막는 실수를 피합니다.

- **NFR4(테스트)**: 11장의 `TestClient` 와 `dependency_overrides` 로 인메모리 DB·인증을 교체해, 운영 환경과 무관하게 결정론적으로 도는 테스트를 만듭니다.
- **NFR5(배포)**: 12장의 `pydantic-settings` 로 시크릿(DB 비밀번호·JWT 키)을 환경변수에서 로드하고, 멀티스테이지 Dockerfile로 컨테이너화합니다.
- **NFR6(문서·보안)**: 1장의 자동 OpenAPI 문서로 모든 응답을 기술하되, 4장의 응답 모델 분리로 `password_hash` 같은 민감 필드는 출력에서 제외합니다.

FR과 NFR의 경계가 헛갈릴 때 "주문을 생성한다"는 FR입니다. "주문 생성이 잘못된 입력에 422를 돌려준다"는 그 FR의 **수용 기준**이면서, 동시에 NFR1(전역 검증 품질)의 한 사례입니다. 하나의 동작이 FR로도 NFR로도 드러날 수 있으며, 추적성 표(6절)가 이 둘을 개념과 연결해 정리합니다.

4. 사용자 스토리와 유스케이스(주문 생성·조회·취소)

요구사항을 시스템 관점(FR/NFR)으로 적었다면, 이제 **사용자 관점**으로도 봐야 합니다. 사용자 스토리는 "누가, 무엇을, 왜"의 짧은 문장이고, 유스케이스는 그 스토리가 실제로 어떤 단계를 밟는지의 시나리오입니다. 둘은 같은 요구사항을 다른 각도에서 비추어, 빠진 흐름을 드러냅니다.

4.1 사용자 스토리(user story)

다섯 명의 역할(구매자·관리자·운영자)이 등장합니다.

1. **구매자로서**, 로그인한 뒤 원하는 상품들을 한 주문으로 담아 결제하고 싶다. (→ FR3·FR4·FR6)
2. **구매자로서**, 재고가 없으면 주문이 거부되고 명확한 사유를 받고 싶다. (→ FR5·FR9)
3. **구매자로서**, 내가 만든 주문만 조회·취소할 수 있길 원한다. (→ FR8)
4. **관리자로서**, 상품과 재고를 등록·수정하고 일반 사용자는 못 바꾸게 하고 싶다. (→ FR1·FR8)
5. **운영자로서**, 핵심 흐름이 자동 테스트로 검증되고 컨테이너로 배포되길 원한다. (→ NFR4·NFR5)

각 스토리 끝에 어느 FR/NFR로 이어지는지를 표시해 두면, "스토리는 있는데 대응 요구사항이 없다"거나 "요구사항은 있는데 그걸 원하는 사용자가 없다" 같은 불일치를 잡아낼 수 있습니다.

4.2 유스케이스(use case)

스토리를 실제 흐름으로 펼친 것이 유스케이스입니다. 이 프로젝트의 세 핵심 유스케이스를 단계로 기술합니다.

UC1 — 주문 생성(가장 복잡한 흐름)

1. 사용자가 로그인해 JWT를 발급받는다.(c09)
2. `POST /orders` 로 품목·수량 목록을 보낸다.(c03 중첩 모델)
3. 서버가 토큰을 검증해 사용자를 식별한다.(c09 `get_current_user`)
4. 각 품목의 재고를 확인하고, 부족하면 409로 거부한다.(c07·c10)
5. 충분하면 트랜잭션 안에서 재고를 차감한다.(c07)
6. 결제 게이트웨이를 비동기로 호출해 결제한다(모킹).(c05)
7. 주문과 주문 품목을 DB에 저장한다.(c07)
8. 확인 메일을 백그라운드 작업으로 등록한다(모킹).(c12)
9. `201 Created` 와 주문 정보를 응답한다.(c04)

UC2 — 상품 탐색

1. 사용자가 `GET /products?category=&min_price=&limit=&offset=` 를 호출한다.(c02)
2. 서버가 필터·페이지네이션을 적용해 목록을 조회한다.(c08)
3. 응답 모델로 직렬화된 페이지네이션 목록을 돌려준다.(c04)

UC3 — 주문 조회/취소

1. 사용자가 `GET /orders/me` 로 본인 주문 목록을 조회한다.(c09)
2. `DELETE /orders/{id}` 로 주문을 취소한다.
3. 서버는 그 주문이 요청자 본인의 것인지 확인하고, 아니면 403으로 막는다.(c09 인가)

UC1이 가장 많은 개념을 한 흐름에서 동시에 요구한다는 점에 주목하세요. 이것이 주문·이커머스 도메인을 종합 프로젝트로 고른 이유입니다. 검증·인증·트랜잭션·비동기·예외·백그라운드가 **한 엔드포인트 안에서** 맞물립니다.

5. 범위 In/Out 확정과 결제·메일 모킹 결정

요구사항을 다 적고 나면 욕심이 생깁니다. "추천도 넣을까", "실시간 알림도?" 분석의 마지막 책임은 **범위(scope)를 칼같이 긋는** 것입니다. 무엇을 넣고(In) 무엇을 뺄지(Out)를 명시하지 않으면, 프로젝트는 끝없이 부풀어 완성되지 않습니다.

5.1 범위 In (이 프로젝트에서 만든다)

범위 In 항목	관련 개념
상품 CRUD와 목록 페이지네이션·필터	c02-c08
사용자 가입·로그인(JWT)·본인 주문 인가	c09
주문 생성 시 재고 차감과 재고 부족 409	c07-c10
결제 처리(모킹 게이트웨이)·주문 확인 메일(백그라운드, 모킹)	c05-c12
SQLModel 기반 DB와 세션 의존성·트랜잭션	c06-c07
예외 핸들러·미들웨어·CORS	c10
TestClient·dependency_overrides 기반 자동 테스트	c11
pydantic-settings 설정·lifespan·Dockerfile	c12

5.2 범위 Out (이 프로젝트에서 빼고, 모킹·확장으로 둔다)

범위 Out 항목	이유
실제 외부 결제 게이트웨이(Stripe 등) 실연동	비결정·API 키 필요 → 모킹으로 결정론 확보
실제 메일 발송(SMTP)	외부 의존·비결정 → 콘솔/스텝 모킹
프론트엔드 UI·관리자 대시보드	이 책은 백엔드 API 범위
분산 작업 큐(Celery)·메시지 브로커	학습 규모 초과, BackgroundTasks로 충분
마이크로서비스 분리·쿠버네티스	단일 서비스 학습 목표 밖
WebSocket·GraphQL·실시간 알림	REST 범위에 집중

5.3 결제·메일을 모킹하는 결정

이 프로젝트에서 가장 중요한 범위 결정은 **결제와 메일을 실제로 연동하지 않고 모킹한다**는 것입니다. 그 근거는 다음 장의 ADR4에서 정식으로 기록하지만, 분석 단계에서 미리 합의해 둡니다.

- **결정론(determinism)**: 실제 결제 게이트웨이는 네트워크·키·외부 상태에 의존해 결과가 매번 달라질 수 있습니다. 교재의 코드는 누가 언제 돌려도 같은 결과여야 하므로(NFR4), 외부 연동을 모킹 인터페이스 뒤로 숨깁니다.

- **인터페이스 동형성:** 모킹은 단순히 "가짜로 때우기"가 아닙니다.
`services/payment.py` · `services/email.py` 라는 **인터페이스**를 두고, 그 안의 구현만 모킹으로 채웁니다. 나중에 실제 게이트웨이로 바꾸려면 인터페이스를 유지한 채 구현만 교체하면 됩니다.
- **MVP 정의:** 최소 기능 제품(MVP)은 "사용자가 로그인해 상품을 조회하고 주문을 생성하면 재고가 차감되고 결제(모킹)가 성공하며 본인 주문을 조회할 수 있는 최소 버전"입니다. 결제 실패 롤백·확인 메일·고급 필터·배포 고도화는 MVP 이후 확장으로 둡니다.

범위에서 흔히 하는 실수

- **모킹을 빼먹고 실연동부터 시도** — 키 발급·네트워크 문제로 학습이 멈춥니다. 인터페이스를 먼저 정하고 모킹으로 채우세요.
- **Out 목록을 비워 둬** — "안 만든다"를 명시하지 않으면 리뷰·평가에서 "왜 빠졌냐"는 지적을 받습니다. 의도적 제외는 반드시 기록합니다.
- **MVP와 전체 범위를 혼동** — 먼저 동작하는 최소 버전을 만들고 확장하는 순서를 잃으면, 한 번에 다 만들려다 아무것도 완성하지 못합니다.

6. 추적성: 요구사항 ↔ 단원 개념 매핑

분석의 결론은 **추적성 매트릭스(traceability matrix)**입니다. 모든 요구사항이 우리가 배운 개념(c01~c12)으로 빠짐없이 구현되는지, 반대로 배운 개념이 프로젝트에서 실제로 쓰이는지를 한눈에 확인합니다. 이 정렬이 깨지면 "배웠는데 안 쓰는 개념" 또는 "쓰는데 안 배운 개념"이 생깁니다. 커리큘럼 우선 설계의 핵심은 바로 이 양방향 정렬입니다.

6.1 요구사항 → 개념 (모든 FR/NFR이 개념으로 덮이는가)

요구사항	c01	c02	c03	c04	c05	c06	c07	c08	c09	c10	c11	c12
FR1 상품 CRUD			O	O				O				
FR2 목록 페이지네이션·필터		O						O				
FR3 가입·로그인 (JWT)									O			
FR4 주문 생성 (여러 품목)			O				O		O			
FR5 재고 차감·409							O			O		
FR6 결제(비동기 모킹)					O							
FR7 확인 메일 (백그라운드)												O
FR8 본인 주문인가									O			

요구사항	c01	c02	c03	c04	c05	c06	c07	c08	c09	c10	c11	c12
FR9 일관 오류 응답				O						O		
NFR1 입력 검증(422)			O									
NFR2 세션·트랜잭션						O	O					
NFR3 비동기 외부 호출					O							
NFR4 자동 테스트·격리											O	
NFR5 시크릿·Docker												O
NFR6 문서·민감필드 제외	O			O								

6.2 개념 → 요구사항 (배운 개념이 모두 쓰이는가)

개념	이름	주로 쓰이는 요구사항
c01	FastAPI 기초와 자동 문서	NFR6(앱 골격·OpenAPI 문서)
c02	경로·쿼리 파라미터와 타입 검증	FR2(필터·페이지네이션 파라미터)
c03	Pydantic v2 모델과 요청 본문 검증	FR1·FR4·NFR1(상품·주문 본문)
c04	응답 모델·상태 코드·직렬화	FR1·FR9·NFR6(출력·상태코드·민감필드)
c05	async/await 비동기 처리	FR6·NFR3(결제·배송 외부 호출)
c06	의존성 주입(Depends)	NFR2(세션·페이지네이션 주입)
c07	데이터베이스 연동(SQLModel)	FR4·FR5·NFR2(영속화·재고·트랜잭션)
c08	CRUD 라우터·페이지네이션·필터	FR1·FR2(상품 라우터·목록)
c09	인증·인가(OAuth2·JWT)	FR3·FR4·FR8(로그인·주문·인가)
c10	에러 처리·예외·미들웨어·CORS	FR5·FR9(도메인 예외·일관 오류)
c11	테스트(pytest·TestClient)	NFR4(자동 테스트·격리)
c12	백그라운드 작업·설정·배포	FR7·NFR5(메일·설정·Docker)

두 방향 표가 모두 빈칸 없이 채워진다는 것은, 12개 단위 개념이 **하나도 남김없이** 프로젝트에 동원되고, 모든 요구사항이 **배운 개념으로** 구현된다는 뜻입니다. 이로써 분석이 끝났습니다. 다음 장에서는 이 요구사항을 데이터 모델·모듈 분해·ADR로 **설계**합니다.

7. 작성형 연습(2종 1세트): 요구사항 표 채우기·자가체크

이제 직접 분석 산출물을 작성해 봅니다. 먼저 모범 답안을 보고, 이어지는 빈 템플릿을 스스로 채운 뒤, 자가체크로 점검합니다.

7.1 모범: 요구사항 한 건을 완전히 기술한 예

아래는 FR5(재고 차감·409)를 분석 표준 양식으로 완전히 기술한 모범입니다.

[요구사항 카드 — 모범]

ID : FR5
 종류 : 기능 요구사항(FR)
 설명 : 주문 생성 시 각 품목의 재고를 확인하고, 부족하면 409로 거부한다
 관련 스토리: "구매자로서, 재고가 없으면 주문이 거부되고 명확한 사유를 받고 싶다"
 수용 기준 : (1) 재고 >= 주문 수량이면 차감 후 주문 저장
 (2) 한 품목이라도 부족하면 409 응답, 어떤 변경도 커밋되지 않음(롤백)
 (3) 응답 본문에 부족한 상품과 사유가 담긴다
 사용 개념 : c07(SQLModel 트랜잭션 재고 차감), c10(도메인 예외 OutOfStock -> 409)
 관련 유스케이스: UC1 4~5단계

모범 카드 읽는 법

- 수용 기준 은 충족 여부를 객관적으로 판정할 수 있게 **검증 가능한 문장**으로 적습니다. "안정하게"가 아니라 "부족하면 409-롤백"처럼 구체적으로 씁니다.
- 사용 개념 은 6절 추적성 표와 일치해야 합니다. 카드와 표가 어긋나면 둘 중 하나가 틀린 것입니다.
- 관련 유스케이스 로 사용자 흐름의 어느 단계인지 연결해, FR이 흐름 속에서 어떻게 발현되는지 확인합니다.

7.2 작성형 빈 템플릿**직접 작성 과제 — 요구사항 카드 2장 채우기**

아래 빈칸을 직접 채워, **FR4(주문 생성)** 와 **NFR2(세션·트랜잭션)** 의 요구사항 카드를 완성하세요. 6절 추적성 표를 근거로 사용 개념을 적습니다.

[요구사항 카드 1 — 빈칸]

ID : FR4
 종류 : _____
 설명 : _____
 관련 스토리: _____
 수용 기준 : (1) _____
 (2) _____
 (3) _____
 사용 개념 : _____, _____, _____ (힌트: 6.1 표의 FR4 행)
 관련 유스케이스: _____

[요구사항 카드 2 — 빈칸]

ID : NFR2
 종류 : _____
 설명 : _____
 수용 기준 : (1) _____
 (2) _____
 사용 개념 : _____, _____ (힌트: 6.1 표의 NFR2 행)

또한 다음 미니 추적성 표의 빈칸(O 표시)을 채우세요.

요구사항	사용 개념(직접 적기)
FR6 결제(비동기 모킹)	—
FR7 확인 메일(백그라운드)	—
NFR4 자동 테스트·격리	—

7.3 자가체크

스스로 점검(자가체크) — 분석 산출물 체크리스트

작성한 카드와 표를 다음 기준으로 점검하세요. 모두 "예"여야 분석이 완결됩니다.

- ☐ 각 카드의 종류가 FR/NFR로 정확히 분류되어 있는가
- ☐ 수용 기준이 "잘", "빠르게" 같은 모호어 없이 판정 가능한 문장으로 적혔는가
- ☐ FR4의 사용 개념이 c03·c07·c09로(6.1 표와 일치) 채워졌는가
- ☐ NFR2의 사용 개념이 c06·c07로 채워졌는가
- ☐ 미니 표의 FR6=c05, FR7=c12, NFR4=c11로 매핑했는가
- ☐ 작성한 카드가 어느 사용자 스토리·유스케이스로 이어지는지 연결했는가
- ☐ 범위 Out에 결제·메일 실연동을 두고 모킹으로 대체하기로 한 결정을 이해했는가

하나라도 "아니오"라면, 해당 장(개념)으로 돌아가 복습한 뒤 카드를 보완하세요. 분석이 정확할수록 다음 장(설계)·그다음 장(구현)이 흔들리지 않습니다.

이 장에서는 드라이빙 퀘스천을 9개 FR·6개 NFR로 분해하고, 사용자 스토리·유스케이스로 주문 흐름을 그렸으며, 범위 In/Out과 결제·메일 모킹 결정을 확정했습니다. 마지막으로 요구사항과 12개 단위 개념을 양방향으로 매핑해 커리큘럼과 프로젝트의 정렬을 확인했습니다. 다음 장(설계)에서는 이 요구사항을 데이터 모델·모듈 분해·ADR 5건·다이어그램으로 구체화합니다.

종합 프로젝트: 주문·이커머스 API — 설계(데이터·모듈·ADR)

학습 목표

- Product·User·Order·OrderItem의 데이터 모델과 관계(1-N·N-1)를 설계할 수 있습니다.
- 라우터·서비스·DB·인증을 모듈로 분해하고 책임을 분리할 수 있습니다.
- 설계 결정을 ADR 5필드(맥락·결정·대안·근거·결과)로 기록할 수 있습니다.
- C4 컴포넌트도·ERD·주문 시퀀스 다이어그램으로 구조를 표현할 수 있습니다.

앞 장(분석)에서 우리는 `shop_api`가 무엇을 해야 하는지를 9개 FR·6개 NFR로 확정하고, 각 요구사항을 1~12장 개념과 매핑했습니다. 이번 장(설계)은 그 요구사항을 어떤 구조로 만들 것인지로 옮깁니다. 데이터를 어떤 테이블로 나눌지(데이터 모델), 코드를 어떤 파일로 쪼갤지(모듈 분해), 그리고 갈림길마다 왜 그 길을 택했는지(ADR)를 기록합니다. 마지막으로 ERD·C4·시퀀스 세 다이어그램으로 구조를 시각화합니다.

설계는 코드를 쓰기 직전의 마지막 안전장치입니다. 여기서 데이터 관계를 잘못 잡거나 모듈 책임을 흐릿하게 두면, 구현 단계에서 비싼 되돌림이 생깁니다. 반대로 설계가 단단하면 다음 장(구현)은 "설계도를 코드로 옮기는 작업"이 되어 한결 수월해집니다.

이 장도 새 FastAPI 개념을 가르치지 않습니다. 7장(SQLModel)·8장(라우터)·9장(인증)·6장(의존성)·4장(응답 모델)에서 배운 개념을 설계 산출물로 통합합니다.

1. 설계 개요: 라우터·서비스·DB·인증의 분리

좋은 백엔드 설계의 출발점은 **관심사 분리(separation of concerns)**입니다. 한 파일에 라우팅·검증·DB·인증·외부 호출을 모두 묶어놓으면, 코드가 길어질수록 무엇이 무엇을 부르는지 추적할 수 없습니다. 우리는 책임을 다음 네 층으로 나눕니다.

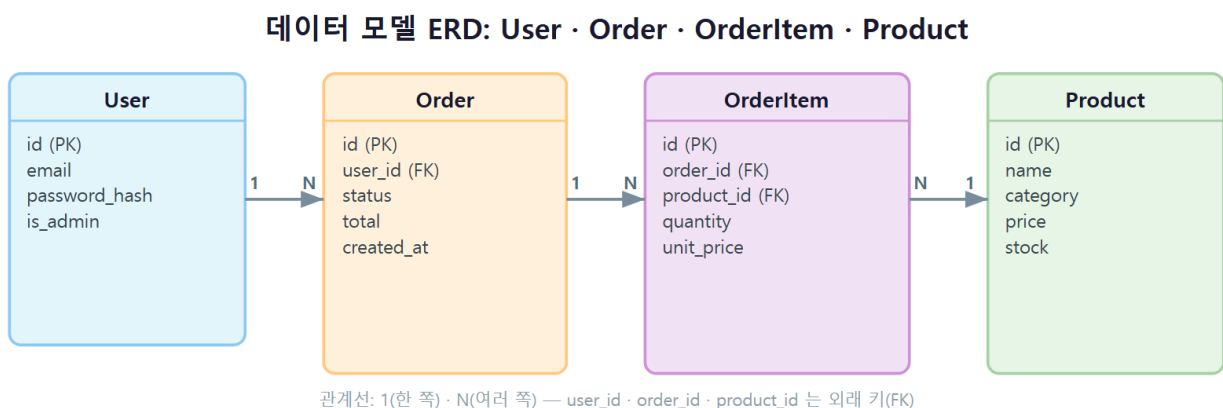
층	책임	관련 개념
라우터(routers)	HTTP 요청을 받아 검증·응답 모델로 입출력을 다룬다	c02·c03·c04·c08
서비스(services)	결제·메일 같은 도메인/외부 연동 로직을 캡슐화한다	c05·c12
DB(models-db)	테이블 정의와 세션·트랜잭션으로 영속화한다	c06·c07
인증(auth)	해시·JWT·현재 사용자 해석으로 인증·인가를 담당한다	c09

이 분리 덕분에 각 층을 독립적으로 이해하고 테스트할 수 있습니다. 예컨대 결제 로직을 `services/payment.py` 로 빼 두면, 11장의 `dependency_overrides` 로 그 서비스만 모킹해 주문 흐름을 격리 테스트할 수 있습니다(NFR4). 인증을 `auth.py` 로 모으면, 보호가 필요한 모든 라우터가 같은 `get_current_user` 의존성을 주입받아 일관됩니다(FR8).

설계 원칙을 세 가지로 요약합니다.

1. **입력·출력 모델 분리** — 테이블 모델을 그대로 노출하지 않고, Create / Read 스키마로 입출력을 통제합니다(4장). 민감 필드(`password_hash`)는 출력에서 제외합니다(NFR6).
2. **요청 범위 세션** — DB 세션은 전역이 아니라 요청마다 `get_session` yield 의존성으로 열고 닫습니다(6·7장). 트랜잭션 경계가 요청 경계와 일치합니다(NFR2).
3. **외부 연동 추상화** — 결제·메일은 `services` 인터페이스 뒤에 두고 모킹으로 구현합니다. 실연동으로 교체해도 호출부는 바뀌지 않습니다(ADR4).

2. 데이터 모델·ERD: User-Order-OrderItem-Product



먼저 데이터부터 설계합니다. 데이터 구조가 정해지면 그 위의 API는 자연스럽게 따라옵니다. 이 도메인의 핵심 엔티티는 넷입니다. **상품(Product)**, **사용자(User)**, **주문(Order)**, **주문 품목(OrderItem)** 입니

다.

2.1 테이블 모델(SQLModel `table=True`)

7장에서 배운 SQLModel `table=True` 모델로 네 테이블을 선언합니다. 각 필드의 타입·제약과 외래 키(FK)를 정합니다.

엔티티	필드	설명
Product	<code>id</code> (PK), <code>name</code> , <code>category</code> , <code>price</code> , <code>stock</code> , <code>is_active</code>	상품과 재고
User	<code>id</code> (PK), <code>email</code> (unique), <code>password_hash</code> , <code>is_admin</code>	사용자와 권한
Order	<code>id</code> (PK), <code>user_id</code> (FK→User), <code>status</code> , <code>total</code> , <code>created_at</code>	주문 헤더
OrderItem	<code>id</code> (PK), <code>order_id</code> (FK→Order), <code>product_id</code> (FK→Product), <code>quantity</code> , <code>unit_price</code>	주문 한 줄

`Order.status` 는 `pending` · `paid` · `cancelled` 세 값을 갖는 상태로, 주문의 수명주기를 나타냅니다.

`OrderItem.unit_price` 는 주문 시점의 가격을 **복제 저장**합니다. 상품 가격이 나중에 바뀌어도 과거 주문의 금액은 그대로 보존되어야 하기 때문입니다(이력 보존).

2.2 엔티티 관계(ERD)

네 엔티티의 관계는 다음과 같습니다.

- **User 1 — N Order**: 한 사용자는 여러 주문을 가질 수 있고, 한 주문은 한 사용자에게 속합니다(`Order.user_id`).
- **Order 1 — N OrderItem**: 한 주문은 여러 품목 줄을 가지며, 한 품목 줄은 한 주문에 속합니다(`OrderItem.order_id`).
- **Product 1 — N OrderItem**: 한 상품은 여러 주문 품목에 등장할 수 있고, 한 품목 줄은 한 상품을 가리킵니다(`OrderItem.product_id`).

```
User (1) —< Order (1) —< OrderItem >— (1) Product
      user_id      order_id  product_id
```

`OrderItem` 은 `Order` 와 `Product` 사이의 **연결 테이블**이면서, `quantity` · `unit_price` 라는 자기 속성을 갖는 점이 특징입니다. 주문은 "어떤 상품을 몇 개, 얼마에" 담았는지를 이 줄들로 표현합니다.

2.3 입출력 스키마(Pydantic, 4장)

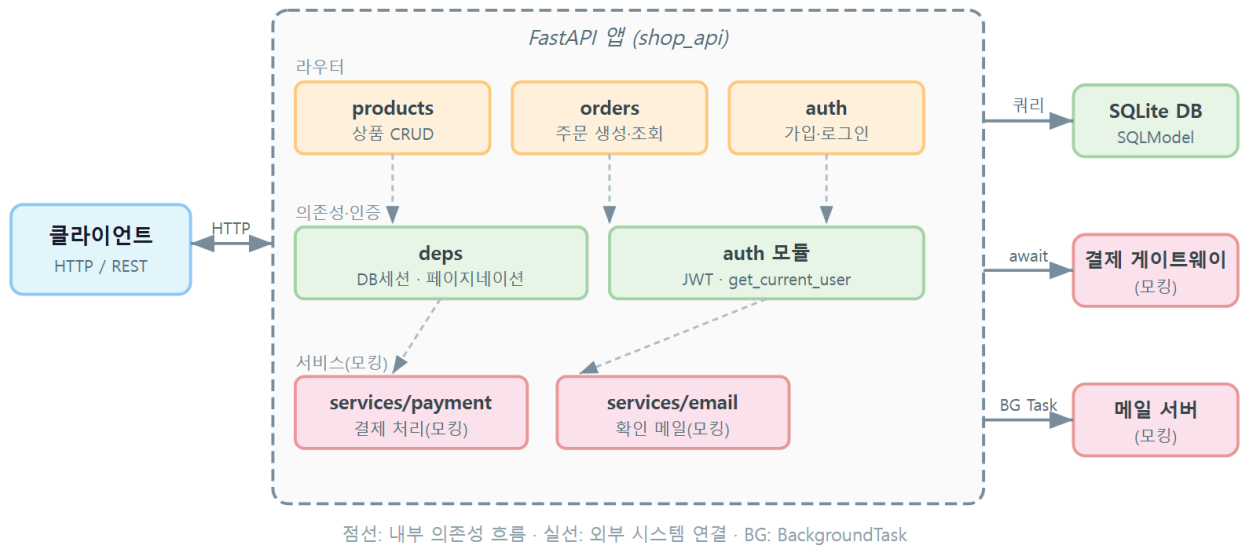
테이블 모델과 별개로, API 입출력은 별도 스키마로 분리합니다(ADR1). 테이블 모델을 그대로 노출하면 내부·민감 필드가 새어 나가기 때문입니다(NFR6).

용도	스키마	비고
상품 입력	<code>ProductCreate</code>	name·category·price·stock
상품 출력	<code>ProductRead</code>	id 포함, 내부 필드 제외
주문 입력	<code>OrderCreate(items: list[OrderItemCreate])</code>	중첩 모델(3장)
주문 출력	<code>OrderRead</code>	주문·품목·합계
사용자 입력	<code>UserCreate</code>	email·password(평문 입력→해시 저장)
사용자 출력	<code>UserRead</code>	password_hash 제외
토큰	<code>Token</code>	access_token·token_type

`UserRead` 가 `password_hash` 를 포함하지 않는다는 점이 보안의 핵심입니다. 입력으로는 평문 비밀번호를 받아 해시로 저장하지만, 출력으로는 해시조차 내보내지 않습니다.

3. 모듈 분해도(C4 컴포넌트): 라우터·서비스·의존성

C4 컴포넌트도: 클라이언트 ↔ FastAPI ↔ 외부 서비스

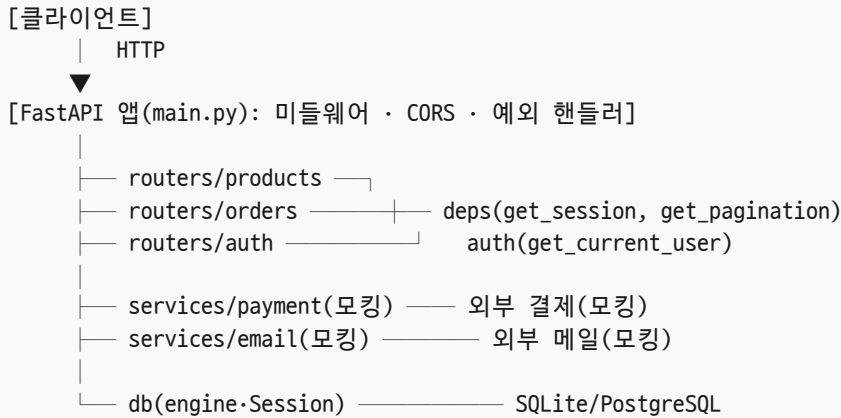


데이터 구조가 정해졌으니, 코드를 어떤 파일로 나눌지 결정합니다. 8장에서 배운 `APIRouter` 와 `include_router` 로 자원별 라우터를 분리하고, 6장의 의존성 주입으로 공통 로직을 모읍니다. 모듈 구성은 다음과 같습니다.

모듈	책임	핵심 개념
main.py	앱 생성·lifespan·미들웨어·CORS·예외 핸들러·라우터 포함	c01·c10·c12
routers/products.py	상품 CRUD·목록(페이지네이션·필터)	c04·c08
routers/orders.py	주문 생성·조회·취소 흐름	c03·c05·c07·c09·c10
routers/auth.py	가입·로그인(/token)·토큰 발급	c09
models.py	SQLModel table=True 테이블 모델 (Product·User·Order·OrderItem)	c07
schemas.py	Pydantic 입출력 스키마(Create·Read·Token)	c03·c04
db.py	engine·create_all·get_session 의존성	c06·c07
auth.py	비밀번호 해시·JWT 생성/검증·get_current_user·관리자 의존성	c09
deps.py	페이지네이션 의존성·세션 타입 별칭(Annotated)	c06
services/payment.py	모킹 결제 게이트웨이(비동기 인터페이스)	c05
services/email.py	모킹 확인 메일 발송	c12
config.py	pydantic-settings 기반 환경설정(DB URL·JWT 시크릿)	c12

3.1 의존성 흐름(C4 컴포넌트 관점)

C4 모델의 컴포넌트 수준에서 보면, 외부 클라이언트가 FastAPI 앱(컨테이너)에 요청하고, 앱 내부의 라우터 컴포넌트가 의존성(인증·세션·페이지네이션)과 서비스(결제·메일)를 거쳐 DB·외부 모킹에 닿습니다.



이 구조에서 라우터는 "얇게" 유지됩니다. 라우터는 입출력 모델로 검증·직렬화만 하고, 실제 외부 연동은 서비스에, DB 접근은 세션 의존성에 위임합니다. 덕분에 각 컴포넌트를 독립적으로 교체·테스트할 수 있습니다.

4. 설계 결정 기록: ADR 5건(모델분리·세션·JWT·모킹·재고)

설계에는 늘 갈림길이 있습니다. "테이블 모델을 그대로 노출할까, 분리할까?", "세션을 전역에 둘까, 요청별로 둘까?" 이런 결정을 머릿속에만 두면 나중에 "왜 이렇게 했지?"를 기억하지 못합니다.

ADR(Architecture Decision Record)은 결정 하나를 다섯 필드로 기록합니다 — 맥락(왜 결정이 필요한가)·결정(무엇을 정했나)·대안(어떤 선택지가 있었나)·근거(왜 그 선택인가)·결과(어떤 대가를 치르나). 이 프로젝트의 핵심 설계 결정 5건을 기록합니다.

ADR1 — 입력·출력 모델 분리

- **맥락:** 테이블 모델을 그대로 응답에 노출하면 내부·민감 필드(`password_hash`·관계)가 샅니다.
- **결정:** `Create / Read` 스키마를 테이블 모델과 분리합니다.
- **대안:** (가) 테이블 모델 직접 노출, (나) `Create/Read` 분리.
- **근거:** `response_model`로 출력을 통제하고 `password_hash` 등을 숨길 수 있어 분리를 택합니다 (c04-NFR6).
- **결과:** 스키마 클래스 수가 늘지만, 보안과 OpenAPI 문서 명확성을 확보합니다.

ADR2 — DB 세션 관리

- **맥락:** 트랜잭션 경계와 자원 정리를 일관되게 해야 합니다.
- **결정:** `get_session` yield 의존성으로 **요청 범위 세션**을 씁니다.
- **대안:** (가) 전역 세션 하나 공유, (나) 요청별 `yield` 세션.
- **근거:** 요청별 세션이 스레드 안전·정리 보장에 유리하고, 11장의 테스트 오버라이드도 쉽습니다(c06-c07-NFR2).

- **결과:** 의존성 그래프 관리가 필요하지만, 격리와 테스트성이 향상됩니다.

ADR3 — 인증 방식

- **맥락:** 상태 없는(stateless) 인증과 표준 흐름이 필요합니다.
- **결정:** OAuth2PasswordBearer + JWT(PyJWT)를 씁니다.
- **대안:** (가) 세션 쿠키, (나) JWT, (다) API 키.
- **근거:** FastAPI 표준 보안 유틸과 잘 맞고 무상태 확장이 쉬워 JWT를 택합니다(c09).
- **결과:** 토큰 만료·갱신 설계가 필요하고, 페이로드에 비밀을 담지 않도록 주의해야 합니다(JWT는 서명이지만 암호화가 아님).

ADR4 — 결제·메일 외부 연동

- **맥락:** 교재 실행이 결정론적·재현 가능해야 합니다(NFR4).
- **결정:** 결제·메일을 services 인터페이스 뒤의 **모킹**으로 추상화합니다.
- **대안:** (가) 실제 게이트웨이 연동, (나) 모킹 인터페이스.
- **근거:** 실연동은 비결정·API 키가 필요하므로, 인터페이스만 동일하게 두고 구현을 모킹합니다(c05·c12).
- **결과:** 실연동은 범위 밖(scope_out)으로 남기되, 인터페이스를 유지해 추후 교체가 쉽습니다.

ADR5 — 재고 차감과 동시성

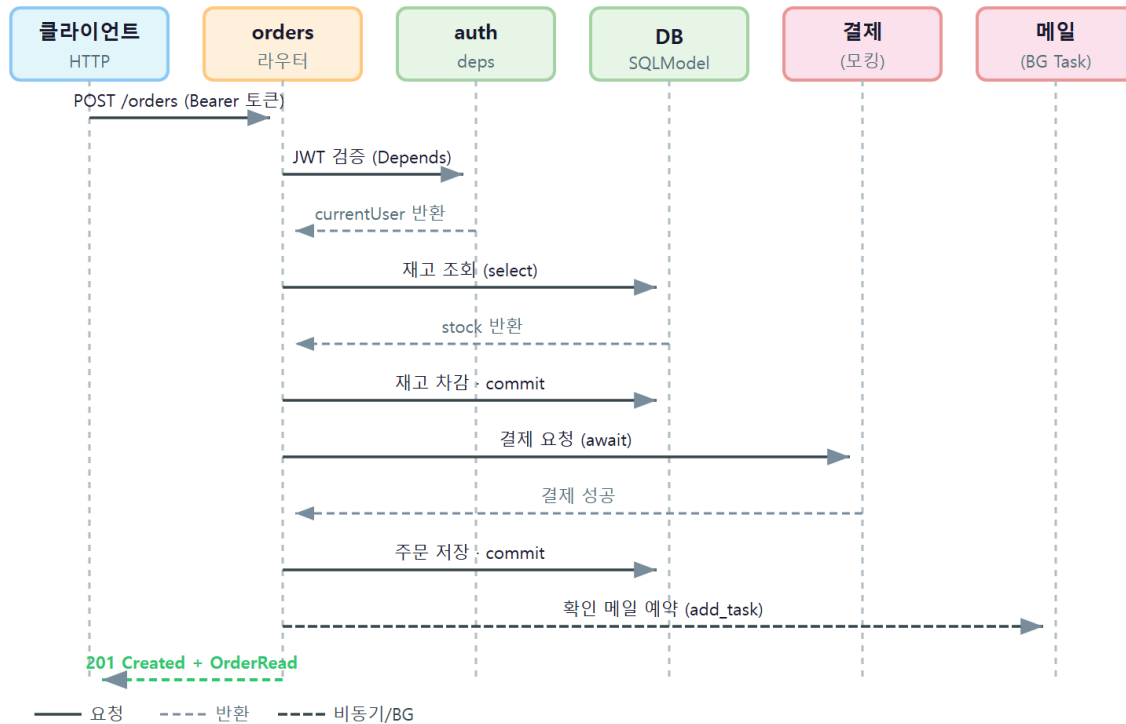
- **맥락:** 동시 주문이 재고를 음수로 만들면 안 됩니다.
- **결정:** 주문 트랜잭션 안에서 재고를 확인·차감하고, 부족하면 409로 거부·롤백합니다.
- **대안:** (가) 조회 후 차감(낙관적), (나) 행 잠금(비관적), (다) 트랜잭션 내 검증.
- **근거:** 교재 범위에서는 단일 트랜잭션 내 검증·차감으로 일관성을 확보합니다(고급 잠금은 확장; c07·c10).
- **결과:** 고부하 경쟁 상황은 추가 잠금이 필요하지만, 학습 규모에서는 충분합니다.

다섯 ADR을 관통하는 주제는 **"단순하되 견고하게, 그리고 교체 가능하게"**입니다. 입력·출력을 분리해 보안을 지키고(ADR1), 세션을 요청 범위로 묶어 일관성을 얻고(ADR2), 표준 무상태 인증을 택하고(ADR3), 외부 연동을 추상화해 결정론을 확보하고(ADR4), 재고를 트랜잭션으로 보호합니다(ADR5).

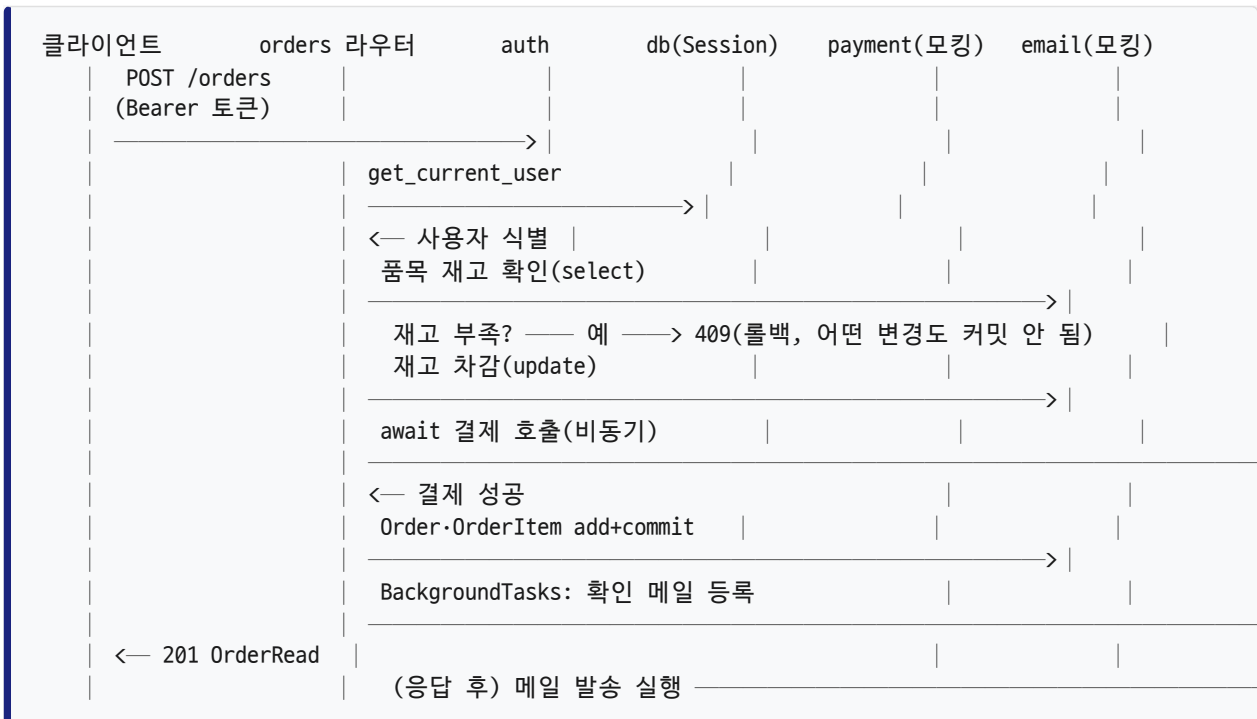
5. 주문 생성 시퀀스도: 인증→재고차감→결제→저장→메일

주문 생성 시퀀스도

인증 → 재고 확인·차감 → 결제(모킹) → 주문 저장 → 백그라운드 메일 → 201



마지막으로 가장 복잡한 흐름인 **주문 생성(UC1)** 을 시퀀스 다이어그램으로 그립니다. 이 한 요청 안에서 인증·검증·트랜잭션·비동기·예외·백그라운드가 모두 맞물립니다. 시간 순으로 단계를 따라가면 다음과 같습니다.



이 시퀀스에서 주목할 설계 포인트가 셋입니다.

1. **인증이 먼저**: 어떤 DB 변경보다 먼저 `get_current_user` 로 사용자를 식별합니다(c09). 토큰이 없거나 유효하지 않으면 401로 즉시 막아 불필요한 작업을 막습니다.
2. **재고 확인·차감·결제·저장은 한 트랜잭션**: 재고가 부족하면 409를 던지고 트랜잭션을 롤백해, 차감·결제·저장이 **전부 일어나거나 전부 안 일어나게** 합니다(ADR5·c07·c10). 결제는 `await` 로 비동기 호출해 이벤트 루프를 막지 않습니다(c05).
3. **메일은 응답 후**: 확인 메일은 `BackgroundTasks` 로 등록만 하고, 실제 발송은 201 응답을 보낸 뒤에 실행됩니다(c12). 사용자는 메일 발송을 기다리지 않고 즉시 응답을 받습니다.

이 세 포인트가 분석의 UC1·FR4~FR7과 정확히 대응하는지 확인하세요. 설계가 요구사항을 빠짐없이 반영했는지는, 이렇게 시퀀스를 요구사항과 대조해 점검합니다.

6. 작성형 연습(2종 1세트): ADR·ERD 작성·자가체크

이제 직접 설계 산출물을 작성합니다. 모범 ADR을 보고, 빈 템플릿을 채운 뒤, 자가체크로 점검합니다.

6.1 모범: ADR 한 건을 5필드로 작성한 예

아래는 "주문 합계(total) 계산을 어디서 할 것인가"라는 추가 결정을 ADR 양식으로 작성한 모범입니다.

```
[ADR — 모범]
ID      : ADR6
제목    : 주문 합계(total) 계산 위치
맥락    : 합계를 클라이언트가 보내면 조작 위험이 있고, 가격이 바뀌면 어긋난다
결정    : 서버가 OrderItem.unit_price * quantity를 합산해 Order.total을 계산한다
대안    : (가) 클라이언트가 합계 전송, (나) 서버가 서버 가격으로 계산
근거    : 서버 권위 가격으로 계산해야 위변조를 막고 이력이 정확하다(c07)
결과    : 클라이언트는 합계를 보낼 필요가 없고, 서버 계산 로직을 테스트로 검증한다
```

모범 ADR 읽는 법

- 맥락 은 "왜 결정이 필요한가"를 적습니다. 문제가 없으면 ADR을 쓸 이유도 없습니다.
- 대안 에 실제로 고려한 선택지를 둘 이상 적어, 결정이 비교를 거친 것임을 남깁니다.
- 근거 는 어느 개념·요구사항과 연결되는지를 함께 적어 추적성을 유지합니다(여기선 c07·이력 보존).
- 결과 에는 좋은 점뿐 아니라 **치르는 대가**(추가 테스트 필요 등)도 솔직히 적습니다.

6.2 작성형 빈 템플릿

직접 작성 과제 1 — ADR 한 건 작성

"비밀번호 해싱 알고리즘으로 무엇을 쓸 것인가"라는 결정을 ADR 5필드로 직접 작성하세요. 9장(c09)의 오개념("평문·단순 해시 금지, 솔트 포함 해시 사용")을 근거로 삼습니다.

```
[ADR — 빈칸]
ID   : ADR7
제목 : 비밀번호 해싱 알고리즘 선택
맥락 : _____
결정 : _____
대안 : (가) _____ (나) _____
근거 : _____ (힌트: c09 오개념)
결과 : _____
```

직접 작성 과제 2 — ERD 관계 채우기

아래 빈칸에 네 엔티티의 관계(1-N:N-1)와 외래 키를 채워 ERD를 완성하세요.

```
User (____) ——< Order (____) ——< OrderItem >—— (____) Product
      FK: _____      FK: _____      FK: _____
```

또한 다음 표에서 각 스키마가 테이블 모델 대비 **제외/추가하는 필드**를 적으세요.

스키마	기준 테이블	제외/주의 필드
UserRead	User	--
ProductRead	Product	--
OrderCreate	(입력)	중첩 __ 필드

6.3 자가체크

스스로 점검(자가체크) — 설계 산출물 체크리스트

작성한 ADR·ERD를 다음 기준으로 점검하세요.

- [] ADR7이 맥락·결정·대안·근거·결과 **5필드를 모두** 채웠는가
- [] ADR7의 근거가 "솔트 포함 해시(bcrypt-argon2)"로, 평문·단순 해시를 배제했는가(c09)
- [] ERD에서 User 1-N Order, Order 1-N OrderItem, Product 1-N OrderItem 관계가 맞는가
- [] 외래 키가 Order.user_id · OrderItem.order_id · OrderItem.product_id 로 정확한가
- [] UserRead 에서 password_hash 를 제외했는가(NFR6·ADR1)
- [] OrderCreate 가 items: list[...] 중첩 모델을 갖는가(3장 c03)
- [] 5절 주문 시퀀스가 인증→재고→결제→저장→메일 순서로, 메일이 응답 후인지 확인했는가

- [] 모듈 분해(3절)에서 라우터·서비스·DB·인증의 책임이 겹치지 않는가

하나라도 "아니오"라면 해당 절·해당 장으로 돌아가 보완하세요. 설계가 정확할수록 다음 장(구현)이 "설계도를 코드로 옮기는 일"이 됩니다.

이 장에서는 Product·User·Order·OrderItem의 데이터 모델과 관계를 ERD로 설계하고, 라우터·서비스·DB·인증을 모듈로 분해했으며, 핵심 설계 결정 5건을 ADR 5필드로 기록하고, 주문 생성 흐름을 시퀀스 다이어그램으로 그렸습니다. 모든 설계 결정은 분석 단계의 요구사항·개념 매핑과 정렬됩니다. 다음 장(구현)에서는 이 설계를 6개 마일스톤으로 나눠 실제 코드로 옮깁니다.

종합 프로젝트: 주문·이커머스 API — 구현(마일스톤 증분)

분석(ch_13)에서 요구사항을, 설계(ch_14)에서 데이터 모델·모듈 분해·ADR을 확정했습니다. 이제 그 설계를 **실제로 동작하는 코드로** 옮깁니다. 이 챕터에서는 새로운 개념을 배우지 않습니다.

ch_01~ch_12에서 익힌 12개 단원의 개념을 통합해 `shop_api` 라는 하나의 FastAPI 애플리케이션을 완성합니다.

이 챕터의 목표

- 앱 골격부터 테스트까지를 6개 마일스톤으로 나누어 증분 구현합니다.
- 각 마일스톤이 어느 단원 개념(c01~c12)을 적용하는지 명시적으로 연결합니다.
- 각 마일스톤 끝에서 실행 결과와 체크포인트(단위 테스트)로 동작을 검증합니다.
- 6개 마일스톤을 통합해 로그인부터 주문 생성까지 end-to-end로 동작하는 API를 완성합니다.

15.1 구현 전략과 마일스톤 개요

큰 프로젝트를 한 번에 작성하면 어디서 깨졌는지 추적하기 어렵습니다. 그래서 **수직 슬라이스 (vertical slice)** 대신 **계층 증분** 전략을 씁니다. 즉 앱이 항상 실행되는 상태를 유지하면서, 기능을 한 겹씩 엮습니다. 각 마일스톤이 끝날 때마다 `pytest` 로 동작을 고정(regression lock)하므로, 다음 마일스톤이 앞 기능을 깨뜨리면 즉시 드러납니다.

전체 모듈 구조는 ch_14의 모듈 분해(module decomposition)를 그대로 따릅니다.

```

shop_api/
  __init__.py
  main.py          # 앱 생성·lifespan·미들웨어·CORS·예외 핸들러·라우터 포함
  config.py        # pydantic-settings 설정(Settings)
  db.py            # engine.create_all.get_session 의존성
  models.py        # SQLAlchemy table 모델(Product·User·Order·OrderItem)
  schemas.py       # Pydantic 입출력 모델(Create/Read/Update/Token)
  auth.py          # 해시·JWT.get_current_user·관리자 의존성
  deps.py          # 페이지네이션·세션 타입 별칭
  errors.py        # 도메인 예외(OutOfStockError)
  routers/
    products.py    # 상품 CRUD
    orders.py      # 주문 생성·조회·취소
    auth.py        # 가입·로그인·me
  services/
    payment.py     # 결제(모킹·비동기)
    email.py       # 메일(모킹)
  tests/
    conftest.py    # 픽스처·dependency_overrides
    test_products.py
    test_orders.py
    test_auth.py
  Dockerfile

```

다음은 6개 마일스톤과 각 마일스톤이 적용하는 단위 개념입니다. 모든 개념이 끼워맞춤 없이 자연스럽게 필요해지는 것이 이 프로젝트를 선택한 이유였습니다(ch_13 선정 매트릭스).

마일스톤	산출물	적용 단위 개념	체크포인트
M1 앱 골격·설정·DB	<code>main · config · db · lifespan</code>	c01(앱·문서)·c06(의존성)·c07(SQLModel)·c12(설정·lifespan)	헬스체크 200·테이블 생성
M2 상품 CRUD	<code>routers/products · schemas</code>	c02(쿼리)·c03(본문 검증)·c04(응답 모델)·c08(라우터·페이지네이션)	CRUD·404·422 테스트 통과
M3 인증·인가	<code>auth · routers/auth</code>	c09(OAuth2·JWT·해시)	무토큰 401·비권한 403·정상 200
M4 주문·재고·결제	<code>routers/orders · services/payment</code>	c05(비동기)·c07(트랜잭션)·c10(도메인 예외)	재고 차감·결제·재고부족 409
M5 예외·미들웨어·배포	예외 핸들러·미들웨어 <code>· services/email · Dockerfile</code>	c10(예외·CORS)·c12(백그라운드·배포)	일반 오류·X-Process-Time·이미지 빌드
M6 테스트 스위트	<code>tests/</code> 전체	c11(pytest·TestClient·오버라이드)	전체 그린·격리 확인

드라이빙 퀘스천 재확인: "실무 백엔드 개발자가 FastAPI로, 상품·재고·주문·인증·결제 흐름을 갖추고 테스트·배포까지 되는 안전하고 견고한 이커머스 주문 API를 어떻게 설계하고 구현할 수 있을까?" 이 챕터의 6개 마일스톤이 이 질문에 대한 실행 가능한 답입니다.

15.2 M1 — 앱 골격·설정·DB (lifespan·세션)

적용 개념: c01(FastAPI 앱·자동 문서), c06(의존성 주입), c07(SQLModel·세션), c12(pydantic-settings·lifespan)

가장 먼저 "항상 실행되는 빈 앱"을 만듭니다. 설정을 환경변수로 주입하고(c12), DB 엔진과 세션의 존성을 구성한 뒤(c07·c06), 앱 시작 시 테이블을 만드는 lifespan을 담니다(c12).

설정: `config.py`

```
from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    """환경변수(.env)로 주입되는 앱 설정. 시크릿은 코드에 하드코딩하지 않습니다."""

    app_name: str = "shop_api"
    database_url: str = "sqlite:///./shop.db"
    jwt_secret: str = "change-me-in-production"
    jwt_algorithm: str = "HS256"
    access_token_expire_minutes: int = 30

    model_config = SettingsConfigDict(env_file=".env", env_prefix="SHOP_")

    @lru_cache
    def get_settings() -> Settings:
        """설정을 한 번만 로드해 캐시합니다(매 요청 재파싱 방지)."""
        return Settings()
```

- `BaseSettings`를 상속하면 각 필드가 환경변수에서 자동으로 채워집니다.
`env_prefix="SHOP_"`이므로 `SHOP_JWT_SECRET` 같은 변수로 주입합니다(c12의 시크릿 외부 주입 오개념 교정 적용).
- `@lru_cache`로 `get_settings()`를 캐시해 의존성으로 주입하면, 설정이 한 번만 로드되고 테스트에서 교체하기도 쉬워집니다.

DB: `db.py`

```

from collections.abc import Generator

from sqlmodel import Session, SQLModel, create_engine

from .config import get_settings

settings = get_settings()
# SQLite를 여러 스레드(테스트·동시 요청)에서 쓰기 위한 옵션입니다.
engine = create_engine(
    settings.database_url, connect_args={"check_same_thread": False}
)

def create_db_and_tables() -> None:
    """등록된 SQLAlchemy 테이블을 모두 생성합니다."""
    SQLModel.metadata.create_all(engine)

def get_session() -> Generator[Session, None, None]:
    """요청 범위 세션을 열고 응답 후 닫는 yield 의존성입니다(ADR2)."""
    with Session(engine) as session:
        yield session

```

- `get_session` 은 c06의 `yield` 의존성입니다. `with Session(engine)` 블록이 요청 종료 시 세션을 닫아 자원 정리를 보장합니다. ADR2(요청별 `yield` 세션)를 코드로 옮긴 것입니다.
- `create_all` 은 테이블이 없을 때만 생성하므로 매 시작 호출해도 안전합니다.

앱 골격: `main.py` (이 단계)

```

from contextlib import asynccontextmanager

from fastapi import FastAPI

from .config import get_settings
from .db import create_db_and_tables

@asynccontextmanager
async def lifespan(app: FastAPI):
    """앱 시작 시 DB 테이블을 만들고, 종료 시 정리합니다."""
    create_db_and_tables() # 시작 단계
    yield
    # 종료 단계: 연결 풀 정리 등이 필요하면 여기에 둡니다.

app = FastAPI(title=get_settings().app_name, lifespan=lifespan)

@app.get("/health")
def health() -> dict[str, str]:
    """헬스체크 — 로드밸런서·배포 점검용 엔드포인트입니다."""
    return {"status": "ok"}

```

실행 결과

```

$ uvicorn shop_api.main:app --reload
INFO:     Will watch for changes in these directories: ['../shop_api']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started server process [12044]
INFO:     Application startup complete.

$ curl http://127.0.0.1:8000/health
{"status":"ok"}

```

http://127.0.0.1:8000/docs 를 열면 c01에서 배운 Swagger UI가 /health 를 자동 문서로 보여 줍니다.

체크포인트 (단위 테스트)

```

def test_health(client):
    """M1: 앱이 뜨고 헬스체크가 200을 돌려주는지 확인합니다."""
    res = client.get("/health")
    assert res.status_code == 200
    assert res.json() == {"status": "ok"}

```

이 테스트가 통과하면 앱 골격·lifespan·DB 초기화가 정상입니다. M1 완료입니다.

15.3 M2 — 상품 CRUD 라우터 (페이지네이션·입출력 모델)

적용 개념: c02(쿼리 파라미터), c03(Pydantic 본문 검증), c04(응답 모델·상태 코드), c08(APIRouter·페이지네이션)

이제 첫 자원인 상품을 다룹니다. 테이블 모델(c07)과 입출력 스키마(c03·c04)를 분리(ADR1)하고, 라우터 모듈(c08)로 CRUD를 구현합니다.

테이블 모델: `models.py`

```
from datetime import datetime, timezone

from sqlmodel import Field, Relationship, SQLModel


class Product(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    category: str = Field(index=True)
    price: float
    stock: int = Field(default=0)
    is_active: bool = Field(default=True)


class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    email: str = Field(index=True, unique=True)
    password_hash: str
    is_admin: bool = Field(default=False)


class Order(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    user_id: int = Field(foreign_key="user.id", index=True)
    status: str = Field(default="pending")
    total: float = Field(default=0.0)
    created_at: datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc)
    )
    items: list["OrderItem"] = Relationship(back_populates="order")


class OrderItem(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    order_id: int = Field(foreign_key="order.id", index=True)
    product_id: int = Field(foreign_key="product.id")
    quantity: int
    unit_price: float
    order: Order | None = Relationship(back_populates="items")
```

- 데이터 모델은 ch_14 ERD(User 1-N Order 1-N OrderItem N-1 Product)를 그대로 코드화했습니다.
- `Relationship / back_populates` 로 주문과 주문 항목을 양방향으로 연결합니다. M4에서 주문을 저장할 때 `order.items` 에 항목을 담으면 함께 영속화됩니다.

입출력 스키마: `schemas.py`

```
from datetime import datetime

from pydantic import BaseModel, EmailStr, Field, field_validator

class ProductCreate(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    category: str
    price: float = Field(gt=0)
    stock: int = Field(ge=0)

    @field_validator("price")
    @classmethod
    def round_price(cls, v: float) -> float:
        return round(v, 2) # 소수 2자리로 정규화

class ProductRead(BaseModel):
    id: int
    name: str
    category: str
    price: float
    stock: int
    is_active: bool

class ProductUpdate(BaseModel):
    name: str | None = None
    category: str | None = None
    price: float | None = Field(default=None, gt=0)
    stock: int | None = Field(default=None, ge=0)
    is_active: bool | None = None
```

- `ProductCreate` 는 c03의 본문 검증입니다. `price: float = Field(gt=0)` 로 0 이하 가격을 422로 거부하고, `field_validator` 로 소수 2자리를 강제합니다(수동 if 검사 불필요).
- `ProductRead` 는 c04의 출력 모델입니다. 입력과 출력을 분리(ADR1)해 응답 스키마를 명시적으로 통제합니다.
- `ProductUpdate` 는 모든 필드가 선택값입니다. c08의 PATCH 부분 갱신에서 `exclude_unset` 과 함께 씁니다.

공통 의존성: `deps.py`

```

from dataclasses import dataclass
from typing import Annotated

from fastapi import Depends, Query
from sqlmodel import Session

from .db import get_session

# 세션 의존성을 타입 별칭으로 만들어 모든 핸들러 시그니처를 단순화합니다(c06).
SessionDep = Annotated[Session, Depends(get_session)]

@dataclass
class Pagination:
    limit: int
    offset: int

def get_pagination(
    limit: Annotated[int, Query(ge=1, le=100)] = 20,
    offset: Annotated[int, Query(ge=0)] = 0,
) -> Pagination:
    """limit/offset 페이지네이션 파라미터 공통 의존성입니다(c02·c08)."""
    return Pagination(limit=limit, offset=offset)

PaginationDep = Annotated[Pagination, Depends(get_pagination)]

```

- `SessionDep` · `PaginationDep` 는 c06의 `Annotated[T, Depends()]` 타입 별칭입니다. 핸들러에서 `session: SessionDep` 처럼 한 단어로 주입해 시그니처가 깔끔해집니다.
- `Query(ge=1, le=100)` 로 페이지 크기 상한을 강제합니다(c02). 위반하면 FastAPI가 자동으로 422를 냅니다.

상품 라우터: `routers/products.py`

```

from typing import Annotated

from fastapi import APIRouter, HTTPException, Query
from sqlmodel import select

from ..auth import AdminUser
from ..deps import PaginationDep, SessionDep
from ..models import Product
from ..schemas import ProductCreate, ProductRead, ProductUpdate

router = APIRouter(prefix="/products", tags=["products"])

@router.post("", response_model=ProductRead, status_code=201)
def create_product(
    payload: ProductCreate, session: SessionDep, _: AdminUser
) -> Product:
    product = Product(**payload.model_dump())
    session.add(product)
    session.commit()
    session.refresh(product) # DB가 채운 id를 읽어 옵니다.
    return product

@router.get("", response_model=list[ProductRead])
def list_products(
    session: SessionDep,
    page: PaginationDep,
    category: str | None = None,
    min_price: Annotated[float | None, Query(ge=0)] = None,
) -> list[Product]:
    stmt = select(Product).where(Product.is_active == True) # noqa: E712
    if category is not None:
        stmt = stmt.where(Product.category == category)
    if min_price is not None:
        stmt = stmt.where(Product.price >= min_price)
    stmt = stmt.offset(page.offset).limit(page.limit)
    return list(session.exec(stmt).all())

@router.get("/{product_id}", response_model=ProductRead)
def get_product(product_id: int, session: SessionDep) -> Product:
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    return product

@router.patch("/{product_id}", response_model=ProductRead)
def update_product(
    product_id: int, payload: ProductUpdate, session: SessionDep, _: AdminUser
) -> Product:
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    for key, value in payload.model_dump(exclude_unset=True).items():
        setattr(product, key, value) # 받은 필드만 갱신합니다(PATCH).

```



```

session.add(product)
session.commit()
session.refresh(product)
return product

@router.delete("/{product_id}", status_code=204)
def delete_product(product_id: int, session: SessionDep, _: AdminUser) -> None:
    product = session.get(Product, product_id)
    if product is None:
        raise HTTPException(status_code=404, detail="상품을 찾을 수 없습니다")
    session.delete(product)
    session.commit()

```

- `APIRouter(prefix="/products")` 로 자원별 모듈을 분리합니다(c08). `prefix` 에 이미 `/products` 가 있으니 경로에는 빈 문자열(`""`)을 써 접두어 중복을 피합니다(c08 오개념 교정).
- `status_code=201` 로 생성 응답을, `204` 로 본문 없는 삭제를 정확히 표현합니다(c04).
- `model_dump(exclude_unset=True)` 로 PATCH가 받은 필드만 바꾸도록 합니다(c08의 PATCH vs PUT 구분).
- `_: AdminUser` 는 M3에서 만들 관리자 의존성으로, 상품 변경을 관리자만 하도록 보호합니다.

실행 결과

```

$ curl -X POST localhost:8000/products -H "Authorization: Bearer <admin>" \
  -H "Content-Type: application/json" \
  -d '{"name":"무선 키보드","category":"office","price":39000,"stock":10}'
{"id":1,"name":"무선 키보드","category":"office","price":39000.0,"stock":10,"is_active":true}

$ curl "localhost:8000/products?category=office&limit=20&offset=0"
[{"id":1,"name":"무선 키보드","category":"office","price":39000.0,"stock":10,"is_active":true}]

$ curl -X POST localhost:8000/products -H "Authorization: Bearer <admin>" \
  -d '{"name":"x","category":"c","price":-5,"stock":3}'
{"detail":[{"type":"greater_than","loc":["body","price"],"msg":"Input should be greater than 0"}]}

```

음수 가격이 자동으로 422로 거부됩니다(c03). 손으로 검사하지 않았는데도 검증이 동작합니다.

체크포인트 (단위 테스트)

```
def test_create_and_get_product(client, admin_headers):
    """M2: 생성 후 조회가 일치하고 201/200 상태코드가 정확한지 확인합니다."""
    res = client.post(
        "/products",
        json={"name": "키보드", "category": "office",
              "price": 39000, "stock": 10},
        headers=admin_headers,
    )
    assert res.status_code == 201
    pid = res.json()["id"]

    got = client.get(f"/products/{pid}")
    assert got.status_code == 200
    assert got.json()["name"] == "키보드"

def test_get_missing_product_404(client):
    assert client.get("/products/9999").status_code == 404
```

CRUD-404-422가 모두 통과하면 M2 완료입니다.

15.4 M3 — 인증·인가 (JWT·get_current_user·관리자)

적용 개념: c09(OAuth2PasswordBearer·JWT·비밀번호 해시·인증/인가)

M2에서 상품 변경을 `AdminUser` 로 막아 두었습니다. 그 의존성을 이제 구현합니다. 로그인 흐름은 c09에서 배운 그대로입니다: 비밀번호 해시 검증 → 서명된 JWT 발급 → 이후 요청의 Bearer 토큰을 검증.

보안 코어: `auth.py`

```

from datetime import datetime, timedelta, timezone
from typing import Annotated

import jwt
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from pwdlib import PasswordHash
from sqlmodel import select

from .config import get_settings
from .deps import SessionDep
from .models import User

settings = get_settings()
password_hash = PasswordHash.recommended() # argon2 기반 권장 해시
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/token")

def hash_password(plain: str) -> str:
    return password_hash.hash(plain)

def verify_password(plain: str, hashed: str) -> bool:
    return password_hash.verify(plain, hashed)

def create_access_token(subject: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=settings.access_token_expire_minutes
    )
    payload = {"sub": subject, "exp": expire}
    return jwt.encode(
        payload, settings.jwt_secret, algorithm=settings.jwt_algorithm
    )

def get_current_user(
    token: Annotated[str, Depends(oauth2_scheme)], session: SessionDep
) -> User:
    credentials_error = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="자격 증명을 검증할 수 없습니다",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(
            token, settings.jwt_secret, algorithms=[settings.jwt_algorithm]
        )
        email = payload.get("sub")
        if email is None:
            raise credentials_error
    except jwt.PyJWTError: # 서명 위조·만료 등
        raise credentials_error
    user = session.exec(select(User).where(User.email == email)).first()
    if user is None:
        raise credentials_error
    return user

```

```

CurrentUser = Annotated[User, Depends(get_current_user)]

def require_admin(user: CurrentUser) -> User:
    """인증된 사용자 중 관리자만 통과시킵니다(인가, 403)."""
    if not user.is_admin:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN, detail="관리자 권한이 필요합니다"
        )
    return user

AdminUser = Annotated[User, Depends(require_admin)]

```

- 비밀번호는 평문이 아니라 `PasswordHash` 로 솔트 포함 해시 저장합니다(c09 오개념 교정).
- `jwt.encode` 로 `exp` (만료)와 서명을 담은 JWT를 발급하고, `jwt.decode` 가 서명·만료를 검증합니다. 검증 실패는 `jwt.PyJWTError` 로 잡아 401로 변환합니다(PyJWT 기준).
- `get_current_user` 는 c09의 인증(401), `require_admin` 은 인가(403)입니다. 두 단계가 분리되어 있다는 점이 c09의 핵심이었습니다. `require_admin` 이 `CurrentUser` 를 하위 의존성으로 받는 것은 c06의 의존성 합성입니다.

인증 라우터: `routers/auth.py`

```

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import OAuth2PasswordRequestForm
from sqlmodel import select

from ..auth import (CurrentUser, create_access_token, hash_password,
                    verify_password)
from ..deps import SessionDep
from ..models import User
from ..schemas import Token, UserCreate, UserRead

router = APIRouter(prefix="/auth", tags=["auth"])

@router.post("/register", response_model=UserRead, status_code=201)
def register(payload: UserCreate, session: SessionDep) -> User:
    if session.exec(select(User).where(User.email == payload.email)).first():
        raise HTTPException(status_code=409, detail="이미 가입된 이메일입니다")
    user = User(email=payload.email,
                password_hash=hash_password(payload.password))
    session.add(user)
    session.commit()
    session.refresh(user)
    return user

@router.post("/token", response_model=Token)
def login(
    form: Annotated[OAuth2PasswordRequestForm, Depends()], session: SessionDep
) -> Token:
    user = session.exec(
        select(User).where(User.email == form.username)
    ).first()
    if user is None or not verify_password(form.password, user.password_hash):
        raise HTTPException(
            status_code=401, detail="이메일 또는 비밀번호가 올바르지 않습니다"
        )
    return Token(access_token=create_access_token(subject=user.email))

@router.get("/me", response_model=UserRead)
def read_me(user: CurrentUser) -> User:
    return user

```

UserRead 에는 password_hash 가 없으므로(c04 입출력 분리), 응답에 해시가 절대 노출되지 않습니다.

실행 결과

```
$ curl -X POST localhost:8000/auth/token \
  -d "username=admin@shop.com&password=password123"
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...","token_type":"bearer"}

$ curl localhost:8000/auth/me          # 토큰 없이
{"detail":"Not authenticated"}         # 401

$ curl localhost:8000/products -X POST -d '{...}' # 일반 사용자 토큰
{"detail":"관리자 권한이 필요합니다"}    # 403
```

체크포인트 (단위 테스트)

```
def test_protected_requires_token(client):
    """M3: 토큰 없이 보호 라우트에 접근하면 401."""
    assert client.get("/auth/me").status_code == 401

def test_admin_only_create_product(client, user_headers):
    """M3: 일반 사용자가 상품 생성 시 403."""
    res = client.post(
        "/products",
        json={"name": "x", "category": "c", "price": 1, "stock": 1},
        headers=user_headers,
    )
    assert res.status_code == 403
```

무토큰 401·비권한 403·정상 200이 모두 통과하면 M3 완료입니다.

15.5 M4 — 주문 생성·재고 차감·결제 (모킹·비동기·409)

적용 개념: c05(async/await 비동기), c07(세션 트랜잭션), c10(도메인 예외→409)

이 마일스톤이 프로젝트의 심장입니다. 주문 생성은 한 트랜잭션 안에서 재고 확인·차감(c07)을 하고, 외부 결제 게이트웨이를 비동기로 호출하며(c05), 재고가 부족하면 도메인 예외로 409를 돌려줍니다(c10·ADR5).

도메인 예외와 결제 모킹

```
# errors.py
class OutOfStockError(Exception):
    """재고 부족 도메인 예외 — main.py의 핸들러가 409로 변환합니다."""

    def __init__(self, product_id: int, requested: int, available: int):
        self.product_id = product_id
        self.requested = requested
        self.available = available
```

```
# services/payment.py
import asyncio
from dataclasses import dataclass

@dataclass
class PaymentResult:
    success: bool
    transaction_id: str

async def charge(amount: float) -> PaymentResult:
    """결제 게이트웨이 호출을 모킹합니다(ADR4). 비동기 I/O를 흉내냅니다."""
    await asyncio.sleep(0.05) # 외부 호출 지연을 await로 양보(c05)
    return PaymentResult(success=True, transaction_id="mock-tx-0001")
```

- 결제는 실제 게이트웨이 대신 `services/payment.py` 로 모킹합니다(ADR4). 인터페이스 (`charge(amount)`)만 같게 유지하면 나중에 실연동으로 교체할 수 있습니다.
- `await asyncio.sleep(0.05)` 은 c05의 핵심입니다. 블로킹 `time.sleep` 대신 `await` 로 이벤트 루프에 제어를 양보해, 결제를 기다리는 동안 다른 요청을 처리할 수 있습니다.

주문 라우터: `routers/orders.py`

```

from fastapi import APIRouter, BackgroundTasks, HTTPException
from sqlmodel import select

from ..auth import CurrentUser
from ..deps import SessionDep
from ..errors import OutOfStockError
from ..models import Order, OrderItem, Product
from ..schemas import OrderCreate, OrderRead
from ..services import email, payment

router = APIRouter(prefix="/orders", tags=["orders"])

@router.post("", response_model=OrderRead, status_code=201)
async def create_order(
    payload: OrderCreate,
    session: SessionDep,
    user: CurrentUser,
    background: BackgroundTasks,
) -> Order:
    order = Order(user_id=user.id, status="pending")
    total = 0.0
    line_items: list[OrderItem] = []

    for line in payload.items:
        product = session.get(Product, line.product_id)
        if product is None or not product.is_active:
            raise HTTPException(
                status_code=404, detail=f"상품 {line.product_id}를 찾을 수 없습니다"
            )
        if product.stock < line.quantity:
            raise OutOfStockError(product.id, line.quantity, product.stock)
        product.stock -= line.quantity
        total += product.price * line.quantity
        line_items.append(
            OrderItem(product_id=product.id, quantity=line.quantity,
                      unit_price=product.price)
        )

    result = await payment.charge(total)
    if not result.success:
        session.rollback()
        raise HTTPException(status_code=402, detail="결제에 실패했습니다")

    order.total = total
    order.status = "paid"
    order.items = line_items
    session.add(order)
    session.commit()
    session.refresh(order)

    background.add_task(email.send_order_confirmation, user.email, order.id)
    return order

@router.get("/me", response_model=list[OrderRead])
def list_my_orders(session: SessionDep, user: CurrentUser) -> list[Order]:

```



```

stmt = select(Order).where(Order.user_id == user.id) # 본인 주문만
return list(session.exec(stmt).all())

@router.delete("/{order_id}", status_code=204)
def cancel_order(order_id: int, session: SessionDep, user: CurrentUser) -> None:
    order = session.get(Order, order_id)
    if order is None:
        raise HTTPException(status_code=404, detail="주문을 찾을 수 없습니다")
    if order.user_id != user.id:
        # 인가(c09)
        raise HTTPException(status_code=403, detail="본인 주문만 취소할 수 있습니다")
    order.status = "cancelled"
    session.add(order)
    session.commit()

```

- 재고 확인·차감·결제·주문 저장에 하나의 세션(트랜잭션) 안에서 일어납니다(c07·ADR5). 결제 실패 시 `session.rollback()` 으로 차감을 되돌립니다.
- `OrderCreate` 의 `items` 는 c03의 중첩 모델(`list[OrderItemCreate]`)입니다. 빈 주문은 `Field(min_length=1)` 로 422 거부됩니다.
- `list_my_orders` · `cancel_order` 는 `user.id` 로 본인 것만 다뤄 FR8(본인 주문 인가)을 구현합니다. 남의 주문 취소 시도는 403입니다(c09).
- `background.add_task(...)` 는 c12의 백그라운드 작업입니다. 응답을 보낸 뒤 확인 메일을 발송합니다.

실행 결과

```

$ curl -X POST localhost:8000/orders -H "Authorization: Bearer <user>" \
  -d '{"items":[{"product_id":1,"quantity":2}]}'
{"id":1,"status":"paid","total":78000.0,"created_at":"2026-06-21T...",
  "items":[{"product_id":1,"quantity":2,"unit_price":39000.0}]}
# 상품 1의 재고가 10 -> 8로 차감됨

$ curl -X POST localhost:8000/orders -H "Authorization: Bearer <user>" \
  -d '{"items":[{"product_id":1,"quantity":999}]}'
{"detail":"재고가 부족합니다","product_id":1,"requested":999,"available":8} # 409

```

체크포인트 (단위 테스트)

```
def test_order_decrements_stock(client, user_headers, session, sample_product):
    """M4: 정상 주문이 재고를 차감하고 결제 성공으로 201."""
    res = client.post(
        "/orders",
        json={"items": [{"product_id": sample_product.id, "quantity": 2}]},
        headers=user_headers,
    )
    assert res.status_code == 201
    assert res.json()["status"] == "paid"

def test_order_out_of_stock_409(client, user_headers, sample_product):
    """M4: 재고보다 많은 수량은 409와 도메인 정보를 돌려준다."""
    res = client.post(
        "/orders",
        json={"items": [{"product_id": sample_product.id, "quantity": 9999}]},
        headers=user_headers,
    )
    assert res.status_code == 409
    assert res.json()["available"] >= 0
```

정상 주문 재고 차감·결제 성공과 재고 부족 409가 통과하면 M4 완료입니다.

15.6 M5 — 예외·미들웨어·백그라운드 메일·Dockerfile

적용 개념: c10(예외 핸들러·미들웨어·CORS), c12(백그라운드·배포)

M4에서 던진 `OutOfStockError` 를 앱 수준에서 일관된 409 응답으로 변환하고(c10), 모든 요청에 처리 시간 미들웨어와 CORS를 적용하며(c10), 메일 모킹과 Dockerfile로 배포를 준비합니다(c12).

완성된 `main.py`

```

import time
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from .config import get_settings
from .db import create_db_and_tables
from .errors import OutOfStockError
from .routers import auth, orders, products

@asynccontextmanager
async def lifespan(app: FastAPI):
    create_db_and_tables()
    yield

settings = get_settings()
app = FastAPI(title=settings.app_name, lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"], # 프론트엔드 출처만 허용
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.middleware("http")
async def add_process_time(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)
    response.headers["X-Process-Time"] = f"{time.perf_counter() - start:.4f}"
    return response

@app.exception_handler(OutOfStockError)
async def out_of_stock_handler(request: Request, exc: OutOfStockError):
    """재고 부족 도메인 예외를 409 응답으로 변환합니다(c10)."""
    return JSONResponse(
        status_code=409,
        content={"detail": "재고가 부족합니다", "product_id": exc.product_id,
                "requested": exc.requested, "available": exc.available},
    )

app.include_router(auth.router)
app.include_router(products.router)
app.include_router(orders.router)

@app.get("/health")

```

```
def health() -> dict[str, str]:
    return {"status": "ok"}
```

- `@app.exception_handler(OutOfStockError)` 로 도메인 예외를 한 곳에서 409로 변환합니다. 라우터는 예외를 던지기만 하고, 변환 책임은 앱이 집니다(c10의 도메인 예외→HTTP 변환 패턴).
- `@app.middleware("http")` 로 모든 응답에 `X-Process-Time` 헤더를 더합니다(c10). 미들웨어는 추가된 역순으로 요청을 감싸므로 등록 순서에 유의합니다.
- `CORSMiddleware` 는 `allow_origins` 에 명시한 출처만 허용합니다. `allow_credentials=True` 와 `allow_origins=["*"]` 는 함께 쓸 수 없다는 c10 오개념을 피해, 구체 출처를 지정했습니다.

메일 모킹과 Dockerfile

```
# services/email.py
def send_order_confirmation(email: str, order_id: int) -> None:
    """주문 확인 메일을 모킹합니다(ADR4). 실제 SMTP 대신 콘솔에 출력합니다."""
    print(f"[email] 주문 #{order_id} 확인 메일을 {email}로 발송")
```

```
# Dockerfile (멀티스테이지)
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY shop_api ./shop_api
EXPOSE 8000
CMD ["uvicorn", "shop_api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

운영에서는 `--reload` 를 쓰지 않고 워커 수를 조정한 프로세스로 띄웁니다(c12 오개념 교정).

실행 결과

```
$ docker build -t shop_api .
[+] Building 18.4s (12/12) FINISHED
$ docker run -p 8000:8000 -e SHOP_JWT_SECRET=prod-secret shop_api
INFO: Application startup complete.

$ curl -i localhost:8000/health
HTTP/1.1 200 OK
x-process-time: 0.0007
{"status":"ok"}
```

체크포인트 (단위 테스트)

```
def test_process_time_header(client):
    """M5: 모든 응답에 처리 시간 헤더가 붙는지 확인합니다."""
    res = client.get("/health")
    assert "x-process-time" in res.headers

def test_order_triggers_background_email(client, user_headers,
                                          sample_product, capsys):
    """M5: 주문 후 백그라운드 메일(모킹)이 실행되는지 확인합니다."""
    client.post(
        "/orders",
        json={"items": [{"product_id": sample_product.id, "quantity": 1}]},
        headers=user_headers,
    )
    assert "확인 메일" in capsys.readouterr().out
```

일관 오류 응답·CORS·처리 시간 헤더·백그라운드 실행·이미지 빌드가 확인되면 M5 완료입니다.

15.7 M6 — 테스트 스위트 통합과 통합 실행 확인

적용 개념: c11(pytest-TestClient-dependency_overrides)

각 마일스톤에서 단위 테스트를 만들어 왔습니다. 이제 그 토대인 픽스처를 정리하고, 인메모리 DB로 격리해 전체 스위트를 한 번에 돌립니다.

픽스처: `tests/conftest.py`

```

import pytest
from fastapi.testclient import TestClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool

from shop_api.auth import create_access_token, hash_password
from shop_api.db import get_session
from shop_api.main import app
from shop_api.models import Product, User

@pytest.fixture(name="session")
def session_fixture():
    """매 테스트마다 격리된 인메모리 SQLite를 만듭니다(c11)."""
    engine = create_engine(
        "sqlite://", connect_args={"check_same_thread": False},
        poolclass=StaticPool,
    )
    SQLModel.metadata.create_all(engine)
    with Session(engine) as session:
        yield session

@pytest.fixture(name="client")
def client_fixture(session):
    """get_session 의존성을 테스트 세션으로 교체합니다(c11)."""
    def override_get_session():
        yield session

    app.dependency_overrides[get_session] = override_get_session
    yield TestClient(app)
    app.dependency_overrides.clear() # 다음 테스트 오염 방지

@pytest.fixture
def admin_headers(client, session):
    admin = User(email="admin@shop.com",
                  password_hash=hash_password("password123"), is_admin=True)
    session.add(admin)
    session.commit()
    token = create_access_token(subject=admin.email)
    return {"Authorization": f"Bearer {token}"}

@pytest.fixture
def user_headers(client, session):
    user = User(email="user@shop.com",
                 password_hash=hash_password("password123"))
    session.add(user)
    session.commit()
    return {"Authorization": f"Bearer {create_access_token(user.email)}"}

@pytest.fixture
def sample_product(session):
    product = Product(name="키보드", category="office", price=39000, stock=10)
    session.add(product)

```

```
session.commit()
session.refresh(product)
return product
```

- `sqlite://` (파일 경로 없음)은 인메모리 DB입니다. `StaticPool` 로 같은 연결을 공유해 테스트 안에서 데이터가 유지됩니다(c11). 운영 DB가 아니라 격리된 테스트 DB에서 도는 것이 핵심입니다.
- `app.dependency_overrides[get_session]` 로 운영 세션을 테스트 세션으로 교체합니다(c11). 끝나면 `clear()` 로 비워 테스트 간 상태 오염을 막습니다(픽스처 초기화 오개념 교정).

통합 실행 결과

```
$ pytest -q
tests/test_auth.py .... [ 25%]
tests/test_products.py ..... [ 62%]
tests/test_orders.py ..... [100%]

16 passed in 1.42s
```

전체 스위트가 그린이면, 6개 마일스톤이 통합되어 로그인 → 상품 조회 → 주문 생성(재고 차감·결제 모킹) → 본인 주문 조회까지 end-to-end로 동작함이 자동으로 보장됩니다.

중분 통합의 효과: M2~M5 각각의 테스트가 M6에서 한 스위트로 모입니다. 이후 누가 코드를 고쳐 어느 마일스톤이 깨지면 `pytest` 가 즉시 알려 줍니다. 이것이 "항상 실행되는 상태를 유지한다"는 전략의 보상입니다.

15.8 작성형 연습 (2종 1세트)

모범: 주문 흐름의 핵심 — 재고 차감과 비동기 결제

15.5의 `create_order` 가 어떻게 c05·c07·c10을 한 핸들러에서 통합하는지 다시 봅니다. 재고 확인 → 차감 → 비동기 결제 → 커밋 → 백그라운드 메일의 순서가 트랜잭션 일관성과 동시성을 모두 만족합니다.

직접 작성: 주문 취소 시 재고 복원 핸들러 채우기

아래 빈칸 코드를 완성하세요. 현재 `cancel_order` 는 상태만 `cancelled` 로 바꿉니다. 취소 시 차감했던 재고를 **복원**하도록 확장하는 것이 과제입니다. 주문 항목(`OrderItem`)을 조회해 각 상품의 재고를 되돌립니다.

과제: 본인 주문 취소 + 재고 복원 (빈칸 코드 채우기)

```

@router.delete("/{order_id}", status_code=204)
def cancel_order(order_id: int, session: SessionDep, user: CurrentUser) -> None:
    order = session.get(Order, order_id)
    if order is None:
        raise HTTPException(status_code=404, detail="주문을 찾을 수 없습니다")

    # 1) 인가: 본인 주문만 취소할 수 있도록 막으세요(403).
    if _____:
        raise HTTPException(status_code=____, detail="본인 주문만 취소할 수 있습니다")

    # 2) 이미 취소·결제완료 상태 처리(선택): pending/paid만 취소 허용
    if order.status == "cancelled":
        return

    # 3) 재고 복원: 이 주문의 OrderItem들을 조회해 상품 재고를 되돌리세요.
    items = session.exec(
        select(OrderItem).where(_____)
    ).all()
    for item in items:
        product = session.get(Product, item.product_id)
        if product is not None:
            product.stock ____ item.quantity # 차감의 반대 연산
            session.add(product)

    # 4) 상태 전이 후 커밋
    order.status = "_____"
    session.add(order)
    session.commit()

```

자가체크 체크리스트

- [] 남의 주문을 취소하려 하면 403이 나는가? (c09 인가)
- [] 취소 후 해당 상품 재고가 주문 수량만큼 다시 늘어나는가? (c07 트랜잭션)
- [] 이미 취소된 주문을 다시 취소해도 재고가 이중으로 복원되지 않는가? (역등성)
- [] pytest 로 "주문 → 취소 → 재고 원복" 흐름을 단언하는 테스트를 작성했는가? (c11)
- [] 응답 상태코드가 204(본문 없음)인가? (c04)

두 번째 작성형: 상품 목록에 가격 정렬 추가하기**과제: list_products 에 정렬 파라미터 추가 (빈칸 코드 채우기)**

목록 API에 `sort` 쿼리 파라미터(`price_asc`·`price_desc`)를 더해 가격 순 정렬을 구현하세요.


```

@router.get("", response_model=list[ProductRead])
def list_products(
    session: SessionDep,
    page: PaginationDep,
    category: str | None = None,
    sort: Annotated[str | None, Query(pattern="^price_(asc|desc)$")] = None,
) -> list[Product]:
    stmt = select(Product).where(Product.is_active == True) # noqa: E712
    if category is not None:
        stmt = stmt.where(Product.category == category)

    # 빈칸: sort 값에 따라 정렬 절을 추가하세요.
    if sort == "price_asc":
        stmt = stmt.order_by(_____)
    elif sort == "price_desc":
        stmt = stmt.order_by(_____)

    stmt = stmt.offset(page.offset).limit(page.limit)
    return list(session.exec(stmt).all())

```

자가체크 체크리스트

- [] `sort=price_asc` 가 가격 오름차순으로 정렬되는가? (c02 쿼리·c08 정렬)
- [] 패턴에 맞지 않는 `sort` 값(예: `name`)에 422가 나는가? (c02 제약)
- [] 페이지네이션(limit/offset)과 정렬이 함께 동작하는가? (c08)
- [] 정렬을 추가해도 기존 목록 테스트가 그대로 통과하는가? (c11 회귀)

이 챕터에서 6개 마일스톤으로 `shop_api` 를 완성했습니다. 다음 `ch_16`에서는 이 산출물을 시연하고, 평가 루브릭으로 점검하며, 12개 단위 개념을 어떻게 통합 적용했는지 성찰합니다.

종합 프로젝트: 주문·이커머스 API — 발표·평가·성찰

ch_15에서 6개 마일스톤으로 `shop_api` 를 완성했습니다. 이제 프로젝트 수명주기의 마지막 단계입니다. 완성한 산출물을 공개적으로 시연하고, 평가 루브릭으로 자신과 동료의 작업을 점검하며, 12개 단원의 개념을 어떻게 하나의 서비스로 통합했는지 회고합니다. 이 단계는 단순한 마무리가 아니라, 흠어져 배운 지식을 자기 것으로 굳히는 **메타인지(metacognition)** 과정입니다.

이 챕터의 목표

- 완성한 주문 API를 OpenAPI 문서(/docs)와 실행으로 시연할 수 있습니다.
- 7개 기준의 평가 루브릭으로 자신과 동료의 작업을 평가할 수 있습니다.
- 12개 단원 개념(c01~c12)을 어떤 기능에 어떻게 통합 적용했는지 성찰할 수 있습니다.
- 한계(동시성·실연동)를 인식하고 구체적 개선안을 제시할 수 있습니다.

16.1 산출물 시연: 주문 흐름 데모와 /docs

발표의 핵심은 "동작한다"를 넘어 "왜 이렇게 설계했고 어떻게 검증했는가"를 보이는 것입니다. 시연은 드라이빙 퀘스천이 요구한 end-to-end 흐름을 따라갑니다: 로그인 → 상품 조회 → 주문 생성(재고 차감·결제 모킹) → 본인 주문 조회.

데모 시나리오 (실행 결과)

```
# 1) 가입·로그인 → JWT 발급 (c09)
$ curl -X POST localhost:8000/auth/register \
  -d '{"email":"buyer@shop.com","password":"password123"}'
{"id":2,"email":"buyer@shop.com","is_admin":false}

$ curl -X POST localhost:8000/auth/token \
  -d "username=buyer@shop.com&password=password123"
{"access_token":"eyJhbGciOiJIc2E1IiwiaXNjaWkiOiJkaWki","token_type":"bearer"}

# 2) 상품 탐색 — 카테고리 필터 + 페이지네이션 (c02·c08)
$ curl "localhost:8000/products?category=office&limit=20&offset=0"
[{"id":1,"name":"무선 키보드","category":"office","price":39000.0,"stock":10,"is_active":true}]

# 3) 주문 생성 — 재고 차감 + 비동기 결제(모킹) + 백그라운드 메일 (c05·c07·c10·c12)
$ curl -X POST localhost:8000/orders -H "Authorization: Bearer <token>" \
  -d '{"items":[{"product_id":1,"quantity":2}]}'
{"id":1,"status":"paid","total":78000.0,"created_at":"2026-06-21T09:12:00Z",
  "items":[{"product_id":1,"quantity":2,"unit_price":39000.0}]}
# 서버 콘솔: [email] 주문 #1 확인 메일을 buyer@shop.com로 발송

# 4) 본인 주문 조회 (c09 인가)
$ curl localhost:8000/orders/me -H "Authorization: Bearer <token>"
[{"id":1,"status":"paid","total":78000.0,...}]
```

/docs 시연 포인트

대화형 문서(<http://127.0.0.1:8000/docs>)는 코드를 쓰지 않고도 전체 API를 보여 주는 가장 강력한 발표 도구입니다(c01). 시연에서 다음을 강조합니다.

보여 줄 것	무엇을 증명하는가	관련 개념
Authorize 버튼으로 토큰 입력	인증이 표준 OAuth2 흐름으로 통합됨	c09
POST /products 스키마	입력 검증(422 예시)이 문서에 자동 기술됨	c03
응답 스키마에 password_hash 없음	출력 모델 분리로 민감 필드 차단	c04
주문 생성 시 409 응답 예시	도메인 예외가 일관된 상태코드로 노출됨	c10
모든 경로가 태그(products·orders·auth)로 묶임	라우터 모듈 분해가 문서에 반영됨	c08

시연 대본 권장 순서: (1) 드라이빙 퀘스천 1문장 → (2) /docs로 전체 구조 한눈에 → (3) 정상 주문 흐름 데모 → (4) 일부러 실패 보이기(재고 부족 409·무토큰 401) → (5) `pytest` 그린 화면 → (6) 한계와 개선안. "실패를 의도적으로 보여 주는" 단계가 견고성을 가장 잘 증명합니다.

한 화면으로 보는 주문 흐름

발표에서 가장 효과적인 슬라이드는 UC1(주문 생성) 한 흐름을 단계별로 짚는 것입니다. 각 단계가 어느 모듈·어느 개념과 맞닿는지 함께 말하면, 청중이 "코드가 설계를 따른다"는 점을 즉시 납득합니다.

단계	호출	동작	모듈 / 개념
1	<code>POST /auth/token</code>	해시 검증 후 JWT 발급	<code>routers/auth.py</code> · c09
2	<code>GET /products?...</code>	필터·페이지네이션 목록	<code>routers/products.py</code> · c02·c08
3	<code>POST /orders</code>	본문 검증(422 방어)	<code>schemas.py</code> · c03
4	(내부)	재고 확인·차감	<code>routers/orders.py</code> · c07
5	(내부)	비동기 결제(모킹)	<code>services/payment.py</code> · c05
6	(내부)	주문·항목 커밋	<code>models.py</code> · c07
7	(응답 후)	확인 메일 백그라운드	<code>services/email.py</code> · c12
8	201 응답	<code>OrderRead</code> 로 직렬화	<code>schemas.py</code> · c04

이 표 한 장이 ch_13의 유스케이스(UC1)·ch_14의 시퀀스 다이어그램·ch_15의 구현을 한 줄로 잇습니다. 발표를 듣는 사람은 "분석에서 말한 흐름이 코드에 그대로 있다"는 추적성을 확인하게 됩니다.

16.2 평가 루브릭

아래 7개 기준의 루브릭으로 자신의 작업을 먼저 평가하고(자기 평가), 이어 동료의 작업을 평가합니다. 각 기준은 미흡·보통·우수의 3수준이며, 가중치를 곱해 가중합으로 총점을 냅니다. 이 평가 기준은 ch_13의 요구사항·ch_14의 설계 결정과 일관됩니다.

기준 (가중치)	미흡	보통	우수
기능 완성도·주문 흐름 (0.25)	상품 조회만 동작	주문 생성·재고 차감 동작	주문·재고·결제(모킹)·인가·취소까지 end-to-end 동작
API 설계·검증 (0.15)	검증 없음·상태코드 부정확	입력 검증·기본 상태코드	입출력 모델 분리·정확한 상태코드·민감 필드 제외
인증·인가 (0.15)	보호 없음	로그인·토큰 검증	JWT·본인/관리자 인가·401/403 구분
데이터·트랜잭션·동시성 (0.15)	영속화 없음	DB 저장만	세션 트랜잭션·재고 차감·부족시 롤백
견고성·예외/미들웨어·비동기 (0.10)	오류에 500	기본 예외 처리	도메인 예외→상태코드·미들웨어·비동기 외부 호출
테스트·배포 (0.10)	테스트 없음	일부 테스트	격리 테스트 스위트·Docker 배포 구성
발표·성찰 (0.10)	동작만 시연	설계 의도 설명	ADR 근거·한계·개선안까지 회고

가중합 계산 예시

각 기준에 미흡=1, 보통=2, 우수=3점을 매기고 가중치를 곱합니다. 예를 들어 모든 기준에서 우수(3점)면 $3 \times (0.25+0.15+0.15+0.15+0.10+0.10+0.10) = 3.0$ 이 만점입니다. 어떤 학습자가 기능 완성도 우수(3), 나머지는 보통(2)이라면 $3 \times 0.25 + 2 \times 0.75 = 0.75 + 1.50 = 2.25 / 3.0$ 입니다.

작성형: 자기 평가 루브릭 채우기 (모범 + 빈 템플릿)

모범(예시 채움)

기준	내 수준	근거(코드/테스트 위치)
기능 완성도	우수	<code>routers/orders.py</code> 의 <code>create_order</code> 가 재고 차감·결제·메일까지 통합, <code>test_orders.py</code> 6건 통과
인증·인가	우수	<code>auth.py</code> 의 <code>get_current_user</code> (401)· <code>require_admin</code> (403), <code>test_auth.py</code> 에서 401/403 검증

과제: 자기 평가 루브릭 직접 작성

아래 빈 표를 자신의 `shop_api` 를 기준으로 채우세요. 각 기준의 수준 판정에 반드시 코드 위치나 통과 테스트를 근거로 적습니다(주장만 하지 말고 증거를 댑니다).

기준	내 수준(미흡/보통/우수)	근거(코드/테스트 위치)
기능 완성도·주문 흐름		
API 설계·검증		
인증·인가		
데이터·트랜잭션·동시성		
견고성(예외·미들웨어·비동기)		
테스트·배포		
발표·성찰		
가중합 총점		/ 3.0

자가체크 체크리스트

- ☐ 각 수준 판정에 코드 파일명·함수명 또는 테스트 이름을 근거로 달았는가?
- ☐ 우수로 매긴 기준이 실제 `pytest` 통과로 뒷받침되는가?
- ☐ 미흡·보통인 기준에 대해 다음 단계 개선안을 한 줄로 적었는가?
- ☐ 가중합 계산이 가중치 합(1.0) 기준으로 맞는가?

16.3 동료 피드백 (코드 리뷰·갤러리 워크)

자기 평가만으로는 사각지대가 남습니다. 동료의 눈으로 보면 내가 당연하게 여긴 결정의 약점이 드러납니다. 두 가지 방식으로 피드백을 주고받습니다.

- **갤러리 워크(gallery walk):** 각자 `/docs`와 `pytest` 결과를 띄워 두고 순회하며 다른 사람의 API 를 실제로 호출해 봅니다. "이 입력에 어떤 응답이 오는가"를 직접 시험합니다.
- **코드 리뷰(code review):** 한 모듈(예: `routers/orders.py`)을 골라 동료가 읽고, 아래 관점으로 구조적 피드백을 남깁니다.

코드 리뷰 관점 표

관점	점검 질문	관련 개념
검증	잘못된 입력이 422로 막히는가, 수동 if 검사로 중복하지 않았는가	c03
응답	내부·민감 필드가 응답에서 빠졌는가	c04
인가	본인/관리자 경계가 모든 변경 경로에 걸려 있는가	c09
트랜잭션	재고 차감·결제·저장이 한 트랜잭션 경계 안에 있는가	c07
비동기	외부 호출이 블로킹 없이 await 되는가	c05
테스트	핵심 실패 경로(409·401·403)가 모두 테스트되는가	c11

피드백 작성 규칙 (2종 1세트 중 모범)

좋은 피드백은 "고쳐라"가 아니라 "무엇을, 왜, 어떻게"를 담습니다. 예시:

(관점: 트랜잭션) `create_order` 에서 결제 실패 시 `session.rollback()` 을 호출하는 점이 좋습니다. 다만 결제 호출 전에 이미 `product.stock -= ...` 로 메모리상 차감이 일어나므로, 동시에 들어온 두 주문이 같은 재고를 보고 둘 다 통과할 여지가 있습니다(ADR5의 한계). 행 잠금(SELECT FOR UPDATE)이나 원자적 UPDATE로 강화하는 것을 제안합니다.

과제: 동료 코드에 피드백 3건 직접 작성

동료의 `shop_api` 한 모듈을 골라 위 6개 관점 중 3개에 대해 피드백을 작성하세요. 각 피드백은 다음 형식을 지킵니다.

[관점] _____
 - 잘한 점(구체적 위치): _____
 - 개선 제안(무엇을·왜·어떻게): _____
 - 관련 단위 개념: c__

자가체크 체크리스트

- ☐ 각 피드백이 "잘한 점"과 "개선 제안"을 모두 담았는가?
- ☐ 개선 제안이 막연한 비난이 아니라 실행 가능한 방법을 제시하는가?
- ☐ 피드백을 특정 코드 위치(파일·함수)에 연결했는가?
- ☐ 받은 피드백 중 하나를 실제로 반영하고 테스트로 검증했는가?

16.4 성찰 회고: 12단원 개념의 통합 적용

이 프로젝트의 가장 큰 교육적 가치는, 12개 단원에서 따로따로 배운 개념이 하나의 서비스 안에서 어떻게 맞물리는지를 직접 체험한 것입니다. 아래 표로 각 개념이 어느 기능으로 이어졌는지 전체를 회고합니다.

단원 개념	주문 API의 어느 기능으로 이어졌는가
c01 FastAPI 기초·자동 문서	앱 골격(<code>main.py</code>)과 <code>/docs</code> 시연·OpenAPI 자동 기술
c02 경로·쿼리 파라미터	상품 단건 조회(<code>/products/{id}</code>)·필터·페이지네이션 쿼리
c03 Pydantic 본문 검증	<code>ProductCreate</code> · <code>OrderCreate(items)</code> 로 입력 검증(422)
c04 응답 모델·상태 코드	<code>ProductRead</code> · <code>UserRead</code> 로 민감 필드 제외, 201/204/409
c05 <code>async/await</code> 비동기	결제 게이트웨이 비동기 호출(<code>await payment.charge</code>)
c06 의존성 주입	<code>SessionDep</code> · <code>PaginationDep</code> · <code>CurrentUser</code> 타입 별칭 주입
c07 <code>SQLModel</code> ·트랜잭션	테이블 모델·세션 의존성·재고 차감 트랜잭션
c08 CRUD 라우터·페이지네이션	<code>routers/products.py</code> 의 표준 CRUD·limit/offset
c09 OAuth2·JWT 인증·인가	로그인·JWT 발급·본인/관리자 인가(401/403)
c10 예외·미들웨어·CORS	<code>OutOfStockError</code> → 409 핸들러·X-Process-Time·CORS
c11 <code>pytest</code> · <code>TestClient</code>	인메모리 DB 격리 테스트 스위트(16건)
c12 백그라운드·설정·배포	확인 메일 백그라운드·Settings·lifespan·Dockerfile

핵심 통찰 회고

ch_13의 통찰(insight)은 "이커머스 주문 흐름은 검증·인증·트랜잭션·동시성·예외가 한 군데서 맞물리는 도메인"이라는 것이었습니다. `create_order` 핸들러 하나가 이 통찰을 그대로 증명합니다. 그 한 함수 안에서 c03(본문 검증)·c07(트랜잭션)·c05(비동기 결제)·c10(도메인 예외)·c09(인증)·c12(백그라운드)가 동시에 동작합니다. 개념을 따로 배울 때는 보이지 않던 "통합의 난점"이 여기서 드러났습니다.

개념이 서로를 떠받친 지점들

통합 프로젝트에서 진짜 학습은 "개념 A가 개념 B를 떠받친다"는 의존을 체험할 때 일어납니다. 몇 가지 대표적인 연결을 회고합니다.

- **c06(의존성 주입)이 c07·c09·c11을 동시에 떠받친다:** SessionDep (c07 세션)·CurrentUser (c09 인증)가 모두 Depends 로 주입되었기 때문에, c11 테스트에서 `app.dependency_overrides` 로 세션을 인메모리 DB로 갈아끼우는 한 줄이 가능했습니다. 의존성 주입을 안 썼다면 테스트 격리가 훨씬 어려웠을 것입니다. 즉 c06은 단독 기능이 아니라 다른 개념들이 교체·합성되게 만드는 "접착제"였습니다.
- **c03·c04(입출력 분리)가 c09(보안)를 떠받친다:** UserRead 에 `password_hash` 가 없다는 단순한 결정(ADR1)이, 인증 기능을 만들면서 비밀번호 해시가 응답으로 새지 않게 막아 주었습니다. 검증과 보안이 별개로 보였지만 출력 모델 한 곳에서 만났습니다.
- **c05(비동기)와 c07(트랜잭션)의 긴장:** 결제를 `await` 하는 동안 세션 트랜잭션이 열린 채 유지된다는 점은 주의가 필요했습니다. 외부 호출이 오래 걸리면 DB 커넥션을 오래 점유하므로, 실서비스에서는 결제를 트랜잭션 밖으로 빼거나 보상 트랜잭션(saga)을 고려해야 한다는 것을 체감했습니다. 두 개념이 만나는 지점에서 새로운 설계 고민이 생긴 것입니다.

전이(transfer) 점검: 좋은 학습의 증거는 "다른 맥락에서도 쓸 수 있는가"입니다. 이 프로젝트로 익힌 수명주기(분석→설계→구현→성찰)와 개념 매핑은 블로그 API·예약 API·파일 저장소 API(ch_13 후보들)에도 그대로 적용됩니다. 예약 API라면 "재고 차감"이 "좌석 충돌 방지"로, "결제 모킹"이 "외부 캘린더 연동 모킹"으로 바뀔 뿐 골격은 같습니다.

한계와 개선안

알려진 한계 (정직하게 기록)

- **동시성·재고 경쟁(ADR5의 한계):** 현재는 단일 트랜잭션 내 조회 후 차감 방식입니다. 동시에 들어온 두 주문이 같은 재고 값을 읽으면 재고가 음수가 될 여지가 있습니다. 개선안: 원자적 UPDATE (`UPDATE product SET stock = stock - :qty WHERE id = :id AND stock >= :qty`)로 갱신 행 수를 확인하거나, 비관적 행 잠금(SELECT FOR UPDATE)을 적용합니다.
- **결제·메일 모킹(ADR4):** 결정론을 위해 외부 연동을 모킹했습니다. 실서비스에서는 결제 실패·타임아웃·재시도·멱등 키(idempotency key)를 다뤄야 하며, 메일은 큐(예: Celery)로 분리합니다.
- **JWT 갱신 부재:** 액세스 토큰 만료만 있고 리프레시 토큰이 없습니다. 개선안: 리프레시 토큰과 토큰 폐기(블랙리스트) 전략 추가.
- **마이그레이션:** `create_all` 로 테이블을 만들 뿐 스키마 변경 이력 관리가 없습니다. 개선안: Alembic 마이그레이션 도입.

16.5 작성형 연습 (2종 1세트): 성찰 템플릿

모범: 성찰 답변 예시

프롬프트 2 — 결제·메일을 모킹으로 둔 결정(ADR4)이 테스트 결정론에 어떤 도움을 주었는가?

결제를 `services/payment.py` 로 모킹했기 때문에 `test_order_decrements_stock` 이 외부 네트워크 없이 항상 같은 결과를 냅니다. 실제 게이트웨이였다면 네트워크 지연·키 발급·실패율 때문에 CI에서 테스트가 불안정(flaky)했을 것입니다. 인터페이스(`charge(amount)`)를 동일하게 유지했으므로, 추후 실연동으로 교체할 때 라우터 코드는 바꾸지 않아도 됩니다. 다만 "모킹이 항상 성공"이라 결제 실패 경로 테스트는 별도로 모킹을 실패 반환으로 바꿔 검증해야 한다는 한계를 배웠습니다.

직접 작성: 성찰 템플릿 채우기

아래 다섯 가지 성찰 프롬프트에 답하세요. 각 답변은 **구체적 코드·결정·테스트에** 근거해야 하며, "잘됐다" 같은 추상적 소감은 피합니다.

과제: 성찰 회고 작성 (작성형 템플릿)

[프롬프트 1] 12개 단원의 어떤 개념이 주문 API의 어느 기능으로 이어졌는가?

- 가장 강하게 통합됐다고 느낀 개념 3개와 그 위치:

- 1) c__ → _____(파일·함수)
- 2) c__ → _____
- 3) c__ → _____

[프롬프트 2] 결제·메일을 모킹으로 둔 결정(ADR4)이 테스트 결정론에 준 도움은?

- 모킹이 없었다면 어떤 테스트가 불안정해졌을까:
- 모킹의 한계로 추가로 검증해야 할 경로:

[프롬프트 3] 동시 주문이 재고를 음수로 만들지 않도록 한 방식의 한계와 강화안은?

- 현재 방식(ADR5)의 위험 시나리오:
- 실서비스 강화 방법(원자적 UPDATE / 행 잠금 등):

[프롬프트 4] JWT 인증을 세션 쿠키로 바꾼다면 어디를 고치고 트레이드오프는?

- 고쳐야 할 모듈·함수:
- 무상태성·CSRF·확장성 측면의 트레이드오프:

[프롬프트 5] 다시 설계한다면 모듈 구조나 트랜잭션 경계를 어떻게 바꾸겠는가?

- 바꾸고 싶은 구조와 이유:

자가체크 체크리스트

- [] 각 답변이 추상적 소감이 아니라 구체적 코드 결정에 근거하는가?
- [] 프롬프트 1에서 개념(c01~c12)과 코드 위치를 정확히 연결했는가?
- [] 한계(프롬프트 3)에 대해 실행 가능한 강화안을 제시했는가?
- [] 트레이드오프(프롬프트 4)를 한쪽 주장이 아니라 양면으로 적었는가?
- [] 다섯 프롬프트에 모두 답했고, 최소 하나는 "다음에 직접 해 볼 개선"으로 이어지는가?

마무리: 드라이빙 퀘스천으로 돌아오기

이 종합 프로젝트는 다음 질문에서 출발했습니다.

드라이빙 퀘스천: "실무 백엔드 개발자가 FastAPI로, 상품·재고·주문·인증·결제 흐름을 갖추고 테스트·배포까지 되는 안전하고 견고한 이커머스 주문 API를 어떻게 설계하고 구현할 수 있을까?"

답: 12개 단원의 개념을 백워드 설계로 시퀀싱해 배우고(ch_01~ch_12), 요구사항·설계·구현·성찰의 수명주기로(ch_13~ch_16) `shop_api`를 만들었습니다. 검증(c03)·응답 통제(c04)·인증/인가(c09)·트랜잭션(c07)·비동기(c05)·예외(c10)·테스트(c11)·배포(c12)가 하나의 주문 흐름 안에서 맞물릴 때, "안전하고 견고한" API가 됩니다.

여기서 배운 것은 특정 프로젝트의 코드가 아니라, **개념을 통합해 신뢰할 수 있는 서비스를 만드는 사고의 틀**입니다. 다음 프로젝트(블로그 API·예약 API 등)에서도 같은 수명주기와 같은 개념 매핑으로 접근할 수 있습니다. 한계로 남긴 동시성·실연동·마이그레이션은 다음 단계의 학습 목표가 됩니다.

다음 단계 로드맵

이 교재의 범위(scope_out, ch_13)는 의도적으로 제한했습니다. 모킹과 단일 트랜잭션으로 결정론을 확보하는 대신, 실서비스에서 마주칠 주제들을 다음 단계로 남겼습니다. 완성한 `shop_api`를 토대로 다음을 하나씩 엮어 보면 좋습니다.

다음 주제	무엇을 바꾸는가	이어지는 개념
Alembic 마이그레이션	<code>create_all</code> 대신 스키마 변경 이력 관리	c07 확장
재고 원자적 차감·행 잠금	<code>create_order</code> 의 경쟁 조건 제거	c07·c10 심화
실 결제 게이트웨이 연동	<code>services/payment.py</code> 모킹을 실 SDK로 교체(인터페이스 유지)	c05 심화
작업 큐(Celery-RQ)	무거운 메일·정산을 BackgroundTasks에서 분리	c12 확장
리프레시 토큰·권한 스코프	토큰 만료·폐기·세분화된 인가	c09 심화
관측성(로깅·메트릭·트레이싱)	미들웨어를 구조화 로깅·OpenTelemetry로 확장	c10 확장

각 주제는 새로운 개념을 더하는 것이 아니라, 이미 배운 개념을 실무 강도로 끌어올리는 일입니다. 모듈 인터페이스(라우터·서비스·의존성)를 일관되게 유지했기 때문에, 한 모듈씩 교체하며 점진적으로 고도화할 수 있습니다. 이것이 ch_14에서 모듈을 분해하고 ADR로 결정을 기록해 둔 보상입니다.

성찰의 마지막 한 문장: "이 프로젝트에서 가장 의외였던 통합 지점은 무엇이었고, 다음에 비슷한 서비스를 만든다면 무엇을 가장 먼저 다르게 하겠는가?" 이 한 문장에 답할 수 있다면, 12개 개념이 단순 지식이 아니라 설계 판단으로 내재화된 것입니다.